From Prompt to Response

An End-to-End Journey Inside LLM and RAG Systems

Table of contents

Welcome 1

I The Art of the True Name 3

1 the intro 5

II The Runes Beneath Language 7

2 APIs and Model Serving 9
- 2.1 The gap between training and deployment 9
- 2.2 The anatomy of an LLM API 10
- 2.3 From checkpoint to serving: the loading pipeline 11
- 2.4 The serving architecture 12
- 2.5 Batching: the key to serving efficiently 13
- 2.6 KV cache management 14
- 2.7 Latency and throughput tradeoffs 15
- 2.8 Key takeaways 15
- 2.9 Further reading 17

3 Tokenization during inference 19
- 3.1 Two tokenization chapters, two different problems 19
- 3.2 The encoding pipeline 19
- 3.3 Token continuation and context window management 21
- 3.4 The decoding pipeline: from numbers back to text 22
- 3.5 Logprobs and their uses at inference time 23
- 3.6 Common failure modes 24
- 3.7 Key takeaways 24
- 3.8 Further reading 25

4 Embeddings and Meaning Representation 27
- 4.1 The problem with integers 27
- 4.2 The embedding table 28
- 4.3 How embeddings are learned 28
- 4.4 The geometry of embedding space 29
- 4.5 Contextual vs. static embeddings 30
- 4.6 Embedding models vs. generation models 30
- 4.7 Similarity metrics 32
- 4.8 Embedding dimensions and matryoshka representations 33
- 4.9 Embeddings in the generative inference pipeline 33
- 4.10 Practical embedding operations 34

4.11 Key takeaways . . . 35
4.12 Further reading . . . 37

III The Labyrinth of Attention **39**

5 Transformer Architecture **41**
5.1 Why the transformer won . . . 41
5.2 The intuition before the math . . . 42
5.3 The high-level forward pass . . . 42
5.4 Token embeddings and positional encoding . . . 43
5.5 Self-attention: the core mechanism . . . 45
5.6 Multi-head attention . . . 47
5.7 The feed-forward network . . . 48
5.8 Residual connections and layer normalization . . . 48
5.9 Stacking transformer blocks . . . 50
5.10 Architecture variants . . . 50
5.11 The output layer . . . 53
5.12 What the model actually learns . . . 54
5.13 Key takeaways . . . 54
5.14 Further reading . . . 54

6 Attention Computation **57**
6.1 The quadratic problem . . . 57
6.2 What the GPU actually does during attention . . . 58
6.3 Standard attention: the naive computation . . . 58
6.4 FlashAttention . . . 59
6.5 Multi-head attention at inference . . . 60
6.6 The KV cache at inference . . . 61
6.7 Sparse and linear attention . . . 61
6.8 Ring attention and sequence parallelism . . . 63
6.9 Attention in the decode loop: step by step . . . 64
6.10 Key takeaways . . . 65
6.11 Further reading . . . 65

7 KV Cache **67**
7.1 The problem that KV cache solves . . . 67
7.2 What is stored . . . 68
7.3 Attention with and without a KV cache . . . 68
7.4 Memory arithmetic . . . 69
7.5 The lifecycle of a KV cache entry . . . 70
7.6 Naive allocation and its problems . . . 71
7.7 PagedAttention . . . 71
7.8 KV cache quantization . . . 73
7.9 KV cache eviction . . . 73
7.10 Key takeaways . . . 74
7.11 Further reading . . . 74

8 GPU Execution **77**
8.1 Why GPU execution deserves its own chapter . . . 77
8.2 GPU architecture fundamentals . . . 78
8.3 The roofline model . . . 79
8.4 Prefill vs. decode: two different regimes . . . 80
8.5 Kernel fusion . . . 81
8.6 Precision and quantization at the kernel level . . . 82

8.7 Tensor parallelism at the kernel level . . . 83
8.8 CUDA graph capture . . . 84
8.9 Key takeaways . . . 84
8.10 Further reading . . . 85

9 Quantization and Model Compression 87
9.1 The memory problem . . . 87
9.2 Floating point formats . . . 88
9.3 Integer quantization . . . 88
9.4 The outlier problem . . . 89
9.5 Post-training quantization methods . . . 89
9.6 GGUF and llama.cpp quantization . . . 90
9.7 Quantization-aware training . . . 91
9.8 Activation quantization . . . 92
9.9 Structured and unstructured sparsity . . . 93
9.10 Combining quantization techniques . . . 93
9.11 Quality evaluation for quantized models . . . 93
9.12 Key takeaways . . . 94
9.13 Further reading . . . 96

10 Decoding and Sampling 97
10.1 The decoding problem . . . 97
10.2 Greedy decoding . . . 98
10.3 Beam search . . . 98
10.4 Temperature sampling . . . 99
10.5 Top-k sampling . . . 99
10.6 Nucleus (top-p) sampling . . . 100
10.7 Min-p sampling . . . 101
10.8 Repetition penalties . . . 101
10.9 Structured decoding and constrained generation . . . 102
10.10 Best-of-n sampling . . . 103
10.11 Speculative sampling . . . 103
10.12 Chain-of-thought and reasoning tokens . . . 105
10.13 Decoding strategy selection guide . . . 105
10.14 Key takeaways . . . 106
10.15 Further reading . . . 108

IV The Forbidden Archive 109

11 Parametric Memory vs External Memory 111
11.1 Two different ways of knowing . . . 111
11.2 What parametric memory is . . . 112
11.3 What external memory is . . . 112
11.4 The complementary structure . . . 113
11.5 When to rely on parametric memory . . . 113
11.6 When to rely on external memory . . . 114
11.7 The interaction between the two . . . 114
11.8 Forms of external memory beyond RAG . . . 115
11.9 Designing the memory split . . . 116
11.10 The emerging continuum . . . 116
11.11 Key takeaways . . . 117
11.12 Further reading . . . 117

12 Document Ingestion 119

12.1 The ingestion problem . . . 119
12.2 Document formats and their challenges . . . 120
12.3 Text cleaning . . . 121
12.4 Document metadata extraction . . . 122
12.5 The ingestion pipeline assembled . . . 123
12.6 Ingestion at scale . . . 125
12.7 Key takeaways . . . 125
12.8 Further reading . . . 126

13 Chunking Strategies **127**
13.1 Why chunking exists and why it is hard . . . 127
13.2 Fixed-size chunking . . . 128
13.3 Sentence-based chunking . . . 129
13.4 Recursive character text splitting . . . 129
13.5 Structure-aware chunking . . . 130
13.6 Semantic chunking . . . 132
13.7 Chunk enrichment . . . 134
13.8 Chunk size selection . . . 136
13.9 Multi-granularity retrieval . . . 136
13.10Key takeaways . . . 137
13.11Further reading . . . 138

14 Embeddings for Search **141**
14.1 This chapter vs. chapter 06 . . . 141
14.2 The retrieval embedding task . . . 142
14.3 Choosing an embedding model . . . 142
14.4 Asymmetric encoding and instruction prefixes . . . 143
14.5 Batching and throughput for corpus embedding . . . 144
14.6 Domain adaptation . . . 146
14.7 Embedding stability and versioning . . . 147
14.8 Embedding quality signals . . . 148
14.9 Late interaction models: ColBERT . . . 150
14.10Key takeaways . . . 150
14.11Further reading . . . 151

15 Vector Databases **153**
15.1 What a vector database does . . . 153
15.2 How ANN Search Works . . . 154
15.3 Vector database landscape . . . 157
15.4 Metadata filtering . . . 161
15.5 Key takeaways . . . 162
15.6 Further reading . . . 163

16 Retrieval Algorithms **165**
16.1 The retrieval problem . . . 165
16.2 Dense retrieval: vector similarity search . . . 166
16.3 Sparse retrieval: BM25 . . . 166
16.4 Hybrid retrieval . . . 167
16.5 Query expansion and reformulation . . . 168
16.6 Contextual and conversational retrieval . . . 168
16.7 Knowledge graph retrieval . . . 169
16.8 Retrieval evaluation . . . 170
16.9 The full retrieval pipeline . . . 171
16.10Key takeaways . . . 172
16.11Further reading . . . 172

17 Reranking **175**
17.1 Why reranking exists as a separate stage . . . 175
17.2 The spectrum of reranking approaches . . . 176
17.3 Score-based reranking . . . 176
17.4 Cross-encoder reranking . . . 177
17.5 LLM-based reranking . . . 179
17.6 Diversity-aware reranking . . . 181
17.7 Contextual compression . . . 183
17.8 The full reranking pipeline . . . 184
17.9 Training custom rerankers . . . 185
17.10 Measuring reranking quality . . . 185
17.11 Key takeaways . . . 186
17.12 Further reading . . . 188

18 Context Construction **189**
18.1 The gap between retrieval and generation . . . 189
18.2 Anatomy of a RAG prompt . . . 190
18.3 The system prompt . . . 190
18.4 Retrieved context formatting . . . 191
18.5 Ordering retrieved chunks . . . 193
18.6 Context window budget management . . . 194
18.7 Deduplication and redundancy handling . . . 196
18.8 Handling contradictions in retrieved context . . . 197
18.9 Query placement and framing . . . 198
18.10 Context sufficiency checking . . . 199
18.11 Structured context for structured queries . . . 200
18.12 Citation and attribution in the response . . . 201
18.13 Putting it all together . . . 202
18.14 Key takeaways . . . 203
18.15 Further reading . . . 203

19 Grounded Generation **205**
19.1 The three failure modes . . . 205
19.2 Extending the system prompt for faithfulness . . . 206
19.3 Chain-of-thought for faithfulness . . . 207
19.4 Faithfulness at the token level . . . 207
19.5 Claim extraction and verification . . . 208
19.6 Self-RAG: faithfulness signals baked into generation . . . 209
19.7 FLARE: active retrieval during generation . . . 210
19.8 Handling insufficient context . . . 211
19.9 Post-hoc citation attribution . . . 212
19.10 Consistency sampling . . . 212
19.11 Post-generation faithfulness filtering . . . 213
19.12 Grounded generation in production . . . 214
19.13 Key takeaways . . . 215
19.14 Further reading . . . 215

V The Forging of Intelligence **217**

20 Data Collection and Curation **219**
20.1 Why data is the real foundation . . . 219
20.2 What scale actually means . . . 220
20.3 The primary sources . . . 220

20.4 The data pipeline . . . 221
20.5 Deduplication: the underrated problem . . . 222
20.6 Quality filtering: what “quality” means . . . 222
20.7 Domain mixing and data proportions . . . 223
20.8 The data contamination problem . . . 224
20.9 The web as a cultural artifact . . . 224
20.10 Key takeaways . . . 224
20.11 Further reading . . . 225

21 Synthetic Data Generation **227**
21.1 Why synthetic data exists . . . 227
21.2 Self-Instruct: the foundational approach . . . 228
21.3 Teacher-student pipelines . . . 228
21.4 Rejection sampling for verifiable tasks . . . 229
21.5 Generating reasoning traces . . . 229
21.6 Constitutional AI and RLAIF . . . 230
21.7 Synthetic conversation data . . . 231
21.8 Data contamination and quality collapse . . . 232
21.9 What synthetic data is good at (and what it is not) . . . 232
21.10 Key takeaways . . . 233
21.11 Further reading . . . 233

22 Tokenization During Training **235**
22.1 The Problem That Tokenization Solves . . . 235
22.2 Byte Pair Encoding . . . 236
22.3 WordPiece . . . 237
22.4 Unigram Language Model Tokenization . . . 238
22.5 SentencePiece . . . 238
22.6 Vocabulary Size: The Fundamental Tradeoff . . . 239
22.7 Special Tokens . . . 239
22.8 Tokenizer Training Data and Its Consequences . . . 240
22.9 The Relationship Between Tokenizer and Model Training . . . 240
22.10 Tokenizer Pathologies and Failure Modes . . . 241
22.11 Tokenizer Compatibility and Model Identity . . . 242
22.12 The tiktoken Implementation . . . 242
22.13 Key Takeaways . . . 242
22.14 Further Reading . . . 243

23 Training Objectives **245**
23.1 What a Training Objective Is . . . 245
23.2 Causal Language Modeling . . . 246
23.3 Why Next-Token Prediction Produces Capable Models . . . 247
23.4 Masked Language Modeling . . . 248
23.5 Sequence-to-Sequence Objectives . . . 249
23.6 The Cross-Entropy Loss in Practice . . . 249
23.7 From Pretraining to Instruction Following . . . 250
23.8 Continued Pretraining and Domain Adaptation . . . 250
23.9 Connecting the Objective to What the Model Learns . . . 251
23.10 Key Takeaways . . . 251
23.11 Further Reading . . . 252

24 The Training Loop **255**
24.1 What learning means for a language model . . . 255
24.2 The objective: cross-entropy loss . . . 256
24.3 The forward pass . . . 256

24.4 The backward pass: backpropagation . . . 256
24.5 The optimizer . . . 257
24.6 The learning rate schedule . . . 258
24.7 Gradient clipping . . . 259
24.8 Mixed precision training . . . 259
24.9 Batch size and gradient accumulation . . . 260
24.10 Activation checkpointing . . . 260
24.11 Checkpointing and fault tolerance . . . 261
24.12 The full training step, assembled . . . 261
24.13 What to monitor during training . . . 262
24.14 The Chinchilla finding: compute-optimal training . . . 262
24.15 Key takeaways . . . 263
24.16 Further reading . . . 263

25 Distributed Training 265
25.1 The Problem: Nothing Fits . . . 265
25.2 Data Parallelism . . . 266
25.3 ZeRO: Sharding Optimizer State, Gradients, and Parameters . . . 267
25.4 Tensor Parallelism . . . 270
25.5 Pipeline Parallelism . . . 272
25.6 3D Parallelism: Combining All Three . . . 272
25.7 Communication Primitives and Interconnects . . . 273
25.8 Key Takeaways . . . 273
25.9 Further Reading . . . 274

26 Distillation 277
26.1 The Deployment Problem That Distillation Solves . . . 277
26.2 Knowledge Distillation: The Core Mechanism . . . 278
26.3 Types of Distillation for Language Models . . . 279
26.4 Distillation at Scale: LLM-to-LLM . . . 280
26.5 Reasoning Distillation . . . 281
26.6 Self-Distillation and Compression Without a Teacher . . . 282
26.7 Distillation vs. Fine-Tuning vs. Training from Scratch . . . 283
26.8 Distillation and Quantization: Stacking Compression . . . 283
26.9 Key Takeaways . . . 283
26.10 Further Reading . . . 284

27 Fine-Tuning and Parameter-Efficient Adaptation 287
27.1 What Fine-Tuning Is and What It Is Not . . . 287
27.2 Full Fine-Tuning . . . 288
27.3 The Case for Parameter Efficiency . . . 289
27.4 LoRA: Low-Rank Adaptation . . . 289
27.5 QLoRA: Quantized LoRA . . . 291
27.6 Prefix Tuning and Prompt Tuning . . . 291
27.7 Adapter Layers . . . 292
27.8 LoRA Variants and Extensions . . . 292
27.9 Instruction Fine-Tuning . . . 293
27.10 Catastrophic Forgetting and How to Manage It . . . 294
27.11 Serving Multiple Fine-Tuned Models . . . 294
27.12 Choosing Between Fine-Tuning Approaches . . . 295
27.13 Key Takeaways . . . 295
27.14 Further Reading . . . 297

28 Alignment and Preference Optimization 299
28.1 The Gap Between Competence and Alignment . . . 299

28.2 Reward Models . . . 300
28.3 RLHF: Reinforcement Learning from Human Feedback . . . 301
28.4 Direct Alignment Methods: Beyond RLHF . . . 308
28.5 RL Without a Value Model: GRPO . . . 310
28.6 Constitutional AI and AI Feedback . . . 310
28.7 Reward Hacking and Alignment Failure Modes . . . 311
28.8 Practical Alignment Pipelines . . . 312
28.9 Key Takeaways . . . 313
28.10 Further Reading . . . 313

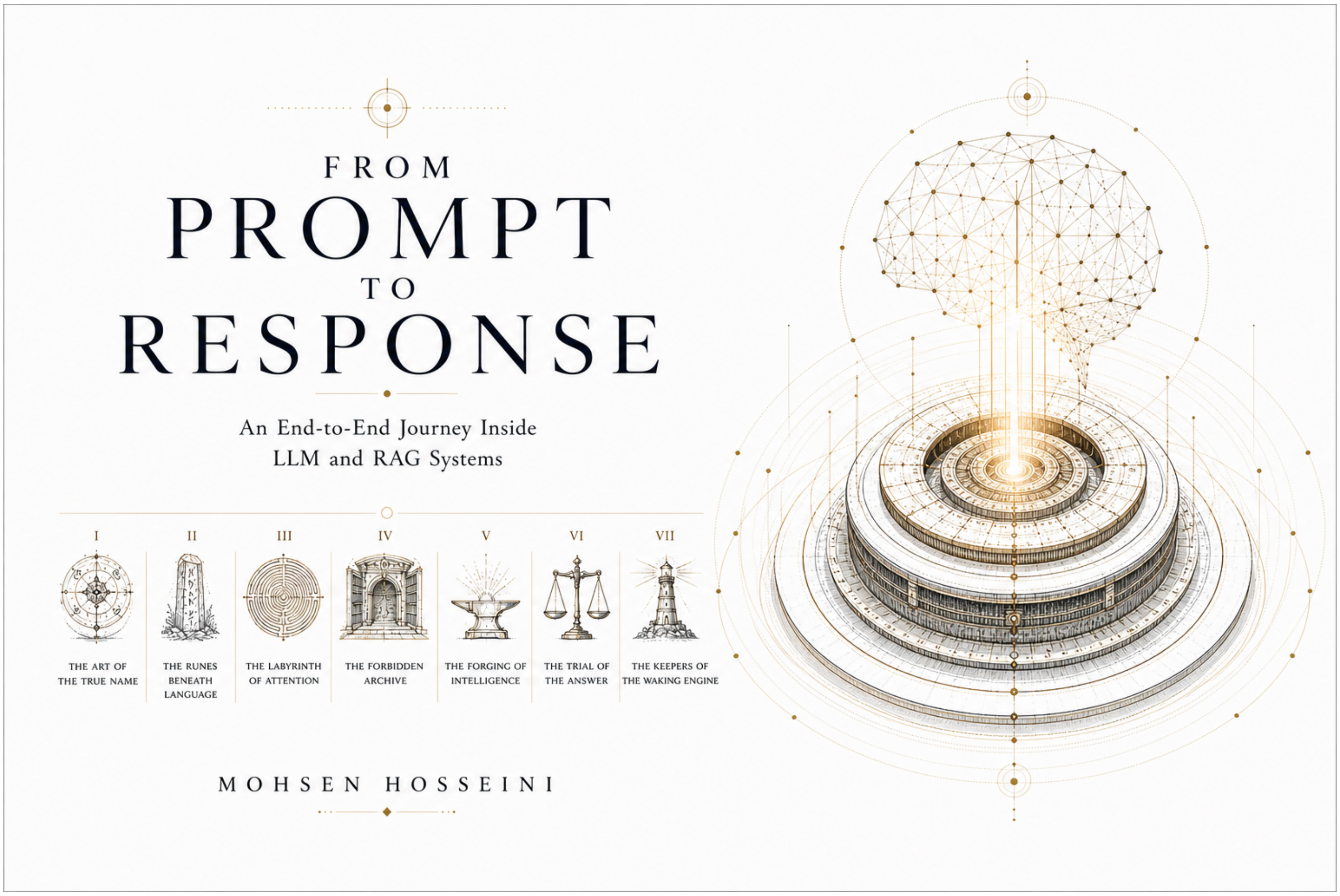
29 Reasoning Models and Test-Time Compute **317**
29.1 The Core Idea: Thinking as Computation . . . 317
29.2 Chain-of-Thought Prompting: The Precursor . . . 318
29.3 From Prompting to Training: Teaching Models to Reason . . . 318
29.4 OpenAI o1 and the Deliberate Reasoning Paradigm . . . 319
29.5 DeepSeek-R1: Open Reasoning at Scale . . . 320
29.6 Test-Time Compute Scaling Strategies . . . 321
29.7 Limitations and Failure Modes . . . 322
29.8 Key Takeaways . . . 322
29.9 Further Reading . . . 323

30 Embedding Model Training **325**
30.1 Two Kinds of Learning . . . 325
30.2 The Contrastive Learning Framework . . . 326
30.3 In-Batch Negatives and Their Limitations . . . 327
30.4 Hard Negative Mining . . . 327
30.5 Why Encode Queries and Documents Differently . . . 328
30.6 Training Data for Embedding Models . . . 329
30.7 Model Architecture Choices . . . 329
30.8 The Two-Stage Training Pipeline . . . 330
30.9 Why Embedding Models Fail . . . 331
30.10 Key Takeaways . . . 332
30.11 Further Reading . . . 332

VI The Trial of the Answer **333**

VII The Keepers of the Waking Engine **335**

Welcome



A technical deep-dive into the full LLM stack, from training and alignment through serving, retrieval, evaluation, and production operation. Every chapter is built around a single question that unlocks a layer of the system.

Designed for engineers and researchers who work with LLM systems and want to understand not just how to use them, but how they behave end-to-end under real constraints.

Part I

The Art of the True Name

Chapter 1

the intro

Part II

The Runes Beneath Language

Chapter 2

APIs and Model Serving

The canonical question for this chapter: *A trained model is a file on a disk. How does it become something millions of people can query simultaneously, with low latency, high reliability, and predictable cost?*

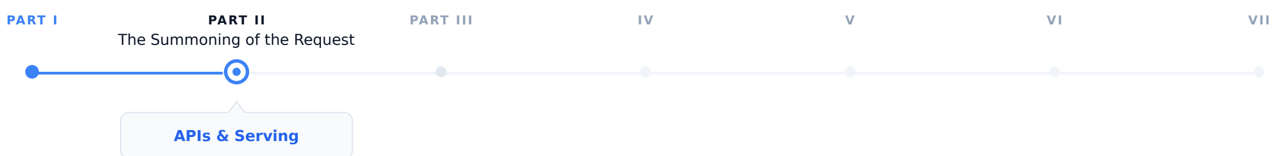


Figure 2.1: The journey through the Model Mind.

The prompt has been written and structured. Now it needs to travel somewhere. This chapter covers everything between the user pressing Enter and the model receiving the request: the wire format, the serving infrastructure, and the engineering decisions that make large-scale LLM deployment economical.

2.1 The gap between training and deployment

Training produces a checkpoint: a collection of weight tensors saved to disk. At this point, the model can do nothing on its own. It has no interface. It cannot receive requests. It does not know what a user is.

Turning that checkpoint into a production service requires solving a completely different set of engineering problems from those that were involved in training. During training, we try to optimize for producing as many tokens as possible per second. On the other hand, during serving, the optimization focuses on latency, concurrency, reliability, and cost, which means we want to respond to requests quickly, serve many users simultaneously, remain available under failure, and do all of this economically.

These requirements require a different approach. The same GPU cluster that runs training efficiently is not automatically an efficient serving infrastructure. Serving is its own engineering discipline, with its own abstractions, its own failure modes, and its own optimization techniques.

This chapter covers the full stack from trained checkpoint to the API endpoint a user or application calls.

2.2 The anatomy of an LLM API

The standard interface for production LLM services is an HTTP API, almost universally following the conventions established by OpenAI's API.

2.2.1 The chat completions endpoint

The primary endpoint for most production use:

`POST /v1/chat/completions`

Request body:

```json
{
  "model": "gpt-4o",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What is the capital of France?"}
  ],
  "max_tokens": 100,
  "temperature": 0.7,
  "stream": false
}
```

Response:

```json
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1699000000,
  "model": "gpt-4o",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "The capital of France is Paris."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 26,
    "completion_tokens": 9,
    "total_tokens": 35
  }
}
```

Several design decisions in this interface iluminates the realities of LLM serving:

Messages array instead of a single string. Conversations have a clear structure consisting of a system prompt, user turns, and assistant turns. The API explicitly defines the structure, instead of relying on concatenated strings and expecting the model to interpret the formatting correctly.

Token counts in the response. Because LLM cost and latency are proportional to token count, the API reports exactly how many tokens were consumed allowing costs to be monitored and controlled programmatically.

`finish_reason`. Why did the model stop generating? `stop` means it produced an end-of-sequence token. `length` means it reached the `max_tokens`. `content_filter` means the response was filtered. These signals help to determine whether the response is complete

`stream` parameter. Whether to stream tokens as they are generated or return the complete response when done. Streaming is architecturally significant and covered in detail in chapter "REFF".

2.2.2 The completions endpoint (legacy)

An older interface that takes the raw string prompt rather than the structured messages array:

```json
{
  "model": "gpt-3.5-turbo-instruct",
  "prompt": "The capital of France is",
  "max_tokens": 10
}
```

This endpoint represents an earlier era of LLM deployment, when models were not chat tuned and the interface was closer to a text continuation service. It is still useful for non-chat use cases but has been superseded by chat completions for most applications.

2.2.3 Key parameters

`temperature` controls the randomness of sampling. At temperature 0, the model always selects the most likely next token (greedy decoding). At temperature 1, it samples from a full distribution. Above 1, the distribution flattens, producing more random and unstable outputs. Temperature interacts with the decoding strategy covered in chapter "REFF".

`top_p` (nucleus sampling) this is an alternative to temperature for controlling randomness. The model samples from the smallest set of tokens in which the cumulative probability exceeds `top_p`. Setting `top_p = 0.9` restricts sampling to tokens that together account for 90% of the probability mass that means the long tail of unlikely tokens is ignored.

`max_tokens` sets the maximum number of tokens to generate. It functions as a cost and latency cap. If the model reaches this limit before producing a stop token, `finish_reason` is `length` and the response may be end mid-sentence.

Stop sequences are one or more strings that cause halt in generation immediately if the model produces them. They are useful for structured generation where you know the output should end at a specific delimiter.

`frequency_penalty` and `presence_penalty` discourage repetition. `frequency_penalty` applies a penalty proportional to how many time a token has already appeared. `presence_penalty` applies a flat penalty to any token that has appeared at all. Both help the model to prevent from getting stuck in repetitive loops.

`n` specifies how many independent completions to generate for a single prompt. Useful for best-of-n sampling where you generate multiple responses and select the best one.

`logprobs` returns the log probabilities of the selected tokens and also the top-k most likely tokens at each position. It is used for calibration, analysis, and classification tasks where you care more about the model's confidence, not just its output.

2.3 From checkpoint to serving: the loading pipeline

Before a model can serve requests, it must be loaded. For a model with hundreds of billions of parameters, this is a nontrivial task.

2.3.1 Checkpoint format and sharding

Training checkpoints are often sharded due to the fact that the full model does not fit in the memory of a single storage node or loading process. A 70B parameter model in BF16 requires approximately 140 GB of storage and Loading it into a GPU memory requires careful orchestration.

Common checkpoint formats:

- **PyTorch `.pt` / `.bin`**: the standard PyTorch format which can be slow to load due to pickle deserialization
- **SafeTensors**: a faster and safer alternative which can be loaded significantly faster via memory-mapped files
- **GGUF**: a format used by llama.cpp, optimized for inference on consumer hardware and it is able to quantize weights natively

2.3.2 Quantization for serving

Training typically utilizes BF16 weights (2 bytes per parameter). For serving, quantization reduces this further:

INT8 quantization stores weights as 8-bit integers (1 byte per parameter). This achieves a 2× memory reduction which causes a small quality loss that is mostly negligible for generation tasks.

INT4 quantization uses 4-bit integers (0.5 bytes per parameter) for a 4× memory reduction. Causes more quality loss and needs careful calibration. Enables running 70B models on consumer GPUs that could not otherwise fit them.

GPTQ is a post-training quantization technique that calibrates 4-bit quantization using a small dataset, minimizing quality loss from reduced precision.

AWQ (Activation-aware Weight Quantization) quantizes weights while accounting for the distribution of activations, producing better results than naive quantization at the same bit width.

The tradeoff is consistent: the quantization reduces memory requirements while increase throughput, at the cost of some output quality. For production serving where cost matters, INT8 is nearly universal. INT4 is used when memory is severely constrained or when we want to serve on consumer hardware.

2.3.3 Tensor parallelism for large models

A 405B parameter model in BF16 requires approximately 810 GB of GPU memory that is far beyond a single GPU's capacity. In these scenarios, loading requires distributing the model across multiple GPUs using tensor parallelism (splitting individual weight matrices across GPUs) or pipeline parallelism (assigning different layers to different GPUs).

The same parallelism strategies used in training apply to serving, but with different constraints: training prioritizes throughput (large batches, high utilization), while serving prioritizes latency (fast first-token generation, smaller batches).

2.4 The serving architecture

2.4.1 The inference server

The core component: a process that holds the model weights in GPU memory and handles requests. The major implementations:

vLLM is the dominant open-source inference server. It introduces PagedAttention (covered in detail in chapter "REFF") for efficient KV cache management, and supports tensor parallelism, continuous batching, and most major model architectures.

TensorRT-LLM (NVIDIA) is NVIDIA's inference optimization library. It applies kernel fusion, INT8/FP8 quantization, and hardware-specific optimizations. Typically the fastest option on NVIDIA hardware but requires compilation for specific model shapes which is a trade between flexibility and performance.

llama.cpp is a pure C++ implementation of LLaMA-family models, optimized for CPU and consumer GPU inference. Not competitive with vLLM for high-throughput serving, but it is unmatched for accessibility and local deployment.

2.4.2 The API gateway

In front of the inference server sits the API layer:

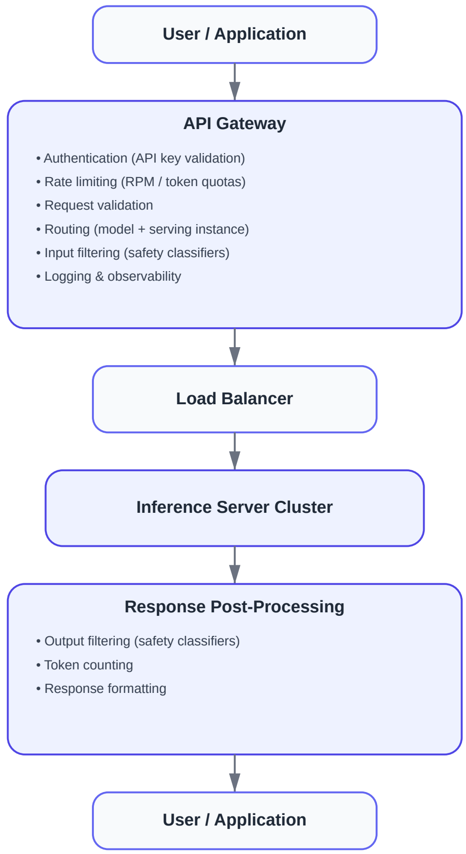


Figure 2.2: The API gateway.

The API gateway is not a performance bottleneck what it does is to process the metadata. It is critical for security (authentication prevents unauthorized access), cost control (rate limiting prevents abuse), reliability (routing distributes traffic and handles failures), and observability (logging every request enables debugging and billing).

2.5 Batching: the key to serving efficiently

A single inference server instance can handle many requests simultaneously. How it batches those requests together affects throughput and GPU utilization.

2.5.1 Naive static batching

The simplest approach is to wait until you have `batch_size` requests, process them together, return all results.

The problem is that LLM generation is variable length. On request might generate 10 tokens while the other might generate 500. In a static batch, all requests must wait for the longest one to finish. GPU utilization drops as shorter requests complete and their slots sit idle, waiting.

2.5.2 Continuous batching

The key innovation for efficient LLM serving, introduced by [1] and popularized by vLLM.

Instead of batching at the request level, continuous batching operates at the iteration level meaning it happens at every token generation step. After each token is generated:

1. Check which sequences in the current batch have finished
2. Remove finished sequences from the batch
3. Add new incoming requests to fill the freed slots
4. Run the next generation step on the updated batch

```
Step 1: [A , B , C ]         ← three requests, first token
Step 2: [A , B , C ]
Step 3: [A , B , C_stop]      ← C finishes after 3 tokens
Step 3: [A , B , D ]         ← D immediately fills C's slot
Step 4: [A , B , D ]
Step 5: [A_stop, B , D ]      ← A finishes
Step 5: [E , B , D ]         ← E fills A's slot
```

The GPU is always processing a full batch. None of the slots sit idle waiting for the longest sequence to finish. On real workloads, this approach can improve throughput by 2 to 3 times compared to a naive static batching. Continuous batching requires careful KV cache management which explains why PagedAttention exists.

2.5.3 Prefill and decode phases

LLM generation has two distinct phases with different compute profiles:

Prefill: processing the input prompt. All prompt tokens are processed in parallel. This phase is compute bound and the bottleneck is GPU arithmetic throughput.

Decode: generating output tokens one at a time. Each step processes one single new token, attending to all previous tokens via the KV cache. This phase is memory-bandwidth-bound and the bottleneck is reading the model weights and KV cache from GPU memory.

Because prefill and decode have different bottlenecks, mixing them in the same batch is not optimal. **Chunked prefill** and **disaggregated serving** are emerging techniques that separate prefill and decode across different hardware or batch slots to maximize utilization of each phase independently.

2.6 KV cache management

The KV cache stores the key and value tensors computed for each previous token, avoiding recomputation on every generation step. It is the central memory management challenge in serving, and will be explained in chapter "REFF".

A brief introduction here, because it directly constrains serving architecture:

For a model with `L` layers, `h` KV heads, `d_k` key/value dimension, and a sequence of length `n`:

```
KV cache size = 2 × L × h × d_k × n × bytes_per_element
```

For LLaMA 2 70B (L=80, h=8 GQA heads, d_k=128) in BF16:

```
Per token: 2 × 80 × 8 × 128 × 2 = 327,680 bytes   320 KB
4,096 tokens: 320 KB × 4,096   1.3 GB
32,768 tokens: 320 KB × 32,768   10.5 GB
```

At high concurrency with long contexts, KV cache competes directly with model weights for GPU memory.

2.6.1 PagedAttention

The key innovation in vLLM, inspired by virtual memory paging in operating systems.

Traditional KV cache reserves a contiguous memory block for each sequence, sized to accommodate the maximum possible length. This causes internal fragmentation (reserved memory that goes unused when sequences are shorter than expected) and external fragmentation (free memory that cannot be used because it is not contiguous).

PagedAttention allocates KV cache in fixed-size pages of tokens, with a page table mapping logical sequence positions to physical memory locations. Memory is allocated and freed one page at a time as sequences grow and complete:

```
Sequence A: [Page 3, Page 7, Page 12]     ← pages need not be contiguous
Sequence B: [Page 1, Page 9]
Sequence C: [Page 2, Page 4, Page 5, Page 11]
Free pages: [Page 6, Page 8, Page 10, ...]
```

This will eliminates fragmentation and enables an almost perfect memory utilization. It also enables prefix sharing which means if two requests share a common system prompt, the KV cache for that prefix can be shared across both sequences without duplication.

2.7 Latency and throughput tradeoffs

Serving performance is characterized by two metrics that pull against each other:

Time to first token (TTFT) is how long the user waits before seeing any output. This is determined primarily by prefill time and since longer prompts take longer to prefill it is the metric users perceive most directly.

Time between tokens (TBT) is how fast tokens arrive after the first one. This is determined by decode speed. For a 70B model on a single A100, 20 to 60 tokens per second is typically expected.

Throughput is total tokens processed per second across all concurrent requests. Maximized by large batch sizes, which directly conflict with low TTFT.

Production systems manage this tradeoff through maximum batch size limits (cap batch size to maintain acceptable latency), priority queuing (prioritize interactive requests over batch workloads), and SLA-based scheduling (different latency targets for different user tiers).

2.8 Key takeaways

- A trained model checkpoint becomes a production service through a pipeline involving quantization, multi-GPU loading, an inference server, and an API gateway, since training and serving are distinct engineering disciplines
- The chat completions API format, including the messages array, key parameters, and token usage reporting, reflects the practical realities of LLM serving rather than arbitrary design choices
- Continuous batching operates at the token level rather than the request level, keeping GPUs fully utilized despite variable-length generation and serving as the main efficiency improvement for high-throughput systems
- KV cache memory is the central serving constraint because it grows with sequence length and batch size and competes directly with model weights for GPU memory, while PagedAttention manages it using virtual memory paging
- Prefill is compute bound and decode is memory bandwidth bound, so they have fundamentally different performance profiles and can be optimized independently
- TTFT and throughput trade off against each other, and the appropriate balance depends on whether the system serves interactive applications or batch workloads

---

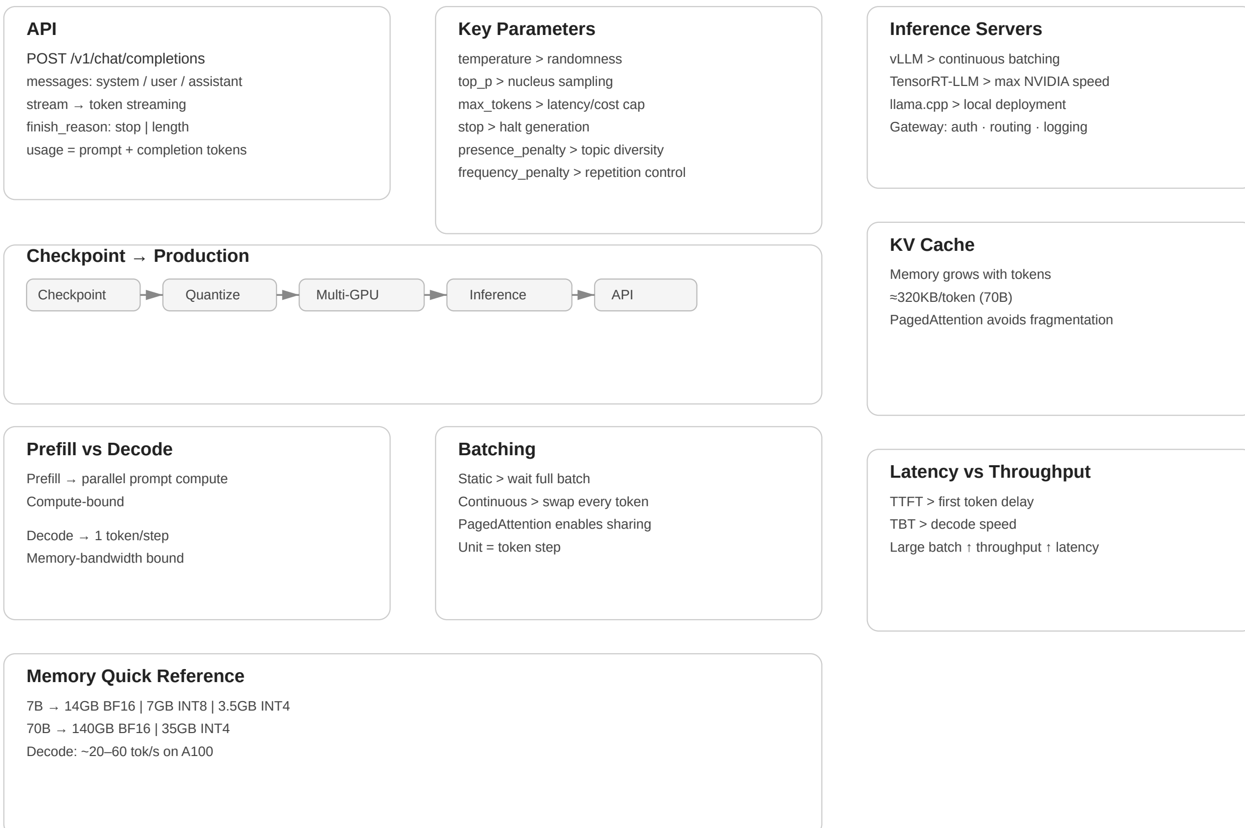


Figure 2.3: Cheat sheet.

2.9 Further reading

- Yu et al. (2022). *Orca: A Distributed Serving System for Transformer-Based Generative Models.*: Introduces continuous batching.
- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.*: The vLLM paper; foundational for modern serving.

Chapter 3

Tokenization during inference

The canonical question for this chapter: *A user sends a message while a model receive and returns numbers. How does text become numbers on the way in, and how do numbers become text on the way out?*

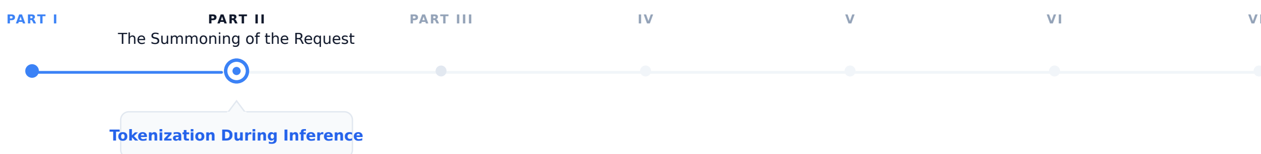


Figure 3.1: The journey through the Model Mind.

The request has arrived at the inference server. Before the model sees a single character, the text must be converted to numbers. This chapter covers that conversion in both directions and describes everything that can go wrong in between.

3.1 Two tokenization chapters, two different problems

If you read the training section first, you have already seen tokenization as in a batch preprocessing environment which means a corpus is tokenized once, offline, before training begins. The vocabulary is fixed. The inputs are known and errors are caught early.

Tokenization at inference is a different story. It runs on arbitrary user input, in real time, and on every request. It supposed to handle every string a user might type which could be multilingual text, source code, emoji, adversarial inputs, or a combination of all four. The encoded output feeds directly into a running model. The decoded output appears on a user's screen, sometimes token by token while they watch.

The concerns here are mostly operational, not theoretical. Understanding them is the difference between shipping reliable LLM applications or debugging corruption failures at 2am.

3.2 The encoding pipeline

When a request arrives, the first operation is encoding: converting the messages array into a flat sequence of token IDs ready for the model. This happens in two steps that are easy to confuse but are conceptually different.

3.2.1 Step 1: chat template application

The messages array has structure which includes roles, turns, and system prompts. The model has no concept of a dictionary. Before tokenization, the structured input must be flattened into a single string using a chat template.

Each model family has its own template. For LLaMA 3:

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>

You are a helpful assistant.<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is the capital of France?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

For Mistral:

```
<s>[INST] What is the capital of France? [/INST]
```

For ChatML (OpenAI and many others):

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
What is the capital of France?<|im_end|>
<|im_start|>assistant
```

The special tokens (`<|begin_of_text|>`, `<|eot_id|>`, `<|im_start|>`) are part of the model's vocabulary and were added during fine-tuning. The model learns to associate these markers with role transitions. Feeding the wrong template to the model causes it to sees an unexpected token sequence which in turn may cause the model to ignore the system prompt, and fail to complete the turn correctly, or produces outputs that look almost right but are systematically wrong in ways that can be hard to diagnose. This is one of the most common sources of bugs: no error is thrown yet the model just quietly misbehaves.

3.2.2 Step 2: token encoding

Once the flat string is constructed, the tokenizer converts it to integer IDs. This is a deterministic step in which the same string always produces the same IDs given the same tokenizer.

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user",   "content": "What is the capital of France?"}
]

# Step 1: flatten with chat template
flat_string = apply_chat_template(messages, tokenizer)
# → "<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n..."

# Step 2: convert to integers
token_ids = tokenizer.encode(flat_string)
# → [151644, 8948, 198, 2610, 525, 264, 10950, 17847, 13, ...]

# Step 3: pass to model
logits = model.forward(token_ids)
```

The tokenizer runs on CPU and for typical prompt lengths it is fast enough to be negligible relative to model inference time. On the other hand, for very long prompts (100k+ tokens) it can become measurable and there are some inference frameworks that cache tokenized system prompts to avoid repeating this work on every request.

3.2.3 Special tokens that require explicit handling

Several special tokens need attention beyond the template:

Beginning of sequence (BOS). Many models expect a BOS token at position 0 and the tokenizer typically adds it automatically while some serving configurations require manual insertion. A missing BOS token causes silent quality degradation which means the model's input distribution does not match training, and early tokens may be erratic.

End of turn markers. The template includes tokens marking the end of each turn and it must appear in the correct positions or the model misidentifies role boundaries.

Tool call tokens. Models fine-tuned for function calling have additional special tokens called tool call syntax. These must be in the tokenizer vocabulary and correctly placed in the template.

The generation prompt. The template typically ends with the opening of the assistant turn (e.g., `<|im_start|>assistant\n`) without closing it. This will signal the model that it should continue from here and omitting it causes the model to generate an incorrect turn structure.

3.3 Token continuation and context window management

The context window is the maximum sequence length the model can process in a single forward pass which is typically a range between 8k to 128k tokens for current models. Managing what fits in this window is a critical operational concern.

3.3.1 Count with the tokenizer, not your intuition

Token counts are not characters divided by four, or words divided by 0.75. They are the actual output of the tokenizer on the specific input string. Heuristics are inaccurate enough to cause real failures:

- Code tokenizes very differently from prose which means identifiers, operators, and whitespace have their own patterns
- Non-English text is often less efficient than English for most tokenizers
- Special tokens (role markers, tool tokens) add counts that character estimates miss entirely

Every production system that needs to stay within a context window must count tokens using the actual tokenizer, not an approximation:

```python
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4o")

def count_tokens(messages: list[dict]) -> int:
    num_tokens = 0
    for message in messages:
        num_tokens += 4  # role, content, separators overhead
        for value in message.values():
            num_tokens += len(enc.encode(value))
    num_tokens += 2  # reply priming tokens
    return num_tokens
```

The exact overhead is varied by model and template. Always validate against the API's reported token counts on a representative sample.

3.3.2 The context window budget

In a typical production request, the context window is shared between several competing claims:

```
Total context window (e.g., 128k tokens)
   System prompt              (fixed, e.g., 500-2,000 tokens)
   Conversation history       (grows with each turn)
   Retrieved context (RAG)    (variable, e.g., 2,000-10,000 tokens)
   Current user message       (variable)
   Reserved for completion    (max_tokens parameter)
```

The sum must not exceed the window and if it does, the request will fail or the serving layer silently truncates something which is usually the earliest conversation history, that could be the exact context the user needs the most. Applications must actively manage this budget rather than hoping it fits.

Rolling window keeps only the most recent N tokens of history. While it is simple, it loses early context which can be fine for short tasks and problematic for long conversations.

Summarization uses the model to compress earlier turns into a compact representation when history approaches the limit. With this approach it preserves semantic content at the cost of an additional model call.

Hierarchical retrieval stores full history externally and only retrieve the most relevant portions for the current turn. This approach requires a retrieval component but it can scale to arbitrarily long conversations.

3.3.3 Prompt caching and its token implications

Some APIs (Anthropic's Claude, Google's Gemini) offer explicit prompt caching: mark a portion of the prompt as cacheable, and subsequent requests that share the same prefix reuse the KV cache from the first request, charged at a reduced rate.

For cost management, structure prompts with the stable content (system prompt, long documents) first and variable content (user messages, dynamic context) in the end. Token counting for cached prompts will distinguishs between newly processed tokens and cache-read tokens, as the API reports these separately and they have completely different cost implications.

3.4 The decoding pipeline: from numbers back to text

After the model selects the next token ID (the selection process is covered in chapter "REFF"), the serving layer converts that ID back to text. This is less straightforward than it appears.

3.4.1 Detokenization

The inverse operation: token IDs to string.

```
token_ids = [15223, 374, 279, 6864, 315, 9822, 13]
text = tokenizer.decode(token_ids)
# → "Paris is the capital of France."
```

While it seems that detokenization simply involves looking up each token's string and concatenating them together, several complications exist:

Byte-level BPE and UTF-8 reconstruction. In byte-level tokenizers, tokens represent raw bytes rather than complete characters. Because UTF-8 characters are able to span multiple bytes, a single character (many Chinese characters or emojis) may be splited across several tokens. Decoding each token independently can therefore produce invalid text or replacement symbols. The correct procedure is to concatenate all token bytes first and then perform a single UTF-8 decode, ensuring the original characters are correctly reconstructed.

Special tokens in output. If the model produces a special token (end of text, role marker, tool call token), detokenization must strip it, convert it to a meaningful representation, or treat it as a halt signal. Which action is correct totally depends on context and must be handled explicitly.

3.4.2 Streaming detokenization

In streaming mode, tokens arrive one at a time and must be decoded incrementally which creates a specific problem: a byte sequence that spans multiple tokens cannot be decoded until all of its bytes have arrived.

Consider a Chinese character requiring 3 UTF-8 bytes, split across three tokens of 1 byte each:

- After token 1: 1 byte received and it is not a valid UTF-8 codepoint yet and cannot emit
- After token 2: 2 bytes received and it is still incomplete
- After token 3: 3 bytes received finally it is completed, decode and emit the character

A streaming decoder must buffer incomplete byte sequences and emit characters only once a complete Unicode codepoint is available. A naive implementations that pass `errors="ignore"` or `errors="replace"` will silently corrupt non-ASCII output while the HuggingFace `TextStreamer` and similar utilities handle this correctly.

3.5 Logprobs and their uses at inference time

Most APIs can optionally return the log probabilities of generated tokens which is more useful than it might initially appear.

3.5.1 What logprobs tell you

For each generated token, logprobs report the log probability the model assigned to the chosen token, and optionally the top-k alternatives with their probabilities:

```
{
  "token": "Paris",
  "logprob": -0.0023,
  "top_logprobs": [
    {"token": "Paris", "logprob": -0.0023},
    {"token": "Lyon",  "logprob": -6.12},
    {"token": "Nice",  "logprob": -7.88}
  ]
}
```

A logprob of -0.0023 means the model assigned probability `exp(-0.0023)` `0.998` to this token which means the model is essentially certain. A logprob of -6.12 means `exp(-6.12)` `0.002` and represent a distant second choice.

3.5.2 Classification without generation

For binary or multiple-choice tasks, logprobs let you skip generating a full response and instead read the probabilities of specific tokens directly which is faster and cheaper:

```
response = client.completions.create(
    model="gpt-4o",
    prompt=f"{text}\n\nIs this sentiment positive? Answer with yes or no.",
    max_tokens=1,
    logprobs=True,
    top_logprobs=5
)

top = response.choices[0].logprobs.top_logprobs[0]
yes_logprob = next(t.logprob for t in top if t.token.strip().lower() == "yes")
no_logprob  = next(t.logprob for t in top if t.token.strip().lower() == "no")
```

```
is_positive = yes_logprob > no_logprob
```

3.5.3 Calibration and uncertainty estimation

The perplexity of a generated response which is represented by the average negative log probability per token is a rough proxy for the model's confidence:

```
import numpy as np

logprobs = [token.logprob for token in response.choices[0].logprobs.content]
perplexity = np.exp(-np.mean(logprobs))
# High perplexity → model was uncertain about this response
```

This is not a hallucination detector which means the model can be confidently wrong, and often is. But it is a useful routing signal: responses with high perplexity can be flagged for human review or follow-up verification without running a separate evaluation model.

3.6 Common failure modes

Tokenization at inference produces specific, reproducible failure patterns. Knowing them saves significant debugging time.

Wrong tokenizer for the model. Using the GPT-2 tokenizer with a LLaMA model produces completely wrong token ID sequences. The model interprets unrelated IDs as completely different tokens which in turn leads to a garbage output.

Wrong chat template. Applying the wrong template produces subtly wrong token sequences. The model may ignore the system prompt, continue the user turn instead of starting an assistant turn, or generate role labels as content. This also fails silently and is especially confusing because outputs look almost right but are systematically off.

BOS token doubling. Some serving frameworks add BOS automatically while some expect the caller to add it. If BOS appears twice, the model sees an unexpected sequence at position 0 and may behave erratically.

Context overflow without truncation. If the total token count exceeds the context window and the serving layer does not truncate explicitly, some APIs return an error while some silently truncate from the beginning.

Encoding/decoding asymmetry. `decode(encode(s))` may not equal `s` for strings containing Unicode normalization differences, special tokens with non-standard decoding, or whitespace that the tokenizer normalizes.

3.7 Key takeaways

- Chat template application: flattening the messages array into a string with model-specific role delimiters it happens before tokenization and is model-specific; the wrong template produces silent quality degradation
- Token counting must use the actual tokenizer, not character or word heuristics; accurate counting is essential for context window management and cost control
- The context window budget must be actively managed across system prompt, conversation history, retrieved context, and completion reservation
- Streaming detokenization must buffer incomplete UTF-8 byte sequences and emit characters only once a complete codepoint is available; naive implementations corrupt non-ASCII output
- Stop sequence detection operates on decoded strings, not token IDs, and must handle sequences that span token boundaries and appear at buffer edges

- Token healing solves the boundary problem for prompts ending at arbitrary character positions, improving completion quality for structured generation
- Logprobs enable efficient classification, uncertainty estimation, and confidence- based routing without generating full responses

---

3.8 Further reading

- HuggingFace Tokenizers documentation: Fast tokenizers and chat templates.
- OpenAI Cookbook: How to count tokens with tiktoken.
- HuggingFace (2023): Chat Templating Guide.

Tokenization at Inference, Cheat Sheet

Encoding Pipeline

1. Apply chat template → flatten structured messages

`<system> ... <user> ... <assistant>`

2. Tokenizer converts text → token IDs

`"Paris is capital" → [1512, 374, 279]`

Critical Special Tokens

- BOS token must exist at position 0
- End-of-turn markers define roles
- Tool tokens required for function calling
- Assistant prompt signals generation start

Context Window Budget

System prompt
Conversation history
RAG context
User message + completion tokens
Always count tokens using tokenizer, never estimate.

History Management Strategies

Rolling window → keep recent tokens only
Summarization → compress earlier turns
Hierarchical retrieval → fetch relevant past context

Decoding Pipeline

Model outputs token IDs → detokenizer reconstructs UTF-8 text

`[15223, 374, 279] → "Paris is the capital"`

Byte-level tokenizers require buffering before emitting characters.

Streaming Decoding

- Tokens arrive one at a time
- Partial UTF-8 bytes must be buffered
- Emit text only when character completes

Logprobs Uses

- Confidence estimation
- Classification without generation
- Perplexity ≈ uncertainty signal

Common Failure Modes

- Wrong tokenizer for model
- Wrong chat template → silent quality loss
- Double BOS token
- Context overflow / silent truncation
- encode(decode(x)) != x due to normalization

Figure 3.2: Cheat sheet.

Chapter 4

Embeddings and Meaning Representation

The canonical question for this chapter: *How does a model convert a token ID into something that has meaning, and how does that representation evolve as it passes through the network?*

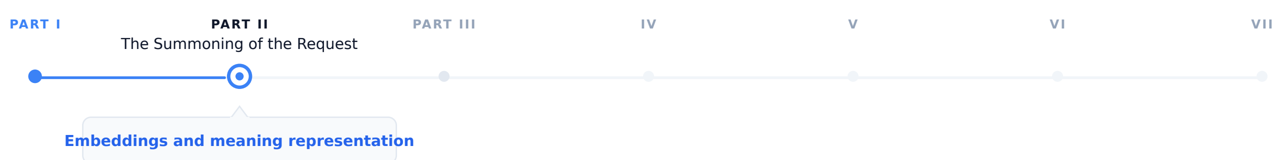


Figure 4.1: The journey through the Model Mind.

Tokenization converts the prompt text into a sequence of integer IDs. These integers has no meaning a neural network on their own. This chapter covers the step that makes them meaningful and it introduces the geometric structure of meaning that the rest of the model operates on.

4.1 The problem with integers

A token ID is an arbitrary index. The token ID for `"Paris"` might be 15342 while the token ID for `"London"` is 9562. These numbers do not carry any information about the relationship between the two cities. They are just indices in a lookup table, nothing more.

An embedding is the thing that you get after that lookup: a dense vector of floating point numbers with hundreds or thousands of dimensions. The embedding for `"Paris"` might be something like `[0.23, -0.87, 0.12, ...]` with 4096 values and the embedding for `"London"` is a completely different vector nevertheless in a well-trained model, it will be close to Paris's vector in this high-dimensional space, because the two tokens appear in similar contexts throughout the training corpus.

This is the core insight: geometric proximity in embedding space convey the semantic relatedness. Tokens that appear in similar contexts have similar embeddings. The model does not learn this rule explicitly it caused by gradient descent on the next-token prediction objective. Training discovers that representing semantically related tokens as nearby vectors produces better predictions, and adjusts accordingly over trillions of examples.

Embeddings are the bridge between the discrete world of tokens and the continuous world of neural network computation. Everything the transformer does which includes every attention operation, and every feed-forward pass operates on these continuous vectors and the model never touches the original token IDs again after the embedding lookup.

4.2 The embedding table

The embedding table is a matrix of shape `[vocab_size, d_model]`. Each row is the learned embedding for one token.

```
Embedding table (simplified, d_model = 4):

Token ID  Token     Embedding vector
0         <pad>     [ 0.00,  0.00,  0.00,  0.00]
1         <eos>     [ 0.12, -0.34,  0.87,  0.23]
2         "the"     [-0.45,  0.67, -0.12,  0.89]
3         "Paris"   [ 0.91,  0.23, -0.67,  0.45]
4         "London"  [ 0.88,  0.19, -0.71,  0.43]
5         "cat"     [-0.23,  0.89,  0.34, -0.67]
```

The lookup is a simple indexed memory access: `embedding = table[token_id]`. Implemented as a matrix multiplication with a one-hot vector in theory; in practice it is O(1) and takes microseconds.

For GPT-3 with `vocab_size = 50,257` and `d_model = 12,288`:

```
Embedding table size = 50,257 × 12,288 × 2 bytes (BF16)
                       1.24 billion parameters
                       2.47 GB
```

This is a significant fraction of total model size and it is also the most memory efficient part of the model relative to its influence: every token the model processes uses exactly one row of this table.

4.2.1 Weight tying

The same embedding table is used twice: once at the input (token IDs → embeddings) and once at the output (final hidden state → logits over vocabulary). This is called weight tying and is standard in most transformer language models.

At the output, the hidden state `h` of shape `[d_model]` is multiplied by the transposed embedding table `E^T` of shape `[d_model, vocab_size]` to produce logits of shape `[vocab_size]`:

```
logits = h @ E^T
```

Weight tying reduces the parameter count by `vocab_size × d_model` and, more importantly it enforces a geometric consistency: the direction that represents a token at the input is the same direction the model uses to predict that token at the output. This constraint improves training efficiency and generalization and it is one of those architectural decisions that looks obvious in hindsight yet it was not obvious at all beforehand.

4.3 How embeddings are learned

At the start of training, the embedding table is initialized from a small random distribution and it does not carry any semantic information they are just noise.

Through training, gradient descent updates each embedding row whenever its token appears in the training data. The update moves the embedding in the direction that reduces prediction loss. Over millions of update steps on millions of tokens, the embeddings converge to representations that capture the statistical structure of language.

The mechanism works by giving tokens that appear in similar contexts, similar gradient updates. If `"Paris"` and `"London"` both appear after `"capital of"` and before `"is beautiful"`, their embeddings are repeatedly pushed in similar directions. Over enough training, they converge to nearby vectors.

This is the distributional hypothesis, operationalized by gradient descent: *you shall know a word by the company it keeps.*

4.3.1 The rare token problem

Embedding parameters receive less frequent updates than other model parameters. A token that appears once in a billion token batch updates once while a common token might get updates in every batch. This creates an implicit learning rate inequality: frequent tokens have well trained embeddings but rare tokens may be poorly trained even in large models.

This is one reason why multilingual models struggle with low-resource languages, and why domain specific jargon may get handled poorly even in a large general model due to this simple fact that the embeddings for rare tokens have not seen enough gradient signal to settle into a useful position.

4.4 The geometry of embedding space

Trained embeddings organize into a rich geometric structure. Understanding this structure is practically useful for interpreting the model behavior and for building a retrieval system.

4.4.1 Semantic neighborhoods

Semantically related tokens cluster together. In embedding space, countries cluster near each other, verbs in the same semantic field cluster together, synonyms are close, and antonyms are often directionally opposite. This is not an artifact of any particular architecture it emerges from the distributional statistics of the language and it is similar across all model families.

4.4.2 Linear structure

The most famous property of word embeddings is the fact that analogical relationships correspond to linear operations.

```
embedding("king") - embedding("man") + embedding("woman")   embedding("queen")
embedding("Paris") - embedding("France") + embedding("Germany")   embedding("Berlin")
embedding("walked") - embedding("walk") + embedding("run")   embedding("ran")
```

Directions in embedding space encode semantic dimensions. The vector from `"man"` to `"woman"` is a gender direction. The vector from `"France"` to `"Paris"` is a capital of direction. Addition and subtraction of embedding vectors correspond to semantic composition and transformation.

This property approximately exist in static embeddings (Word2Vec, GloVe) and it continues to presist in the token embeddings of large transformers, though it might not be as clean as static embedding. The transformer layers above the embedding table further transform these representations in ways that create richer structure yet it makes it harder to extract clean linear relationships.

4.4.3 Isotropy and representation collapse

Embeddings occupy only a narrow cone of the available space, with most directions unused which makes it one of the failure modes in embedding training which is called anisotropy. When embeddings are anisotropic, cosine similarity between random embeddings is high even for semantically unrelated tokens and causes the embedding based retrieval unreliable.

Modern large models are less prone to this compared older smaller models, yet it remains a concern in fine-tuning models where the embedding distribution can shift significantly during training.

4.5 Contextual vs. static embeddings

The embedding table produces a single fixed vector for each token, regardless of context. The token "bank" gets the same embedding whether the surrounding context is about finance or about rivers.

This is the limitation of static embeddings an the transformer layers above the embedding table are able to solve it. After the initial embedding lookup, each transformer block modifies the token representations through attention and feed-forward operations and when it gets to the final layer, the representation of "bank" in a financial context is completley different from "bank" in a geological context. This happens not because the embedding table has changed, but because the transformer layers incorporated contextual information from surrounding tokens.

These contextual representations are called "contextual embeddings" to distinguish them from static lookup embeddings at the first layer. They are what BERT family models were designed to expose, and what RAG retrieval systems use when they need semantically rich representations.

4.5.1 The residual stream

This is a useful model for understanding how representations evolve through the network:

```
x_0 = token_embedding + positional_encoding

x_1 = x_0 + Attention_1(LayerNorm(x_0))
x_1 = x_1 + FFN_1(LayerNorm(x_1))

x_2 = x_1 + Attention_2(LayerNorm(x_1))
x_2 = x_2 + FFN_2(LayerNorm(x_2))

...

x_L = final hidden state → logits
```

Each layer adds a refinement to the running representation while the early layers tend to encode syntactic features, middle layers encode semantic relationships and the final layers encode task specific features that is used for prediction.

The residual structure means the original token embedding is always present in the stream and it gets added to every layer and never replaced. This is why the embedding table remains relevant even in very deep models: its contribution persists all the way through computation, like a base note that harmonics are built on top of.

4.5.2 Which layer to extract from

For tasks that use hidden states as embeddings, such as classification, retrieval, or similarity, a practical question that could arise is which layer should be used?

The last layer contains the most task specific information but it may over optimize for next token prediction at the expense of general semantic content. The second to last layer is a common heuristic that often outperforms the last layer for semantic similarity tasks. An average across all layers captures information at all levels of abstraction can be considered robust yet expensive. Middle layers often encode the richest structural information for syntactic tasks.

For dedicated embedding models (text-embedding-3-large, E5, BGE), this question is moot, these models are specifically trained to produce useful representations at the last layer using contrastive objectives rather than next token prediction. For retrieval tasks these models should be used instead of the generative models.

4.6 Embedding models vs. generation models

Generative models are trained to predict the next token although their internal hidden states can be extracted and used for retrieval, this is not what they are optimized for. On the other hand dedicated embedding models form a separate class. They are trained with different objectives and are designed specifically to produce useful vector representations of text.

4.6.1 Contrastive training

Contrastive training is the dominant training approach for embedding models. The model is trained in a way that semantically similar texts have nearby embeddings and semantically different texts have distant embeddings.

The InfoNCE loss, used by models like E5, GTE, and many others:

$$\mathcal{L} = -\log\left(\frac{\exp\left(\text{sim}(q, k^+)/\tau\right)}{\sum_i \exp\left(\text{sim}(q, k_i)/\tau\right)}\right)$$

Where q is the query embedding, k^+ is the positive (semantically similar) embedding, k_i are all keys including negatives, sim is cosine similarity, and τ is a temperature parameter.

The model learns to pull positive pairs together and push negative pairs apart. Training data consists of (query, positive document) pairs which is derived from naturally occurring sources: question-answer pairs, search query logs, natural language inference datasets.

4.6.2 Hard negatives

The quality of negative examples is critical. Random negatives, which mostly means arbitrary documents from the corpus, are too easy. The model learns to distinguish clearly unrelated texts without developing fine-grained discrimination.

Hard negatives are documents that are topically related but semantically different from the positive:

```
Query:         "What is the capital of France?"
Positive:      "Paris is the capital and most populous city of France."
Easy negative: "The mitochondria is the powerhouse of the cell."
Hard negative: "France is a country in Western Europe with Paris as a major city."
               (related topic, but does not directly answer the question)
```

Mining hard negatives is itself a retrieval problem. Modern embedding model training pipelines use a previous model version or a BM25 index to find hard negatives and train a better model on those negatives, and iterate.

4.6.3 Asymmetric encoding

Many retrieval tasks are asymmetric which means that the query and the document are different in length, style, and information density. A question is short and underspecified yet the document is long and detailed.

Some embedding models handle this by encoding queries and documents with different instruction prefixes:

```
Query encoding:    embed("Represent this query for retrieval: " + query)
Document encoding: embed("Represent this document for retrieval: " + document)
```

The prefix signals to the model what role the text plays, allowing it to produce better representations for each. Models like E5 and Instructor were designed around this approach.

4.6.4 Embedding model families

Model	Dimensions	Context	Notes
text-embedding-3-small	1536	8,191 tokens	OpenAI; fast and cheap
text-embedding-3-large	3072	8,191 tokens	OpenAI; best quality in family

Model	Dimensions	Context	Notes
text-embedding-ada-002	1536	8,191 tokens	OpenAI legacy; widely deployed
E5-large-v2	1024	512 tokens	Open-source; strong performance
BGE-large-en-v1.5	1024	512 tokens	BAAI; strong on MTEB
Nomic-embed-text-v1	768	8,192 tokens	Open-source; long context
voyage-large-2	1536	16,000 tokens	Voyage AI; strong retrieval

The MTEB (Massive Text Embedding Benchmark) leaderboard is the standard reference for comparing embedding models across retrieval, classification, clustering, and semantic similarity tasks.

4.7 Similarity metrics

Embeddings are only useful if you can compare them so the choice of metric matters.

4.7.1 Cosine similarity

The standard metric for text embedding comparison:

`cosine_sim(a, b) = (a · b) / (||a|| × ||b||)`

Cosine similarity measures the angle between two vectors, ignoring magnitude. Values range from -1 (opposite directions) to 1 (identical direction). The appeal for this metric is the invariance to magnitude. A document and its summary should be similar even if the document produces a larger magnitude vector due to greater information density.

For normalized embeddings (unit vectors), cosine similarity equals the dot product: `cosine_sim(a, b) = a · b` when `||a|| = ||b|| = 1`. Most embedding APIs return normalized vectors, making the dot product the efficient choice at scale.

4.7.2 Dot product

Without normalization, the dot product combines angle and magnitude. For some retrieval tasks, magnitude carries useful signal which means a more specific or central document might have higher magnitude and it makes the raw dot product a better metric than cosine similarity.

Modern embedding models and vector databases support both; check the model documentation for which metric the model was trained to optimize.

4.7.3 Euclidean distance (L2)

This is a less common metric for text embeddings andor normalized vectors, L2 distance and cosine similarity are monotonically related:

$$\|a - b\|_2^2 = 2 - 2\,\cos(a, b)$$

L2 is mostly specific for applications (k-means clustering, certain index structures) where the metric properties are desirable.

4.8 Embedding dimensions and matryoshka representations

The dimensionality of an embedding determines its representational capacity and its cost. While a 3072-dimension embedding can represent finer grained semantic distinctions, it requires 12 KB per vector in float32 and more compute for similarity search. On the other hand a 768-dimension embedding has less capacity but 4× lower storage and faster search. For retrieval over millions to billions of vectors, the difference is significant.

4.8.1 Matryoshka Representation Learning (MRL)

This is a training technique that allows a single model to produce useful embeddings at multiple dimensions from the same forward pass. Instead of training separate models for different embedding sizes, MRL organizes the embedding space in a way that **useful representations exist at progressively larger prefixes of the same vector**.

MRL trains the model so that the first `d` dimensions of the full embedding are as useful as a `d`-dimensional embedding trained from scratch and simultaneously, for multiple values of `d`:

4.8.1.1 Core Idea

Let the model output a full embedding:

$$\text{embed} \in \mathbb{R}^{1536}$$

MRL trains the network such that **every prefix of the embedding** is independently meaningful:

```
- `embed[:64]` behaves like a strong 64-dim embedding
- `embed[:128]` behaves like a strong 128-dim embedding
- `embed[:256]` behaves like a strong 256-dim embedding
- ...
- `embed[:1536]` remains the highest-quality representation
```

This is analogous to **Russian matryoshka dolls**, where smaller representations are nested inside larger ones.

4.8.1.2 Training Objective

During training, the same embedding is evaluated at multiple truncation levels:

```
L = L(embed[:64]) + L(embed[:128]) + L(embed[:256]) + L(embed[:512]) + L(embed[:1536])
```

Each term computes the task loss (e.g., contrastive similarity loss) using only the first `d` dimensions.

The result is that you can use the full 1536-dimension embedding for high-accuracy search, or truncate to 256 dimensions for a 6× speedup at a small quality cost, using the same model. Dimensionality becomes a deployment parameter rather than a model choice.

OpenAI's text-embedding-3 series uses MRL, allowing callers to specify output dimensions at request time.

4.9 Embeddings in the generative inference pipeline

Within a generative LLM's inference pipeline, embeddings participate at two specific points.

4.9.1 Input: the embedding lookup

After tokenization, token IDs are converted to embeddings via the table lookup. Positional encodings are added (either as additive vectors or via RoPE modifications to attention), and result enters the first transformer block. This step is computationally trivial and takes microseconds even for long sequences.

4.9.2 Output: projection back to vocabulary

The final hidden states are projected to logit space via the transposed embedding table. This is a matrix multiplication of shape `[seq_len, d_model] @ [d_model, vocab_size]` which is the largest matrix multiply in a single forward pass for models with large vocabularies.

For a model with `d_model = 4096` and `vocab_size = 128,000`:

```
Output projection: [seq_len, 4,096] @ [4,096, 128,000]
                 = seq_len × 524,288,000 multiplications
```

For a sequence of 1,000 tokens, this is over 500 billion operations it is significant enough that some models apply the output projection only to the final token position during generation rather than to all positions.

4.10 Practical embedding operations

4.10.1 Embedding a batch of texts

```python
from openai import OpenAI

client = OpenAI()

texts = [
    "The capital of France is Paris.",
    "Paris is a city in Europe.",
    "The mitochondria is the powerhouse of the cell."
]

response = client.embeddings.create(
    input=texts,
    model="text-embedding-3-large",
    dimensions=1024  # MRL truncation
)

embeddings = [item.embedding for item in response.data]
# embeddings[0] and embeddings[1] will be much closer than embeddings[0] and [2]
```

4.10.2 Computing similarity

```python
import numpy as np

def cosine_similarity(a: list[float], b: list[float]) -> float:
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

sim_01 = cosine_similarity(embeddings[0], embeddings[1])
sim_02 = cosine_similarity(embeddings[0], embeddings[2])

print(f"Paris sentences:       {sim_01:.3f}")
print(f"Paris vs mitochondria: {sim_02:.3f}")
```

4.10.3 Semantic search over a corpus

```
import numpy as np

corpus = ["doc 1 text...", "doc 2 text...", ...]
corpus_embeddings = np.array([
    client.embeddings.create(
        input=doc, model="text-embedding-3-large"
    ).data[0].embedding
    for doc in corpus
])  # shape: [n_docs, 1024]

query = "What is the capital of France?"
query_embedding = np.array(
    client.embeddings.create(
        input=query, model="text-embedding-3-large"
    ).data[0].embedding
)  # shape: [1024]

# Cosine similarity (assuming normalized embeddings from the API)
similarities = corpus_embeddings @ query_embedding  # shape: [n_docs]
top_k_indices = np.argsort(similarities)[::-1][:5]

for idx in top_k_indices:
    print(f"Score: {similarities[idx]:.3f} | {corpus[idx][:80]}")
```

For production retrieval over large corpora, replace the numpy dot product with a vector database (Pinecone, Weaviate, Qdrant, pgvector) that handles indexing and approximate nearest neighbor search at scale. The vector database is the subject of chapter “REFF”.

4.11 Key takeaways

- An embedding converts a discrete token ID into a continuous vector; geometric proximity in embedding space encodes semantic relatedness, emerging from the distributional statistics of training rather than any explicit rule
- The embedding table is a `[vocab_size, d_model]` matrix shared between input lookup and output projection (weight tying), enforcing geometric consistency between how tokens are represented and how they are predicted
- Static embeddings give every token a fixed vector regardless of context; contextual embeddings (hidden states at later layers) incorporate surrounding context through attention, the residual stream shows how representations accumulate refinements from syntactic to semantic to task-specific
- Dedicated embedding models are trained with contrastive objectives (InfoNCE loss on positive/negative pairs) rather than next-token prediction, producing representations optimized for similarity search
- Hard negatives, topically related but semantically wrong examples, are essential for training models that make fine-grained distinctions; mining them is itself a retrieval problem
- Cosine similarity is the standard metric for comparing embeddings; always use the metric the model was trained to optimize, using the wrong one degrades retrieval results in ways that are hard to diagnose
- Matryoshka Representation Learning allows a single model to produce useful embeddings at multiple dimensions, making dimensionality a deployment parameter
- The output projection (final hidden state to logits) is the most compute- intensive embedding operation in the generative forward pass; the input lookup is negligible by comparison

---

Figure 4.2: Cheat sheet.

4.12 Further reading

- Elhage et al. (2021). *A Mathematical Framework for Transformer Circuits.*: The residual stream model and how information flows through transformer layers.
- Wang et al. (2022). *Text Embeddings by Weakly-Supervised Contrastive Pre-training.*: The E5 paper; representative of modern embedding model training with hard negatives.
- Kusupati et al. (2022). *Matryoshka Representation Learning.*: MRL for flexible-dimension embeddings from a single model.
- Muennighoff et al. (2023). *MTEB: Massive Text Embedding Benchmark.*: The standard benchmark for comparing embedding models across tasks and domains.

Part III

The Labyrinth of Attention

Chapter 5

Transformer Architecture

The canonical question for this chapter: *Given a sequence of token embeddings, how does the transformer convert them into a prediction about what comes next and why is this architecture the one that won?*

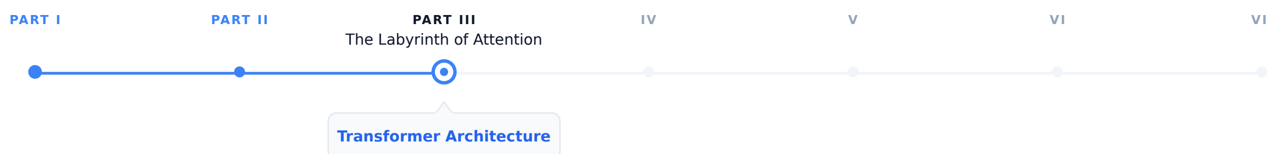


Figure 5.1: The journey through the Model Mind.

The prompt has been tokenized, the tokens have been converted to embedding vectors, and the request has arrived at the inference server. Now the model begins to think. This chapter covers the architecture that does the thinking (what it is, why it is built this way, and what each component actually does.)

5.1 Why the transformer won

Before 2017, sequence modeling was dominated by recurrent neural networks (LSTMs and GRUs). These architectures processed text one token at a time, left to right, maintaining a hidden state that carried information forward through the whole sequence. They worked, but they had two fundamental problems.

The first was the bottleneck problem, all the information about the entire input had to be compressed into a single fixed-size hidden state vector before being used to generate output. For the long sequences, early information was inevitably lost or distorted and the model had to decide what to remember and what to forget at every step.

The second was parallelization. Because each step depended on the previous step's hidden state, RNNs could not be parallelized across the sequence dimension. While the model GPU hardware are heavily tuned for parallelisim training these models were fundamentally bottlenecked by sequential computation.

The transformer, introduced in Vaswani et al. (2017), eliminated both problems with a single architectural decision: replace recurrence with attention. Every token attends directly to every other token in a single step, resulting in neither sequential dependency nor an information bottleneck, and the training can be fully parallelized across the sequence.

This is not a minor optimization but a different computational paradigm that happens to be perfectly matched to the hardware of the last decade.

5.2 The intuition before the math

Before working through the mechanics, it helps to have a mental model of what the transformer is doing at a high level.

Think of each token as a person sitting in a room with everyone else who has ever appeared in the same sequence. At each layer, every person can ask a question of every other person, receive weighted answers based on relevance, and update their understanding accordingly. After many rounds of this, each person's understanding has been shaped by the entire room.

The questions are learned queries, the relevance weighting is attention, the updating occurs in the residual stream, and the many rounds correspond to the layers.

5.3 The high-level forward pass

A transformer language model takes a sequence of token IDs as input and produces a probability distribution over the vocabulary as output in other word a prediction of what token comes next.

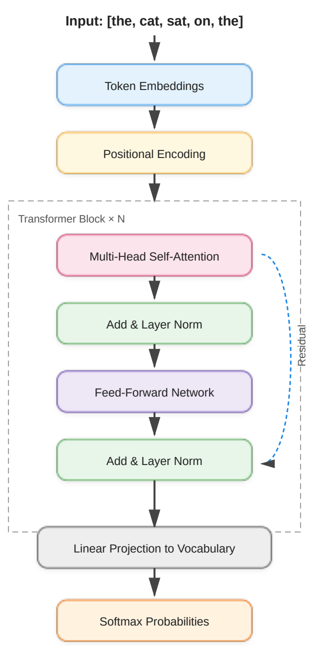


Figure 5.2: Transformer block.

Each component serves a specific function. In the following sections we work through them in order.

5.4 Token embeddings and positional encoding

5.4.1 Token embeddings

The first operation maps integer token IDs to continuous vectors using an embedding table, which is a matrix with shape `[vocab_size, d_model]` and Each row is a learned vector for that token. Looking up token ID 42 retrieves row 42 of the embedding table.

For GPT-3, `d_model = 12,288` and the vocabulary has roughly 50,000 tokens. The embedding table therefore has about 600 million parameters which is a significant fraction of the total model. These embeddings are learned end-to-end during training; the model discovers what representation of each token is most useful for predicting the next one.

The embedding table is shared with the output layer. The same matrix used to look up input embeddings is used (transposed) to project the final hidden state back to vocabulary space. This weight tying reduces parameters and improves performance by enforcing geometric consistency between how tokens are represented and how they are predicted.

5.4.2 Positional encoding

Attention, as we will see, is permutation-invariant which means if you shuffle the tokens in the input, attention produces the same output for each token regardless of order. This is a problem: `"the cat sat"` and `"sat cat the"` would be processed identically. Positional encodings inject order information into the representations by adding a position-dependent signal to each token embedding.

5.4.3 Sinusoidal encoding (the original transformer)

It uses a deterministic functions:

```
PE(pos, 2i)   = sin(pos / 10000^(2i / d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))
```

Different dimensions oscillate at different frequencies that produces a unique fingerprint for each position. These encodings are fixed and generalizes to sequence lengths not seen during training.

5.4.4 Learned positional embeddings**

they replace the formula with a second learned matrix of shape `[max_sequence_length, d_model]`, each position gets its own trained vector, updated by gradient descent.

5.4.5 Rotary Positional Embeddings (RoPE)

They are widely used in modern transformer architectures such as LLaMA, PaLM, and many recent open-weight large language models. Unlike absolute positional encodings, which are added directly to token embeddings before the attention mechanism, RoPE injects positional information directly into the self-attention computation itself.

The core idea behind RoPE is to encode position by applying a rotation to the query and key vectors in each attention head. For each token position, the representation is split into pairs of dimensions, and each pair is rotated by an angle that depends on the token's absolute position. This rotation is structured so that when computing the dot product between a query at position `m` and a key at position `n`, the result depends on the relative offset `(m - n)` rather than their absolute positions.

More formally, RoPE treats each pair of dimensions in a query or key vector as a 2D plane and applies a rotation matrix whose angle increases linearly with position. For a given position `p`, each 2D subspace is rotated by:

$$\theta_p = p \cdot \omega_i$$

where ω_i is a frequency term that depends on the embedding dimension index. This design closely relates to sinusoidal encodings, but differs fundamentally in that the positional information is applied as a transformation of the query and key vectors rather than an additive signal.

In practice, each pair of dimensions is rotated using sine and cosine functions:

$$\begin{pmatrix} x_1' \\ x_2' \end{pmatrix} = \begin{pmatrix} \cos(\theta_p) & -\sin(\theta_p) \\ \sin(\theta_p) & \cos(\theta_p) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

This rotation preserves vector magnitude while changing orientation, embedding positional structure geometrically into the representation space.

A key consequence of this formulation is that the dot product between a query at position `m` and a key at position `n` naturally depends on the relative distance `(m - n)`. This emerges from trigonometric identities that cause the absolute positional components to cancel out during the inner product. As a result, RoPE encodes relative position information implicitly without explicitly computing pairwise distances.

This property provides several important benefits:

- Firstly, it captures relative position information directly, which is often more meaningful for language modeling than absolute position. Language understanding typically depends on relationships between tokens rather than their absolute locations in a sequence.
- Secondly, RoPE improves extrapolation to longer sequences. Because the rotation is a smooth, continuous function of position, it generalizes beyond the training context window more gracefully than learned absolute embeddings. However, performance can still degrade for very long contexts if not trained appropriately.
- Thirdly, RoPE introduces no additional learned parameters for positional encoding. The positional structure is fully deterministic, making it parameter-efficient and reducing the risk of overfitting positional patterns.

In multi-head attention, each head applies the same rotational mechanism but operates in different learned subspaces. This allows different heads to specialize in different positional behaviors. Some heads naturally focus on local dependencies, while others capture long-range relationships due to accumulated phase differences.

5.4.6 ALiBi (Attention with Linear Biases)

It takes a very different approach to positional information. Instead of explicitly encoding positions into token representations, ALiBi modifies the attention mechanism itself by adding a fixed bias to attention scores based on the distance between tokens.

In ALiBi, the attention score between a query at position `m` and a key at position `n` is penalized linearly according to their distance `|m - n|`. This means that tokens that are farther apart receive a progressively stronger negative bias, encouraging the model to focus more on nearby context while still allowing access to long-range dependencies when needed.

A key advantage of ALiBi is that it requires no positional embeddings at all—neither learned nor fixed sinusoidal ones. Instead, positional information is entirely encoded through the attention bias structure. This simplicity makes ALiBi highly efficient and robust, particularly for long-context extrapolation. It has been used in models such as MPT and has shown strong performance in settings where generalization to longer sequences is important.

For a query at position m and a key at position n, the attention score is adjusted as:

$$\text{score}(m, n) = q_m \cdot k_n - \alpha_h \cdot |m - n|$$

Here: - $q_m \cdot k_n$ is the standard dot-product attention score - $| m - n |$ is the absolute distance between tokens - α_h is a head-specific slope (each attention head gets a different decay rate)

One of the key design choices in ALiBi is that different attention heads use different slopes. Some heads have a steep penalty (they focus very locally), while others have a shallow penalty (they can attend further away). This creates a natural diversity of attention ranges without explicitly designing multiple mechanisms. The slopes are not learned, Instead, they are fixed using a deterministic scheme that assigns geometrically spaced values. With this method it avoids additional parameters and ensures predictable extrapolation behavior.

5.5 Self-attention: the core mechanism

Self-attention is an operation that allows every token to gather information from every other token in the sequence. It is the defining innovation of the transformer.

5.5.1 The intuition

Consider: `"The cat slept on the bed because it was dark."`

What does `"it"` refer to? For a human, it's clear that `"it"` refers to the environment, not `"the cat"` or `"the bed"`. A model needs to make this connection to process the sentence correctly. Self-attention allows the token `"it"` to look at every other token, assign an attention weight to each one (higher weight to the context suggesting darkness, lower weight to unrelated tokens like `"bed"`), and blend their representations into its own. After attention, the representation of `"it"` carries information about the surrounding environment being dark.

5.5.2 Queries, keys, and values

Self-attention uses three learned linear projections of each token's embedding: Query (Q), Key (K), and Value (V).

For a sequence of `n` tokens with embedding dimension `d_model`:

$$
\begin{aligned}
Q &= XW_Q \qquad shape : [n, d_k] \\
K &= XW_K \qquad shape : [n, d_k] \\
V &= XW_V \qquad shape : [n, d_v]
\end{aligned}
$$

Where `X` is the matrix of token embeddings and W_Q, W_K, W_V are learned weight matrices.

The attention scores are computed as:

$$
\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V
$$

Breaking this down:

1. QK^T is dot product between every query and every key. The shape `[n, n]` and an entry `[i, j]` measures how much token `i` should attend to token `j`.

2. $\sqrt{d_k}$ scales the dot products by the square root of the key dimension. Without this, dot products in high dimensions become very large and pushes the softmax into regions with near-zero gradients and destabilizing training.

3. $softmax()$ converts scores to probabilities. Each row sums to 1. This is the attention weight matrix.

4. $\times V$ is the weighted sum of value vectors. Each token's output is a blend of all value vectors, weighted by how much it attends to each position.

The result is a new representation for each token that has been informed by the entire sequence. The attention chapter REFF goes deeper into the computational mechanics and what attention heads actually compute.

5.5.3 The causal mask

For language model training, the model must not attend to future tokens. When predicting the token at position `i`, only positions `0` through `i-1` should be visible, otherwise the model is simply reading the answer.

This is enforced by a causal mask: before softmax, a large negative value (effectively `-∞`) is added to attention scores where `j > i`. After softmax, these positions have attention weight 0.

```
Tokens:
Position:   0     1     2     3
           the   cat   sat    on

Causal Attention Matrix (upper triangle masked)

                 Key tokens →
              the   cat   sat    on
Query ↓
the           1.0   0.0   0.0   0.0
cat           0.3   0.7   0.0   0.0
sat           0.1   0.5   0.4   0.0
on            0.2   0.1   0.3   0.4
```

Each token attends to itself and all previous tokens and it will never see the future tokens. This masking enables a decoder-only Transformer to function as an autoregressive language model. It allows the model to be trained on the full sequence in a single forward pass, while ensuring that each position can attend only to preceding tokens.

5.5.3.1 Bidirectional attention (encoder models)

Used in encoder-only models such as BERT, RoBERTa, and most embedding models.

Here, **no causal mask is applied**:

- each token can attend to all other tokens
- both left and right context are visible

This enables richer representations because every token is contextualized by the full sequence.

However, it breaks autoregressive generation:

> if a model can see future tokens, it cannot learn to predict them

Instead, encoder models are trained with masking objectives (e.g., masked language modeling), where some tokens are hidden and must be reconstructed.

5.5.3.2 Prefix masking (prefix-LM / partial bidirectionality)

A hybrid scheme used in some long-context and instruction models.

The sequence is split into two parts:

- **prefix (context)**: fully visible to itself and often to the entire sequence

- **generation region (suffix)**: causal masking applies

This allows:

- bidirectional understanding of the prompt/context
- autoregressive generation for outputs

It is particularly useful for: - long document conditioning - retrieval-augmented generation - structured prompt formats

---

5.5.3.3 Encoder–decoder (cross-attention masking)

The encoder–decoder transformer is designed for tasks where an input sequence must be *understood* before a new sequence is generated. Models such as **T5** and **BART** follow this structure.

The architecture separates the model into two cooperating systems:

- The **encoder** reads the entire input at once using bidirectional self-attention. Every token can attend to every other token, allowing the model to construct a contextual representation which is often described as a *memory* of the input.
- The **decoder** then generates the output sequence step by step. Unlike the encoder, its self-attention is **causally masked**, meaning each token can only attend to previously generated tokens. This preserves autoregressive generation.

  A second attention mechanism (**cross-attention**) connects the two halves of the model. Here, the decoder forms queries while the encoder provides keys and values, allowing generation to remain grounded in the encoded input representation.

This separation of *understanding* and *generation* makes the encoder–decoder architecture especially effective for transformation tasks such as translation, summarization, and structured rewriting. This creates a structured information flow:

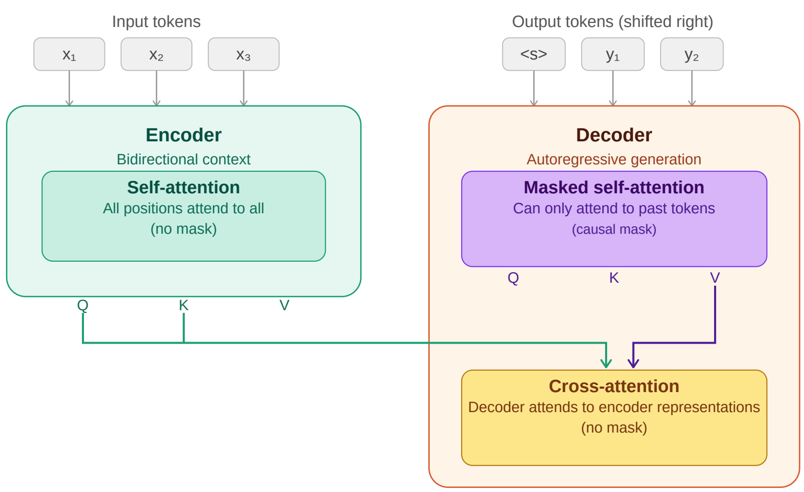


Figure 5.3: encoder-decoder information flow

5.6 Multi-head attention

A single attention operation gives each token one way of relating to every other token nevertheless tokens simultaneously relate to each other in multiple ways.

In `"John gave Mary the book"`, `John` relates to `gave` syntactically (as subject), to `Mary` semantically (as giver to recipient), and to `the book` as the transferred object. A single attention head cannot capture all of these simultaneously and it results in production of a single weighted blend over the sequence.

Multi-head attention runs multiple attention operations in parallel, each with its own learned Q, K, V projections. With this format each head can specialize in a different type of relationship. Finally, the outputs of all heads are concatenated and projected through a learned output matrix W_O back to d_{model} dimensions.

For GPT-3: `d_model = 12,288`, `h = 96` heads, `d_k = 128` per head. The 96 heads run in parallel, each attending to the sequence from a different perspective.

5.7 The feed-forward network

After the attention sublayer, each token passes through a position-wise feed-forward network (FFN) independently:

$$FFN(x) = GeLU(xW_1 + b_1)W_2 + b_2$$

The intermediate dimension is typically 4× the model dimension: for `d_model = 1,024`, the FFN expands to 4,096 dimensions, applies the activation, then projects back to 1,024.

Crucially, the FFN operates on each token independently and identically. The attention layer mixes information across tokens; the FFN processes each token in isolation. Together they form a complementary pair: attention gathers the context and FFN processes it.

5.8 Residual connections and layer normalization

5.8.1 Residual connections

```
x = x + Attention(LayerNorm(x))
x = x + FFN(LayerNorm(x))
```

The output of each sublayer is added back to its input which gives the gradients a direct path backward through the network, enabling training of very deep models. Without residual connections, training a 96-layer network is extremely difficult due to gradients vanishing or exploding before reaching the earlier layers.

Residual connections make the default behavior of each sublayer simply pass its input through unchanged. As a result, sublayers only need to learn small refinements on top of the identity function, which is a much easier optimization problem than learning the full transformation from scratch.

5.8.2 Layer normalization

Layer normalization is used to stabilize the activations of a transformer by normalizing each token's representation across its feature dimensions. Unlike normalization methods that have been used in convolutional networks, it operates independently on each token rather than across the batch or sequence.

Formally, for a hidden state vector $x \in \mathbb{R}^d$:

$$\text{LayerNorm}(x) = \frac{x - \text{mean}(x)}{\sqrt{\text{var}(x) + \varepsilon}} \cdot \gamma + \beta$$

where: - `mean(x)` and `var(x)` are computed over the feature dimension - ε is a small constant for numerical stability - γ and β are learned scale and shift parameters and allow the model to restore or reshape the normalized representation if needed.

This normalization keeps activations in a controlled range, which prevents instability as signals propagate through many stacked layers. Without it, deep transformers become difficult to train due to exploding or vanishing activations in the residual stream.

5.8.2.1 Why not Batch Normalization?

Batch Normalization is rarely used in transformer language models for a fundamental reason: it depends on batch-level statistics.

In BatchNorm, normalization is computed across examples in a minibatch. This creates several problems for language modeling:

- **Inference mismatch**: at inference time, batch sizes are often 1 (especially in autoregressive decoding), making batch statistics unreliable or unavailable
- **Sequence variability**: token sequences have variable lengths, making batch-wise statistics inconsistent across steps
- **Autoregressive constraint**: generation happens one token at a time, so there is no meaningful "batch" context during decoding

Because of these issues, BatchNorm introduces instability and is incompatible with the sequential nature of language model inference. LayerNorm avoids this entirely by normalizing within each token independently.

5.8.2.2 Pre-LN vs Post-LN

The original transformer used **Post-LN**, where normalization is applied after the residual connection:

```
x = x + Sublayer(x)
x = LayerNorm(x)
```

Modern large language models instead use **Pre-LN**, where normalization happens before the sublayer:

```
x = x + Sublayer(LayerNorm(x))
```

Pre-LN is now the standard architecture because it significantly improves optimization stability in deep networks. While it can sometimes slightly reduce raw performance compared to Post-LN in certain setups, it provides much more reliable training at scale. It ensures that each sublayer receives well-conditioned inputs while maintaining a clean gradient flow through the residual stream. This stability benefit becomes increasingly important as models grow to hundreds of layers, where training dynamics would become difficult to control.

5.8.2.3 RMSNorm (modern simplification)

Some recent architectures replace LayerNorm with **RMSNorm (Root Mean Square Normalization)**, used in models such as LLaMA.

RMSNorm simplifies LayerNorm by removing the mean-centering step:

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\mathrm{mean}(x^2) + \varepsilon}} \times \gamma$$

Key differences: - no subtraction of the mean - normalization is based only on vector magnitude - slightly cheaper computationally - empirically comparable or better performance in large-scale models

Despite its simplicity, RMSNorm preserves the key property needed for deep transformers: controlling the scale of activations in the residual stream.

5.9 Stacking transformer blocks

A single transformer block gives the model one round of attention and one round of FFN computation while modern LLMs stack many of them:

Model	Layers	d_model	Heads	Parameters
GPT-2 Small	12	768	12	117M
GPT-2 XL	48	1,600	25	1.5B
GPT-3	96	12,288	96	175B
LLaMA 2 70B	80	8,192	64	70B
LLaMA 3 405B	126	16,384	128	405B

Each layer refines the representation produced by the previous layer. Early layers tend to encode syntactic features while middle layers encode semantic relationships and final layers encode task-specific features relevant to prediction. This hierarchy

---

5.10 Architecture variants

The standard transformer has been modified in various ways by recent models.

5.10.1 Grouped Query Attention (GQA)

Standard multi-head attention uses separate Q, K, and V projections for each head. In contrast, Multi-Query Attention (MQA) shares a single set of K and V projections across all heads, while keeping separate Q projections per head. Grouped Query Attention (GQA) works in a middle ground: K and V are shared across groups of heads, while each head (or each group of heads) retains its own Q projections.

LLaMA 2 70B, LLaMA 3, and Mistral use GQA. The motivation is inference efficiency: KV cache memory (chapter REFF) scales with the number of K, V heads and fewer KV heads is equal to smaller cache, resulting in longer sequences and larger batches at the same memory budget.

5.10.2 Sliding window attention

Used by Mistral 7B. Instead of attending to all previous tokens, each token attends only to a local window of the most recent `w` tokens. For long sequences this reduces attention complexity from $O(n^2)$ to $O(n \cdot w)$. Information from beyond the window propagates through the layered structure of the network and multiple layers of local attention can capture long-range dependencies indirectly.

5.10.3 Activation functions (ReLU → Tanh → Sigmoid → GELU → Swish → GLU → SwiGLU)

The feed-forward network (FFN) in transformers is where most of the non-linear feature transformation happens and over time, activation functions have evolved from simple thresholding operations to smooth functions and finally to gated multiplicative mechanisms.

5.10.3.1 ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

ReLU is the simplest activation function, zeroing out all negative values while leaving positive values unchanged, which makes it computationally efficient and encourages sparse activations. However, this abrupt cutoff at zero can discard useful information, making it a useful starting point but not ideal for training deep transformer models.

5.10.3.2 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The sigmoid function was one of the earliest nonlinearities used in neural networks, mapping inputs into the range ((0, 1)) in a smooth and differentiable way that can be interpreted as a probability or gating mechanism; intuitively, it acts like a soft switch that controls how much of a signal passes through, from fully blocked at 0 to fully allowed at 1.

5.10.3.3 Tanh (Hyperbolic Tangent)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tanh was widely used in early neural networks and recurrent models before transformers, producing outputs in the range ([-1, 1]) with a smooth, zero-centered, symmetric shape that preserves both positive and negative signals; unlike ReLU, it does not discard negative information but instead compresses inputs into a bounded range, allowing both excitatory and inhibitory effects to be represented.

5.10.3.4 Why sigmoid and tanh are rarely used in transformers

Both sigmoid and tanh share a key limitation in that they saturate for large absolute values of (x), causing gradients to vanish in those regions and making optimization difficult in deep architectures such as transformers. Despite this, sigmoid remains important in practice, especially in gating mechanisms like GLU and LSTM-style components, as well as in attention-related soft selection functions where smooth probabilistic control is useful.

5.10.3.5 GELU (Gaussian Error Linear Unit)

$$\text{GELU}(x) = x \cdot \Phi(x)$$

GELU replaces hard thresholding with a smooth probabilistic gating function.

Intuition: Instead of "drop or keep", it asks: > "how likely is this feature to be useful?"

This smoothness improves optimization stability and is used in BERT and early GPT-style models.

5.10.3.6 Swish

Swish is a simpler, learned-smooth alternative to GELU:

$$\text{Swish}(x) = x \cdot \sigma(x)$$

where ((x)) is the sigmoid function.

Swish is a smooth, fully differentiable activation similar in spirit to GELU, but simpler in form and notable for being non-monotonic, meaning it can both slightly suppress or enhance features depending on the input value. Intuitively, rather than using hard gating like ReLU or probabilistic gating like GELU, Swish applies a self-gated mechanism where each feature determines how much of itself to pass forward, which is why it often matches or slightly improves GELU's performance while remaining conceptually simpler.

5.10.3.7 GLU (Gated Linear Unit)

instead of simply modifying the activation function, GLU introduces explicit gating between two learned projections, where the input is split into a content stream and a gate stream:

$$\text{GLU}(x) = (xW_a) \odot \sigma(xW_b)$$

This design separates "what to pass" from "how much to pass," enabling multiplicative feature control that is substantially more expressive than standard pointwise activations.

5.10.3.8 SwiGLU (Swish-Gated Linear Unit)

Modern LLMs (LLaMA, PaLM, Mistral) use SwiGLU, which replaces the sigmoid gate with Swish:

$$\text{FFN}(x) = (\text{Swish}(xW_1) \odot (xW_2))W_3$$

SwiGLU combines the smooth, self-gated nonlinearity of Swish with the multiplicative two-stream structure of GLU, so rather than simply applying a nonlinearity to a feature, it learns how two projected feature streams interact through multiplicative gating. This shift from pointwise transformation to learned feature interaction makes it significantly more expressive than standard activation functions.

5.10.3.9 Visual intuition: from tanh to GELU-like behavior

A useful way to build intuition for modern activation functions is to see how they evolve from simple bounded nonlinearities into smooth, ReLU-like gating mechanisms that combine stability with adaptive feature control.

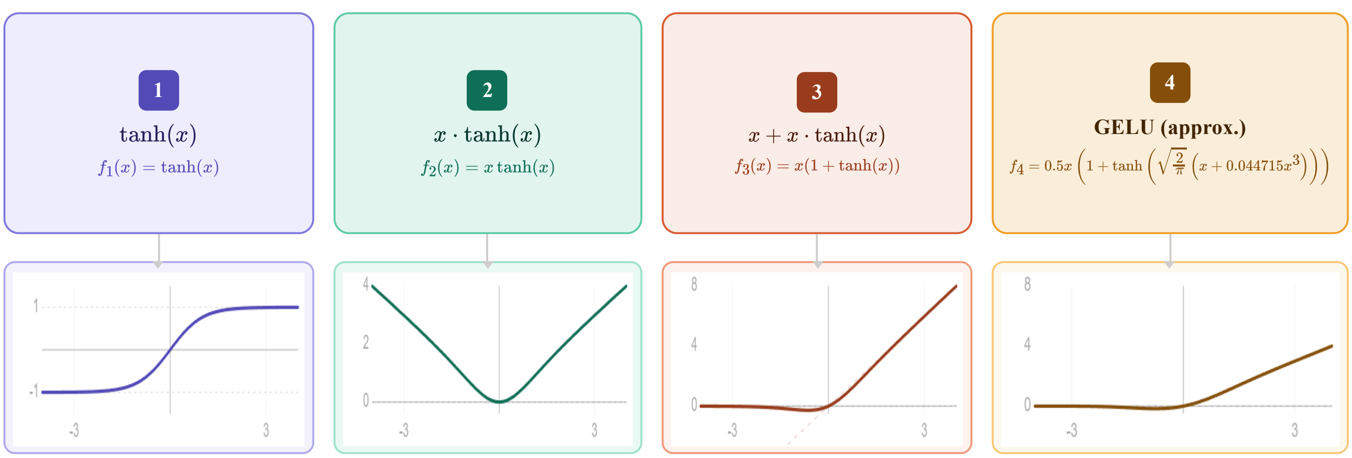


Figure 5.4: Tanh to GELU.

5.10.3.10 Visual intuition: from sigmoid to Swish-like behavior

A useful way to build intuition for modern activation functions is to see how they evolve from simple probabilistic gating functions into smooth, self-modulated nonlinearities. Sigmoid starts as a soft on–off switch, squeezing inputs into ((0,1)), while Swish generalizes this idea by letting the input itself be modulated by a smooth gate, producing a function that behaves like a continuous, input-dependent scaling mechanism rather than a fixed probabilistic filter.

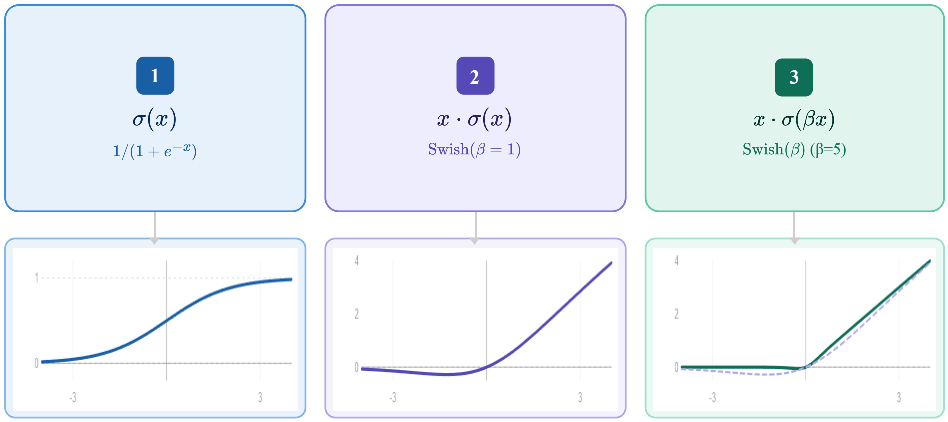


Figure 5.5: Sigmoid to swish.

5.10.4 Mixture of Experts (MoE)

Mixture of Experts (MoE) is used by Mixtral, reportedly GPT-4, it is a sparse Transformer architecture that increases model capacity without proportional increase in computation. It replaces the standard Feed-Forward Network (FFN) with multiple parallel FFNs (experts) and uses a learned routing mechanism to activate only a small subset per token.

Formulation

Given a set of experts $\mathrm{FFN}_1, \dots, \mathrm{FFN}_N$, a gating function computes routing scores:

$$g(x) = \mathrm{softmax}(W_r x)$$

MoE enables sparse computation, where only a fraction of parameters are active per token. This allows model capacity to scale significantly while keeping per-token compute similar to dense models. For example, a model with 8 experts of 7B parameters (~47B total) may activate only 1–2 experts per token, resulting in compute comparable to a much smaller dense model.

5.11 The output layer

After the final transformer block, the hidden state at the last token position is projected back to vocabulary size:

$$\mathrm{logits} = h_{\mathrm{final}} W_{\mathrm{embed}}^{\top} \quad \text{shape: [vocab\_size]}$$

This produces one unnormalized score per vocabulary token. Softmax converts logits to probabilities:

$$P(\mathrm{token}_i) = \frac{\exp(\mathrm{logits}_i)}{\sum_j \exp(\mathrm{logits}_j)}$$

The training objective (cross-entropy loss) maximizes the probability assigned to the actual next tokenand that is the only thing the model is explicitly trained to do. Chapter REFF covers training objectives in detail.

5.12 What the model actually learns

The transformer is trained to predict the next token. That is all. There is no explicit objective to learn grammar, facts, reasoning, or common sense, yet models trained at scale acquire all of these.

Why? Because predicting the next token well requires understanding everything that determines what comes next. Next-token prediction is a seems to be a simple objective that implicitly rewards a comprehensive world model. In order to predict `"Paris"` after `"The capital of France is"`, the model must have encoded the fact that Paris is the capital of France or to predict grammatically correct continuations, it must have internalized syntactic rules.

5.13 Key takeaways

- The transformer replaced recurrence with attention, enabling full parallelism during training and direct token-to-token interaction at any distance
- Token embeddings map integer IDs to continuous vectors; positional encodings inject order information; RoPE encodes relative rather than absolute position
- Self-attention allows every token to gather information from every other token via query-key dot products, scaled and softmaxed into attention weights over value vectors
- The causal mask enforces that tokens cannot attend to future positions, enabling autoregressive training on the full sequence in a single forward pass
- Multi-head attention runs multiple attention operations in parallel, each specializing in a different type of relationship.
- The feed-forward network processes each token independently after attention, acting as associative memory for facts learned during training
- Residual connections and layer normalization stabilize training of very deep networks; Pre-LN and RMSNorm are the current standard
- Modern variants (RoPE, GQA, SwiGLU, MoE) improve efficiency and performance without changing the fundamental architecture
- The next-token prediction objective implicitly rewards a comprehensive world model because everything that makes text predictable (grammar, facts, reasoning) improves prediction quality

---

5.14 Further reading

- Vaswani et al. (2017). *Attention Is All You Need.*: The original transformer paper.
- Alammar, J. (2018). *The Illustrated Transformer.*: The clearest visual explanation of the architecture available. Essential companion to this chapter.

Transformer Architecture — Cheat Sheet

Why Transformers Won

- RNNs: sequential → no parallelism
- Long-range info bottleneck in hidden state
- Transformers: attention → full parallelism + direct token interaction

High-Level Pipeline

Tokens → Embeddings → Positional Encoding → Transformer Blocks → Logits → Softmax

$P(\text{next token} \mid \text{context})$

Embeddings & Position

Token embedding: lookup table (vocab → vectors)

Weight tying: input embedding = output projection

Positional encodings:

- Sinusoidal (fixed, original transformer)
- Learned embeddings
- RoPE: rotates Q/K → relative position

Self-Attention

$Q = XW_q,\ K = XW_k,\ V = XW_v$

$\text{Attention} = \text{softmax}(QK^T / \sqrt{d})\, V$

Each token attends to all previous tokens

Creates contextualized representation

Attention Masks

- Causal (decoder): only past tokens visible
- Bidirectional (encoder): full sequence visible (BERT)
- Encoder-decoder: cross-attention between memory + generation
- Prefix LM: prefix bidirectional, suffix causal

Multi-Head Attention

Multiple parallel attention heads

Each head learns different relation type

Outputs concatenated + projected

Specialization emerges from training

Feed-Forward Network

$\text{FFN}(x) = W_2 \cdot \text{activation}(W_1 x)$

Per-token independent transformation

Modern activations:

- GELU (default GPT/BERT)
- SwiGLU (modern LLaMA, Mistral)

Residual + Normalization

x = x + Sublayer(LayerNorm(x)) (Pre-LN)

Stabilizes deep training

LayerNorm → RMSNorm (modern models)

Pre-LN = stable gradients at scale

Modern Variants

- RoPE: relative position encoding
- GQA: shared K/V across heads
- MoE: sparse expert FFNs
- Sliding window attention (Mistral)

Output Layer

$\text{logits} = h W^T \rightarrow \text{softmax} \rightarrow P(\text{next token})$

Training objective: next-token prediction only

Figure 5.6: Cheat sheet.

Chapter 6

Attention Computation

The canonical question for this chapter: *When the model processes your prompt, how does each token decide what to look at and what does the hardware actually do to compute that?*

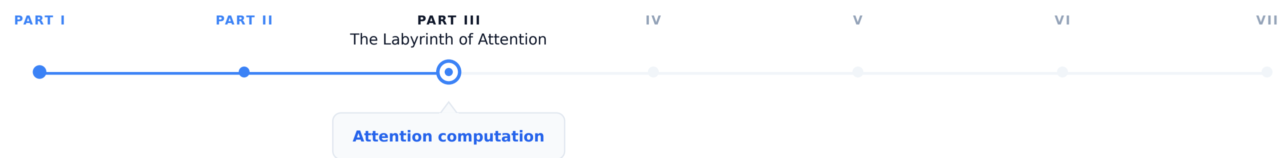


Figure 6.1: The journey through the Model Mind.

In the previous chapter we introduced attention conceptually: queries, keys, values, dot products, softmax, weighted sum. This chapter covers what attention computation actually looks like on real hardware, under the constraints of latency, memory bandwidth, and concurrent requests.

6.1 The quadratic problem

Standard self-attention has a fundamental scaling property where both computation and memory grow quadratically with sequence length. For a sequence of `n` tokens:

$$\begin{array}{ll} \text{Attention score matrix:} & n \times n \\ \text{Memory for scores:} & n^2 \times 2 \text{ bytes (BF16)} \\ \text{Compute for scores:} & n^2 \times d_k \text{ multiplications } (QK^\top) \\ \text{Compute for output:} & n^2 \times d_v \text{ multiplications } (\text{scores} \times V) \end{array}$$

At small sequence lengths this is manageable:

Sequence length	Score matrix	Memory
512 tokens	512 × 512	0.5 MB
2,048 tokens	2,048 × 2,048	8 MB
8,192 tokens	8,192 × 8,192	128 MB
32,768 tokens	32,768 × 32,768	2 GB

Sequence length	Score matrix	Memory
131,072 tokens	131,072 × 131,072	32 GB

At 128k tokens on the other hand which is currently supported by several models the attention score matrix alone would require 32 GB of GPU memory per layer per head which means a model with 96-layer and 96 heads would need $96 \times 96 \times 32$ GB = 295 TB and that is clearly impossible.

In reality the score matrix is never actually materializes in practice with the help of FlashAttention but the compute still scales quadratically. Doubling the sequence length quadruples the attention computation and it is central cost driver for long-context inference.

6.2 What the GPU actually does during attention

Understanding attention at the hardware level requires understanding two main bottlenecks in GPU computation, arithmetic throughput and memory bandwidth. **Arithmetic throughput** refers to how many floating-point operations per second the GPU can perform (an H100 delivers approximately 1,000 TFLOPS for BF16 matrix multiplications). **Memory bandwidth** refers to how fast data can be read/write from/to GPU high bandwidth memory (an H100 provides approximately 3.35 TB/s). The ratio between these, known as arithmetic intensity, determines whether a computation is compute-bound or memory-bandwidth-bound.

For matrix multiplication, the main operation in attention and FFNs, large matrices have high arithmetic intensity and are compute-bound, while small matrices or element-wise operations have low arithmetic intensity and are memory-bandwidth-bound.

During the decoding phase (processing one new token against the KV cache), attention is severely memory-bandwidth-bound. In this phase GPU must read the entire KV cache (potentially gigabytes of data) to compute attention scores for a single new token. The arithmetic work is tiny relative to the memory access while the GPU's massive arithmetic throughput sits idle, waiting for data. This is the reason why decoding throughput scales with memory bandwidth, not arithmetic throughput and why increasing sequence length primarily increases memory pressure and not the compute pressure, during generation.

6.3 Standard attention: the naive computation

Before FlashAttention, attention was computed in the obvious way:

```
# Standard attention (naive)
# Q: [seq_len, d_k]
# K: [seq_len, d_k]
# V: [seq_len, d_v]

scores = Q @ K.T                           # [seq_len, seq_len] - written to HBM
scores = scores / math.sqrt(d_k)           # scaling - read/write from/to HBM
scores = scores + causal_mask              # masking - read/write from/to HBM
scores = softmax(scores, dim=-1)           # softmax - read/write from/to HBM
output = scores @ V                        # [seq_len, d_v] - read/write from/to HBM
```

Each step read/write from/to HBM. The score matrix of shape `[seq_len, seq_len]` is materialized in HBM multiple times during scaling, masking, and softmax.

For a sequence of 8,192 tokens in BF16:

- Score matrix: $8{,}192 \times 8{,}192 \times 2$ bytes = 128 MB

- Each read/write of the score matrix: 128 MB / 3.35 TB/s 38 microseconds
- Across multiple reads and writes: hundreds of microseconds in memory traffic alone

The actual arithmetic is fast, while the movement of data between compute units and HBM is the bottleneck, which making this a classic I/O-bound computation.

6.4 FlashAttention

FlashAttention (Dao et al., 2022) computes exact attention which is basically same numerical result as standard attention, while using dramatically less memory and running significantly faster. The trick is reorganizing the computation to minimize HBM traffic.

6.4.1 The key insight: tiling

FlashAttention divides the Q, K, and V matrices into tiles that fit in SRAM, the fast on-chip memory sitting directly next to the compute units, with much higher bandwidth than HBM but much smaller capacity:

```
SRAM bandwidth:  ~20 TB/s   (fast, ~50 MB on H100)
HBM bandwidth:   ~3.35 TB/s (slower, 80 GB on H100)
```

By loading tiles into SRAM and performing the full attention computation on those tiles before writing results back to HBM, FlashAttention reduces HBM reads and writes from $O(n^2)$ to $O(n)$ while the score matrix is never written to HBM at all.

6.4.2 Online softmax

The challenge: softmax requires knowing the maximum value across all scores in a row before normalizing while in the tiled computation, you process one tile at a time without seeing the full row. How do you compute correct softmax without materializing the full score matrix?

FlashAttention uses an online softmax algorithm that maintains running statistics (the running maximum and running sum of exponentials) and updates them as each tile is processed. After all tiles, the statistics are complete and the output is correctly normalized:

```
running_max = -infinity
running_sum = 0
running_output = zeros(d_v)

for each key/value tile (K_tile, V_tile):
    scores_tile = Q_row @ K_tile.T / sqrt(d_k)
    tile_max = max(scores_tile)

    # Update running statistics
    new_max = max(running_max, tile_max)
    running_sum = (running_sum * exp(running_max - new_max)
                  + sum(exp(scores_tile - new_max)))
    running_output = (running_output * exp(running_max - new_max)
                     + exp(scores_tile - new_max) @ V_tile)
    running_max = new_max

# Final normalization
output = running_output / running_sum
```

This procedure produces the identical result to naive attention while never storing more than one tile's worth of scores at a time.

6.4.3 FlashAttention in practice

FlashAttention is 2–4× faster than standard attention on typical sequence lengths, with the speedup increasing as sequences grow longer and more importantly, its memory usage is O(n) rather than O(n^2).

6.5 Multi-head attention at inference

Multi-head attention runs `h` independent attention operations in parallel. For a model with `h = 96` heads and `d_model = 12,288`:

$$d_k = d_v = \frac{d_{\text{model}}}{h} = 128 \quad \text{(per head)}$$

Projections:

$$Q : [n,\ 12288]\ @\ [12288,\ 96 \times 128]\ \rightarrow\ [n,\ 96,\ 128]$$
$$K : [n,\ 12288]\ @\ [12288,\ 96 \times 128]\ \rightarrow\ [n,\ 96,\ 128]$$
$$V : [n,\ 12288]\ @\ [12288,\ 96 \times 128]\ \rightarrow\ [n,\ 96,\ 128]$$

Per-head attention:

$$\text{scores} = \frac{Q_h K_h^\top}{\sqrt{128}}\ \rightarrow\ [n,\ n]$$

$$\text{output} = \text{softmax}(\text{scores})\, V_h\ \rightarrow\ [n,\ 128]$$

Concatenate: $[n,\ 96,\ 128]\ \rightarrow\ [n,\ 12288]$

Output proj: $[n,\ 12288]\ @\ [12288,\ 12288]\ \rightarrow\ [n,\ 12288]$

In practice, all 96 heads are computed in parallel by treating the head dimension as a batch dimension. The Q, K, V projections produce batched tensors processed simultaneously by the matrix multiplication kernel.

6.5.1 MHA vs. MQA vs. GQA

Multi-Head Attention (MHA) uses separate Q, K, and V projection matrices for each head. While this increases expressiveness, it is memory-intensive because each head requires its own stored K and V tensors in the KV cache.

Multi-Query Attention (MQA) (Shazeer, 2019): All attention heads share a single set of K and V projections, while Q remains separate per head. This significantly reduces KV cache usage by a factor of h, for example, a 96-head model achieves a ~96× smaller cache and in general, this comes with only a modest impact on output quality.

MHA: $Q \in \mathbb{R}^{n\times h\times d_k},\ K \in \mathbb{R}^{n\times h\times d_k},\ V \in \mathbb{R}^{n\times h\times d_v}\ \Rightarrow\ \text{KV cache} \propto 2 \cdot h \cdot d_k$

MQA: $Q \in \mathbb{R}^{n\times h\times d_k},\ K \in \mathbb{R}^{n\times 1\times d_k},\ V \in \mathbb{R}^{n\times 1\times d_v}\ \Rightarrow\ \text{KV cache} \propto 2 \cdot d_k$

Grouped-Query Attention (GQA) (Ainslie et al., 2023): the middle ground in which Heads are divided into groups; each group shares one K and V projection:

Component	Shape	Details
Q (Query)	$[n, 96, d_k]$	per head
K (Key)	$[n, 8, d_k]$	per group

Component	**Shape**	**Details**
V (Value)	$[n, 8, d_v]$	per group
KV Cache	$\propto 2 \times 8 \times d_k$	12× smaller than MHA

LLaMA 2 70B uses GQA with 8 KV heads and Mistral 7B uses GQA with 8 KV heads. Quality loss compared to MHA is negligible on standard benchmarks while the KV cache reduction is significant for serving.

6.6 The KV cache at inference

During decoding, the model generates one token at a time and at each step, this new token must attend to all previous tokens. Recomputing the K and V vectors for all previous tokens at every step would be extremely wasteful due to the fact that those tokens have not changed, so their projections have not changed either.

The KV cache stores K and V tensors for all previous tokens and reuses them at each generation step. Only the new token's Q, K, V vectors need to be computed fresh. This is covered in full detail in chapter REFF nevertheless the momory cost understanding will be discussed here

6.6.1 KV cache memory per token

For LLaMA 2 70B (L=80 layers, 8 KV heads, d_k=128, BF16):

Per token per layer: $2 \times 8 \times 128 \times 2$ bytes = 4 KB

Per token total: 4×80 layers = 320 KB

For a 4,096-token sequence: 320 KB × 4,096 **1.28 GB of KV cache** per sequence. At batch size 64 concurrent sequences: 64 × 1.28 GB = 82 GB which is more than the 80 GB on an H100. This is the reason KV cache management is the central serving constraint for long-context workloads.

6.6.2 Prefill vs. decode attention

During prefill (processing the input prompt), all tokens' K and V vectors are computed simultaneously and written to the KV cache it is a batch matrix multiplication, efficient and compute-bound. On the othe hand during decoding (generating output), each new token's K and V are computed and appended to the cache, then attention is computed between that single new token's Q and the full KV cacheand it is a vector-matrix multiply, memory-bandwidth-bound:

```
Prefill: [n_prompt, d_k] @ [d_k, n_prompt]       → large matmul, compute-bound
Decode:  [1, d_k] @ [d_k, n_prompt + n_gen]      → vector-matrix, bandwidth-bound
```

This asymmetry is why decoding is slower per token than prefill, and why disaggregated serving (separate hardware for prefill and decode phases) is considered as an active area of optimization.

6.7 Sparse and linear attention

The quadratic cost of standard attention has motivated significant research into more efficient alternatives yet None has been able to displace full attention in frontier models, but several are worth understanding.

6.7.1 Sparse attention

Instead of every token attending to every other token, sparse attention restricts each token to a subset of positions:

Local window attention: each token attends only to the surrounding `w` tokens which reduces the complexity to O(n × w). With this approach the information still can propagate across long distances through multiple layers of local attention.

Strided attention: alternating layers use different patterns in which some layers use local windows and some use strided patterns (attend every k steps) allowing attention to distant tokens. Used in Longformer and BigBird.

Sliding window with global tokens: certain tokens (typically the first ones) attend to all other tokens and are attended to by all others. These global tokens propagate long-range information while the rest of the sequence uses local attention.

The limitation: the sparsity pattern must be specified in advance. If it does not match the actual information dependencies in the data, quality degrades. Full attention is flexible that helps the model to learn any pattern which is why it has remained dominant.

6.7.2 Linear attention

The key idea behind **linear attention** is to reinterpret softmax attention as a kernel operation whose structure allows the order of computation to change.

Ignoring scaling and normalization constants for clarity, the softmax attention weight between tokens can be written as

$$A_{ij} = \exp(q_i^\top k_j).$$

The attention output for token i therefore becomes

$$\text{output}_i = \frac{\sum_j \exp(q_i^\top k_j)\, v_j}{\sum_j \exp(q_i^\top k_j)}.$$

This reveals that attention is fundamentally a **kernel similarity** $\kappa(q_i, k_j) = \exp(q_i^\top k_j)$ meaning attention computes a weighted kernel regression over values.

With linear attention we can assume that this exponential kernel can be approximated by an inner product in a feature space, $\exp(q^\top k) \approx \phi(q)^\top \phi(k)$, where the feature map $\phi : \mathbb{R}^d \to \mathbb{R}^{d_\phi}$ embeds queries and keys into a space where their similarity become linear and substituting this approximation into attention gives

$$\text{output}_i = \frac{\sum_j \phi(q_i)^\top \phi(k_j)\, v_j}{\sum_j \phi(q_i)^\top \phi(k_j)}.$$

Because $\phi(q_i)$ does not depend on j, it can be factored outside the sums:

$$\text{output}_i = \frac{\phi(q_i)^\top \sum_j \phi(k_j) v_j^\top}{\phi(q_i)^\top \sum_j \phi(k_j)}.$$

All interactions with the sequence are now contained in two global statistics,

$$S = \sum_j \phi(k_j) v_j^\top, \qquad z = \sum_j \phi(k_j),$$

so the output becomes

$$\text{output}_i = \frac{\phi(q_i)^\top S}{\phi(q_i)^\top z}.$$

Stacking all queries yields the linear attention form

$$\boxed{\mathrm{Attn}(Q,K,V) = \phi(Q)(\phi(K)^\top V)} \quad \text{(up to normalization).}$$

The crucial consequence is that computation can be reordered:

$$(QK^\top)V \quad \longrightarrow \quad \phi(Q)(\phi(K)^\top V),$$

allowing the key-value product to be computed once and reused for all queries.

Method	Formula	Complexity
Standard attention	$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$	$O(n^2)$
Linear attention	$\phi(Q)(\phi(K)^\top V)$	$O(n)$

The limitation is that the approximation works for some tasks but degrades on others, particularly tasks requiring precise comparison of specific tokens. Softmax produces a peaky distribution (concentrating attention on a small number of tokens) which is important for retrieval-like tasks that linear approximations cannot replicate faithfully.

6.7.3 State space models and hybrid architectures

Another direction for scaling sequence models is to move away from attention completely and instead model sequences through **learned dynamical systems**. State space models (SSMs) replace pairwise token interaction with a recurrent state that evolves over time, allowing sequences to be processed in linear time.

Rather than computing interactions between all tokens, an SSM maintains a compact hidden state updated sequentially,

$$h_t = Ah_{t-1} + Bx_t, \qquad y_t = Ch_t,$$

so information is carried forward through the state instead of recomputed via an attention matrix. Models such as Mamba achieve $O(n)$ computation and memory while retaining long-range dependency modeling, since past context is compressed into the evolving state.

Hybrid architectures combine both paradigms. Instead of choosing between attention and recurrence, models like Jamba and Griffin interleave attention layers with SSM layers. Attention provides flexible global reasoning when precise token interactions matter, while SSM blocks handle long-context processing efficiently by propagating information through state updates.

These hybrid designs represent an active research direction: as context lengths grow, future architectures may rely less on dense attention and more on mixtures of attention, recurrence, and continuous-time sequence modeling.

6.8 Ring attention and sequence parallelism

For extremely long sequences, even FlashAttention with $O(n)$ memory may exceed a single GPU's capacity with the help of Ring attention the sequence distributions can go across multiple GPUs.

Each GPU holds a slice of the Q, K, and V matrices. The attention computation proceeds in a ring: each GPU computes a partial attention using its local Q slice against the current K/V slice, then passes the K/V to the next GPU while receiving the previous GPU's K/V:

```
Ring of 4 GPUs:

Step 1: GPU0 computes Q0×K0V0, GPU1 computes Q1×K1V1, ...
        K/V rotate: K0V0 → GPU1, K1V1 → GPU2, ...

Step 2: GPU0 computes Q0×K3V3 (received from GPU3)
        GPU1 computes Q1×K0V0 (received from GPU0), ...

Step 4: Each GPU has accumulated contributions from all K/V slices
        → complete attention output for its Q slice
```

After one full revolution, each GPU has computed its complete attention output using K/V from all other GPUs and enables sequence lengths that scale linearly with the number of GPUs.

6.9 Attention in the decode loop: step by step

Here is exactly what happens during one attention computation step during decoding, after prefill is complete and the model is generating token by token:

```
State at step t:
  KV cache contains K, V for tokens 0 through t-1
  New token t has been embedded with positional encoding applied

1. Compute Q, K, V for token t:
   Q_t = x_t @ W_Q    → [1, d_k] per head
   K_t = x_t @ W_K    → [1, d_k] per head  (stored in KV cache)
   V_t = x_t @ W_V    → [1, d_v] per head  (stored in KV cache)

2. Append K_t, V_t to KV cache:
   K_cache[:t+1] = [K_0, K_1, ..., K_{t-1}, K_t]

3. Compute attention scores:
   scores = Q_t @ K_cache[:t+1].T / sqrt(d_k)    → [1, t+1] per head

4. Softmax:
   attn_weights = softmax(scores)    → [1, t+1] per head

5. Weighted sum:
   attn_output = attn_weights @ V_cache[:t+1]    → [1, d_v] per head

6. Concatenate heads and project:
   output = concat(attn_output_per_head) @ W_O    → [1, d_model]

7. Add to residual stream:
   x_t = x_t + output
```

Step 3 is the memory-bandwidth bottleneck: it reads the entire KV cache from HBM. At 4,096 tokens for LLaMA 2 70B, step 3 reads approximately 1.28 GB of KV cache per generation step. At 3.35 TB/s bandwidth, that is 0.38 milliseconds per step (for attention alone, before FFN and other operations). This is why long-context generation is slower per token than short-context generation.

6.10 Key takeaways

- Attention computation scales quadratically with sequence length in both memory and compute; this is the central cost driver for long-context inference, at 128k tokens, the naive score matrix would require 32 GB per layer per head
- Standard attention materializes the full $n \times n$ score matrix in HBM, causing excessive memory traffic; FlashAttention eliminates this by tiling the computation into SRAM and using online softmax, reducing HBM access from $O(n^2)$ to $O(n)$
- During decode, attention is memory-bandwidth-bound: the GPU reads the entire KV cache for a single new token; throughput scales with HBM bandwidth, not arithmetic throughput
- GQA shares K and V projections across groups of heads, reducing KV cache by the ratio of total heads to KV heads, the primary reason GQA is now the default for new models
- Prefill is compute-bound (large batch matrix multiply); decode is memory- bandwidth-bound (vector-matrix multiply against the growing KV cache), the same model, two different hardware bottlenecks
- Sparse attention and linear attention reduce $O(n^2)$ cost but sacrifice flexibility; state space models and hybrid architectures are competitive alternatives for extreme sequence lengths

---

6.11 Further reading

- Dao et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.*: The foundational paper.
- Shazeer (2019). *Fast Transformer Decoding: One Write-Head is All You Need.*: The original Multi-Query Attention paper.
- Ainslie et al. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.*: Grouped-Query Attention.
- Liu et al. (2023). *Ring Attention with Blockwise Transformers for Near-Infinite Context.*: Ring attention for distributed long-context computation.
- Gu & Dao (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.*: The leading SSM alternative to attention.

Core Attention Equation

`Attention(Q,K,V) = softmax(QKᵀ / √dₖ) V`

QK^T → similarity scores between tokens

softmax → attention weights (probability distribution)

V → content being mixed based on weights

Tensor Shapes

Q: $[n, d_k]$

K: $[n, d_k]$

V: $[n, d_v]$

QK^T: $[n, n]$ (attention matrix)

Output: $[n, d_v]$

Intuition

Each token asks: “what should I look at?”

Q = query (what I want)

K = key (what I offer)

V = value (content to pass)

Output = weighted memory retrieval

Causal Mask (Decoder Attention)

Prevents access to future tokens

Implemented by adding $-\infty$ to invalid positions before softmax

`j > i → mask(i,j) = −∞ → softmax = 0`

Ensures autoregressive generation: predict next token only from past

GPU Reality

Two bottlenecks:

- Compute (TFLOPS) → fast matrix math
- Memory bandwidth → KV cache dominates decode

Decode is memory-bound: GPU waits for KV cache, not compute

KV Cache

Stores past Keys & Values

Avoids recomputation every step

Memory grows with sequence length:

$O(n \times \text{layers} \times \text{heads} \times d_k)$

Bottleneck for long-context inference

FlashAttention

Key idea: never store full $n \times n$ matrix

Tiling into SRAM (fast on-chip memory)

Online softmax computes statistics incrementally

Result: same output, much less memory traffic

$O(n^2)$ memory → $O(n)$ HBM traffic)

Multi-Head Attention

Parallel attention heads

Each head learns different relation type

Concat heads → linear projection

`Specialization emerges from training`

Attention Variants

- MHA: full expressivity, high KV cost
- MQA: shared K/V, minimal cache
- GQA: grouped sharing (LLaMA, Mistral)
- Sparse/Linear: efficiency vs quality tradeoff

Figure 6.2: Cheat sheet.

Chapter 7

KV Cache

The canonical question for this chapter: *Generating one token requires attending to every previous token. How do we avoid recomputing all of that work on every single generation step and what does managing that memory look like in a production system?*

Where are we?

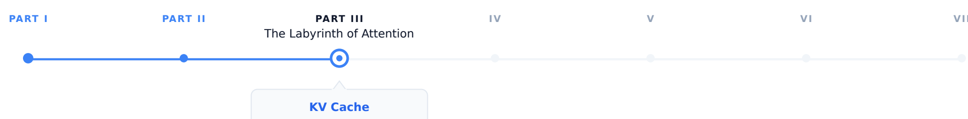


Figure 7.1: The journey through the Model Mind.

The previous chapter showed how attention is computed in inference and this chapter focuses on the engineering implications of that process when generation happens sequentially: at each step, the model must retain access to the keys and values from all previously generated tokens. While the KV cache makes this computationally practical, it becomes the primary memory management challenge in production scale LLM serving.

7.1 The problem that KV cache solves

Autoregressive generation is fundamentally sequential and generating each token requires attending to all previously generated tokens. To generate token `t`, the model must attend to all preceding tokens `0` through `t-1`, and generating each subsequent token expands the context by one, requiring the model to repeatedly process the entire accumulated sequence.

Without caching, generating a 500 token response to a 100 token prompt would require:

```
Step 1:   process 101 tokens → generate token 101
Step 2:   process 102 tokens → generate token 102
...
Step 500: process 600 tokens → generate token 600
```

Total compute proportional to $101 + 102 + ... + 600 \approx 175,000$ token processing steps. For large models, this process is highly inefficient: by step 500, most of the computation power is spent to reprocess tokens that were already handled at step 1, recomputing the same key and value vectors even though the underlying inputs have not changed.

The KV cache eliminates this redundancy by computing key and value vectors only once and stores them in memory. At each new generation step, the model computes the new token's Q, K, and V vectors, retrieves the cached keys and values for all previous tokens, and performs attention over the full cached context.

With the KV cache:

```
Prefill:          process all 100 prompt tokens once, store their K and V
Each decode step: compute K, V for one new token, append to cache, attend
Total work:       proportional to 100 (prefill) + 500 (one step per token)
```

The savings grow with sequence length. Without the KV cache, practical LLM deployment at today's context lengths would be economically infeasible.

7.2 What is stored

For each token in the sequence, for each transformer layer, for each KV head, the cache stores two vectors: the key and value produced by the linear projections of that token's hidden state.

```
KV cache structure:

Layer 0:
  K: [seq_len, n_kv_heads, d_k]   ← key vectors for all tokens so far
  V: [seq_len, n_kv_heads, d_v]   ← value vectors for all tokens so far

Layer 1:
  K: [seq_len, n_kv_heads, d_k]
  V: [seq_len, n_kv_heads, d_v]

...

Layer L-1:
  K: [seq_len, n_kv_heads, d_k]
  V: [seq_len, n_kv_heads, d_v]
```

Query vectors are not cached because they are used only once: the query for token `t` is needed solely to compute attention at generation step `t` and becomes irrelevant afterward. On the other hand, the key and value vectors for token `t` are reused at every subsequent step until generation ends. The KV cache exploit this asymmetry by storing persistent keys and values while recomputing only the transient query vectors.

7.3 Attention with and without a KV cache

Consider a single attention head. After processing a sequence of length (t),

$$K_t = \begin{bmatrix} k_1 \\ k_2 \\ \vdots \\ k_t \end{bmatrix}, \quad V_t = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_t \end{bmatrix}.$$

To generate the next token, the model first computes the hidden state (h_{t+1}) and projects it to

$$q_{t+1} = W_Q h_{t+1}, \quad k_{t+1} = W_K h_{t+1}, \quad v_{t+1} = W_V h_{t+1}.$$

The attention output is

$$o_{t+1} = \text{softmax}\left(\frac{1}{\sqrt{d_k}} q_{t+1} \begin{bmatrix} K_t & k_{t+1} \end{bmatrix}^T \right) \begin{bmatrix} V_t & v_{t+1} \end{bmatrix}.$$

The important observation is that all previous keys and values already exist:

$$K_t = [k_1, \dots, k_t], \quad V_t = [v_1, \dots, v_t].$$

Only the final column k_{t+1}, v_{t+1} must be computed and appended.

7.3.1 Without caching

At step $t+1$, the model recomputes

$$\{k_1, \dots, k_t, k_{t+1}\}, \quad \{v_1, \dots, v_t, v_{t+1}\}.$$

The cost of the key/value projections is therefore $O(t+1)$. Across an entire generation of length T,

$$\sum_{t=1}^{T} O(t) = O(T^2).$$

7.3.2 With caching

The projections

$$k_i = W_K h_i, \quad v_i = W_V h_i$$

are computed only once when token i is created. During generation of step $t+1$, the model only needs to compute the new projections k_{t+1}, v_{t+1} which results in $O(1)$ projection cost per step. Over a full sequence of length T, this accumulates to $\sum_{t=1}^{T} O(1) = O(T)$.

The attention computation itself still grows linearly with context length, but the repeated projection cost has been eliminated.

7.4 Memory arithmetic

KV cache memory is deterministic and computable from model hyperparameters and sequence length:

```
KV cache bytes = 2 × L × n_kv_heads × d_k × seq_len × bytes_per_element
```

Where `2` is for K and V, `L` is the number of transformer layers, `n_kv_heads` is the number of KV heads (equals `n_heads` for MHA, fewer for GQA), `d_k` is the key/value dimension per head, and `bytes_per_element` is 2 for BF16.

7.4.1 Examples

LLaMA 2 7B (L=32, n_kv_heads=32, d_k=128, BF16):

```
Per token: 2 × 32 × 32 × 128 × 2 = 524,288 bytes = 512 KB
4,096 tokens: 512 KB × 4,096 = 2 GB
```

LLaMA 2 70B (L=80, n_kv_heads=8, d_k=128, BF16):

```
Per token: 2 × 80 × 8 × 128 × 2 = 327,680 bytes = 320 KB
4,096 tokens:  320 KB × 4,096  = 1.28 GB
32,768 tokens: 320 KB × 32,768 = 10 GB
```

GPT-3 175B (L=96, n_kv_heads=96, d_k=128, BF16):

```
Per token: 2 × 96 × 96 × 128 × 2 = 4,718,592 bytes   4.5 MB
4,096 tokens: 4.5 MB × 4,096 = 18 GB
```

The GPT-3 numbers explain why GQA was adopted: reducing `n_kv_heads` from 96 to 8 reduces the per-token cache from 4.5 MB to 375 KB (12 times reduction) with modest quality loss. GQA is now standard in nearly every new model architecture for exactly this reason.

7.4.2 The memory budget

A serving instance has a fixed GPU memory budget, divided between competing claims:

```
Total GPU memory (e.g., 80 GB H100)
   Model weights     (fixed, e.g., ~35 GB for LLaMA 2 70B in BF16 with GQA)
   Activation memory (fixed per batch, ~1-3 GB)
   KV cache          (variable that grow with sequence length and batch size)
   Reserved          (CUDA overhead, fragmentation buffer, ~3 GB)

Available for KV cache: 80 - 35 - 2 - 3   40 GB
```

For LLaMA 2 70B with 40 GB available:

```
Max tokens in cache: 40 GB / 320 KB   125,000 tokens

Batch size 32, avg sequence 4,096 tokens:
  32 × 4,096 = 131,072 tokens → barely fits

Batch size 32, avg sequence 8,192 tokens:
  32 × 8,192 = 262,144 tokens → does not fit
```

This is the core serving constraint: KV cache capacity limits concurrency at long sequence lengths. Every additional token in context competes directly with additional concurrent users for the same GPU memory.

7.5 The lifecycle of a KV cache entry

Understanding the sequence of operations that touch the KV cache during a single request makes the memory management problem concrete.

7.5.1 Prefill

When a request arrives, the full prompt is tokenized and processed in a single forward pass (the prefill phase). For each layer, K and V are computed for every prompt token simultaneously and written to the KV cache:

```
Input:  [tok_0, tok_1, tok_2, ..., tok_n]  ← full prompt
Output: KV cache populated for positions 0 through n
        Model output: logits at position n → first generated token
```

Prefill is compute-bound and processing all prompt tokens in parallel is a large batch matrix multiply that saturates GPU arithmetic throughput. It is the phase where prompt length most directly affects latency: time to first token scales with prompt length.

7.5.2 Decode

Each decode step generates exactly one new token:

```
1. Embed new token → hidden state
2. Compute K, V for new token → append to KV cache
3. Compute Q for new token
4. Attend: Q against full KV cache (all positions 0 through current)
5. FFN and residual → logits → sample next token
```

Step 4 is memory-bandwidth-bound. The GPU reads the entire KV cache to compute attention for a single query vector. Arithmetic work is trivial compared to the memory transfer. This is why decode throughput is limited by HBM bandwidth, not by compute, and why longer sequences are slower per token.

7.5.3 Request completion

When generation ends (stop token, max_tokens, stop sequence), the KV cache entries for that request are freed. In a naive contiguous allocator this means marking a region of memory as available. In PagedAttention it means returning individual pages to the free pool.

7.6 Naive allocation and its problems

The simplest KV cache allocation strategy: at request admission, allocate a contiguous block of memory sized for the maximum possible output length. Hold it until generation completes, then free it.

This produces two forms of waste:

Internal fragmentation. If the request was allocated 8,192 token positions but only generated 200 tokens, the remaining 7,992 positions were reserved but unused for the lifetime of the request. At 320 KB per token for LLaMA 2 70B, that is 2.55 GB of unusable memory per such request.

External fragmentation. As requests arrive and complete with variable lengths, the free memory becomes fragmented into non-contiguous gaps. A new request requiring 1 GB may fail to be admitted even if 2 GB is technically free, because no single contiguous 1 GB region is available.

On real workloads with variable request lengths, naive allocation wastes 60–80% of available KV cache memory.

7.7 PagedAttention

PagedAttention (Kwon et al., 2023) solves the fragmentation problem by borrowing the virtual memory paging idea from operating systems.

Instead of allocating a contiguous block per request, PagedAttention divides the KV cache into fixed-size pages which typically has 16 tokens per page. A page table maps logical token positions to physical page locations. Pages are allocated on demand as generation proceeds and freed immediately on completion.

```
Page table for three concurrent requests:

Request A (generating, 48 tokens so far):
  Logical positions 0-15  → Physical page 3
  Logical positions 16-31 → Physical page 7
  Logical positions 32-47 → Physical page 12

Request B (generating, 32 tokens so far):
  Logical positions 0-15  → Physical page 1
  Logical positions 16-31 → Physical page 9

Request C (generating, 64 tokens so far):
  Logical positions 0-15  → Physical page 2
  Logical positions 16-31 → Physical page 4
  Logical positions 32-47 → Physical page 5
  Logical positions 48-63 → Physical page 11

Free pages: [0, 6, 8, 10, 13, 14, 15, ...]
```

Physical pages are non-contiguous. Logical positions map to whatever physical page is available. A new request needs four pages for a 64-token sequence; it gets four pages from the free pool regardless of where they are in physical memory.

Internal fragmentation is bounded to at most one partially filled page per request (the current page being written). External fragmentation is eliminated entirely and all free pages are usable regardless of their physical location. PagedAttention achieves over 90% KV cache utilization on typical workloads, compared to 20–40% for naive contiguous allocation.

7.7.1 Prefix sharing

PagedAttention enables a further optimization: memory sharing between requests with a common prefix.

If two requests share the same system prompt like a 1,000-token system prompt that appears in every request to a production API, their KV cache pages for that prefix are identical. There is no reason to store them twice.

With prefix sharing, the pages for the common prefix are marked as shared and reference-counted. Multiple requests point to the same physical pages via their page tables:

```
Request A page table:
  Positions 0-15   → Physical page 7  (shared system prompt, page 1)
  Positions 16-31  → Physical page 8  (shared system prompt, page 2)
  Positions 32-47  → Physical page 12 (private request A's user message)

Request B page table:
  Positions 0-15   → Physical page 7  (shared same physical pages)
  Positions 16-31  → Physical page 8
  Positions 48-63  → Physical page 15 (private request B's user message)
```

When both requests complete, the shared pages are not freed until all references are released. This is copy-on-write semantics: pages are shared until a write is required, at which point a private copy is made.

For a 1,000-token system prompt with 16 tokens per page: 63 pages of KV cache are shared across all concurrent requests using that prompt. At 320 KB per token for LLaMA 2 70B, this is 320 MB of KV cache that does not need to be recomputed or stored per-request.

7.7.2 Radix attention

Radix attention (SGLang, Zheng et al., 2023) generalizes prefix sharing to arbitrary prefix trees rather than just common prefixes.

In applications with structured multi-step prompts system like agentic workflows, few-shot templates, and multi-document retrieval, requests may share prefixes at multiple points in their structure, not just at the beginning. Radix attention maintains a trie (prefix tree) of all cached KV sequences. When a new request arrives, the longest matching prefix in the trie is found and reused:

```
Prefix tree:
  [System prompt] → shared by all requests
      [Document A] → shared by requests querying document A
          [Query 1] → private to request 1
          [Query 2] → private to request 2
      [Document B] → shared by requests querying document B
          [Query 3] → private to request 3
```

This turns prefix sharing from a single-level optimization into a general caching system for the KV cache. For agentic applications where many requests share substantial common context, radix attention can dramatically reduce both memory and compute.

7.8 KV cache quantization

The KV cache can be stored in reduced precision to trade a small amount of quality for significant memory savings.

INT8 KV cache stores key and value vectors as 8-bit integers rather than 16-bit floats. This provides a 2× memory reduction. Quality loss is typically negligible for standard generation tasks K and V vectors have bounded ranges as outputs of linear projections of layer-normalized hidden states, making them well-suited to quantization.

FP8 KV cache uses 8-bit floating point rather than integer quantization. H100 GPUs have native FP8 support. Better precision characteristics than INT8 for values with non-uniform distributions, with the same 2× memory reduction.

INT4 KV cache reduces to 4 bits for a 4× memory reduction. More quality loss; requires careful calibration. Used in memory-constrained scenarios where the alternative is rejecting requests.

The quality tradeoff for KV cache quantization is typically smaller than for weight quantization for two reasons: quantization errors in the KV cache are averaged across many attention heads (reducing their impact), and modern calibration techniques can minimize the effect on attention score distributions.

INT8 KV cache quantization is now standard in production and is supported by vLLM, TensorRT-LLM, and most major inference frameworks. The 2× memory savings effectively doubles serving concurrency on fixed hardware.

7.9 KV cache eviction

Some applications require sequences longer than the KV cache can hold. Rather than rejecting these requests, The serving system can evict cached entries to make room for new ones, at the cost of a quality trade-off.

7.9.1 Eviction policies

Sliding window evicts the oldest tokens, keeping only the most recent `w` token positions in cache. Simple to implement. Degrades quality for any task requiring long-range context, the model simply cannot attend to evicted positions.

H2O (Heavy-Hitter Oracle) evicts tokens with the lowest cumulative attention scores. Tokens that have received little attention weight in recent steps are unlikely to be needed in future steps. Empirically preserves quality better than sliding window eviction at the same cache budget.

Attention sink preservation always keeps the BOS (beginning of sequence) token regardless of other eviction decisions. The BOS token functions as an attention sink and many heads route excess attention weight to it when no other token is highly relevant. Evicting the BOS token causes attention distributions to become erratic and output quality to degrade unpredictably. Any eviction policy should treat the BOS token as eviction-exempt.

SnapKV identifies important KV vectors per head using attention patterns from a recent observation window, then evicts the remaining positions. Can maintain quality at 10–20% of the full KV cache size for long documents make it useful for very long context workloads where even INT8 quantization is insufficient.

7.10 Key takeaways

- The KV cache stores key and value vectors for all previous tokens, avoiding quadratic recomputation cost; query vectors are not cached because they are used exactly once per step
- KV cache memory is precisely computable: `2 × L × n_kv_heads × d_k × seq_len × bytes_per_element`; GQA's reduction in KV heads is the primary architectural lever for reducing this cost
- Naive contiguous allocation wastes 60–80% of KV cache memory through internal and external fragmentation; PagedAttention eliminates both by allocating fixed-size pages on demand, achieving 90%+ utilization
- Prefix sharing lets multiple requests reference the same physical KV cache pages for a shared prompt prefix — identical K/V vectors computed once, stored once; radix attention generalizes this to arbitrary shared structure
- INT8 KV cache quantization provides a 2× memory reduction with negligible quality loss, effectively doubling serving concurrency on fixed hardware
- Eviction policies (sliding window, H2O, SnapKV) enable sequences longer than the cache capacity; attention sink preservation is mandatory — evicting the BOS token causes unpredictable quality degradation
- StreamingLLM enables unbounded generation on a fixed cache budget by combining BOS token preservation with a sliding recent window
- CPU offloading moves inactive request caches to DRAM, freeing GPU HBM for active requests at the cost of PCIe transfer latency on resumption
- KV cache metrics — utilization, prefix hit rate, eviction rate, throughput vs. sequence length — are the primary production signals for diagnosing memory pressure and serving efficiency

---

7.11 Further reading

- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* — the foundational work on paged KV cache management.
- Zheng et al. (2023). *SGLang: Efficient Execution of Structured Language Model Programs.* — Introduces radix attention for generalized prefix tree sharing.
- Xiao et al. (2023). *Efficient Streaming Language Models with Attention Sinks.* — StreamingLLM and the attention sink phenomenon; directly relevant to eviction strategy design.
- Zhang et al. (2023). *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models.* — Attention-score-based eviction.
-

Core Idea

- Autoregressive decoding recomputes attention over all previous tokens
- KV cache stores Keys & Values once and reuses them for all future steps
- Eliminates redundant projection + recomputation during generation

Without cache: $O(T^2)$ recomputation → With cache: $O(T)$

Why It Matters in Production

Each new token attends to all previous tokens in KV cache
Decode cost becomes memory-bandwidth bound (HBM), not compute bound
Long context → KV cache dominates GPU memory and throughput

What is Stored

For each layer:
`K: [seq_len, heads, d_k]`
`V: [seq_len, heads, d_v]`

Stored per token, per layer, per head
Q is NOT stored (used once per step)

Growth: linear in sequence length

Attention with KV Cache

$o_t = \mathrm{softmax}(Q_t K_{cache}^\top) V_{cache}$

At step t:

- Q_t computed fresh
- K/V appended once per token
- All past K/V reused

No recomputation of history

Without vs With KV Cache

Without cache:
Recompute K,V for all tokens at every step → $O(T^2)$

With cache:
Compute K,V once → reuse → $O(T)$

Memory Cost

KV bytes = 2 × L × n_kv_heads × d_k × seq_len × bytes

Key drivers:

- Layers (L)
- KV heads (GQA reduces this)
- Context length (linear growth)

Decode Bottleneck

Each token requires reading full KV cache
→ HBM bandwidth bound (not FLOPs)

At long context:
1–10 GB memory read per token step

Result: longer context = slower generation

Prefill vs Decode

Prefill:
Parallel matmul → compute-bound

Decode:
Vector × matrix → bandwidth-bound

Same model, different bottleneck regime

Key Optimizations

- GQA: reduce KV heads → smaller cache
- PagedAttention: avoid fragmentation (paging instead of contiguous memory)
- Prefix sharing: reuse KV for shared prompts
- INT8/FP8 KV cache: 2× memory reduction
- Eviction policies: sliding window / H2O / SnapKV

Goal: maximize GPU utilization under fixed HBM budget

Figure 7.2: Cheat sheet.

Chapter 8

GPU Execution

The canonical question for this chapter: *When the model runs, what is GPU actually doing? Why does the same operation take wildly different amounts of time depending on how it is scheduled, and how the data is laid out in memory?*

Where are we?

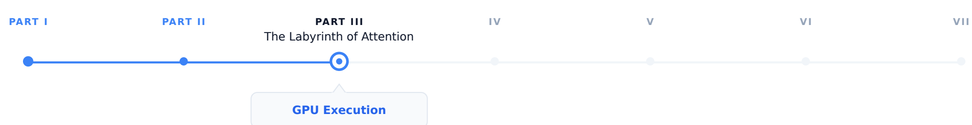


Figure 8.1: The journey through the Model Mind.

The previous chapters established what the model computes: embeddings, attention, feed-forward networks, the KV cache. This chapter covers *where* that computation actually happens and why it behaves the way it does. Every optimization discussed so far (FlashAttention, PageAttention, continuous batching, quantization) exists because of specific GPU execution constraints. This chapter makes those constraints explicit.

8.1 Why GPU execution deserves its own chapter

Every previous chapter has mentioned GPU constraints in passing: memory bandwidth, arithmetic throughput, HBM capacity. This chapter makes those concepts precise.

Understanding GPU execution has direct impact ob system performance and it is not just an academic exercise for LLM engineers. A serving system operating at 60% of a GPU's theoretical peak throughput can deliver 50% more throughput than one running at just 40%, without any additional hardware. This knowledge also explains why certain operations become bottlenecks, why quantization improves performance, why kernel fusion is so effective, and why the prefill and decode phases show different latency characteristics.

More directly: every optimization technique in serving (FlashAttention, continuous batching, tensor parallelism) exists because of specific GPU execution constraints. Understanding those constraints explains why the optimizations take the form they do, rather than appearing as a collection of unrelated tricks.

8.2 GPU architecture fundamentals

A modern GPU is not a fast CPU. It is a fundamentally different compute architecture optimized for a different class of problems.

8.2.1 The execution model

A CPU has a small number of powerful cores (typically 8–128), each capable of complex out-of-order execution, branch prediction, and deep caches. CPUs are optimized for latency and completing a single thread of execution as fast as possible.

A GPU has a massive number of simpler cores (an H100 has 16,896 CUDA cores) organized into streaming multiprocessors (SMs). Each SM is not particularly fast for a single operation, but thousands of them executing simultaneously produce enormous total throughput. GPUs are optimized to complete as many simple operations as possible per second, not completing any one operation quickly.

The programming model reflects on this: GPU programs are written as kernels that execute the same operation on many data elements simultaneously. A matrix multiplication kernel runs the same multiply operation on thousands of elements in parallel. If there is data parallelism in your problem, a GPU exploits it. If there is not, a GPU is slower than a CPU for that specific task.

LLM inference has enormous data parallelism: the same transformer operations apply to every token in the sequence, and every element of every weight matrix participates in the same type of computation. This structural match is why GPUs dominate LLM inference.

8.2.2 The memory hierarchy

GPU memory has multiple tiers with very different bandwidth and capacity:

```
Memory tier       Bandwidth        Capacity      Latency

Registers         ~80 TB/s         256 KB/SM     ~1 cycle
L1/SRAM           ~20 TB/s         256 KB/SM     ~30 cycles
L2 cache          ~5 TB/s          50 MB         ~200 cycles
HBM (VRAM)        ~3.35 TB/s       80 GB         ~600 cycles
PCIe (CPU RAM)    ~64 GB/s         TBs           ~microseconds
NVLink (GPU-GPU)  ~900 GB/s        -             ~microseconds
```

SRAM (often called shared memory or L1 cache) is the GPU's fast on-chip memory. It offers extremely low latency but limited capacity. HBM (High Bandwidth Memory), is the GPU's large off-chip memory although it provides massive bandwidth, accessing HBM is still significantly more expensive than accessing on-chip memory.

Every GPU computation ultimately reads data from HBM and writes results back to it. As a result, the central challenge of GPU kernel optimization is reducing HBM traffic by keeping frequently accessed data in SRAM for as long as possible which is the exact key insight behind FlashAttention.

8.2.3 Streaming multiprocessors and warps

The Streaming Multiprocessor (SM) is the fundamental compute unit of an NVIDIA GPU. An H100 GPU contains many SMs operating in parallel, with each SM responsible for executing thousands of lightweight threads concurrently.

Each SM on an H100 contains:

- 128 CUDA cores for FP32 arithmetic operations.
- 4 Tensor Cores specialized for matrix multiplication.
- 256 KB of registers for storing temporary values used by active threads.
- 228 KB of on-chip SRAM, configurable between:
  - L1 cache for automatically caching memory accesses.

  - Shared memory for explicitly sharing data between threads in the same thread block.
- Warp schedulers that determine which groups of threads execute next.

GPU threads execute in groups called warps, where each warp consists of 32 threads. Unlike CPU threads, which execute independently, all threads within a warp execute the same instruction at the same time on different pieces of data. NVIDIA refers to this execution model as SIMT (Single Instruction, Multiple Threads).

For example, consider the following code executed by a warp:

```
if x > 0:
    y = a + b
else
    y = c + d
```

Suppose that among the 32 threads in the warp 20 threads satisfy $x > 0$ and 12 threads satisfy $x <= 0$ The GPU cannot execute both branches simultaneously within the same warp. Instead, it execute $y = a + b$ for the 20 relevant threads while the other 12 threads remain temporarily inactive and than execute $y = c + d$ for the remaining 12 threads while the first 20 threads remain inactive. Conceptually, execution looks like:

```
Cycle 1:
[Active]   Threads 0-19  -> y = a + b
[Inactive] Threads 20-31

Cycle 2:
[Inactive] Threads 0-19
[Active]   Threads 20-31 -> y = c + d
```

Although each thread only needs one branch, the warp executes both branches sequentially, reducing parallel efficiency whcih is known as branch divergence.

An H100 has 132 SMs. With 4 warp schedulers per SM and 32 threads per warp, the H100 can keep $132 \times 4 \times 32 =$ 16,896 threads active simultaneously.

8.2.4 Tensor cores

Standard CUDA cores perform scalar floating point operations. Tensor cores perform matrix multiplications directly in hardware, at much higher throughput.

An H100 tensor core performs a 16×16×16 matrix multiplication in a single operation. The H100 has 528 tensor cores, delivering approximately:

- 1,000 TFLOPS for BF16/FP16 matrix multiplications
- 1,979 TFLOPS for FP8 matrix multiplications
- 66 TFLOPS for FP32 scalar operations

The gap between tensor core throughput and scalar throughput explains why matrix multiplication is fast and why all other operations are relatively expensive on modern GPUs.

8.3 The roofline model

The roofline model is a simple framework for understanding whether a given operation is compute-bound or memory-bandwidth-bound, and what the theoretical maximum performance is:

```
Attainable performance = min(Peak FLOPS, Memory bandwidth × Arithmetic intensity)
```

Where arithmetic intensity = FLOPs executed / bytes read from memory.

The ridge point is where the roofline transitions from memory-bound to compute-bound which is approximately 300 FLOPs/byte for the H100 (1,000 TFLOPS / 3.35 TB/s). Operations with arithmetic intensity above 300 are compute-bound and those below are memory-bandwidth-bound.

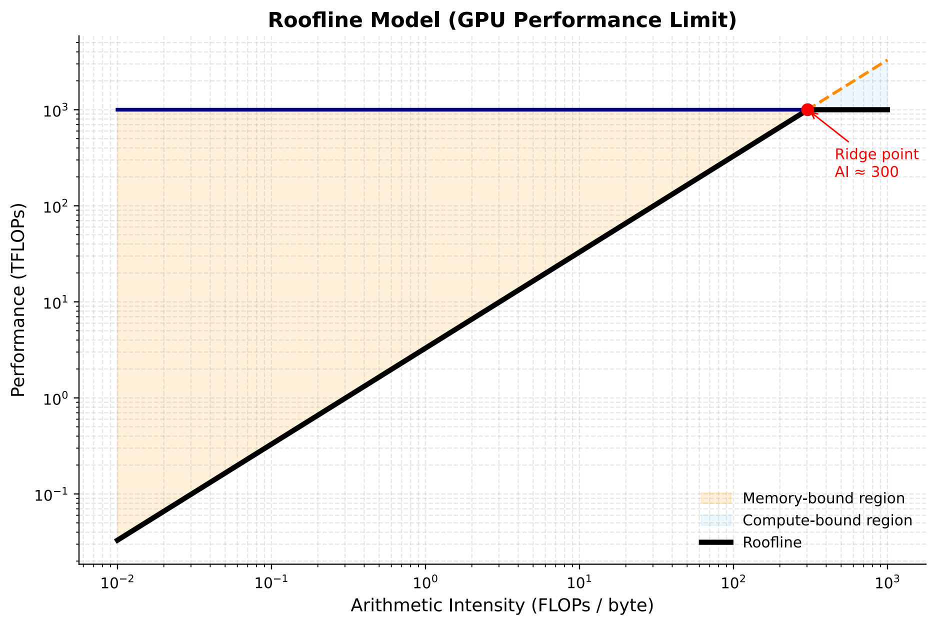


Figure 8.2: The roofline Model.

8.3.1 LLM operations on the roofline

Different operations in LLM inference have very different arithmetic intensities:

Large matrix multiplications (prefill, large batches): Matrix multiplication achieves high arithmetic intensity by reusing each byte of weight data for every sequence in the batch. For example, with a batch size of 32 and $d_model = 4,096$, the arithmetic intensity is approximately 32 FLOPs/byte. This value approaches the roofline model's ridge point, indicating that performance is primarily limited by computational throughput rather than memory bandwidth.

Vector-matrix products (decode, batch size 1): The Q projection at a batch size of 1 exhibits low arithmetic intensity because each byte of weight data is used in only one multiply operation. Consequently, the arithmetic intensity is approximately 1 FLOP/byte, which is about 300× below the roofline model's ridge point. As a result, performance is severely limited by memory bandwidth, leaving the tensor cores underutilized.

Attention with KV cache (decode): extremely low arithmetic intensity. Reading the full KV cache and computing a dot product with the query vector. Memory- bandwidth-bound with near-zero compute utilization.

Softmax, layer norm, element-wise ops: essentially zero arithmetic intensity. Read data, apply a simple function, write it back. Pure memory bandwidth operations. No tensor core involvement.

The practical implication: for decode at small batch sizes, the GPU is almost entirely idle arithmetically, waiting for data from HBM. Increasing batch size is the primary lever for improving GPU utilization during decode, because it amortizes weight reads across multiple sequences and increases arithmetic intensity toward the ridge point.

8.4 Prefill vs. decode: two different regimes

The roofline analysis explains why prefill and decode behave so differently and they are not just slow and fast versions of the same operation. They are different hardware bottlenecks entirely.

8.4.1 Prefill: compute-bound

During prefill, all prompt tokens are processed simultaneously. The Q projection operates on a matrix of shape `[n_prompt, d_model]`:

```
Q projection: [n_prompt, d_model] @ [d_model, d_model]
FLOPs:        2 × n_prompt × d_model²
Bytes:        d_model² × 2    (weight matrix, loaded once)

Arithmetic intensity = n_prompt FLOPs/byte
```

For n_prompt = 1,024 and d_model = 4,096: arithmetic intensity = 1,024 FLOPs/byte which is above the ridge point. Prefill is compute-bound. The H100 runs near its theoretical FLOPs ceiling. In order to make prefill faster it requires more FLOPs (more GPUs, higher clock speeds, or more efficient kernels).

8.4.2 Decode: memory-bandwidth-bound

During decode, only one new token is processed at a time. The Q projection operates on a vector of shape `[1, d_model]`:

```
Q projection: [1, d_model] @ [d_model, d_model]
FLOPs:        2 × 1 × d_model²
Bytes:        d_model² × 2    (same weight matrix)

Arithmetic intensity = 1 FLOPs/byte
```

For d_model = 4,096: arithmetic intensity = 1 FLOPs/byte which makes decode memory-bandwidth-bound. The H100 runs at approximately 1/300th of its peak FLOPs utilization. Making decode faster requires loading weights faster, higher bandwidth memory, quantization (fewer bytes per parameter), or larger batches (sharing weight reads across sequences).

8.5 Kernel fusion

Each operation in a transformer block, such as layer normalization, Q/K/V projection, softmax, and residual addition, can be implemented as a separate CUDA kernel. In this approach, every kernel reads its inputs from HBM and writes its outputs back to HBM before the next kernel executes. While this design is straightforward to implement, it is inefficient because intermediate results are repeatedly transferred to and from HBM despite being needed only by subsequent operations.

Kernel fusion addresses this inefficiency by combining multiple operations into a single CUDA kernel. Intermediate results are retained in registers or on-chip shared memory (SRAM) rather than being written to HBM after each operation. Only the final output is stored in HBM, reducing memory traffic, increasing arithmetic intensity, and improving overall performance.

8.5.1 Example: fused layer norm and linear projection

Without fusion:

```
x_norm = layer_norm(x)      # read x from HBM, write x_norm to HBM
q = x_norm @ W_Q            # read x_norm from HBM, write q to HBM
```

Two HBM reads, two HBM writes, for data that is only an intermediate result.

With fusion:

```
Single kernel:
  Read x from HBM once
```

```
Compute layer norm in registers/SRAM
Apply W_Q projection immediately
Write q to HBM once
```

One HBM read, one HBM write. Memory traffic halved for this pair of operations. The intermediate `x_norm` never leaves the chip.

8.5.2 FlashAttention as kernel fusion

FlashAttention is the most impactful example of kernel fusion in LLM inference. The naive attention computation reads and writes the n×n score matrix multiple times (scaling, masking, softmax, and the final weighted sum). FlashAttention fuses all of these into a single kernel that keeps intermediate scores in SRAM.

The memory traffic reduction is from $O(n^2)$ to $O(n)$. At long sequence lengths, this is the difference between attention fitting in available memory and not fitting at all. Chapter REFF covers FlashAttention in detail; the point here is that it is fundamentally a kernel fusion optimization, not an algorithmic approximation. The result is mathematically identical to naive attention.

8.5.3 Kernel fusion in practice

Production inference frameworks apply kernel fusion throughout the model:

- Layer norm fused into linear projections (Q, K, V, output projections)
- Attention scores, masking, and softmax fused (FlashAttention)
- Residual addition fused with layer norm
- Activation functions fused into FFN computation
- RoPE application fused into Q and K projection

The cumulative impact is typically 1.5–2× throughput improvement over naively separate kernels, primarily by reducing HBM traffic.

8.6 Precision and quantization at the kernel level

Quantization affects GPU execution at the kernel level, not just memory footprint. Understanding this distinction separates knowing what quantization does from knowing why it helps.

8.6.1 Weight-only quantization

The most common serving quantization scheme: model weights are stored in INT4 or INT8, but computation happens in BF16/FP16.

During a weight-only quantized matrix multiplication:

```
1. Load INT4 weights from HBM          (4× fewer bytes than BF16)
2. Dequantize to BF16 in registers     (fast, negligible compute overhead)
3. Multiply with BF16 activations via tensor cores
4. Accumulate in FP32, write BF16 output to HBM
```

The benefit: decode is memory-bandwidth-bound. Reducing weight bytes by 4× means 4× more weights can be streamed per second, directly improving decode throughput by approximately 4× for bandwidth-limited operations.

The cost: dequantization adds compute overhead. For bandwidth-bound operations (decode), this overhead is negligible. For compute-bound operations (prefill, large batches), dequantization adds meaningful overhead. Weight-only quantization therefore improves decode more than prefill, which is precisely where the improvement is most needed.

8.6.2 Activation quantization: INT8 and FP8

For compute-bound operations, quantizing both weights and activations to INT8 or FP8 allows tensor cores to operate at higher throughput:

- BF16 tensor cores: 1,000 TFLOPS
- FP8 tensor cores: 1,979 TFLOPS on H100 (approximately 2× faster)

Activation quantization is more complex than weight quantization because activations have more dynamic range and they can contain outlier values that cause large quantization errors when mapped to a narrow integer range.

The outlier problem. LLM activations contain outlier values in a small fraction of channels, values that are orders of magnitude larger than typical. These outliers are consistent across inputs and are a known property of large language models. Naive quantization sets the scale by the outlier, compressing all normal values to near-zero and destroying information.

LLM.int8() (Dettmers et al., 2022) addresses this by decomposing the matrix multiplication: outlier channels are computed in full BF16 precision, and the remaining channels (typically 99%+) are computed in INT8. The results are combined. Quality is preserved while most of the memory savings are retained.

SmoothQuant (Xiao et al., 2023) migrates the quantization difficulty from activations to weights, mathematically equivalent to scaling the activation channels down and the corresponding weight rows up before quantization. The result is more uniform activation distributions that quantize cleanly to INT8 without special outlier handling.

FP8 inference is available on H100 GPUs and has become the recommended precision for large-scale serving: near-BF16 quality at roughly 2× the arithmetic throughput. The H100's Transformer Engine handles FP8 scaling automatically, making it practical without manual calibration.

8.7 Tensor parallelism at the kernel level

When a model is too large for a single GPU, tensor parallelism distributes weight matrices across multiple GPUs. The kernel-level implementation requires communication between GPUs at each layer.

8.7.1 Column and row parallelism

A linear projection `Y = X @ W` can be split in two ways:

Column parallelism splits W by columns:

```
W split into [W  | W ] across 2 GPUs
GPU 0: Y  = X @ W    (partial output - subset of output dimensions)
GPU 1: Y  = X @ W    (partial output - complementary subset)
Combine: Y = [Y  | Y ]    (concatenation, no communication needed here)
```

Row parallelism splits W by rows and requires split input:

```
W split into [W ; W ], X split into [X , X ] across 2 GPUs
GPU 0: Y_partial  = X  @ W
GPU 1: Y_partial  = X  @ W
Combine: Y = Y_partial  + Y_partial    (all-reduce required)
```

In a transformer layer: Q, K, V projections use column parallelism, each GPU computes a subset of attention heads. The output projection uses row parallelism. Attention within each GPU is independent; an all-reduce synchronizes after the output projection. The FFN follows the same pattern: column parallelism on the first layer, row parallelism on the second, with one all-reduce between them.

8.7.2 Pipeline parallelism and bubble overhead

Pipeline parallelism splits a transformer model across multiple GPUs by assigning different sets of layers to different devices. Instead of each GPU holding the full model, computation flows through GPUs sequentially, like an assembly line and each GPU processes a contiguous block of layers. To keep all GPUs busy, the input batch is divided into microbatches, which are then streamed through the pipeline:

```
Time step:        1    2    3  |  4    5    6    7    8  | 9   10   11
                 <-- Fill -->  | <---- Steady state ---> | <-- Drain -->

GPU 0 (L0 -L24): [1]  [2]  [3]   [4]  [5]  [6]  [7]  [8]    ·    ·    ·
GPU 1 (L25-L48):  ·   [1]  [2]   [3]  [4]  [5]  [6]  [7]  [8]    ·    ·
GPU 2 (L49-L72):  ·    ·   [1]   [2]  [3]  [4]  [5]  [6]  [7]  [8]    ·
GPU 3 (L73-L96):  ·    ·    ·    [1]  [2]  [3]  [4]  [5]  [6]  [7]  [8]
```

This creates a wave-like execution pattern where different GPUs work on different microbatches at different stages of the model. However, pipeline parallelism introduces idle time, known as the pipeline bubble. This occurs during pipeline fill (startup) and pipeline drain (shutdown), when not all GPUs are fully utilized.

Increasing the number of microbatches reduces bubble overhead by improving pipeline utilization, but it also increases memory usage and scheduling complexity. In practice, systems use 8–32 microbatches to keep bubble overhead under 10% while maintaining efficiency.

8.8 CUDA graph capture

Every GPU operation requires a kernel launch: the CPU sends an instruction to the GPU specifying which kernel to run with which parameters. For small operations, kernel launch overhead can dominate the actual execution time.

For a transformer decode step at batch size 1:

- Total arithmetic work: small (bandwidth-bound, most of step time is HBM reads)
- Number of kernel launches: hundreds (one per operation across all layers)
- Kernel launch overhead: ~5–10 microseconds per launch

Hundreds of launches × 5–10 microseconds = milliseconds of CPU overhead per decode step, for a step that may itself take only a few milliseconds of actual GPU work. The CPU becomes the bottleneck.

CUDA graphs capture the sequence of kernel launches and their data dependencies as a static graph, replayed without CPU involvement for each step. After the initial capture, subsequent replays bypass CPU overhead entirely.

For decode at batch size 1, CUDA graph capture reduces per-step latency by 30–50%. The tradeoff: the graph is captured for a specific batch size and sequence length; changing these requires recapture. Production inference frameworks use CUDA graphs for decode (fixed batch size and one new token per step) and not for prefill (variable sequence length per request).

8.9 Key takeaways

- GPUs are throughput-optimized processors with thousands of simple cores; LLM inference maps well to GPUs because transformer operations are highly data- parallel — the same computation applied to every token and every weight element
- The GPU memory hierarchy spans registers (80 TB/s) to HBM (3.35 TB/s) to PCIe (64 GB/s); kernel optimization is the art of keeping hot data in fast memory and minimizing HBM round trips
- Tensor cores perform 16×16×16 matrix multiplications in hardware at ~15× the throughput of scalar CUDA cores; every operation expressible as matrix multiplication should be

- The roofline model predicts whether an operation is compute-bound (arithmetic intensity above ~300 FLOPs/byte for H100) or memory-bandwidth-bound (below); prefill is compute-bound, decode is severely memory-bandwidth-bound at ~1 FLOPs/byte
- MFU for decode at batch size 1 is approximately 0.3% — the GPU is nearly idle arithmetically; batching and quantization are the primary levers for improving this
- Kernel fusion eliminates intermediate HBM reads and writes between operations; FlashAttention reduces attention memory traffic from $O(n^2)$ to $O(n)$ by keeping scores in SRAM throughout computation
- Weight-only INT4/INT8 quantization reduces decode latency proportionally to the bit reduction, with negligible dequantization overhead when bandwidth-bound; FP8 activation quantization improves prefill throughput by ~2×
- CUDA graph capture eliminates CPU kernel launch overhead — 30–50% latency reduction for decode at small batch sizes — by replaying a static kernel sequence without CPU involvement

---

8.10 Further reading

- Williams et al. (2009). *Roofline: An Insightful Visual Performance Model for Multicore Architectures.* — The roofline model; essential reading for GPU performance analysis regardless of domain.
- Dao et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* — The most important kernel fusion example in LLM inference; the IO complexity analysis is the key insight.
- Dettmers et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.* — The outlier problem and mixed-precision INT8 quantization.
- Xiao et al. (2023). *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models.* — Migrating quantization difficulty from activations to weights for clean INT8 inference.
- Shoeybi et al. (2019). *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism.* — Tensor and pipeline parallelism; the foundational implementation reference.

Transformer Architecture — Cheat Sheet

Why Transformers Won

- RNNs: sequential → no parallelism
- Long-range info bottleneck in hidden state
- Transformers: attention → full parallelism + direct token interaction

High-Level Pipeline

Tokens → Embeddings → Positional Encoding → Transformer Blocks → Logits → Softmax

`P(next token | context)`

Embeddings & Position

Token embedding: lookup table (vocab → vectors)

Weight tying: input embedding = output projection

Positional encodings:

- Sinusoidal (fixed, original transformer)
- Learned embeddings
- RoPE: rotates Q/K → relative position

Self-Attention

`Q = XWq, K = XWk, V = XWv`

`Attention = softmax(QKᵀ / √d) V`

Each token attends to all previous tokens

Creates contextualized representation

Attention Masks

- Causal (decoder): only past tokens visible
- Bidirectional (encoder): full sequence visible (BERT)
- Encoder-decoder: cross-attention between memory + generation
- Prefix LM: prefix bidirectional, suffix causal

Multi-Head Attention

Multiple parallel attention heads

Each head learns different relation type

Outputs concatenated + projected

`Specialization emerges from training`

Feed-Forward Network

`FFN(x) = W2 · activation(W1 x)`

Per-token independent transformation

Modern activations:

- GELU (default GPT/BERT)
- SwiGLU (modern LLaMA, Mistral)

Residual + Normalization

x = x + Sublayer(LayerNorm(x)) (Pre-LN)

Stabilizes deep training

LayerNorm → RMSNorm (modern models)

Pre-LN = stable gradients at scale

Modern Variants

- RoPE: relative position encoding
- GQA: shared K/V across heads
- MoE: sparse expert FFNs
- Sliding window attention (Mistral)

Output Layer

`logits = h Wᵀ → softmax → P(next token)`

Training objective: next-token prediction only

Figure 8.3: Cheat sheet.

Chapter 9

Quantization and Model Compression

The canonical question for this chapter: *A 70B parameter model in full precision requires 140 GB of GPU memory and costs a fortune to serve. How do you make it smaller, faster, and cheaper without making it useless?*

Where are we?

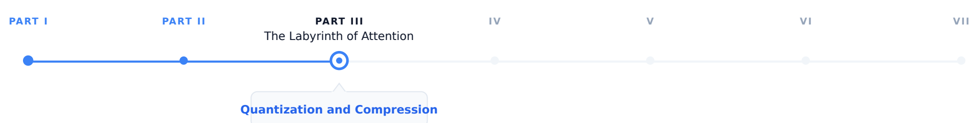


Figure 9.1: The journey through the Model Mind.

This chapter covers the numerical representation of the model itself: how reducing the precision of weights and activations affects memory, speed, and quality, and which techniques have become standard in production. Quantization is not an afterthought for most deployments, it is the difference between a model that fits and one that does not.

9.1 The memory problem

A language model's parameters are floating-point numbers. The precision of those numbers (how many bits are used to represent each one) determines how much memory the model occupies.

For a model with `N` parameters:

```
FP32 (4 bytes per parameter):   N × 4 bytes
BF16 (2 bytes per parameter):   N × 2 bytes
INT8 (1 byte per parameter):    N × 1 byte
INT4 (0.5 bytes per parameter): N × 0.5 bytes
```

For a 70B parameter model:

```
FP32:  70B × 4  = 280 GB   -> requires 4× H100s just for weights
BF16:  70B × 2  = 140 GB   -> requires 2× H100s
INT8:  70B × 1  =  70 GB   -> fits on 1× H100 with room for KV cache
```

```
INT4:  70B × 0.5 =  35 GB  -> fits on 1× H100 with substantial headroom
```

The difference between BF16 and INT8 is not just cost it is also whether a model can be served on a single GPU at all. INT4 opens models to consumer hardware: a 70B model at INT4 fits on a single 40 GB A100 or on two consumer RTX 4090s. This is the practical motivation for quantization and everything else follows from it.

9.2 Floating point formats

Understanding quantization requires understanding the number formats involved.

9.2.1 IEEE 754 floating point

Standard floating point represents numbers as:

```
value = (-1)^sign × mantissa × 2^exponent

FP32 (float):  1 sign bit + 8 exponent bits + 23 mantissa bits = 32 bits
               Range: ±3.4 × 10^38, precision: ~7 decimal digits

FP16 (half):   1 sign bit + 5 exponent bits + 10 mantissa bits = 16 bits
               Range: ±65,504, precision: ~3 decimal digits

BF16:          1 sign bit + 8 exponent bits + 7 mantissa bits  = 16 bits
               Range: same as FP32, precision: ~2 decimal digits
```

The key difference between FP16 and BF16: BF16 keeps FP32's exponent range (8 bits) but reduces the mantissa. This matters for training, large language models encounter large gradient values that overflow FP16's limited range (max   65,504) but fit comfortably in BF16's range. BF16 became the standard training format for large models after FP16 was found to cause instability in very deep networks.

9.2.2 FP8

FP8 is a family of 8-bit floating point formats introduced with the H100 GPU:

```
E4M3: 1 sign + 4 exponent + 3 mantissa bits -> more precision, less range
E5M2: 1 sign + 5 exponent + 2 mantissa bits -> more range, less precision
```

The H100's Transformer Engine supports FP8 matrix multiplications natively, roughly doubling throughput compared to BF16 on the same hardware. FP8 is now the recommended training and inference format for frontier models on H100-class hardware.

9.3 Integer quantization

Integer formats represent numbers in a fundamentally different way, as scaled integers rather than floating point values.

9.3.1 The quantization mapping

To quantize a floating point value to INT8:

```
1. Find the range of values to quantize: [x_min, x_max]
2. Compute scale: s = (x_max - x_min) / 255
3. Compute zero point: z = round(-x_min / s)
4. Quantize: x_int8 = clip(round(x / s) + z, 0, 255)
```

5. Dequantize: `x_approx = s × (x_int8 - z)`

The round-trip introduces quantization error: `x_approx   x`, not `x_approx = x`. The magnitude of this error depends on the precision of the scale factor and the distribution of values being quantized.

For INT8, values are mapped to the range [0, 255] (unsigned) or [-128, 127] (signed). For INT4, the range is [0, 15] or [-8, 7]. The smaller the integer range, the coarser the quantization, and the larger the potential error.

9.3.2 Symmetric vs. asymmetric quantization

Symmetric quantization uses a single scale factor with zero point at 0:

```
x_int = round(x / s)
```

Simple and fast. Works well when the distribution is symmetric around zero.

Asymmetric quantization uses both scale and zero point, as shown above. More accurate for asymmetric distributions (e.g., activations after ReLU, which are always non-negative).

9.4 The outlier problem

LLM quantization is harder than quantizing small networks for edge deployment. The reason: outlier values.

LLM activations (the values produced by the model's hidden layers, not the weights) contain a small fraction of channels with values that are orders of magnitude larger than typical. In a layer where most activation values are in the range [-1, 1], a few channels may have values in the range [-100, 100].

If you set the quantization scale based on these outlier channels, all the normal-range values are compressed into a tiny fraction of the integer range, losing most of their precision. If you clip the outlier channels, you introduce large errors for those specific values.

This outlier phenomenon is a consistent property of large language models, it appears in GPT-3, LLaMA, and their successors. It is not a training bug but a structural feature of how the models represent information.

The major INT8 and INT4 quantization schemes are largely distinguished by how they handle outliers.

9.5 Post-training quantization methods

Post-training quantization (PTQ) quantizes a trained model without additional training. It uses a small calibration dataset to measure activation statistics and find optimal scale factors.

9.5.1 LLM.int8()

Dettmers et al. (2022) introduced a mixed-precision decomposition specifically for LLM outliers:

1. Identify the small fraction of "outlier" feature dimensions (typically less than 1% of channels, but containing most of the activation magnitude)
2. Extract those channels and compute them in full BF16 precision
3. Quantize all remaining channels to INT8 and compute in INT8
4. Sum the two partial results

```
W × X = W_outlier × X_outlier  (BF16)
      + W_normal  × X_normal    (INT8)
```

This preserves accuracy in the outlier channels while achieving INT8 efficiency for the remaining 99%+ of computation. The memory savings are slightly less than pure INT8 (due to the BF16 outlier channels), but the quality preservation is close to BF16.

LLM.int8() enabled serving 175B+ parameter models on a single GPU for the first time. It is integrated into HuggingFace Transformers as `load_in_8bit=True`.

9.5.2 GPTQ

Frantar et al. (2022) introduced GPTQ, a one-shot weight quantization method for INT4 quantization:

1. Process the weight matrix layer by layer
2. For each column (output neuron), quantize the weights to INT4
3. Compute the quantization error for this column
4. Propagate the error to unquantized columns using the inverse Hessian of the weight matrix, updating them to compensate for the quantized column's error
5. Repeat for each column in sequence

The Hessian-based error compensation is the key innovation. Instead of quantizing each weight independently, GPTQ adjusts remaining weights to compensate for already-quantized weights, minimizing cumulative error.

GPTQ typically achieves INT4 quantization with perplexity degradation of less than 0.5 on standard benchmarks for models above 30B parameters. Below 30B, degradation is more noticeable. Larger models quantize more gracefully, a well-known empirical observation without a fully satisfying theoretical explanation.

GPTQ requires a calibration dataset (typically 128–512 samples from the training data) and takes several hours to run for large models. The result is stored as INT4 weights with FP16/BF16 scale factors.

9.5.3 AWQ

Lin et al. (2023) introduced AWQ (Activation-aware Weight Quantization) with a different approach to the outlier problem: instead of handling outlier activations differently at inference time, AWQ identifies the weight channels that correspond to the most important activations and protects them during quantization.

The insight: not all weights are equally important. Weights that multiply high-activation channels contribute more to the output than weights that multiply low-activation channels. Quantizing the important weights with less error and the unimportant weights with more error is better than treating all weights equally.

AWQ achieves this through per-channel scaling: multiply important weight channels by a scale factor > 1 before quantization (expanding their range relative to the quantization step size, reducing quantization error) and divide the corresponding activations by the same factor (maintaining the mathematical equivalence of the operation):

```
W × X = (W × s) × (X / s)   for any scale s
```

If `s > 1` for the important channels, quantizing `W × s` to INT4 introduces less error in the important channels than quantizing `W` directly. The activation scaling by `1/s` is a free operation at inference time (absorbed into the previous layer's output scaling).

AWQ is faster to apply than GPTQ (minutes rather than hours), does not require Hessian computation, and achieves comparable or slightly better accuracy on most benchmarks. It has become the preferred method for INT4 quantization in many production pipelines.

9.6 GGUF and llama.cpp quantization

GGUF (GPT-Generated Unified Format) is a model format developed by the llama.cpp project, optimized for inference on consumer hardware, CPUs, Apple Silicon, and consumer GPUs. It supports a range of quantization schemes with different quality-size tradeoffs:

Quantization	Bits per weight	Memory (70B)	Quality loss
Q2_K	~2.6	~23 GB	Significant
Q3_K_M	~3.4	~30 GB	Moderate
Q4_K_M	~4.8	~43 GB	Small
Q5_K_M	~5.7	~50 GB	Very small
Q6_K	~6.6	~58 GB	Minimal
Q8_0	~8.5	~75 GB	Near-lossless

The naming convention: Q4 = 4-bit quantization base, K = k-quants (a more accurate block quantization scheme), M = medium size variant.

GGUF uses mixed-precision block quantization: weights are divided into blocks (typically 32 values), each block is quantized with its own scale factor, and additional "super-block" scale factors quantize the block scales themselves. This reduces the overhead of storing per-block scale factors while maintaining accuracy.

GGUF is the dominant format for local inference with Ollama, LM Studio, and llama.cpp. It is not typically used in high-throughput serving (vLLM and TensorRT-LLM use their own formats), but it is how most self-hosted models run on consumer hardware.

9.7 Quantization-aware training

Post-training quantization applies quantization after the fact, introducing error that cannot be corrected. Quantization-aware training (QAT) incorporates the quantization into the training process, allowing the model to adapt to quantization during training.

9.7.1 Simulated quantization

QAT uses simulated quantization: during the forward pass, weights and activations are quantized to the target format; during the backward pass, gradients flow through as if the quantization had not occurred (the straight- through estimator). The model learns weight values that produce low loss even after quantization.

```
def quantize_forward(w, bits=8):
    # Quantize weights for forward pass
    scale = w.abs().max() / (2**(bits-1) - 1)
    w_int = torch.round(w / scale).clamp(-(2**(bits-1)), 2**(bits-1) - 1)
    w_quantized = w_int * scale  # Dequantize for computation
    # Straight-through: gradient passes through as if w_quantized = w
    return w + (w_quantized - w).detach()
```

The result: weights that, when quantized, produce minimal performance loss. QAT typically achieves better quality than PTQ at the same bit width, at the cost of requiring a full training run rather than a single post-processing step.

9.7.2 QLoRA

QLoRA (Dettmers et al., 2023) combines QAT with parameter-efficient fine-tuning:

1. Load the base model quantized to INT4 using NF4 (Normal Float 4, a 4-bit floating point format designed for normally distributed weights)
2. Add trainable LoRA adapters in BF16
3. Fine-tune only the LoRA adapters, with the quantized base model frozen
4. Store gradients and optimizer states only for the adapter parameters

QLoRA enables fine-tuning a 70B model on a single 48 GB GPU, something that would require multiple A100s with full-precision fine-tuning. It has become the standard approach for fine-tuning large models on limited hardware.

9.7.3 NF4: the normal float format

NF4 is a data type designed specifically for neural network weights. Unlike INT4 which maps values uniformly, NF4 maps values so that each quantization bin contains an equal fraction of weights from a normal distribution. Since pretrained model weights follow approximately normal distributions, NF4 minimizes quantization error for typical weight distributions:

```
INT4 bins (uniform):     [-8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7]
NF4 bins (non-uniform): denser near zero, sparser at extremes
                         matches normal distribution of pretrained weights
```

NF4 typically outperforms INT4 for weight quantization at 4-bit precision because it allocates more precision where the weights actually are (clustered near zero) rather than allocating precision uniformly across the possible range.

9.8 Activation quantization

Weight quantization stores weights at reduced precision but typically performs the actual matrix multiplications in BF16 or FP16 (dequantizing first). Activation quantization goes further: both weights and activations are in reduced precision during computation, allowing the use of integer or FP8 arithmetic units that are faster than BF16 arithmetic.

9.8.1 SmoothQuant

Xiao et al. (2023) introduced SmoothQuant to address the outlier problem for activation quantization. The key insight: quantization difficulty can be migrated between activations and weights.

If activations have outlier channels (hard to quantize), and weights do not (easy to quantize), we can:

1. Scale down the outlier activation channels by a factor `s_j`
2. Scale up the corresponding weight rows by the same factor `s_j`
3. The matrix product is unchanged: `(X / s) × (W × s) = X × W`
4. The scaled activations have a more uniform distribution which is easy to quantize
5. The scaled weights have larger values in specific rows and still quantizable

The scale factors are computed from activation statistics gathered on a calibration dataset. The weight scaling is applied offline (absorbed into the stored weights) and the activation scaling is applied as a single per-channel multiply at inference time.

SmoothQuant enables W8A8 (8-bit weights and activations) quantization with accuracy close to BF16, at roughly 1.5× the throughput of BF16 on hardware with INT8 tensor cores.

9.8.2 FP8 inference

FP8 (E4M3 and E5M2) is increasingly the preferred quantization format for high-throughput serving on H100-class hardware. Unlike INT8, FP8 is a floating point format with dynamic range, making it more robust to outliers without the outlier-handling complexity of LLM.int8() or SmoothQuant.

The H100's Transformer Engine supports FP8 matrix multiplications with automatic dynamic scaling: it monitors activation ranges during inference and adjusts the FP8 scale factors to minimize quantization error in real time. This makes FP8 practically zero-configuration compared to INT8 calibration.

FP8 inference on H100 achieves approximately 2× the throughput of BF16, with quality typically within 0.5% of BF16 on standard benchmarks.

9.9 Structured and unstructured sparsity

Quantization reduces the precision of each value. Sparsity takes a different approach: set some values to exactly zero and skip their computation entirely.

9.9.1 Unstructured sparsity

Prune individual weights based on magnitude or importance:

```
W = [0.23, -0.91, 0.04, 0.67, -0.02, 0.88, ...]
      keep   keep  zero  keep   zero   keep

50% sparse: approximately half the weights are zero
```

Unstructured sparsity requires sparse matrix arithmetic, computing only the multiplications where the weight is non-zero. In theory, 50% sparsity halves the computation. In practice, sparse arithmetic is harder to accelerate on GPU than dense arithmetic, and the speedup is often much less than 50%. Unstructured sparsity is difficult to exploit efficiently on current hardware.

9.9.2 2:4 structured sparsity (NVIDIA)

NVIDIA introduced 2:4 sparsity with the A100: in every group of 4 consecutive weights, exactly 2 must be zero. The non-zero pattern must be this specific structured form.

```
Dense: [w1, w2, w3, w4]
2:4:   [w1,  0, w3,  0]  or  [w1, w2,  0,  0]  or  [0, w2, w3,  0]  ...
```

This specific structure allows the hardware to represent the sparse matrix compactly (store only the 2 non-zero values plus a 2-bit index indicating which positions they occupy) and compute sparse matrix multiplications at roughly 2× the throughput of dense computation with nearly no quality loss.

Blackwell-generation GPUs (B100, B200) extend this to 2:4 sparsity combined with FP4 quantization, potentially achieving 4× throughput compared to BF16 dense computation.

9.10 Combining quantization techniques

Production deployments typically combine multiple techniques:

Weight-only INT4 + BF16 compute (most common for serving): - Store weights as INT4 with per-group scales - Dequantize to BF16 on-the-fly before matrix multiplication - Compute in BF16 using tensor cores - Memory savings: 4× over BF16; throughput gain: primarily from reduced memory bandwidth for weight loading

FP8 weights + FP8 activations (H100 production): - Both weights and activations in FP8 - Compute using FP8 tensor cores (~2× throughput of BF16) - Dynamic scaling via Transformer Engine - Near-BF16 quality for most tasks

INT4 weights + 2:4 sparsity (Blackwell): - Weights quantized to INT4 and pruned to 2:4 sparsity - Compute using sparse FP4 tensor cores - Potential 4-8× throughput improvement over dense BF16 - Quality impact requires careful calibration

QLoRA for fine-tuning: - Base model in NF4 - Adapters in BF16 - Enables fine-tuning models 4× larger than would otherwise fit - Merge adapters back to BF16 for serving, or serve with adapters

---

9.11 Quality evaluation for quantized models

Quantization quality is measured on the same metrics as full-precision models, but the comparison requires care.

Perplexity on a held-out dataset is the most reliable automatic metric for quantization quality. A perplexity increase of less than 0.5 is typically imperceptible in generated text while increases above 1.0 are noticeable.

Downstream task benchmarks (MMLU, HellaSwag, HumanEval) catch task- specific degradation that perplexity may miss. Some tasks are more sensitive to quantization than others, math and code tend to degrade before general language.

Outlier-sensitive tasks: question answering tasks that require precise recall of specific facts tend to be more sensitive to quantization than open-ended generation. This is consistent with the outlier hypothesis, outlier channels may be important for retrieving specific facts.

Human evaluation: for the final quality gate before production, human preference evaluation (does this response feel worse?) is the most reliable signal. Perplexity improvements from aggressive quantization do not always translate to imperceptible quality in practice.

9.11.1 The quantization degradation curve

Quantization quality degrades non-linearly with bit width and model size:

```
                      Quality (relative to BF16)

100%                        BF16
 99%                    FP8
 98%                  INT8
 96%                INT4 (large models, e.g., 70B)
 90%            INT4 (medium models, e.g., 13B)
 80%      INT4 (small models, e.g., 7B)
 60%   INT3 (typically unacceptable)
                           → bit width
```

Large models are more robust to quantization than small models. A 70B model at INT4 often outperforms a 7B model at BF16 while using similar memory. This is the primary argument for quantizing large models rather than serving small ones: you may get better quality at lower cost.

9.12 Key takeaways

- Quantization reduces parameter precision from BF16 (2 bytes) to INT8 (1 byte) or INT4 (0.5 bytes), directly halving or quartering memory requirements — the difference between serving a 70B model on 2 GPUs vs. 1 GPU vs. a consumer card
- BF16 is the standard training format (matches FP32 range, handles gradient magnitudes that overflow FP16); FP8 on H100 is the emerging standard for inference (2× throughput with near-BF16 quality)
- The outlier problem — a small fraction of activation channels with very large values — is the core challenge of LLM quantization; LLM.int8(), AWQ, and SmoothQuant all address it differently
- GPTQ uses Hessian-based error compensation to minimize quantization error layer by layer; AWQ uses activation-aware weight scaling; both achieve near- lossless INT4 quantization for large models
- GGUF/llama.cpp quantization enables local inference on consumer hardware; Q4_K_M is the practical default for local deployment
- QLoRA combines INT4 base model quantization with BF16 LoRA adapters, enabling fine-tuning of 70B models on a single GPU — the standard approach for practitioners without datacenter hardware
- Larger models quantize more gracefully than smaller ones; a quantized 70B often outperforms a full-precision 13B while using similar memory
- 2:4 structured sparsity enables ~2× throughput on A100/H100 without quality loss; combined with FP4 on Blackwell GPUs, this is the next frontier of inference efficiency

---

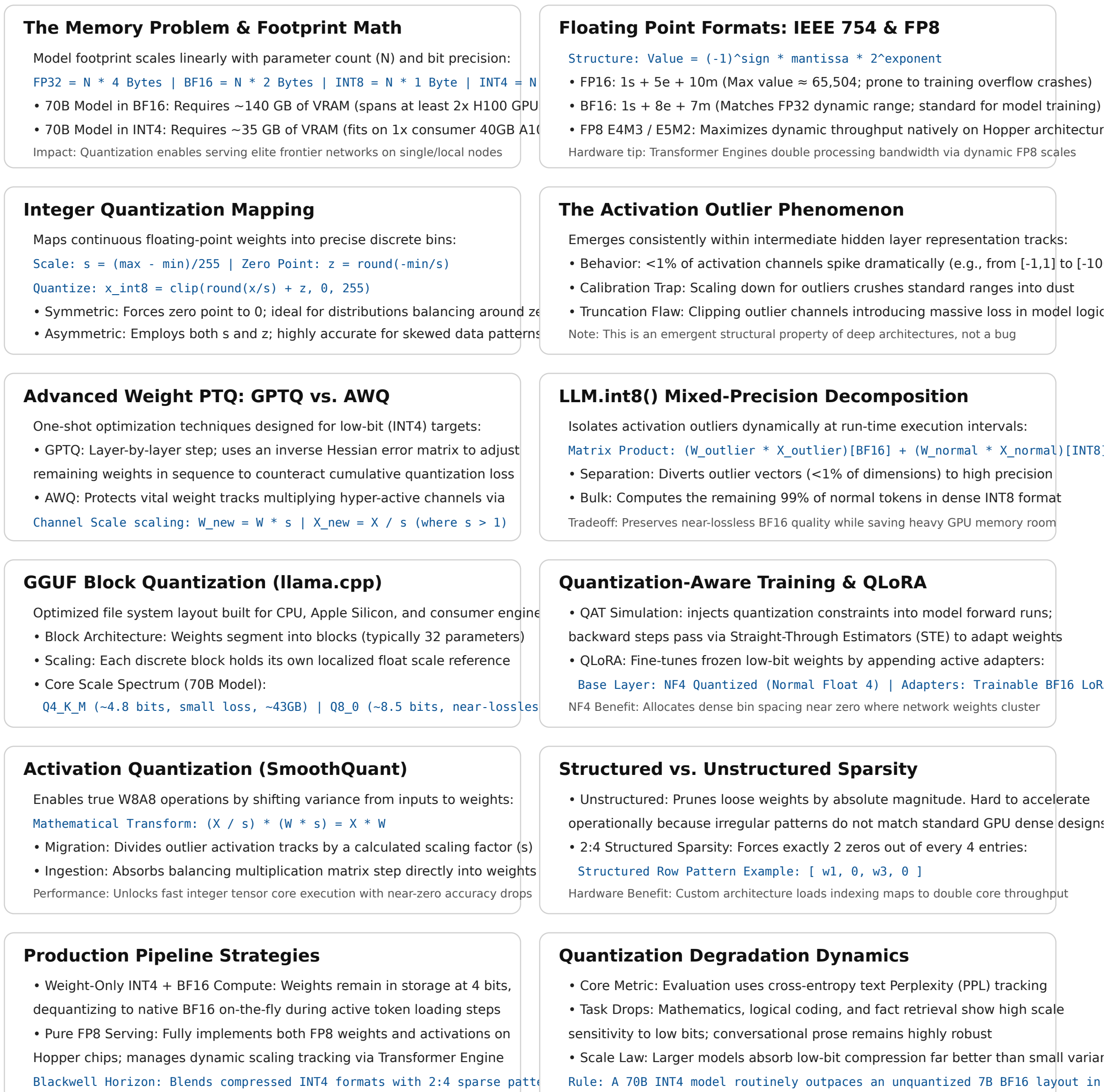


Figure 9.2: Cheat sheet.

9.13 Further reading

- Dettmers et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.* — The foundational paper on LLM quantization; identified the outlier problem and introduced mixed-precision decomposition.
- Frantar et al. (2022). *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.* — Hessian-based INT4 quantization; the first practical 4-bit quantization for models above 30B parameters.
- Lin et al. (2023). *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration.* — Faster and often more accurate than GPTQ; now the preferred INT4 method in many production systems.
- Xiao et al. (2023). *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models.* — Migrating quantization difficulty from activations to weights for W8A8 quantization.
- Dettmers et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs.* — NF4 quantization combined with LoRA for fine-tuning on consumer hardware; introduced the NF4 data type and double quantization.
- Mishra et al. (2021). *Accelerating Sparse Deep Neural Networks.* — NVIDIA's 2:4 structured sparsity and the hardware support for it in A100 and later.
- NVIDIA (2024). *FP8 Formats for Deep Learning.* — Technical specification of E4M3 and E5M2 formats and the H100 Transformer Engine's dynamic scaling.

Chapter 10

Decoding and Sampling

The canonical question for this chapter: *The model produces a probability distribution over 100,000 tokens. How do you turn that distribution into the actual text the user sees, and how does that choice shape everything from creativity to factuality to cost?*

Where are we?

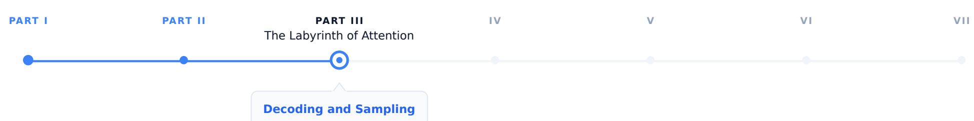


Figure 10.1: The journey through the Model Mind.

The transformer forward pass is complete. The model has produced a vector of logits one score per vocabulary token. This chapter covers what happens next: how those logits become a token, and how that single choice, made thousands of times, produces the text the user reads. The decoding strategy is the last step before output also one of the most consequential.

10.1 The decoding problem

After the transformer forward pass, the output is a vector of logits one score per token in the vocabulary. Applying softmax converts these to a probability distribution:

```
logits:        [2.3,  -1.2,  0.8,  4.1,  -0.3,  ...]   shape: [vocab_size]
probabilities: [0.04,  0.001, 0.01, 0.33,  0.004, ...]   shape: [vocab_size]
```

The highest-probability token is not always the right choice. Selecting it every time known as greedy decoding produces text that is repetitive, overconfident, and often worse than sampling from the distribution. Nevertheless sampling randomly from the full distribution produces incoherent text that assigns probability to every token, including terrible ones.

Decoding is the set of strategies for navigating this tradeoff: selecting a token (or sequence of tokens) from the distribution in a way that produces output that is coherent, diverse, and appropriate for the task. It runs once per generated token, which for a 500 token response means it runs 500 times. Small differences in strategy compound into large differences in output.

10.2 Greedy decoding

The simplest strategy: always select the token with the highest probability.

```python
def greedy_decode(logits):
    return logits.argmax(dim=-1)
```

Greedy decoding is deterministic the same prompt always produces the same output. It is fast (no sampling required) and produces coherent local choices at each step.

The problem: local optimality does not imply the global optimality. The highest- probability token at step `t` may lead to a sequence that is less probable overall than a sequence that takes a lower probability token at step `t` and recovers. As a result, greedy decoding can become trapped in a local optimum.

The most visible symptom is repetition. Once a model enters a repetitive pattern, greedy decoding has no escape mechanism, the most likely next token at each step continues the pattern. The model knows, in some sense, that it is repeating itself (it assigns lower probability to the repetition than to alternatives), but greedy decoding ignores that signal and repeats anyway.

Greedy decoding is appropriate for tasks with a single correct answer (extracting specific information, classification, structured output generation where diversity is undesirable) and for debugging, where the deterministic output makes it easier to isolate model behavior from sampling noise.

10.3 Beam search

Beam search maintains a set of `k` candidate sequences and extends all of them at each step, keeping only the `k` most probable overall:

```
Beam width k=2, step 1:
  Candidates: ["The cat", "A dog"]
  Scores:     [log P("The") + log P("cat"|"The"),
               log P("A") + log P("dog"|"A")]

Step 2: extend each candidate:
  "The cat sat", "The cat ran", "The cat is", ...   (vocab_size options)
  "A dog ran",   "A dog sat",   "A dog is", ...     (vocab_size options)

Keep top k=2 by cumulative log probability:
  ["The cat sat", "A dog ran"]
```

At the end of generation, the candidate with the highest cumulative log probability is returned. Beam search finds sequences with higher overall probability than greedy decoding because it can recover from locally suboptimal choices.

10.3.1 The beam search curse

Beam search dominated sequence-to-sequence tasks (translation, summarization) from roughly 2015 to 2020. For tasks with constrained output spaces, it performs well. For open-ended generation, it has a well-documented failure mode: it produces output that is generic and boring.

High-probability sequences tend to be safe, common phrases. The beam converges to the statistical mean of training data rather than producing specific, vivid text. This is the beam search curse: the sequences with highest probability under the model are not the sequences humans prefer. Human language is not the highest- probability sequence, people regularly choose moderately probable, interesting words over the most probable, expected ones.

Beam search is still used for translation (k=4 is typical), structured data extraction, and code generation with hard constraints. For conversational and creative generation, sampling-based methods have largely replaced it.

10.4 Temperature sampling

Temperature is one of the most important hyperparameters in decoding. It adjusts the probability distribution over the next token before sampling, making the distribution either sharper or flatter.

10.4.1 The temperature transformation

Before sampling, the logits are divided by a temperature parameter (T):

```
adjusted_probs = softmax(logits / T)
```

T < 1.0 (cold): the distribution become sharper and high-probability tokens receive relatively more probability mass, while low-probability tokens receive less. At $T \to 0$, the distribution approaches a point mass on the highest-probability token. Consequently, stochastic sampling becomes deterministic, recovering the behavior of greedy decoding.

T = 1.0: the distribution is unchanged from the model's output.

T > 1.0 (hot): the distribution becomes flatter and probabilities are spread more evenly across tokens. At $T \to \infty$, the distribution becomes uniform.

```
Logits:           [4.0,  2.0,  1.0,  0.5]

T=0.5 (cold):     softmax([8.0, 4.0, 2.0, 1.0]) → [0.86, 0.12, 0.02, 0.01]
T=1.0 (normal):   softmax([4.0, 2.0, 1.0, 0.5]) → [0.68, 0.17, 0.08, 0.07]
T=2.0 (hot):      softmax([2.0, 1.0, 0.5, 0.25])→ [0.45, 0.27, 0.16, 0.12]
```

10.4.2 Choosing temperature

T = 0 (greedy): factual retrieval, classification, structured output. Maximizes accuracy for tasks with correct answers.

T = 0.1–0.4: code generation, factual Q&A, tasks where accuracy matters but some flexibility is useful. Concentrated distribution that rarely samples improbable tokens.

T = 0.7–1.0: conversational responses, general-purpose assistants. Balanced diversity and coherence. Most production systems operate in this range.

T = 1.0–1.4: creative writing, brainstorming, poetry. Higher diversity, more surprising choices. Coherence may occasionally suffer.

T > 1.5: experimental. Useful for exploring the model's probability space, not for production output.

Temperature is not a global constant and it should vary by task. A system that writes creative fiction and also answers factual questions should use different temperatures for each, ideally set automatically based on query classification.

10.5 Top-k sampling

Temperature sampling from the full distribution can still sample very unlikely tokens. If there are 100,000 tokens in the vocabulary, even after temperature adjustment there may be thousands of tokens with non-negligible probability, including tokens that produce incoherent output.

Top-k sampling restricts sampling to the k most probable tokens:

```
def top_k_sample(logits, k, temperature=1.0):
    logits = logits / temperature
    top_k_values, top_k_indices = torch.topk(logits, k)
    filtered_logits = torch.full_like(logits, float('-inf'))
    filtered_logits.scatter_(0, top_k_indices, top_k_values)
    probs = torch.softmax(filtered_logits, dim=-1)
    return torch.multinomial(probs, num_samples=1)
```

The remaining probability mass is renormalized across the top-k tokens before sampling. Tokens outside the top-k are masked with `-inf`, giving them zero probability after softmax.

Top-k has a structural limitation: `k` is fixed regardless of the distribution shape. When the model is very confident (one token at probability 0.95, the rest near zero) top-k=50 forces sampling from 50 tokens when 49 of them are clearly wrong choices. When the model is genuinely uncertain (500 tokens each at probability ~0.002) top-k=50 discards 450 plausible options. A fixed `k` handles neither scenario well. This motivates nucleus sampling.

10.6 Nucleus (top-p) sampling

Nucleus sampling adapts the number of candidates to the distribution shape. Instead of a fixed count `k`, it takes the smallest set of tokens whose cumulative probability exceeds `p`:

```
def nucleus_sample(logits, p, temperature=1.0):
    logits = logits / temperature
    probs = torch.softmax(logits, dim=-1)
    sorted_probs, sorted_indices = torch.sort(probs, descending=True)
    cumulative_probs = torch.cumsum(sorted_probs, dim=-1)
    sorted_indices_to_remove = cumulative_probs - sorted_probs > p
    sorted_probs[sorted_indices_to_remove] = 0
    sorted_probs /= sorted_probs.sum()
    sampled_index = torch.multinomial(sorted_probs, num_samples=1)
    return sorted_indices[sampled_index]
```

With `p = 0.9`: include the fewest tokens whose probabilities sum to at least 0.9. Renormalize and sample from those tokens only.

When the model is confident (one token at 0.95 probability), the nucleus contains only that token which is effectively greedy decoding. When the model is uncertain (probability spread widely), the nucleus includes many tokens and sampling is diverse. The nucleus size adapts to the situation rather than being fixed in advance which makes it ideal for production.

10.6.1 Choosing p

p = 0.9–0.95: the most common range. Excludes the long tail of implausible tokens while preserving diversity for plausible choices.

p = 1.0: sample from the full distribution there aren't any nucleus restriction. Equivalent to pure temperature sampling.

p = 0.5: very concentrated. Appropriate for high-accuracy tasks but risks becoming too deterministic.

Top-p and temperature are typically used together. Temperature shapes the distribution; top-p determines the cutoff. A common production configuration: `temperature=0.7, top_p=0.9`.

10.7 Min-p sampling

A more recent alternative to top-p that addresses a subtle failure mode.

Top-p's cutoff is cumulative: if the nucleus already covers 90% of probability mass, tokens beyond that are excluded regardless of their absolute probability. In practice, this can include tokens with very low absolute probability if they accumulate to push the total past `p`, especially when the distribution is flat.

Min-p sets a minimum absolute probability threshold instead: only sample from tokens whose probability exceeds `min_p × max_token_probability`.

```
def min_p_sample(logits, min_p, temperature=1.0):
    probs = torch.softmax(logits / temperature, dim=-1)
    max_prob = probs.max()
    threshold = min_p * max_prob
    filtered_probs = probs.clone()
    filtered_probs[probs < threshold] = 0
    filtered_probs /= filtered_probs.sum()
    return torch.multinomial(filtered_probs, num_samples=1)
```

With `min_p = 0.05`: if the most likely token has probability 0.4, only tokens with probability ≥ 0.02 are considered. If the most likely token has probability 0.9, the threshold rises to 0.045 automatically restricting sampling when the model is confident.

Min-p adapts to distribution shape like top-p but uses the maximum token probability as the reference rather than the cumulative sum. Empirically, it produces more coherent output than top-p for creative tasks while maintaining diversity.

10.8 Repetition penalties

A common failure mode of sampling is that the model becomes trapped in repetitive loops. This issue is not solely caused by the decoding algorithm, it often reflects the model operating in a low-quality mode or producing a poorly calibrated probability distribution. Nevertheless, appropriate decoding strategies can help mitigate this problem.

10.8.1 Frequency penalty

Reduces the logit of any token proportional to how many times it has appeared in the generated output so far:

```
adjusted_logit[t] = logit[t] - frequency_penalty × count(t, generated_so_far)
```

A token that has appeared 5 times with `frequency_penalty = 0.5` has its logit reduced by 2.5. Frequently used tokens become progressively less likely.

10.8.2 Presence penalty

Reduces the logit of any token that has appeared at all, by a flat amount:

```
adjusted_logit[t] = logit[t] - presence_penalty × (1 if t in generated_so_far else 0)
```

Unlike frequency penalty, presence penalty does not scale with count, it applies a flat reduction on first appearance and no additional reduction after that. It discourages reusing any token at all, not repeatedly using a token many times.

10.8.3 Repetition penalty (multiplicative)

Used in HuggingFace and most open-source implementations:

```
adjusted_logit[t] = logit[t] / repetition_penalty    if logit[t] > 0
adjusted_logit[t] = logit[t] × repetition_penalty    if logit[t] < 0
```

For `repetition_penalty > 1.0`, this divides positive logits and multiplies negative logits, making previously generated tokens less likely regardless of their sign.

10.8.4 Practical considerations

Repetition penalties must be carefully calibrated. If the penalty is too strong, the model may avoid repeating necessary words, including common function words such as the, is, and and, which naturally occur multiple times in fluent text. Conversely, if the penalty is too weak, repetitive outputs may persist.

Most production systems apply penalties only to the generated portion, not the prompt, you do not want tokens appearing in the prompt to be penalized in the response. Context window position also matters: penalizing tokens from many turns ago that have no relationship to the current generation is rarely useful.

10.9 Structured decoding and constrained generation

For many production use cases, the output must conform to a specific format (a valid JSON, a Python function, a response matching, a defined schema). Unconstrained sampling may produce syntactically invalid output, requiring expensive retry loops. Structured decoding guarantees valid output by constraining the logits at each step.

10.9.1 How constrained generation works

At each decoding step, a parser tracks the current state of the partially generated output and computes the set of tokens that are valid at this position, tokens that would not cause the output to become unparseable. All invalid tokens are masked out (their logits set to `-inf`) before sampling.

```
Target format: JSON object with "name" (string) and "age" (integer)

After generating: {"name": "
  Valid next tokens:   any character that could appear in a JSON string
  Invalid next tokens: }, ], :, numbers, structural tokens

After generating: {"name": "Alice", "age":
  Valid next tokens:   digits (0-9), negative sign
  Invalid next tokens: letters, quotes, braces
```

The parser can be regex-based (valid tokens match the regex at the current position), grammar-based (tokens permitted by a context-free grammar at the current parse state), or schema-based (tokens consistent with a JSON schema or Pydantic model).

10.9.2 Implementations

Outlines (Willard & Louf, 2023): builds a finite state machine from the schema or regex at compile time. At inference, the FSM advances after each token and the valid next-token set is read from the FSM in O(1). The precomputed FSM makes constraint checking negligible overhead.

Guidance (Microsoft): the earlier widely-used structured generation library. Intercepts the generation loop and applies constraints at each step with a broader feature set including branching and conditional generation.

llama.cpp GBNF grammars: built-in grammar-based constrained generation using a BNF-like notation. Ships with llama.cpp without additional dependencies.

10.9.3 Overhead

Constrained generation adds overhead in logit masking (computing the valid token set and applying the mask which is O(1) per step with precomputed FSMs) and in reduced sampling diversity (smaller effective vocabulary). Latency overhead is typically 5–15% relative to unconstrained generation with FSM-based approaches. The alternative (parsing failures and retry loops) is far more expensive.

10.10 Best-of-n sampling

Generate `n` independent completions for the same prompt and select the best one according to some criterion:

```
Generate n=8 completions for the same prompt
Score each:
  - Reward model score (human preference prediction)
  - Verifier (run the code, check the math)
  - Self-consistency (most common answer across completions)
  - Model perplexity (model's own confidence in its output)
Return the highest-scoring completion
```

Best-of-n is a simple yet effective strategy for improving output quality by allocating additional computation during inference. The model generates n candidate outputs and selects the best one according to a scoring criterion. Performance typically scales log-linearly with n: increasing the number of samples from 1 to 4 often yields substantial improvements, whereas increasing it from 8 to 16 provides smaller, diminishing gains.

Verifiable tasks are where best-of-n works best. For math problems, the answer can be verified for correctness. For code, it can be executed and tested. Generate `n` solutions, keep those that pass verification, return the best or majority-vote answer.

Reward model reranking generates `n` responses and scores them with a reward model trained to predict human preference, effectively the RLHF reward signal applied at inference time rather than training time.

Self-consistency (Wang et al., 2023) generates `n` answers with chain-of- thought, returns the most common final answer. This improves accuracy on multi- step reasoning tasks significantly without requiring a separate verifier.

10.11 Speculative sampling

Autoregressive generation has an uncomfortable property: producing token 50 requires having already produced tokens 1 through 49, one at a time, each requiring a full forward pass through the model.

Speculative decoding is a way to avoid paying that tax token-by-token. A small, cheap **draft model** runs ahead and proposes a candidate sequence of `k` tokens. The target model then evaluates all `k` candidates in a *single* forward pass. The target model isn't generating here; it's grading a draft that's already been written.

Drafting versus grading raises the obvious question: how does the target model decide which of the draft's guesses to keep? The naive answer is a threshold: keep what the target model agrees with and discard the rest. This naive answer is wrong, and understanding *why* it's wrong is the key to understanding the whole algorithm.

10.11.1 The acceptance rule

For each draft token `x_t`, accept it with probability:

```
P(accept x_t) = min(1, p_target(x_t) / p_draft(x_t))
```

Two regimes:

- **The target model likes the token at least as much as the draft did** ($p_{target} \geq p_{draft}$): accept with probability 1.
- **The target model likes it less** ($p_{target} < p_{draft}$): accept only with probability p_{target}/p_{draft}. The draft model was, in effect, overconfident relative to the target, so the token survives only in proportion to how much the target actually endorses it.

10.11.2 What happens on rejection and why the obvious fix is wrong

Suppose a draft token is rejected. The intuitive move is to sample a replacement directly from `p_target` at that position after all, the target model already computed that full distribution during verification. This turns out to *distort* the final output, and a small numeric example makes the distortion concrete.

Take a two-token vocabulary, `{A, B}`, where:

```
p_target(A) = 0.5,  p_target(B) = 0.5
p_draft(A)  = 0.9,  p_draft(B)  = 0.1
```

The draft model samples from its own distribution, so it proposes A 90% of the time. When it does, the acceptance probability is `min(1, 0.5/0.9) = 0.556`. If we reject and then naively resample straight from `p_target`:

```
P(final = A) = P(draft=A)·P(accept) + P(draft=A)·P(reject)·P(naive resample = A)
             = 0.9 × 0.556 + 0.9 × 0.444 × 0.5
             = 0.5 + 0.2 = 0.7
```

The final probability of A comes out to **0.7**, even though the target model only wanted A 50% of the time. Token A is getting double-counted: it wins a first chance through acceptance, and then (because a naive resample doesn't know that) a second, independent chance through rejection. Any token the draft model over-favored relative to the target ends up over-represented in the output. This is precisely the kind of quality drift speculative decoding is supposed to avoid.

10.11.3 The residual distribution

The fix is to correct only into the probability mass that acceptance *hasn't already claimed*:

```
p_correction(x) = max(0, p_target(x) - p_draft(x)) / Z
```

This is **not** the target's raw distribution, it is the target's distribution with the draft's contribution subtracted out, clipped at zero, and renormalized by `Z`. Any token the draft over-proposed is zeroed out here, since it already had its fair shot via acceptance and cannot be picked a second time. Any token the draft under-proposed keeps some mass, since that's the only place where the target's true preference hasn't yet been accounted for.

Reworking the example: `p_correction(A) = max(0, 0.5 - 0.9) = 0`, so A can never be selected on correction. `p_correction(B) = max(0, 0.5 - 0.1) = 0.4`, and since it's the only token with nonzero mass, `Z = 0.4` and correction always yields B. Recomputing:

```
P(final = A) = 0.9 × 0.556 = 0.5                             matches p_target(A)
P(final = B) = 0.1 × 1 (direct accept) + 0.9 × 0.444 × 1 (correction) = 0.1 + 0.4 = 0.5   matches p_target(B
```

Both tokens land exactly on the target's true probabilities.

10.11.4 Walking through a full sequence

The acceptance/correction step happens *per position*, scanned left to right, and stops at the first rejection. Suppose the draft model proposes five tokens continuing "The cat sat on the":

```
Draft sequence:  x1="mat"   x2="and"   x3="looked"  x4="very"   x5="happy"
```

One target-model forward pass over the prompt plus all five draft tokens yields the target's distribution at every position simultaneously, thanks to causal masking position 1 conditioned only on the real prompt, position 2 on the real prompt plus `x1`, and so on.

```
Position 1: x1="mat"     p_draft=0.7, p_target=0.8  → accept (prob. 1)
Position 2: x2="and"     p_draft=0.5, p_target=0.5  → accept (prob. 1)
Position 3: x3="looked"  p_draft=0.6, p_target=0.2  → accept w.p. 0.33 → suppose REJECTED
```

Once position 3 is rejected, positions 4 and 5 are discarded without ever being checked `x4` and `x5` were drafted under the assumption that "looked" was the accepted prefix, and that assumption no longer holds. Position 3 is then resolved by drawing from the residual distribution at that position, which excludes "looked" (already rejected) and favors whatever the target under-weighted relative to the draft say this yields "leapt."

```
Accepted this round:   "mat", "and", "leapt"
Discarded:              "looked", "very", "happy"
```

Three tokens survive from five drafted, produced by a single target forward pass rather than three sequential ones. The next round starts fresh: the draft model proposes another `k` tokens continuing from "…sat on the mat and leapt," and the cycle repeats.

10.12 Chain-of-thought and reasoning tokens

A decoding-time technique that improves performance on reasoning tasks: before producing the final answer, the model generates intermediate reasoning steps.

```
Without chain-of-thought:
  Q: "If a train travels 120 miles in 2 hours, what is its speed?"
  A: "60 miles per hour."

With chain-of-thought:
  Q: "If a train travels 120 miles in 2 hours, what is its speed?"
  A: "To find speed, I divide distance by time.
      Speed = 120 miles / 2 hours = 60 miles per hour."
```

Chain-of-thought is a decoding strategy in that it requires generating more tokens before reaching the answer. The intermediate tokens are not just scaffolding, they change the probability distribution over the final answer. When the model writes out an intermediate calculation, that calculation is encoded into the KV cache. Subsequent tokens attend to it. The final answer is generated with the full reasoning chain as context, information that was not available at the first token.

This is in-context computation: the model uses its own generated text as working memory, offloading complex reasoning to the sequence rather than encoding it entirely within the forward pass. The context window is not just for input but also it is the model's scratch pad.

10.13 Decoding strategy selection guide

A practical reference for choosing decoding parameters by task:

Task	Strategy	Temperature	Top-p	Notes
Factual Q&A	Greedy or low-T	0–0.3	1.0	Consistency over diversity
Code generation	Low temperature	0.2–0.4	0.95	Correctness matters; some diversity for alternatives

Task	Strategy	Temperature	Top-p	Notes
JSON / structured output	Constrained	0.0–0.3	—	Use constrained decoding; redundant with high-T
Translation	Beam or low-T	0.1–0.3	—	Beam k=4 or low-T sampling
Summarization	Low-medium	0.3–0.6	0.9	Some diversity; avoid repetition penalties for common words
Conversation	Medium	0.7–0.9	0.9	Naturalness and variety
Creative writing	High	0.9–1.2	0.95	Maximize diversity; consider min-p
Reasoning / math	Very low or CoT	0.0–0.1	—	Greedy for extraction; chain-of-thought for reasoning
Brainstorming	High	1.0–1.4	0.95	Divergent thinking; coherence secondary

---

10.14 Key takeaways

- The model outputs a probability distribution over the vocabulary at each step; decoding selects a token from that distribution — this choice, made thousands of times, determines the character of the output
- Greedy decoding (always pick the most probable token) is deterministic and accurate for factual tasks but produces repetitive, generic output for open- ended generation because local optimality does not imply global optimality
- Beam search maintains `k` candidate sequences and returns the highest cumulative probability sequence; better than greedy for constrained tasks but suffers the beam search curse for open-ended generation — highest-probability most interesting
- Temperature scales logits before softmax: below 1.0 sharpens the distribution (more deterministic), above 1.0 flattens it (more random); vary temperature by task rather than using a single global value
- Top-p (nucleus sampling) adaptively samples from the smallest token set covering probability mass `p` — the nucleus shrinks when the model is confident, expands when uncertain; it is the dominant production sampling strategy
- Min-p uses the maximum token probability as a reference threshold rather than cumulative mass, producing more coherent output in cases where top-p includes low-probability tokens
- Structured decoding masks invalid tokens at each step using FSMs or parsers, guaranteeing format correctness with ~5–15% overhead — far cheaper than retry loops
- Best-of-n generates multiple completions and selects the best; most effective for verifiable tasks (math, code) where correctness can be checked programmatically
- Chain-of-thought uses the KV cache as working memory — intermediate reasoning tokens change the probability distribution over the final answer; reasoning models extend this to thousands of deliberation tokens
- Higher temperature increases hallucination rate; factual and structured tasks should use low temperature or constrained decoding to respect the model's already-correct probability signal

---

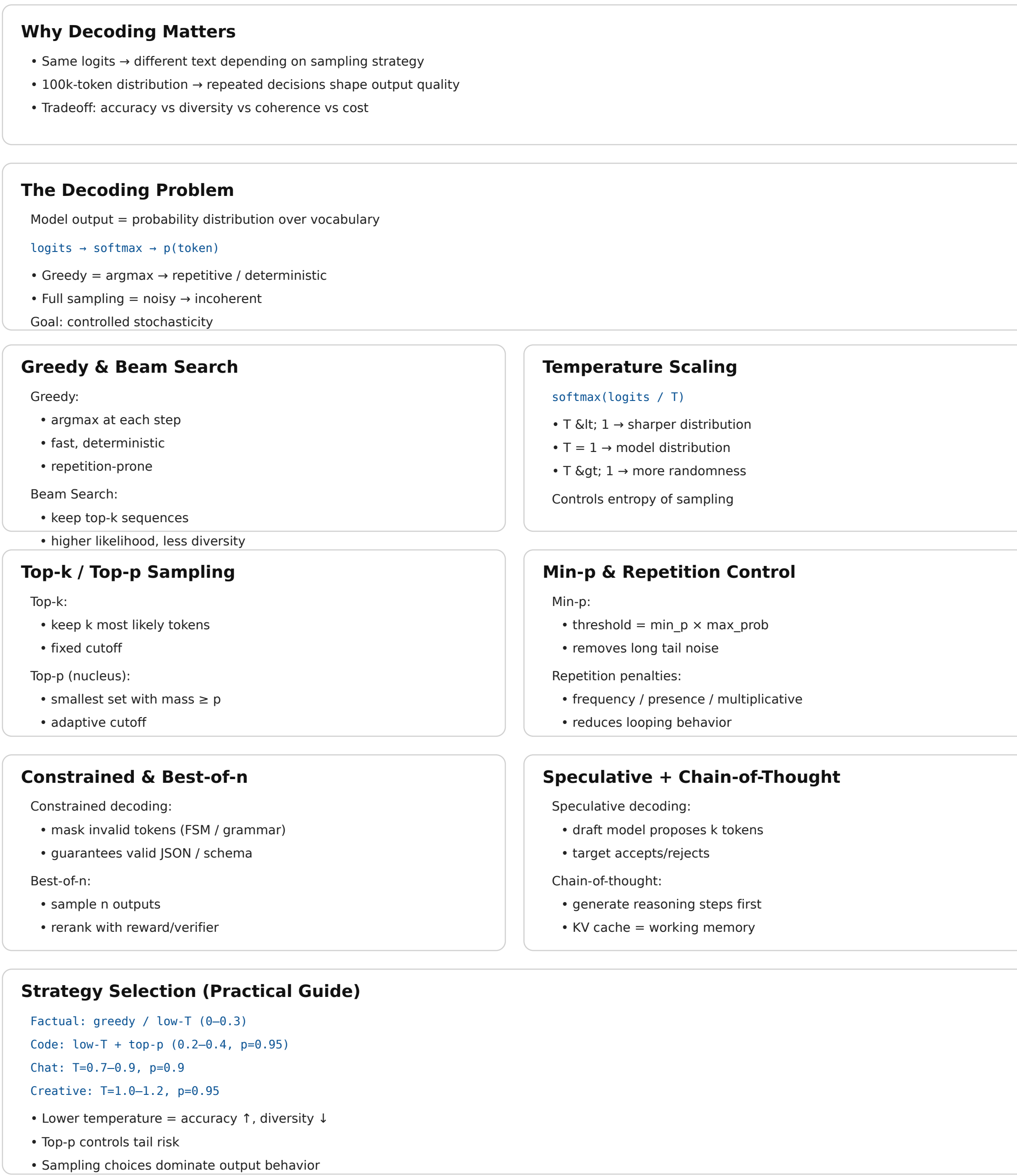


Figure 10.2: Cheat sheet.

10.15 Further reading

- Holtzman et al. (2020). *The Curious Case of Neural Text Degeneration.* — Established why greedy and beam search fail for open-ended generation and introduced nucleus sampling. The analysis of probability maximization failure modes is essential.
- Fan et al. (2018). *Hierarchical Neural Story Generation.* — Introduced top-k sampling for neural text generation; the original motivation and evaluation.
- Leviathan et al. (2023). *Fast Inference from Transformers via Speculative Decoding.* — Speculative sampling with exact distribution preservation; the proof that the correction step maintains the target distribution is the key contribution.
- Wei et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* — Established chain-of-thought as a decoding-time technique.
- Willard & Louf (2023). *Efficient Guided Generation for Large Language Models.* — The Outlines paper; FSM-based constrained decoding with O(1) per-step overhead.

Part IV

The Forbidden Archive

Chapter 11

Parametric Memory vs External Memory

The canonical question for this chapter: *A language model knows things. A retrieval system finds things. These are different cognitive acts and understanding exactly how they differ is what lets you design systems that use each one where it works best.*

Where are we?

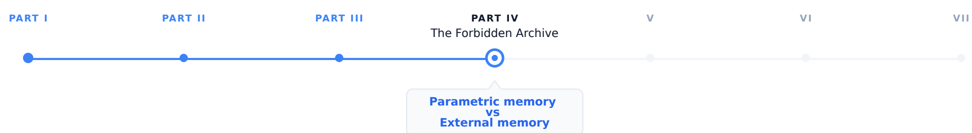


Figure 11.1: The journey through the Model Mind.

Retrieval exists because parametric memory has fundamental limits. This chapter explores those limits in greater depth by examining the nature of parametric memory, the role of external memory, how the two interact, and how to design systems that leverage both effectively.

11.1 Two different ways of knowing

Consider two employees at a company. The first has been there for twenty years. He knows the company's history, culture, and procedures intuitively. When someone asks a question, he answers from internalized understanding, he does not need to look anything up. Her knowledge is fast, fluid, and deeply integrated. It is also frozen: she knows things as they were when she last learned them, and it takes effort to update.

The second is a researcher who joined last week. She does not know the company's history, but she has access to every document the company has ever produced. When someone asks a question, she searches the archives, finds the relevant documents, and reads the answer. Her knowledge is current, precise, and verifiable but it depends entirely on what is in the archives and whether she can find it.

A language model with retrieval is both employees working together.

The first employee is **parametric memory** (knowledge encoded in the model's weights through training). The second is **external memory** (knowledge stored in a document corpus, retrieved at inference time). Neither is sufficient alone. Together, they cover each other's weaknesses.

11.2 What parametric memory is

Parametric memory is the knowledge encoded in a model's parameters basically its weights. It is everything the model learned by predicting the next token across a training corpus of trillions of words.

This knowledge is not stored as facts in labeled locations. It is distributed across billions of floating-point numbers, each contributing fractionally to the model's behavior. When the model "knows" that Paris is the capital of France, that knowledge is not a database entry, it is a statistical pattern spread across the embedding table, the attention heads, and the feed-forward layers of every transformer block.

11.2.1 Properties of parametric memory

Always available. Parametric memory requires no lookup. Every piece of knowledge the model has is accessible on every forward pass, with no retrieval latency.

Deeply integrated. Parametric knowledge is woven into the model's reasoning. When the model answers a question about French geography, it can simultaneously apply its knowledge of European history, political structures, and linguistic patterns without explicit retrieval of any of them. The integration is seamless because all of it lives in the same substrate.

Compressed and approximate. The training corpus contains far more information than fits in the parameter count. What gets encoded is statistical regularities, patterns that appeared consistently enough across training examples to survive gradient descent. Specific facts, especially rare ones, are encoded imprecisely. The model reconstructs them from learned patterns rather than retrieving them verbatim.

Fixed at training time. Parametric memory cannot be updated without retraining. Events after the training cutoff, information not in the training corpus, corrections to facts the model learned incorrectly, none of these can be incorporated without a new training run.

Opaque. There is no mechanism to ask parametric memory for its source. When the model produces a fact, it cannot say which document that fact came from. The provenance is lost in the compression.

Confident under uncertainty. When the model does not know something, its parametric memory does not respond with silence. It responds with whatever continuation is statistically plausible in context. This is hallucination, the model generating text that sounds like knowledge but is not grounded in actual facts.

11.3 What external memory is

External memory is a structured store of information that the model can query at inference time (a vector database, a document corpus, a knowledge graph, or a traditional database). It is external to the model: stored separately, maintained independently, and accessed through a retrieval interface.

In a RAG system, the external memory is typically a collection of documents chunked into passages, embedded as vectors, and stored in a vector database. A query arrives, it is embedded, the most similar passages are retrieved, and they are inserted into the prompt as context.

11.3.1 Properties of external memory

Updatable in real time. New documents can be added, old documents expired, and corrections made without touching the model's weights. The knowledge base can be current as of minutes ago.

Precise and verbatim. Retrieved documents contain exact text from the source. The model does not reconstruct the content, it reads it. Specific numbers, dates, names, and quotations can be retrieved and cited exactly as they appear in the source.

Verifiable and citable. Every retrieved passage has a source. The system knows which document a fact came from, can provide citations, and can display the source for user verification. The provenance is preserved.

Bounded by what was indexed. External memory can only surface what is in the corpus. Knowledge not indexed (information not added, documents not processed) is invisible to retrieval. The corpus boundary defines the limits of what can be found.

Quality-dependent on retrieval. If retrieval fails to surface the relevant passage, the model has no external knowledge to work with. A RAG system is only as good as its retrieval.

Slower than parametric access. External memory requires embedding the query, searching the index, and retrieving documents before the model can use the information. This adds latency (typically 50–500ms) that is absent for parametric knowledge.

11.4 The complementary structure

Parametric and external memory have complementary strength profiles. A well- designed system delegates to each where it excels:

	Parametric Memory	External Memory
Speed	Fast	Slower
Update cost	High	Low
Currency	Stale	Current
Coverage	Training data	Indexed corpus
Precision	Approximate	Exact
Verifiability	None	Full
Private data	No	Yes
Reasoning ability	High	None (the model reasons)
Integration depth	Deep	Shallow
Failure mode	Hallucination	Retrieval miss

11.5 When to rely on parametric memory

Certain categories of knowledge are better served by parametric memory than by retrieval.

11.5.1 General reasoning and language

Grammar, syntax, logical inference, mathematical operations, programming patterns, writing style, none of these are facts to be retrieved. They are capabilities that emerged from training and are deeply integrated into the model's generation process. Retrieval cannot help with "write a recursive function" in the way it helps with "what is the current interest rate."

11.5.2 Stable, well-documented knowledge

The capital of France, the speed of light, the date of World War II, the syntax of Python, facts that are stable, widely documented, and unlikely to have changed since training. For these, retrieval adds latency and complexity without adding information. The model's parametric memory is sufficient.

11.5.3 Synthesis and inference

When a question requires combining multiple pieces of knowledge in ways that are not explicitly stated in any document, parametric memory's deep integration is an advantage. "What are the implications of X for Y?" may not have a document that answers it directly. The model's ability to reason across integrated knowledge (drawing on history, economics, and science simultaneously) is a parametric capability.

11.5.4 Common knowledge about common things

Facts about widely documented subjects that appeared many times in the training corpus are more reliably stored in parametric memory than rare or specialized facts. The model's reconstruction of common knowledge is accurate; its reconstruction of rare knowledge is not.

11.6 When to rely on external memory

11.6.1 Current information

Anything that changes after the training cutoff: news, prices, personnel, policies, software versions, research findings. Parametric memory cannot help here. External memory with regular updates is the only option.

11.6.2 Private and proprietary data

Enterprise documents, customer records, internal policies, proprietary research, none of this is in the training corpus. Retrieval is the only mechanism to give the model access to organization-specific knowledge.

11.6.3 Precise facts and citations

Specific numbers, quotations, contractual terms, regulatory requirements, cases where "approximately right" is not acceptable. External memory provides the exact text; parametric memory approximates.

11.6.4 Accountability and auditability

Cases where the source of information matters, legal, medical, financial, regulatory. External memory provides citable sources. Parametric memory provides assertions without provenance.

11.6.5 Long-tail and specialized knowledge

Domain-specific jargon, rare technical standards, specialized procedures, topics underrepresented in the training corpus. The model's parametric knowledge of these is unreliable. A well-indexed specialized corpus outperforms parametric reconstruction for niche domains.

11.7 The interaction between the two

When both memories are available (a model with access to a retrieval system) they interact in ways that require careful system design.

11.7.1 Which memory does the model prefer?

Research shows that the model's preference for parametric vs. retrieved knowledge depends on the relevance and quality of the retrieved context:

- When retrieved context is highly relevant and clearly addresses the question, the model generally uses it

- When retrieved context is irrelevant or contradicts the model's parametric memory, the model may ignore the retrieved content and answer from parametric knowledge
- When parametric memory is strong (the subject was well-represented in training), the model may underweight retrieved context even when it is relevant

This is not a binary choice the model makes consciously. It is an emergent property of how attention weights retrieved context against parametric associations. The model does not "decide" to trust one over the other it attends to the full context and generates the most likely continuation.

11.7.2 Parametric memory as noise

A counterintuitive failure mode: for questions where the correct answer is in the retrieved context, a model with strong parametric associations to a different answer may generate the wrong answer despite the retrieved evidence.

Example: a company changed its refund policy. The new policy is in the retrieved document. The old policy appeared many times in the training data. The model may generate the old policy from parametric memory rather than the new one from the retrieved document.

This is one of the strongest arguments for grounded generation techniques explicitly instructing the model to prefer retrieved context over internal assumptions. But even with careful prompting, the failure mode persists for strongly-encoded parametric beliefs.

11.7.3 Parametric memory as context interpreter

The model's parametric knowledge is also what allows it to use retrieved context intelligently. When a retrieved document contains technical jargon, the model can interpret it because of parametric knowledge of the domain. When a document is ambiguous, the model uses parametric context to resolve the ambiguity. External memory provides the raw content; parametric memory provides the interpretive framework.

A model with poor parametric knowledge of a domain will retrieve relevant documents and still answer poorly, because it cannot reason about the content effectively. This is why domain-specific fine-tuning (to improve parametric knowledge of the domain) and domain-specific retrieval are complementary rather than substitutes.

11.8 Forms of external memory beyond RAG

RAG (retrieving passages from a vector database) is one form of external memory. Others exist and are increasingly used in production systems.

11.8.1 Structured databases

SQL databases, key-value stores, and graph databases provide structured external memory. A model with tool use can query a SQL database to retrieve exact records (sales figures, customer attributes, product specifications) with guarantees of precision that vector search cannot provide.

For questions with exact, structured answers, a database query is superior to vector search: "What was the revenue in Q3 2024?" is better answered by a database query than by retrieving a passage that mentions Q3 revenue.

11.8.2 Episodic memory stores

For agentic systems that operate over extended periods, episodic memory stores record what the agent has done, observed, and learned in previous sessions. This is a form of external memory that accumulates over time and can be retrieved to inform current decisions.

11.8.3 Tool-accessed knowledge

Search engines, APIs, calculators, and code executors are forms of external memory accessed through tools rather than vector retrieval. A model that can search the web has access to a form of external memory that is more current and more comprehensive than any prebuilt corpus. The tradeoff is latency, reliability, and the need to process retrieved results rather than receiving them in a controlled format.

11.8.4 The knowledge graph

Knowledge graphs represent facts as typed relationships between entities: (Paris, capital_of, France), (France, located_in, Europe). They are more structured than document corpora and support logical inference over stored relationships.

11.9 Designing the memory split

For any application, the decision of what goes in parametric memory and what goes in external memory is an architectural one that affects quality, cost, maintainability, and update frequency.

11.9.1 The update frequency heuristic

Knowledge that changes frequently → external memory. Knowledge that is stable → parametric memory (or cached retrieval).

A customer support application for a product with rapidly evolving features needs product documentation in external memory which changes with every release. The general reasoning and communication style it needs from the model is parametric.

11.9.2 The precision heuristic

Knowledge that must be exact (legal terms, financial figures, technical specs) → external memory. Knowledge that can be approximate (general explanations, background context) → parametric memory.

11.9.3 The privacy heuristic

Knowledge that is confidential or proprietary → external memory with access controls. Knowledge that is public and stable → parametric memory is fine.

11.9.4 The volume heuristic

A small number of highly-accessed facts can be effectively encoded through fine-tuning (moving them into parametric memory). A large, diverse, or growing corpus must go in external memory, there is no practical way to encode millions of documents into model weights through fine-tuning.

11.9.5 The accountability heuristic

When the source of information must be traceable → external memory. When the answer is general knowledge without attribution requirement → parametric memory.

11.10 The emerging continuum

The sharp distinction between parametric and external memory is increasingly a simplification. Several techniques blur the boundary:

Fine-tuning for domain knowledge moves external knowledge into parametric memory through additional training. A model fine-tuned on a company's internal documentation partially encodes that documentation into its weights. The result is faster access and deeper integration, but at the cost of update agility.

Memory-augmented models like RETRO (Borgeaud et al., 2022) integrate retrieval directly into the model architecture, retrieved documents are processed by dedicated cross-attention layers rather than being inserted into the context. The line between the model's parameters and the retrieval corpus is architecturally blurred.

Caching retrieved content in the context creates a form of working memory that persists across turns, more dynamic than parametric memory (it can change with each session) but more structured than ad-hoc retrieval (it is explicitly managed). Long-context models with large KV caches make this increasingly practical.

Continuous learning progressively updates parametric memory from new data, approaching the update agility of external memory while maintaining the access speed of parametric memory. This remains computationally expensive but is an active research direction.

11.11 Key takeaways

- Parametric memory is knowledge encoded in model weights — always available, deeply integrated, fast, but fixed at training time, approximate, and opaque
- External memory is knowledge stored in a retrieval corpus — current, precise, verifiable, and updatable, but dependent on retrieval quality and slower to access
- The two memory types have complementary strength profiles: parametric memory handles reasoning, stable knowledge, and synthesis; external memory handles current information, private data, precise facts, and citable sources
- When both are available, the model does not make a binary choice — it attends to both, and strong parametric associations can override retrieved evidence even when retrieval is correct; grounded generation techniques address but do not fully eliminate this
- Structured databases, episodic memory stores, tool-accessed knowledge, and knowledge graphs are forms of external memory beyond vector search, each better suited to different knowledge types
- The decision of what goes in parametric vs. external memory is an architectural one driven by update frequency, required precision, privacy constraints, volume, and accountability requirements
- The boundary is blurring: fine-tuning moves external knowledge into parametric memory; memory-augmented architectures integrate retrieval into the model; long-context caching creates working memory that spans the two

---

11.12 Further reading

- Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* — The original RAG paper; frames the parametric/non-parametric distinction explicitly and demonstrates complementarity.
- Borgeaud et al. (2022). *Improving Language Models by Retrieving from Trillions of Tokens (RETRO).* — Memory-augmented architecture integrating retrieval into the model rather than the context.
- Mallen et al. (2023). *When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories.* — Empirical study of when models rely on which memory type and when each is more accurate.
- Shi et al. (2023). *Large Language Models Can Be Easily Distracted by Irrelevant Context.* — Retrieved context is not always used; irrelevant retrieval can degrade performance below the parametric-only baseline.
- Guu et al. (2020). *REALM: Retrieval-Augmented Language Model Pre-Training.* — Integrating retrieval into pre-training itself; the predecessor to RAG.

Chapter 12

Document Ingestion

The canonical question for this chapter: *You upload a PDF, a Word document, a webpage, or a database export. Before any of it can be retrieved or reasoned over, it must be transformed into something a retrieval system can work with. What actually happens during that transformation?*

Where are we?

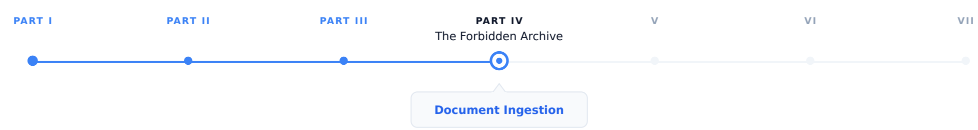


Figure 12.1: The journey through the Model Mind.

Before a document can be retrieved, it must be ingested, parsed, cleaned, structured, and prepared for the pipeline that follows. This is where most RAG systems fail, and it happens long before any query is ever issued.

12.1 The ingestion problem

A language model's knowledge is frozen at training time. It knows what was in its training data and nothing more. For most production applications, this is insufficient, users ask about documents the model has never seen, databases that change daily, proprietary internal knowledge that was never part of any public corpus.

Retrieval-Augmented Generation solves this by giving the model access to external knowledge at inference time. But before any document can be retrieved, it must be ingested: parsed, cleaned, structured, and stored in a form that retrieval systems can query efficiently.

Ingestion is the foundation of every RAG system and where most RAG failures actually originate. It is not in the retrieval algorithm or the language model, but in poor parsing, inadequate cleaning, or naive chunking that destroys the coherence of the source material. Getting ingestion right is a prerequisite for optimizing anything else.

12.2 Document formats and their challenges

The first step in ingestion is parsing: extracting usable text from whatever format the document arrives in. Different formats present different challenges.

12.2.1 PDF

The most common and most problematic format. PDF is a presentation format, not a semantic one. It describes where each character is placed on a page, not what structure the document has.

A PDF does not have "paragraphs" or "sections" in any structural sense. It has positioned text elements, each with a font, size, and (x, y) coordinate. Reconstructing reading order, identifying headings, distinguishing body text from captions, and separating columns requires heuristics that work well on simple documents and poorly on complex ones.

Specific PDF challenges:

Multi-column layouts. Text from different columns is interleaved in the raw extraction order. Naive extractors produce text that mixes columns: "The company revenue | left column second line | right column second line | grew significantly." Correct column separation requires spatial analysis.

Tables. Tabular data in PDFs is often just positioned text with no structural information. Extractors must infer table structure from spatial relationships between text elements. This fails on complex merged cells, rotated headers, and tables that span pages.

Scanned PDFs. PDFs that contain scanned images rather than text contain no extractable text at all, what appears as "text" is actually just pixels. Extraction requires OCR, which adds its own error rate. A 98% OCR accuracy rate sounds high until you realize it means roughly one error every fifty words in a dense technical document.

Headers and footers. Page numbers, document titles, and section headers in running headers and footers are often extracted inline with body text. A 100-page document might have "CONFIDENTIAL Page N of 100" injected into the text every few paragraphs.

Mathematical notation. Equations in PDFs are often stored as positioned symbols rather than structured mathematical expressions. Extraction typically produces garbled sequences of symbols.

PDF libraries for Python:

- **PyMuPDF (fitz)**: fast, good text extraction for standard PDFs, reasonable layout analysis
- **pdfplumber**: better table extraction than PyMuPDF, slower
- **pypdf**: pure Python, simpler, adequate for basic extraction
- **Adobe PDF Extract API**: commercial, highest quality for complex layouts

For high-stakes RAG applications (legal documents, financial filings, technical documentation) investing in high-quality PDF parsing pays off significantly. A poorly parsed PDF produces retrieval failures that are invisible at the extraction stage but manifest as hallucinations and incorrect answers at inference time.

12.2.2 Word documents (DOCX)

Structurally richer than PDF. DOCX files have an explicit document model: paragraphs, headings (with levels), tables (with cell structure), lists, and styles. Extraction can respect this structure rather than reconstructing it from layout.

Challenges: embedded objects (images, embedded PDFs, charts) are opaque blobs whose content is inaccessible without separate processing. Tracked changes and revision marks can clutter extracted text.

12.2.3 HTML and web pages

HTML is semantically rich when used correctly. Heading tags (`<h1>` through `<h6>`), paragraph tags, list tags, table tags, and semantic elements like `<article>`, `<section>`, and `<nav>` provide structural information that text extractors can use.

In practice, web HTML is rarely so clean. Navigation menus, cookie banners, advertisements, and footers must be removed. JavaScript-rendered content requires a headless browser to extract.

12.2.4 Plain text and Markdown

The easiest cases. Text files require no parsing; they are read directly. Markdown adds lightweight semantic structure (headings via `#`, code blocks via backticks, lists) that parsers can respect.

Markdown is increasingly common as a RAG source format because it is both human-readable and machine-parseable. Documents intentionally written for RAG ingestion are sometimes authored in Markdown for this reason.

12.2.5 Spreadsheets (XLSX, CSV)

Tabular data presents a different challenge: how do you convert a table into text that can be retrieved and understood by a language model?

Options:

- **Row-by-row serialization**: "Row 3: Name=Alice, Age=32, Department=Engineering, Salary=95000"
- **Markdown table format**: convert to a Markdown table that the LLM can parse
- **Schema + sample rows**: describe the schema and include representative examples; retrieve the schema for schema-level questions and specific rows for value-level questions

For large spreadsheets (thousands of rows), row-by-row serialization produces too much text to embed meaningfully. SQL or structured databases with text-to-SQL generation are often a better architecture than RAG for tabular data. Retrieval finds relevant rows; the model generates SQL; the database executes it exactly.

12.2.6 Images and multimodal documents

Documents with meaningful visual content (diagrams, charts, photographs with captions, infographics) require multimodal processing.

OCR extracts text from images. Tesseract is the standard open-source OCR library; AWS Textract, Google Cloud Vision, and Azure Computer Vision are commercial alternatives with higher accuracy on complex layouts.

Vision-language models can describe image content, answer questions about figures, and extract structured information from charts. For documents where figures carry significant information, routing image content through a VLM before ingestion produces much better retrieval coverage than ignoring figures or relying on captions alone. A technical diagram with no caption is invisible to text-based retrieval without VLM description.

12.3 Text cleaning

After parsing, raw extracted text often contains artifacts that reduce retrieval quality and cleaning is used to remove or normalize them.

12.3.1 Common cleaning steps

Whitespace normalization. Collapse multiple spaces to one, remove leading/trailing whitespace from paragraphs, normalize line endings. Necessary because PDF extraction produces variable whitespace depending on character positioning.

Header and footer removal. Identify and remove page numbers, document titles repeated on each page, copyright notices, and other non-content text. Pattern matching (regex for "Page N of M") handles simple cases and spatial analysis during PDF parsing handles complex ones.

Hyphenation repair. PDF extraction often breaks hyphenated words at line breaks: `trans-\naction` appears as two tokens rather than "transaction". Detecting and joining these is straightforward for end-of-line hyphens but ambiguous for legitimate compound hyphenation.

Encoding normalization. Documents from multiple sources may have different encodings (UTF-8, Latin-1, Windows-1252). Normalize to UTF-8. Handle common encoding artifacts: curly quotes that appear as `â€œ`, em dashes that appear as `â€"`.

Boilerplate removal. Repeated passages that appear across many documents (terms of service, disclaimer language, standard legal notices) add noise to embeddings and can dominate retrieval for questions that accidentally match boilerplate patterns. Deduplication at the passage level or explicit blocklisting removes this.

Table of contents removal. Tables of contents in PDFs and Word documents are often extracted as body text, producing a list of section titles that duplicates content the model will encounter in the actual sections. Identifying and removing ToC passages prevents this duplication from dominating retrieval.

Reference list handling. Bibliographies and reference lists at the end of academic papers are often extracted as body text. Whether to include or exclude them depends on whether the references themselves are useful for retrieval. For scientific RAG, including references can help answer "what papers support this claim" questions.

12.3.2 Cleaning for specific domains

Legal documents. Case citations, statutory references, and defined terms have specific patterns. Preserving these patterns rather than normalizing them is important for legal search. Removing citation formatting destroys the semantic signal that distinguishes a reference to a case from a discussion of it.

Medical records. PHI (Protected Health Information) removal is legally required before storing patient records in retrieval systems. Names, dates, locations, and identifiers must be de-identified before ingestion. This is not optional and not a post-processing step, it must happen before any content is stored.

Code. Programming language syntax should not be normalized like prose. Tab/space preservation matters for Python. Comment blocks, docstrings, and variable names are semantically important and should not be striped. Code is often better chunked by function or class boundary than by token count.

Financial filings. Numerical tables in SEC filings have specific conventions (negative numbers in parentheses, thousands vs. millions) which should be preserved or explicitly normalized consistently across the corpus.

12.4 Document metadata extraction

Every ingested document should be accompanied by metadata that enables filtering, ranking, and attribution during retrieval.

12.4.1 Core metadata fields

```
{
    "document_id": "uuid-abc123",
    "source_url": "https://example.com/docs/api-reference.pdf",
    "source_type": "pdf",
    "title": "API Reference Guide v3.2",
    "author": "Engineering Team",
    "created_at": "2024-01-15T10:30:00Z",
    "modified_at": "2024-03-22T14:15:00Z",
    "language": "en",
    "page_count": 48,
    "word_count": 23500,
```

```
    "section": "Technical Documentation",
    "tags": ["api", "reference", "v3"],
    "access_level": "internal",
}
```

12.4.2 Why metadata matters

Filtering. Retrieval can be limited to documents matching metadata criteria. "What does our API documentation say about authentication?" should retrieve from technical documentation, not from marketing materials or HR policies. Without metadata filtering, vector similarity is the only selection mechanism, and it will surface whatever is most semantically similar regardless of source.

Recency ranking. For questions about current state, recently modified documents should rank higher. Metadata enables this without modifying embeddings.

Access control. Metadata carries authorization information. Users should not retrieve documents their access level does not permit, even if those documents are semantically relevant to their query. Access control must be enforced at retrieval time, not just at ingestion time.

Attribution. When the model uses retrieved content to generate a response, metadata provides the citation: "According to the API Reference Guide v3.2, section 4…" Users who need to verify information can follow the citation to the source.

12.4.3 Automatic metadata extraction

Some metadata fields must be inferred rather than extracted directly:

Language detection. langdetect and fastText language identification models classify language from text samples. Important for multilingual corpora where language-specific retrieval is needed, you do not want a French query retrieving German documents.

Topic classification. Embedding the document title and first paragraph and comparing to topic embedding clusters can assign broad topical categories. Useful for filtering and for routing queries to the right sub-corpus.

Entity extraction. Named entity recognition (NER) identifies people, organizations, locations, and other entities mentioned in the document. These can be stored as metadata fields enabling entity-based filtering: "find documents that mention Acme Corp."

12.5 The ingestion pipeline assembled

Putting the components together, a production document ingestion pipeline:

```
Raw document (PDF, DOCX, HTML, etc.)


Format detection


Parser selection and execution
    PDF: PyMuPDF / pdfplumber / LlamaParse
    DOCX: python-docx
    HTML: trafilatura + BeautifulSoup
    Image: OCR + VLM captioning
    Plain text: direct read
```

```
Raw text + structural information
  (paragraphs, headings, tables, page numbers)


Text cleaning
    Whitespace normalization
    Header/footer removal
    Encoding normalization
    Boilerplate removal
    Domain-specific cleaning


Metadata extraction
    Core fields (title, source, timestamps)
    Language detection
    Entity extraction (optional)


Chunking


Embedding


Vector store + metadata store
```

12.5.1 Idempotency and incremental updates

A production ingestion pipeline runs continuously as new documents arrive and existing documents are updated. Two properties are essential:

Idempotency. Ingesting the same document twice should produce the same result as ingesting it once. Without idempotency, re-running the pipeline on a partially-updated corpus duplicates documents and corrupts the retrieval index. Implementation: checksums enable detection of already-ingested documents. If a document's checksum matches an existing entry, skip re-ingestion. If the checksum differs (document has been updated), delete the old chunks and embeddings and re-ingest.

Atomic updates. When a document is updated, the retrieval index should not be in a mixed state (some chunks from the old version, some from the new). Atomic updates require: ingesting the new version to a staging area, deleting all old chunks and embeddings for this document ID, and promoting the new chunks and embeddings from staging to production. These steps are typically transactional in a properly designed metadata store.

12.5.2 Error handling and monitoring

Document ingestion fails silently in ways that are hard to detect. A document that fails to parse produces no chunks and no embeddings, it simply does not exist in the retrieval index. Users asking questions about that document get answers based on whatever other documents are retrieved instead. Without logging and monitoring, these failures are invisible.

Instrument the pipeline to track: documents processed, failed, and skipped per run; parse failures by format and error type; quality gate rejection rates; embedding failures; and processing latency by document type and size.

Alert on anomalies: if the PDF parse failure rate suddenly spikes, the parser library may have a bug or the input format may have changed. If the quality gate rejects an unusually large fraction of documents, the source may be producing degraded content.

12.6 Ingestion at scale

For large corpora (millions of documents) the ingestion pipeline must be designed for scale from the start.

12.6.1 Parallelization

Document parsing is CPU-bound and embarrassingly parallel, each document can be processed independently. Use a task queue (Celery, Ray, AWS SQS + Lambda) to distribute documents across multiple workers. Each worker handles parsing, cleaning, chunking, and embedding for its assigned documents.

The embedding step often calls an external API (OpenAI, claude). API rate limits impose a ceiling on parallelism. Design the pipeline to batch embedding requests at the maximum allowed batch size and implement rate limit handling with exponential backoff.

12.6.2 Incremental ingestion

For corpora that change frequently, full re-ingestion on every update is too slow. Incremental ingestion processes only new and modified documents:

1. Maintain a manifest of all ingested documents and their checksums
2. On each run, compare new document checksums to the manifest
3. Process only documents with new or changed checksums
4. Update the manifest after successful ingestion

For web crawls, compare last-modified headers or ETags to detect changes without re-downloading content.

12.6.3 Storage tiering

Ingested documents at various stages should be stored durably:

- **Raw documents**: store the original files in object storage (S3, GCS). Enables re-ingestion if the pipeline changes without retrieving the original from the source.
- **Parsed text**: store the cleaned extracted text alongside metadata. Enables re-chunking and re-embedding without re-parsing.
- **Chunks**: store chunk text and metadata. Enables re-embedding without re-chunking.
- **Embeddings**: stored in the vector database. The final retrieval index.

Each tier enables rerunning from that point forward without rerunning earlier stages. When you improve your embedding model, you only need to re-embed, not re-parse. When you improve your chunking strategy, you only need to re-chunk and re-embed, not re-parse. This separation pays off every time a component of the pipeline changes.

12.6.4 Versioning the pipeline

The ingestion pipeline is code that runs over data to produce embeddings. Changes to any component (parser version, cleaning rules, chunking strategy, embedding model) invalidate some or all of the stored artifacts.

Track the pipeline version with each stored artifact. When the pipeline version changes, identify which artifacts need to recompute. This is analogous to dependency tracking in build systems and is essential for maintaining a coherent retrieval index. Without it, a corpus may silently contain chunks embedded with an old model alongside chunks embedded with a new one, making similarity comparisons meaningless.

12.7 Key takeaways

- Document ingestion is the foundation of every RAG system; most RAG failures originate in poor parsing, inadequate cleaning, or naive structure destruction rather than in retrieval algorithms or language models

- PDF is the most common and most problematic format: a presentation format with no semantic structure, requiring heuristic reconstruction of reading order, headings, columns, and tables; for high-stakes applications, invest in high-quality PDF parsing
- Different formats require different parsers and present different challenges; text files and Markdown are easy; scanned PDFs and multi-column layouts are hard
- Text cleaning removes artifacts introduced by parsing — whitespace, headers and footers, encoding issues, boilerplate — that degrade retrieval quality without conveying content; domain-specific cleaning (PHI removal, citation preservation, code formatting) requires domain knowledge
- Metadata is not optional: it enables filtering by source, date, and access level; provides attribution for citations; and allows quality gating and deduplication before embedding
- A production ingestion pipeline must be idempotent (re-running is safe) and support atomic updates (document updates replace all old chunks, not mix old and new)
- Silent failures — documents that fail to parse or are rejected by quality gates — are invisible without explicit logging and monitoring; instrument the pipeline before you need to debug it
- At scale, ingestion must be parallelized, incremental, and tiered: store artifacts at each stage so that pipeline improvements can be applied from that stage forward without rerunning from scratch
- Track pipeline versions with stored artifacts; a corpus that mixes chunks embedded with different models has corrupted similarity comparisons

---

12.8 Further reading

- Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* — The foundational RAG paper; establishes the ingestion → retrieval → generation framework.
- Edge et al. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization.* — GraphRAG; extends ingestion to extract entities and relationships, not just text chunks.
- Unstructured.io (2023). *Unstructured: Document parsing for LLM applications.* — The leading open-source document parsing library for RAG; handles many formats with layout analysis.
- Shi et al. (2023). *REPLUG: Retrieval-Augmented Black-Box Language Models.* — Analysis of how retrieval quality affects end-to-end RAG performance; ingestion quality is the upstream dependency.

Chapter 13

Chunking Strategies

The canonical question for this chapter: *A document might be 50,000 words while an embedding model can process 512 tokens. A retrieved context window might be limited to 5,000. How do you divide the document into pieces that are retrievable, coherent, and useful to the language model?*

Where are we?

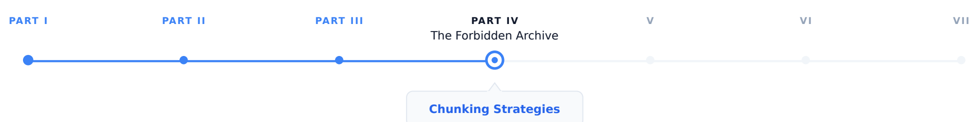


Figure 13.1: The journey through the Model Mind.

Document ingestion produced clean text. Now that text must be divided into retrievable chunks which is the most consequential preprocessing decision in a RAG system. Poor chunking produces retrieval failures that no downstream component can fix.

13.1 Why chunking exists and why it is hard

Embedding models have a maximum input length which is typically 512 to 8,192 tokens. Embedding an entire document as a single vector loses granularity: a 10,000-word technical manual embedded as one vector will match queries about any topic covered in the manual, making retrieval imprecise. You want the chunk about authentication to be retrieved for authentication questions and the chunk about rate limiting to be retrieved for rate limiting questions, not the entire manual for both.

At the same time, chunks must be semantically coherent. A chunk that cuts mid-sentence, mid-argument, or mid-table is harder for the language model to use correctly. The model receives a fragment without the context that makes it interpretable.

The fundamental tension: small chunks are precise for retrieval but too small to be self-contained. Large chunks are self-contained but imprecise for retrieval. The right chunk size is a function of the document type, the query distribution, the embedding model, and the language model's context window.

Getting chunking wrong is one of the most common RAG failure modes. A retrieval system with perfect relevance ranking that retrieves incoherent chunks will produce worse answers than a system with imperfect ranking that retrieves coherent chunks. The language model cannot compensate for fragmented context.

13.2 Fixed-size chunking

The simplest strategy: split the document into chunks of a fixed number of tokens, with an optional overlap between adjacent chunks.

```python
def fixed_size_chunk(text: str, chunk_size: int, overlap: int) -> list[str]:
    tokens = tokenize(text)
    chunks = []

    start = 0
    while start < len(tokens):
        end = min(start + chunk_size, len(tokens))
        chunk_tokens = tokens[start:end]
        chunks.append(detokenize(chunk_tokens))
        start += chunk_size - overlap

    return chunks
```

For `chunk_size=512` and `overlap=50`: chunk 1 covers tokens 0–511, chunk 2 covers tokens 462–973, chunk 3 covers tokens 924–1435. Each chunk overlaps with its neighbors by 50 tokens.

13.2.1 Why overlap exists

Without overlap, a sentence that straddles a chunk boundary is split: the first half is at the end of one chunk, the second half at the start of the next. Neither chunk contains the complete sentence, and a query that matches the complete sentence may not match either fragment.

Overlap ensures that every point in the document appears in at least one complete chunk. For overlap of `k` tokens, any contiguous span of `chunk_size - k` tokens or fewer is guaranteed to appear entirely within at least one chunk.

The cost of overlap: the same content is embedded and stored multiple times, increasing storage and compute cost. An overlap of 10% (50 tokens on a 512- token chunk) increases storage by approximately 10% which is a small price for significantly better boundary coverage.

13.2.2 When fixed-size chunking is appropriate

Fixed-size chunking is fast, simple, and predictable. It works reasonably well for homogeneous documents without meaningful structure (transcripts, continuous prose), corpora where you cannot rely on structural markers (inconsistent heading styles, poorly formatted PDFs), and prototyping.

It is inappropriate for documents with meaningful structure (technical documentation, legal contracts) where semantic boundaries do not align with token count boundaries, for tables and code which should not be split arbitrarily, and for documents where context continuity is critical.

13.2.3 Token-based vs. character-based splitting

Splitting on token count is more precise than splitting on character count because token count directly maps to embedding model input length. Character- based splitting produces chunks of unpredictable token length, a chunk of 2,000 characters might be 400 tokens for English prose or 700 tokens for code with many short tokens.

13.3 Sentence-based chunking

Instead of splitting at fixed token counts, split at sentence boundaries. This preserves semantic units and avoids splitting mid-sentence.

```python
import spacy

nlp = spacy.load("en_core_web_sm")

def sentence_chunk(text: str,
                   max_tokens: int,
                   overlap_sentences: int) -> list[str]:
    doc = nlp(text)
    sentences = [sent.text.strip() for sent in doc.sents]

    chunks = []
    current_chunk = []
    current_length = 0

    for sentence in sentences:
        sentence_tokens = len(tokenize(sentence))

        if current_length + sentence_tokens > max_tokens and current_chunk:
            chunks.append(" ".join(current_chunk))
            current_chunk = current_chunk[-overlap_sentences:]
            current_length = sum(len(tokenize(s)) for s in current_chunk)

        current_chunk.append(sentence)
        current_length += sentence_tokens

    if current_chunk:
        chunks.append(" ".join(current_chunk))

    return chunks
```

Sentence detection requires a sentence boundary detector, spaCy, NLTK's Punkt tokenizer, or regex-based heuristics for simple cases. Sentence detectors are imperfect: abbreviations ("Dr.", "U.S."), decimal points, and ellipses can confuse them.

Sentence-based chunking works well for dense prose where individual sentences carry complete thoughts. It works poorly for bullet points and numbered lists (items may not be complete sentences), tables (rows are not sentences), code (statements are not sentences in the linguistic sense), and very long sentences that exceed the chunk size on their own.

13.4 Recursive character text splitting

LangChain's `RecursiveCharacterTextSplitter` is the most widely used chunking implementation in production RAG systems. It splits on a hierarchy of separators, trying each in order until chunks are within the size limit:

```
Separator hierarchy (default):
  1. "\n\n"   (paragraph break)
  2. "\n"     (line break)
  3. " "      (word boundary)
  4. ""       (character)
```

The algorithm tries to split on `"\n\n"`. If all resulting pieces are within the size limit, done. If some pieces are still too large, it recursively splits those on `"\n"`. If still too large, on `" "`. If still too large, on individual characters.

This preserves the largest meaningful structure possible within the size limit. For a well-formatted document, most chunks will be split at paragraph boundaries. The character-level fallback is rarely triggered for natural text.

Custom separator hierarchies for specific document types:

```
# For Markdown documents
markdown_separators = [
    "\n# ",    # H1 heading
    "\n## ",   # H2 heading
    "\n### ",  # H3 heading
    "\n#### ", # H4 heading
    "\n\n",    # Paragraph
    "\n",      # Line break
    " ",       # Word
]

# For code
code_separators = [
    "\nclass ",    # Class definition
    "\ndef ",      # Function definition
    "\n\n",        # Blank line
    "\n",          # Line
    " ",           # Token
]
```

The key insight: splitting at paragraph boundaries is almost always better than splitting at arbitrary token counts, and the recursive approach tries for paragraph boundaries first.

13.5 Structure-aware chunking

For documents with explicit structure, the best chunking strategy uses that structure directly rather than approximating it with text splitting.

13.5.1 Heading-based chunking

For documents with hierarchical headings (HTML, Markdown, DOCX with proper heading styles), chunk at heading boundaries. Each chunk is one section: from a heading to the next heading at the same or higher level.

```
def heading_chunk(document: ParsedDocument) -> list[Chunk]:
    chunks = []
    current_section = []
    current_heading = None
    current_level = 0

    for element in document.elements:
        if element.type == "heading":
            if current_section:
                chunks.append(Chunk(
                    text="\n".join(current_section),
                    heading=current_heading,
                    heading_level=current_level,
```

```
                metadata=document.metadata
            ))
        current_section = [element.text]
        current_heading = element.text
        current_level = element.level
    else:
        current_section.append(element.text)

if current_section:
    chunks.append(Chunk(
        text="\n".join(current_section),
        heading=current_heading,
        heading_level=current_level,
        metadata=document.metadata
    ))

return chunks
```

This produces chunks that correspond to meaningful sections rather than arbitrary text blocks. The heading becomes part of the chunk metadata, enabling heading-based filtering and citation.

The limitation: sections vary enormously in length. A one-paragraph section produces a tiny chunk; a multi-page section produces a huge one. Post-processing applies fixed-size splitting to chunks that exceed the embedding model's context limit while keeping chunks below the limit intact.

13.5.2 Hierarchical chunking

Documents with nested structure (chapters contain sections contain subsections) can be chunked at multiple levels simultaneously:

```
Chapter level:     "Chapter 4: Authentication"
  Section level:   "4.1 API Keys"
  Section level:   "4.2 OAuth 2.0"
    Subsection:    "4.2.1 Authorization Code Flow"
    Subsection:    "4.2.2 Client Credentials Flow"
  Section level:   "4.3 JWT Tokens"
```

A query about "authentication" might retrieve the chapter-level chunk. A query about "OAuth authorization code flow" retrieves the specific subsection. Both are indexed; retrieval chooses the appropriate level. Hierarchical chunking requires a retrieval system that can handle multi-granularity documents, typically by indexing all levels and using metadata to filter or re-rank by granularity.

13.5.3 Table handling

Tables should almost never be split across chunks. A partial table (some rows at the end of one chunk, remaining rows at the start of the next) is nearly useless to the language model. Two strategies:

Keep tables whole. Treat each table as an atomic chunk regardless of size. If the table exceeds the embedding model's limit, summarize it or split into row groups that include the header row in each group.

Serialize by row group. Convert the table to serialized text and chunk by row groups, including the header in each chunk:

```
def serialize_table_chunk(table: Table, rows_per_chunk: int) -> list[str]:
    header = "| " + " | ".join(table.headers) + " |"
    separator = "| " + " | ".join(["---"] * len(table.headers)) + " |"
```

```
    chunks = []
    for i in range(0, len(table.rows), rows_per_chunk):
        row_group = table.rows[i:i + rows_per_chunk]
        rows_text = "\n".join(
            "| " + " | ".join(str(cell) for cell in row) + " |"
            for row in row_group
        )
        chunks.append(f"{header}\n{separator}\n{rows_text}")

    return chunks
```

13.5.4 Code chunking

Code has its own natural units: functions, classes, modules. Chunking at these boundaries is almost always preferable to fixed-size splitting.

For Python, the AST provides exact function and class boundaries:

```
import ast

def chunk_python_file(source_code: str) -> list[Chunk]:
    tree = ast.parse(source_code)
    chunks = []

    for node in ast.walk(tree):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef)):
            start_line = node.lineno - 1
            end_line = node.end_lineno
            chunk_text = "\n".join(source_code.split("\n")[start_line:end_line])

            chunks.append(Chunk(
                text=chunk_text,
                metadata={
                    "type": type(node).__name__,
                    "name": node.name,
                    "start_line": node.lineno,
                    "end_line": node.end_lineno,
                }
            ))

    return chunks
```

13.6 Semantic chunking

Rather than splitting on structural or syntactic boundaries, semantic chunking splits where the meaning changes. Adjacent sentences discussing the same topic are grouped together; a split is introduced when the topic shifts.

13.6.1 Embedding-based semantic chunking

Embed each sentence independently, then compute the cosine similarity between adjacent sentence embeddings. A significant drop in similarity indicates a topic shift and marks the boundary for a new chunk.

```python
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def semantic_chunk(sentences: list[str],
                   embedding_model,
                   percentile: float = 95) -> list[str]:

    embeddings = embedding_model.encode(sentences)

    similarities = [
        cosine_similarity([embeddings[i]], [embeddings[i+1]])[0][0]
        for i in range(len(embeddings) - 1)
    ]

    # Adaptive threshold: split where similarity is in the bottom percentile
    breakpoint_threshold = np.percentile(similarities, 100 - percentile)
    breakpoints = [
        i for i, sim in enumerate(similarities)
        if sim < breakpoint_threshold
    ]

    chunks = []
    start = 0
    for bp in breakpoints:
        chunks.append(" ".join(sentences[start:bp+1]))
        start = bp + 1
    chunks.append(" ".join(sentences[start:]))

    return chunks
```

13.6.2 Advantages and costs of semantic chunking

Semantic chunking produces chunks that are more topically coherent than fixed- size or structure-based chunking. Queries match chunks about the same topic as the query, not just chunks that happen to contain the query terms at an arbitrary text position.

The cost is significant: every sentence must be embedded individually during chunking. For a document with 1,000 sentences and an embedding model that takes 10ms per sentence, that is 10 seconds per document. At corpus scale (millions of documents), this is a substantial offline cost that is usually justified for high-value corpora where retrieval quality is critical.

13.6.3 LLM-based semantic chunking

A more sophisticated approach: use a language model to identify thematic boundaries directly.

```python
def llm_chunk(document: str, llm) -> list[str]:
    prompt = f"""Identify the major thematic sections in the following document.
Return a JSON list of character indices where each new section begins.

Document:
{document}

Return only the JSON array of character indices."""

    response = llm.complete(prompt)
```

```
    section_starts = json.loads(response)

    chunks = []
    for i, start in enumerate(section_starts):
        end = section_starts[i+1] if i+1 < len(section_starts) else len(document)
        chunks.append(document[start:end])

    return chunks
```

LLM-based chunking is expensive (one model call per document) and slow but produces the highest-quality semantic boundaries. It is practical for high- value documents (contracts, research papers, technical specifications) where chunking quality directly affects critical business outcomes.

13.7 Chunk enrichment

Raw chunks often lack context that was available in the document but is not present in the chunk itself. Chunk enrichment adds this context before embedding, making the resulting embeddings more informative.

13.7.1 Prepending metadata

Include the document title, section heading, and other metadata in the chunk text before embedding:

```
def enrich_chunk(chunk: Chunk) -> str:
    parts = []

    if chunk.metadata.get("document_title"):
        parts.append(f"Document: {chunk.metadata['document_title']}")

    if chunk.metadata.get("section_heading"):
        parts.append(f"Section: {chunk.metadata['section_heading']}")

    if chunk.metadata.get("source_date"):
        parts.append(f"Date: {chunk.metadata['source_date']}")

    parts.append(chunk.text)

    return "\n".join(parts)
```

This changes what the embedding represents: instead of just the chunk text, the embedding represents the chunk in the context of its document and section. Queries about "authentication" will now match chunks in authentication sections more reliably, because the section heading "Authentication" is part of the embedded text.

13.7.2 Contextual retrieval

Anthropic's contextual retrieval technique uses a language model to generate a brief context description for each chunk, prepended before embedding:

```
def add_chunk_context(document: str, chunk: str, llm) -> str:
    prompt = f"""Given the following document:
<document>
{document}
</document>
```

```
And this chunk from the document:
<chunk>
{chunk}
</chunk>

Provide a brief (2-3 sentence) description of what this chunk is about and
how it relates to the broader document. This will be prepended to the chunk
to improve retrieval."""

    context = llm.complete(prompt)
    return f"{context}\n\n{chunk}"
```

The result: instead of embedding "The token expires after 3600 seconds by default," you embed "This chunk describes the default expiration time for authentication tokens in the API. It is from the Authentication section of the API reference guide. The token expires after 3600 seconds by default."

The enriched embedding retrieves more accurately for queries about token expiration, authentication configuration, or API defaults. The cost: one LLM call per chunk during ingestion. Batch the calls, use a smaller model for context generation, and cache results so that re-chunking does not require regenerating contexts.

13.7.3 Hypothetical document embeddings (HyDE)

A related technique applied at query time rather than ingestion time: instead of embedding the raw query, use a language model to generate a hypothetical document that would answer the query, then embed that hypothetical document for retrieval.

The intuition: a query ("what is the default token expiration time?") and a document chunk ("The token expires after 3600 seconds by default") may be phrased very differently and have dissimilar embeddings. A hypothetical answer ("The default token expiration time is 3600 seconds, configurable via the token_ttl parameter") is phrased similarly to the actual document content and embeds more similarly.

```
def hyde_retrieve(query: str, llm, retriever) -> list[Chunk]:
    hypothetical_doc = llm.complete(
        f"Write a brief passage that answers this question:\n{query}"
    )
    return retriever.retrieve(hypothetical_doc, top_k=5)
```

HyDE consistently improves retrieval quality for factual queries where the query phrasing differs significantly from the document phrasing. It adds one LLM call per query which is acceptable for most production systems where answer quality is the priority.

13.7.4 Query rewriting

A related technique operates at query time rather than ingestion time: rewrite the user's query so that it more closely matches the language, terminology, and writing style of the underlying document corpus before retrieval.

The motivation is the same as HyDE: retrieval systems operate in embedding space, and semantically identical concepts may be expressed very differently by users and documents. A user might ask:

> "How long does a login session last?"

while the documentation states:

> "Authentication tokens expire after 3600 seconds."

Although these refer to the same concept, the phrasing differs substantially. A query rewriting model transforms the user query into language that more closely resembles the corpus:

> "What is the default expiration time for authentication tokens?"

The rewritten query is then embedded and used for retrieval.

```
def rewrite_retrieve(query: str, llm, retriever) -> list[Chunk]:
    rewritten_query = llm.complete(
        f"""Rewrite the following search query so that it matches the
terminology and writing style of technical documentation while preserving
its original meaning.

Query: {query}

Return only the rewritten query."""
    )

    return retriever.retrieve(rewritten_query, top_k=5)
```

Query rewriting differs from HyDE in an important way. HyDE generates a hypothetical answer document and embeds that document for retrieval. Query rewriting preserves the query format but adapts its vocabulary and style to better align with the corpus.

This approach is particularly effective for domain-specific corpora where users and documents use different terminology, such as medical records, legal documents, internal company jargon, and technical documentation.

13.8 Chunk size selection

There is no universal optimal chunk size. The right value depends on the embedding model's context window (chunks cannot exceed the maximum input length), the query type (short factual queries match short precise chunks; long analytical queries match longer contextual chunks), the document type (technical documentation with short precise sections warrants smaller chunks; legal contracts with long interconnected clauses warrant larger ones), and the retrieval top-k and context window (if you retrieve top-5 chunks into a 128k context window, chunks can be large; if the context window is 4k tokens, keep chunks under 1,000 tokens each).

13.8.1 Empirical chunk size selection

The right approach: treat chunk size as a hyperparameter and evaluate empirically.

1. Build a small evaluation set: 50–100 questions with known answers from your document corpus
2. Ingest the corpus with multiple chunk sizes (256, 512, 1024, 2048 tokens)
3. For each chunk size, measure retrieval recall (does the answer appear in the top-k retrieved chunks?) and answer quality (does the LLM produce a correct answer?)
4. Select the chunk size that maximizes answer quality on the evaluation set

This takes time but produces chunk sizes calibrated to your specific documents and queries rather than arbitrary defaults. The defaults in most frameworks (512 tokens, 10% overlap) are reasonable starting points, not optimal values.

13.9 Multi-granularity retrieval

A single chunk size is a compromise. A better approach: index documents at multiple granularities and retrieve from the most appropriate level.

13.9.1 Parent-child chunking

Index small chunks for precise retrieval, but retrieve and pass larger parent chunks to the language model for context:

```
Document
    Parent chunk (1,000 tokens): "Section 4.2: OAuth 2.0"
        Child chunk (200 tokens): "Authorization Code Flow"
        Child chunk (200 tokens): "Client Credentials Flow"
        Child chunk (200 tokens): "Token Refresh"
```

Retrieval operates on child chunk embeddings (precise matching). When a child chunk is retrieved, the system returns its parent chunk (rich context). The language model receives the full parent chunk, not the narrow child chunk.

This combines the retrieval precision of small chunks with the contextual richness of large chunks which is the best of both without the tradeoff.

13.9.2 Small-to-big retrieval

Similar to parent-child, but the relationship is sentence-to-paragraph: embed sentences for retrieval, return the surrounding paragraph to the language model.

```python
def small_to_big_retrieve(query: str, retriever, document_store) -> list[str]:
    sentence_chunks = retriever.retrieve(query, top_k=10)

    paragraphs = set()
    for chunk in sentence_chunks:
        paragraph_id = chunk.metadata["paragraph_id"]
        paragraphs.add(document_store.get_paragraph(paragraph_id))

    return list(paragraphs)
```

13.9.3 Summary indexing

For very long documents, index a summary of each section alongside the sections themselves. Broad queries ("what is this document about?") match the summary. Specific queries ("what is the default timeout?") match the section chunks. The summary provides a navigational layer above the detailed content.

13.10 Key takeaways

- Chunking is the most consequential preprocessing decision in a RAG system; incoherent chunks cannot be compensated for by better retrieval or a better language model
- Fixed-size chunking with overlap is simple and works reasonably well for homogeneous prose; 10% overlap costs 10% more storage and dramatically reduces boundary failures
- Recursive character splitting preserves the largest meaningful structure (paragraphs, then lines, then words) within the size limit — better than pure fixed-size splitting for most documents
- Structure-aware chunking at heading boundaries produces the most semantically coherent chunks for well-formatted documents; tables and code should always be treated as atomic units with splitting only at row or function boundaries
- Semantic chunking (embedding-based similarity breakpoints) produces the most topically coherent chunks at the cost of significant offline compute — worth it for high-value corpora, expensive at scale
- Chunk enrichment — prepending metadata, section headings, or LLM-generated context descriptions — improves retrieval by making embeddings more informative about the chunk's place in the document; contextual retrieval is high-impact when quality justifies the extra LLM calls
- HyDE improves retrieval at query time by embedding a hypothetical answer rather than the raw query, bridging the vocabulary gap between queries and documents
- Parent-child chunking combines retrieval precision (embed small child chunks) with contextual richness (return large parent chunks to the language model)

- Treat chunk size as a hyperparameter: build an evaluation set of 50–100 representative questions and select chunk size empirically rather than using arbitrary defaults

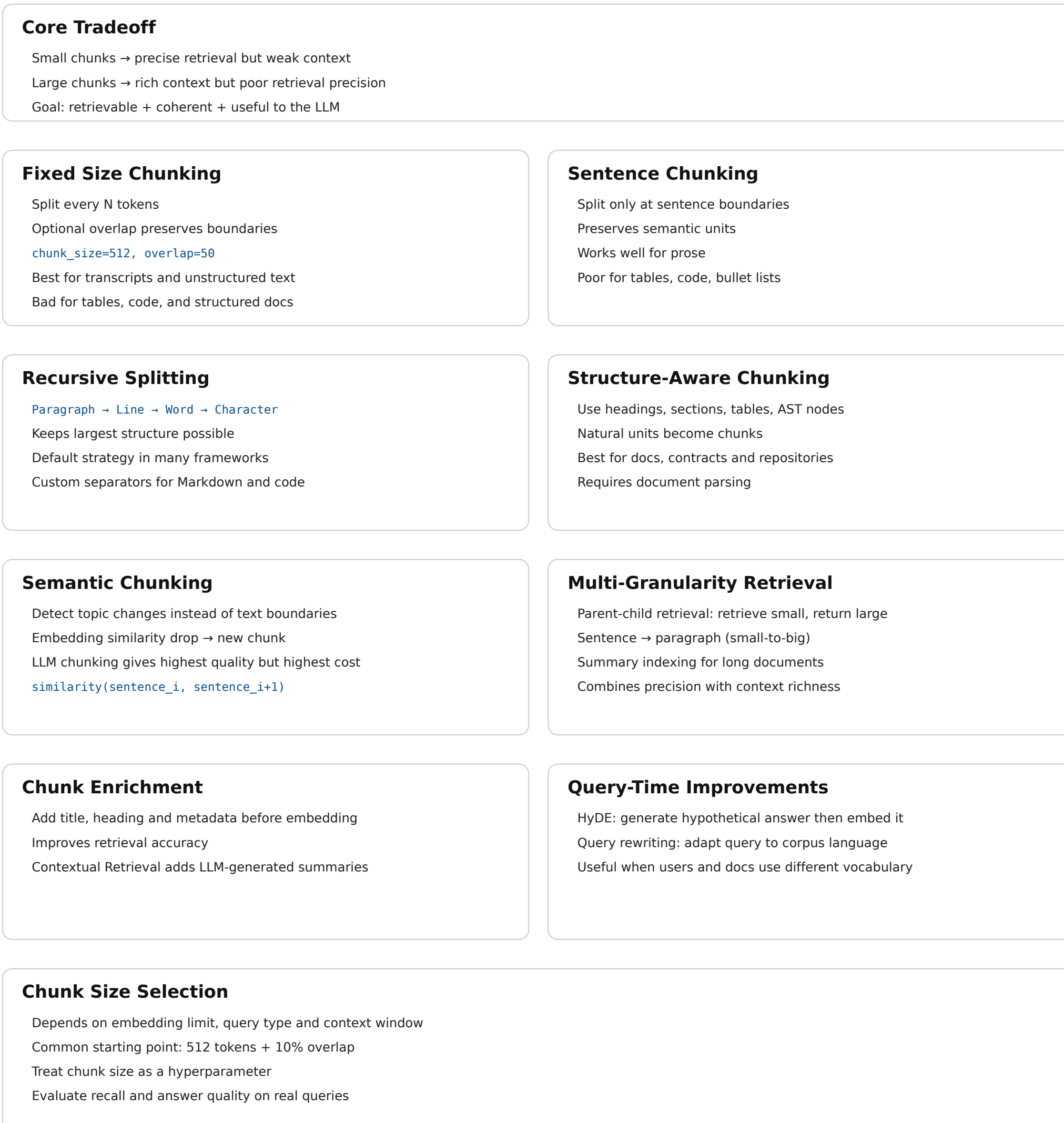


Figure 13.2: Cheat sheet.

13.11 Further reading

- Anthropic (2024). *Introducing Contextual Retrieval.* — The contextual chunk enrichment technique; one of the highest-impact chunking improvements available with documented retrieval quality gains.
- Gao et al. (2023). *Precise Zero-Shot Dense Retrieval without Relevance Labels.* — The HyDE paper; generating

hypothetical documents for retrieval.
- Liu et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts.* — Establishes that chunk position within the context window affects how well LLMs use retrieved content; relevant for context assembly decisions in chapter 24.
- LangChain Documentation. *Text Splitters.* — Comprehensive reference for RecursiveCharacterTextSplitter and other splitters with configuration examples.
- LlamaIndex Documentation. *Node Parsers / Text Splitters.* — Alternative implementation reference with hierarchical chunking and parent-child strategies.

Chapter 14

Embeddings for Search

The canonical question for this chapter: *You have 500,000 chunks from your document corpus. How do you convert them into vectors that make retrieval actually work, choosing the right model, encoding correctly for your domain, and handling the practical realities of embedding at scale?*

Where are we?

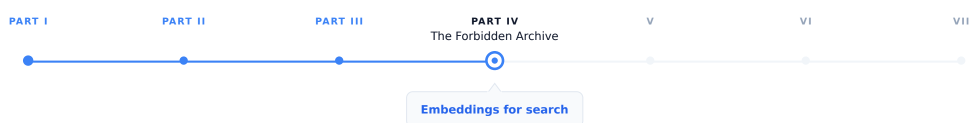


Figure 14.1: The journey through the Model Mind.

Chunking produced a list of text fragments. Now each fragment must become a vector, a point in a high-dimensional space where semantic similarity corresponds to geometric proximity. This chapter covers everything that happens between a text chunk and a vector stored in your index: choosing the right embedding model, encoding queries and documents correctly, managing the operational complexity of embedding at scale, and knowing when your embeddings are failing.

14.1 This chapter vs. chapter 06

Chapter 06 covered embeddings from the perspective of what happens inside a generative LLM: the embedding table, the residual stream, how token representations evolve through transformer layers, and what contextual embeddings are.

This chapter covers embeddings from the perspective of building a retrieval system: choosing and evaluating embedding models for your specific corpus and query distribution, encoding documents and queries correctly, handling domain shift, updating embeddings as documents change, and managing the operational complexity of embedding a large corpus.

The underlying mathematics is the same. The engineering concerns are entirely different.

14.2 The retrieval embedding task

Retrieval embedding is a specific task with requirements that differ from other embedding uses such as classification, clustering, or general semantic similarity.

In retrieval, you have two types of inputs with asymmetric properties:

Queries are short, often incomplete, phrased as questions or keyword phrases, written by users who may not use the same terminology as the documents. A user asking "how long before my session times out" is asking about the same thing as a document that says "the default token expiration is 3600 seconds."

Documents (chunks) are longer, complete, written by authors using precise domain terminology, often containing the answer to many possible queries rather than to one specific query.

A retrieval embedding model must map these two types of inputs which are stylistically and lexically different to nearby vectors when they are semantically related. This is the core challenge, and it is different from general semantic similarity, where both inputs have similar style.

The best retrieval embedding models are trained specifically for this asymmetric task, using query-document pairs rather than symmetric text-text pairs. Models trained only for general semantic similarity perform measurably worse at retrieval even if they produce better representations for other tasks.

14.3 Choosing an embedding model

The embedding model is one of the highest-impact decisions in a RAG system. Switching models requires re-embedding the entire corpus, so the choice is sticky and getting it right matters.

14.3.1 The MTEB leaderboard

The Massive Text Embedding Benchmark (MTEB) is the standard reference for comparing embedding models. It evaluates models across 56 datasets covering 8 task categories including retrieval, clustering, classification, reranking, and semantic similarity.

For RAG systems, the retrieval subcategory of MTEB is most relevant. Key retrieval datasets include BEIR (18 heterogeneous retrieval datasets covering biomedical, financial, scientific, and general-purpose text), TREC (standard IR benchmarks), and MIRACL (multilingual retrieval).

Critical caveat: MTEB scores are averages across many datasets. A model that ranks first on average may perform worse than a fifth-place model on your specific domain. Always evaluate on data representative of your actual use case.

14.3.2 Key selection criteria

Retrieval-specific training. Prefer models trained with query-document contrastive objectives over general sentence encoders. Models explicitly listed as "retrieval" or "embedding search" models in their documentation are trained for this task.

Embedding dimension. Higher dimensions (1,536, 3,072) generally provide better retrieval quality but cost more in storage and computation. For millions of documents, the cost difference is significant. Matryoshka models allow dimension reduction post-hoc which could be preferable for flexibility.

Context window. Must accommodate your chunk size. A model with a 512-token context window cannot embed 1,000-token chunks without truncation. Models with 8,192-token contexts handle longer chunks but may not use the extra context efficiently for all document types.

Language coverage. If your corpus is multilingual, use a multilingual model. English-centric models tokenize non-English text inefficiently and produce degraded embeddings for non-English content. mE5, multilingual-e5-large, and similar models cover 100+ languages.

Inference cost and latency. Embedding 500,000 chunks at $0.0001 per 1,000 tokens costs $50. At $0.00002 per 1,000 tokens, it costs $10. For large corpora embedded frequently, cost compounds. Latency also matters for query-time embedding: if queries must be embedded before retrieval, embedding latency adds directly to TTFT.

Self-hosted vs. API. API-based embedding (OpenAI, Cohere, Voyage) avoids infrastructure management but creates a runtime dependency. Self-hosted models (BGE, GTE, E5 via HuggingFace) require GPU infrastructure but give full control over latency, cost, and data privacy.

14.3.3 Evaluating on your data

The only reliable evaluation is on your own data:

1. Sample 200–500 representative queries from your expected query distribution.
2. For each query, identify the ground-truth relevant chunks.
3. Embed your corpus with each candidate embedding model.
4. For each query, retrieve top-k chunks and measure Recall@k (fraction of ground-truth chunks appearing in the top-k), MRR (Mean Reciprocal Rank), and NDCG (Normalized Discounted Cumulative Gain).
5. Select the model that maximizes these metrics on your evaluation set.

14.4 Asymmetric encoding and instruction prefixes

Many retrieval embedding models require different encoding for queries vs. documents. Using the same encoding for both is a common mistake that silently degrades retrieval quality.

14.4.1 Instruction-based models (E5, Instructor)

E5 and Instructor models prepend a task-specific instruction to each input. The instruction tells the model what type of text it is encoding and what the embedding will be used for:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("intfloat/e5-large-v2")

# Queries: prepend "query: "
queries = ["query: how long before session timeout?"]
query_embeddings = model.encode(queries, normalize_embeddings=True)

# Documents: prepend "passage: "
documents = [
    "passage: The default token expiration is 3600 seconds, "
    "configurable via the token_ttl parameter in the authentication settings."
]
doc_embeddings = model.encode(documents, normalize_embeddings=True)
```

The instruction prefix shifts the model's representation toward the appropriate mode. Query embeddings and passage embeddings are trained to be similar when semantically related, even though they are encoded differently.

14.4.2 Cohere and Voyage asymmetric encoding

Cohere and Voyage AI models expose the asymmetry through explicit `input_type` parameters:

```
import cohere

co = cohere.Client(api_key)

```

```
# Encode queries
query_response = co.embed(
    texts=["how long before session timeout?"],
    model="embed-english-v3.0",
    input_type="search_query",       # ← query encoding
)

# Encode documents
doc_response = co.embed(
    texts=["The default token expiration is 3600 seconds..."],
    model="embed-english-v3.0",
    input_type="search_document",    # ← document encoding
)
```

The `input_type` parameter is not optional and omitting it or using the wrong value produces incorrect embeddings. This is a common source of subtle retrieval bugs where the system appears to work (it returns results) but quality is silently degraded.

14.4.3 OpenAI embeddings

OpenAI's text-embedding-3 models do not require explicit query/document differentiation. They can be used for both with the same API call:

```
from openai import OpenAI

client = OpenAI()

def embed(texts: list[str]) -> list[list[float]]:
    response = client.embeddings.create(
        input=texts,
        model="text-embedding-3-large",
        dimensions=1024,  # MRL truncation
    )
    return [item.embedding for item in response.data]
```

The simplicity of the OpenAI interface makes it easier to use correctly, at the cost of potentially lower retrieval quality compared to models with explicit asymmetric training on specialized retrieval tasks.

14.5 Batching and throughput for corpus embedding

Embedding a large corpus efficiently requires batching. Embedding one document at a time is 10–100× slower than batching due to GPU underutilization and per-request overhead.

14.5.1 Optimal batch size

For GPU-hosted models:

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("BAAI/bge-large-en-v1.5")

def embed_corpus(chunks: list[str], batch_size: int = 256) -> np.ndarray:
```

```
    all_embeddings = []

    for i in range(0, len(chunks), batch_size):
        batch = chunks[i:i + batch_size]
        batch_embeddings = model.encode(
            batch,
            normalize_embeddings=True,
            show_progress_bar=False,
            convert_to_numpy=True,
        )
        all_embeddings.append(batch_embeddings)

    return np.vstack(all_embeddings)
```

For API-based embedding, respect provider limits: OpenAI allows up to 2,048 inputs per request (max 8,191 tokens per input); Cohere allows up to 96 inputs per request. Rate limits are typically 1,000–10,000 RPM depending on tier.

Implement retries with exponential backoff for API calls to handle transient failures, which are unavoidable in distributed systems operating at scale.

```
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=1, max=60)
)
def embed_batch_with_retry(
    texts: list[str], client, model: str
) -> list[list[float]]:
    response = client.embeddings.create(input=texts, model=model)
    return [item.embedding for item in response.data]
```

14.5.2 Incremental embedding

For corpora that change continuously, embedding only new and updated documents is essential and re-embedding the full corpus on each run is prohibitively expensive at scale:

```
import hashlib

def compute_chunk_hash(chunk_text: str) -> str:
    return hashlib.sha256(chunk_text.encode()).hexdigest()

def embed_incremental(
    new_chunks: list[Chunk],
    existing_hashes: set[str],
    embedding_store
) -> int:
    """Embed only chunks not already in the store. Returns count of new chunks."""
    chunks_to_embed = [
        chunk for chunk in new_chunks
        if compute_chunk_hash(chunk.text) not in existing_hashes
    ]

    if not chunks_to_embed:
        return 0
```

```
    texts = [chunk.text for chunk in chunks_to_embed]
    embeddings = embed_corpus(texts)

    for chunk, embedding in zip(chunks_to_embed, embeddings):
        embedding_store.upsert(
            id=chunk.id,
            vector=embedding,
            metadata=chunk.metadata,
            text=chunk.text,
        )

    return len(chunks_to_embed)
```

Content hashing ensures idempotency: the same chunk embedded twice produces one entry, not two.

14.6 Domain adaptation

General-purpose embedding models are trained on diverse web text. For specialized domains (medical literature, legal documents, financial filings, source code) general models often underperform domain-specific alternatives.

14.6.1 When domain adaptation is needed

Signs that general models are insufficient: retrieval recall on domain-specific evaluation sets is significantly lower than on general benchmarks; queries use domain terminology that general models do not represent well ("nociceptor," "collateralized debt obligation," "amortization schedule"); documents use domain conventions (citation formats, abbreviations, structured fields) that general models have not seen frequently.

14.6.2 Domain-specific pre-trained models

Several domains have dedicated embedding models: BioMedBERT, PubMedBERT, and BioBERT for biomedical text; Legal-BERT for legal documents; FinBERT for financial text; CodeBERT and StarEncoder for source code. These models start from domain-specific pretraining, meaning their tokenizer and base representations are calibrated for domain vocabulary.

For retrieval specifically, domain-pretrained models still need retrieval fine-tuning and pretraining alone is not sufficient for strong asymmetric retrieval performance.

14.6.3 Fine-tuning an embedding model

If no suitable domain model exists, or if retrieval quality remains insufficient after trying domain models, fine-tune a general embedding model on domain- specific query-document pairs.

Training data: pairs of (query, relevant_document_chunk) where the query is representative of real user questions and the document chunk contains the correct answer. Sources for training pairs include human annotation (most reliable, expensive), synthetic generation (use an LLM to generate queries for each chunk for example: "write 3 questions this passage answers" scales easily and works well), and mined pairs (Stack Overflow questions and accepted answers, FAQ pages, support tickets with resolution notes).

```
from sentence_transformers import SentenceTransformer, losses
from sentence_transformers.training_args import SentenceTransformerTrainingArguments
from datasets import Dataset

train_data = Dataset.from_dict({
```

```
    "anchor": ["how long before session timeout?", ...],          # queries
    "positive": ["The default token expiration is 3600...", ...], # relevant chunks
})

model = SentenceTransformer("BAAI/bge-large-en-v1.5")

train_loss = losses.MultipleNegativesRankingLoss(model)

training_args = SentenceTransformerTrainingArguments(
    output_dir="./fine-tuned-retrieval-model",
    num_train_epochs=3,
    per_device_train_batch_size=32,
    learning_rate=2e-5,
    warmup_ratio=0.1,
)

model.fit(
    train_objectives=[(train_data, train_loss)],
    args=training_args,
)
```

Fine-tuned models consistently outperform general models on domain-specific retrieval by 5–20% on recall metrics. The investment (data collection plus training) pays off quickly for high-value retrieval applications.

14.7 Embedding stability and versioning

Embeddings are not stable across model versions, as a result updating a model changes the vector representation of the same input text. A corpus embedded with `text-embedding-ada-002` is not compatible with `text-embedding-3-large`: they live in different vector spaces, and similarity scores between the two are meaningless.

14.7.1 The model update problem

If you embed your corpus with model version V1 and switch to V2: - Similarity scores between V1 document embeddings and V2 query embeddings are garbage - Retrieval quality can degrade silently: results are still returned, but they may be incorrect without any errors being raised. - The only fix is to re-embed the entire corpus with V2

This creates a strong operational coupling between your corpus embeddings and your query encoder. They must always use the same model version.

14.7.2 Versioning strategy

Track the embedding model version with every stored embedding:

```
{
    "chunk_id": "chunk-abc123",
    "text": "The default token expiration is 3600 seconds...",
    "embedding": [0.23, -0.87, ...],
    "embedding_model": "text-embedding-3-large",
    "embedding_model_version": "2024-02-01",
    "embedding_dimensions": 1024,
    "embedded_at": "2024-03-15T10:30:00Z",
}
```

When switching embedding models, you can either perform a big-bang migration by re-embedding the entire corpus with the new model (simpler but requiring downtime or a parallel index), or use a dual-index approach that maintains both old and new indices, routes new documents to the new index, and gradually migrates data while merging results at query time; this enables zero downtime but adds significant complexity.

14.8 Embedding quality signals

How do you know if your embeddings are working without running full end-to-end evaluation?

14.8.1 Nearest-neighbor inspection

Sample 50 chunks from your corpus. For each, find the 5 nearest neighbors in embedding space. If the neighbors are semantically related to the sample chunk, embeddings are working correctly. If neighbors are unrelated embeddings are failing.

```python
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def inspect_nearest_neighbors(
    sample_chunks: list[str],
    all_chunks: list[str],
    all_embeddings: np.ndarray,
    embedding_model,
    k: int = 5
):
    sample_embeddings = embedding_model.encode(
        sample_chunks, normalize_embeddings=True
    )

    for chunk, embedding in zip(sample_chunks, sample_embeddings):
        similarities = cosine_similarity([embedding], all_embeddings)[0]
        top_k_indices = np.argsort(similarities)[::-1][1:k+1]  # exclude self

        print(f"\nQuery chunk: {chunk[:100]}...")
        print("Nearest neighbors:")
        for idx in top_k_indices:
            print(f"  [{similarities[idx]:.3f}] {all_chunks[idx][:80]}...")
```

14.8.2 Embedding space health check

```python
def check_embedding_health(embeddings: np.ndarray):
    # Norm statistics - should be near 1.0 for normalized embeddings
    norms = np.linalg.norm(embeddings, axis=1)
    print(f"Norm stats: mean={norms.mean():.3f}, std={norms.std():.3f}")

    # Pairwise similarity - high average indicates embedding collapse
    sample_indices = np.random.choice(
        len(embeddings), min(1000, len(embeddings)), replace=False
    )
    sample = embeddings[sample_indices]
    pairwise_sims = cosine_similarity(sample)
```

```python
    mask = ~np.eye(len(sample), dtype=bool)
    avg_sim = pairwise_sims[mask].mean()
    print(f"Average pairwise similarity: {avg_sim:.3f}")
    print("(< 0.1 is healthy, > 0.3 suggests potential collapse)")
```

High average pairwise similarity (> 0.3) suggests embedding collapse. When all chunks are being mapped to similar vectors it means retrieval failed. This can happen with domain shift (the model has not seen your document type), poor text quality (chunks too short or too noisy), or model configuration issues such as wrong prefix format.

14.8.3 Query-document alignment evaluation

For a sample of known query-document pairs, compute the rank of the correct document:

```python
def evaluate_retrieval(
    query_doc_pairs: list[tuple[str, str]],
    all_chunks: list[str],
    all_embeddings: np.ndarray,
    embedding_model
) -> dict:
    queries = [pair[0] for pair in query_doc_pairs]
    correct_chunks = [pair[1] for pair in query_doc_pairs]

    query_embeddings = embedding_model.encode(
        [f"query: {q}" for q in queries],
        normalize_embeddings=True
    )

    ranks = []
    for q_emb, correct in zip(query_embeddings, correct_chunks):
        similarities = cosine_similarity([q_emb], all_embeddings)[0]
        sorted_chunks = [all_chunks[i] for i in np.argsort(similarities)[::-1]]
        rank = (
            sorted_chunks.index(correct) + 1
            if correct in sorted_chunks
            else len(all_chunks)
        )
        ranks.append(rank)

    return {
        "recall@1":  sum(r <= 1  for r in ranks) / len(ranks),
        "recall@5":  sum(r <= 5  for r in ranks) / len(ranks),
        "recall@10": sum(r <= 10 for r in ranks) / len(ranks),
        "mrr":       sum(1/r for r in ranks) / len(ranks),
        "median_rank": np.median(ranks),
    }
```

Run this evaluation before deploying to production, and re-run whenever the embedding model or corpus changes. Recall@5 above 0.8 is a reasonable bar for most retrieval tasks and when it is below 0.6, investigate chunking and model choice before blaming retrieval algorithms.

14.9 Late interaction models: ColBERT

Standard embedding models produce one vector per document, and retrieval is a single vector comparison. Late interaction models like ColBERT produce one vector per token, and retrieval involves matching token-level representations.

14.9.1 How ColBERT works

ColBERT encodes both the query and the document into sequences of token-level embeddings (not pooled to a single vector). Similarity is computed via MaxSim: for each query token, find its most similar document token; sum these maximum similarities across all query tokens.

```
Query: ["how", "long", "session", "timeout"]
  → 4 query token embeddings: [q1, q2, q3, q4]

Document: ["default", "token", "expiration", "3600", "seconds"]
  → 5 doc token embeddings: [d1, d2, d3, d4, d5]

MaxSim score = max_sim(q1, {d1..d5}) + max_sim(q2, {d1..d5})
             + max_sim(q3, {d1..d5}) + max_sim(q4, {d1..d5})
```

This allows fine-grained matching: "session timeout" in the query matches "token expiration" in the document because the token-level embeddings for "session" and "token" are similar in context, and "timeout" and "expiration" are similar. Standard single-vector models must capture all of this in one pooled vector, which is a lossy compression.

14.9.2 ColBERT tradeoffs

Better retrieval quality. ColBERT consistently outperforms single-vector models on retrieval benchmarks, often by significant margins on tasks requiring multi-token matching.

Higher storage cost. Instead of one 1,024-dimensional vector per chunk, ColBERT stores N vectors (one per token). A 200-token chunk requires 200 × 128-dimensional vectors.

More complex retrieval. MaxSim requires comparing all query token vectors against all document token vectors. In practice, ColBERT uses approximate nearest-neighbor retrieval to find candidate documents, then applies MaxSim for precise re-ranking.

14.10 Key takeaways

- Retrieval embedding is an asymmetric task — queries and documents have different styles and vocabulary; use models trained specifically for retrieval, not general semantic similarity models
- Always evaluate embedding models on your own domain data before committing; MTEB rankings reflect average performance across many datasets, not performance on your specific corpus
- Asymmetric encoding is required for models like E5, Instructor, Cohere, and Voyage: queries and documents must be encoded differently; using the wrong prefix or input_type silently degrades retrieval quality with no error thrown
- Batch embedding efficiently using appropriate batch sizes; implement retry with exponential backoff for API-based embedding; use content hashing for incremental updates to avoid re-embedding unchanged chunks
- Domain-specific models and fine-tuning on domain query-document pairs improve retrieval recall by 5–20% for specialized corpora; synthetic query generation from chunks is an effective and scalable way to create training data
- Embedding model versions are incompatible: switching models requires re-embedding the entire corpus; track model version with every stored embedding and plan migrations carefully

- Inspect embedding quality before deploying: nearest-neighbor inspection, pairwise similarity health checks, and known query-document pair evaluation catch failures that would otherwise appear as degraded answers in production
- ColBERT's token-level late interaction achieves higher retrieval quality than single-vector models at the cost of significantly more storage and retrieval complexity — worth it when single-vector recall is insufficient

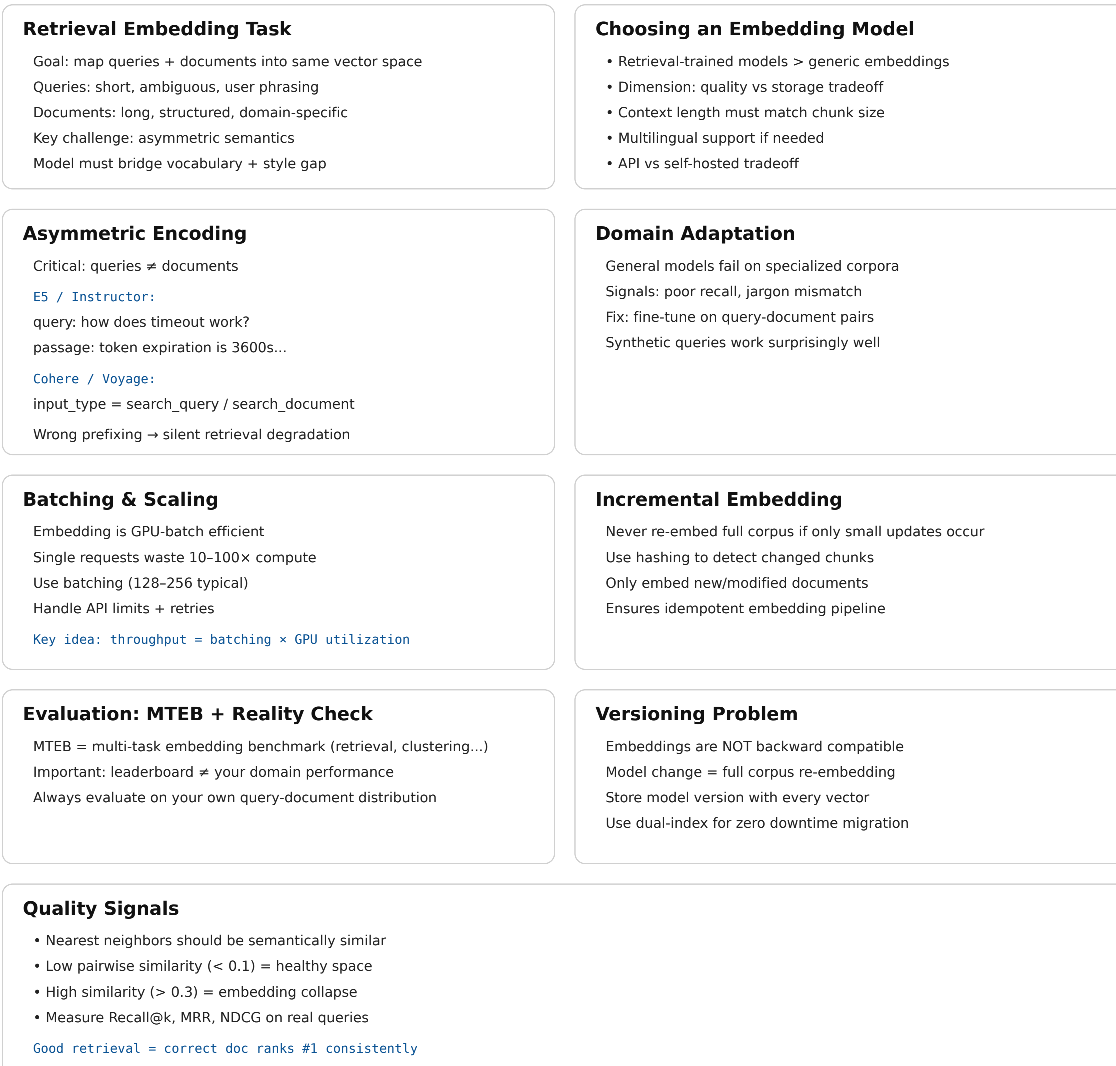


Figure 14.2: Cheat sheet.

14.11 Further reading

- Karpukhin et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering.* — The foundational dense retrieval paper establishing the query-document asymmetric training paradigm.
- Khattab & Zaharia (2020). *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT.* — Token-level late interaction retrieval.

- Wang et al. (2022). *Text Embeddings by Weakly-Supervised Contrastive Pre-training.* — E5; the instruction-prefix asymmetric encoding approach.
- Muennighoff et al. (2023). *MTEB: Massive Text Embedding Benchmark.* — The benchmark; essential reference for comparing embedding models.

Chapter 15

Vector Databases

The canonical question for this chapter: *You have 10 million embeddings and a query arrives. You need the 10 most similar vectors in under 50 milliseconds how does that actually work and which system should you use?*

Where are we?

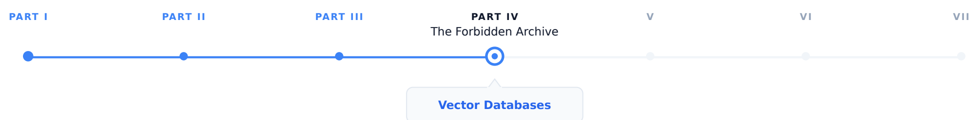


Figure 15.1: The journey through the Model Mind.

Embeddings converted your chunks into vectors. Now those vectors need a home, a system that stores them durably, indexes them for fast search, and returns the most similar ones for any query in interactive latency. This chapter covers how that works algorithmically and which systems implement it, and how to choose between them.

15.1 What a vector database does

A vector database stores high-dimensional vectors and answers nearest-neighbor queries: given a query vector, return the k vectors in the database most similar to it, along with their associated metadata and document text.

Although conceptually simple, efficient vector search at scale is far from trivial. Exact nearest-neighbor search over 10 million 1,536-dimensional vectors requires computing 10 million dot products per query. At 1,536 dimensions and float32 precision, each dot product requires 1,536 multiplications and 1,535 additions. For 10 million vectors, that is 30 billion floating-point operations per query. On a CPU doing 100 GFLOPS, that is 300 milliseconds too slow for an interactive system.

The solution is approximate nearest neighbor (ANN) search: algorithms that find vectors very likely to be among the true top-k, fast enough for interactive latency, with a small probability of missing some true nearest neighbors. The accuracy/speed tradeoff is the central engineering parameter of every vector search system.

Beyond search, a vector database also handles persistent storage of vectors and metadata, metadata filtering (retrieve only vectors where `source == "legal_docs"`), CRUD operations, consistency and durability guarantees, horizontal scaling as the corpus grows, and access control.

Choosing the right system requires understanding how ANN algorithms work, what the operational tradeoffs are, and what your actual requirements are.

15.2 How ANN Search Works

ANN search is not one algorithm but a family of approaches with different tradeoffs between build time, query latency, memory footprint, and recall. Production vector databases typically implement several of these and let you choose (or choose automatically based on your data). Understanding the whole family, is what lets you diagnose why a system is slow, why recall is lower than expected, or which index type to reach for at a given scale.

15.2.1 The exact search baseline

Before approximation makes sense, it helps to understand what you are approximating away from. Exact nearest-neighbor search is brute-force: compute the similarity between the query vector and every stored vector, sort by similarity, return top-k. This is $O(n \times d)$ per query where n is the number of vectors and d is the embedding dimension.

```python
import numpy as np

def exact_search(
    query_embedding: np.ndarray,
    corpus_embeddings: np.ndarray,
    k: int = 10
) -> tuple[list[int], list[float]]:
    # Assumes embeddings are normalized (unit vectors)
    # dot product = cosine similarity for normalized vectors
    similarities = corpus_embeddings @ query_embedding
    top_k_indices = np.argsort(similarities)[::-1][:k]
    top_k_scores = similarities[top_k_indices]
    return top_k_indices.tolist(), top_k_scores.tolist()
```

For small corpora (under ~100,000 vectors), exact search is fast enough. FAISS's flat index, NumPy's matrix multiplication, and pgvector's exact search all use this approach. It is the right choice when correctness matters more than scale yet for larger corpora, you need ANN.

The approximate algorithms fall into four families based on how they narrow the search space: graph-based, cluster-based (partitioning), hashing-based, and tree-based. Each makes a different bet about the structure of your data.

15.2.2 Graph-based methods

Graph-based ANN builds a proximity graph over the vectors (nodes are vectors, edges connect "nearby" vectors) and answers queries by greedily walking the graph toward the query.

HNSW (Hierarchical Navigable Small World). The most widely used ANN algorithm in production vector databases. Weaviate, Qdrant, Milvus, and pgvector all offer it as their primary index type.

The core idea: build a layered graph where each node is a vector and edges connect nearby vectors. Higher layers have fewer nodes but longer-range connections; lower layers have all nodes with short-range connections. Search starts at the top layer (coarse navigation) and descends to lower layers (fine-grained search).

```
Layer 2 (sparse):    o --------- o --------- o
                            ↓ descend
Layer 1 (medium):    o -- o -- o -- o -- o -- o
```

```
                          ↓ descend
Layer 0 (dense):    o-o-o-o-o-o-o-o-o-o-o-o-o
```

Search starts at the entry point at the top layer. Greedily navigate to the nearest neighbor. Descend to the next layer. Repeat until layer 0. At layer 0, perform a beam search over the local neighborhood. Return the top-k candidates found. This gives O(log n) expected search time for well-distributed data, compared to O(n) for exact search.

Key HNSW parameters:

`M` (max connections per node) — higher M means better recall but more memory and slower build time. Typical values: 16–64.

`ef_construction` (beam width during index build) — higher values mean a higher-quality index at the cost of longer build time. Typical values: 100–400.

`ef` (beam width during search, also called `ef_search`) — the primary recall/speed tradeoff lever, adjustable at query time without rebuilding the index. Higher ef means more nodes examined per query, better recall, higher latency. Typical values: 50–500.

Representative recall-latency numbers at different `ef` settings (always benchmark on your own data):

ef	Recall@10	Latency (p99)
50	0.92	8 ms
100	0.97	15 ms
200	0.99	28 ms
500	0.999	65 ms

NSG (Navigating Spreading-out Graph). Predates and influenced HNSW's practical adoption. Builds a single-layer graph (no hierarchy) but constructs edges more carefully, starting from a k-NN graph and pruning edges to minimize graph diameter while preserving navigability. NSG indexes are smaller than HNSW for comparable recall because there is no multi-layer overhead, but build time is longer and the algorithm is less forgiving of insertions after the initial build. Used in some high-performance research systems more than in mainstream managed databases.

Vamana / DiskANN. Developed by Microsoft Research specifically for billion-scale search where the full index cannot fit in RAM. Vamana builds a single flat graph (like NSG) but is explicitly designed so the graph can live on SSD: each node's neighbor list is small and the graph has low diameter, so a query touches only a small number of disk pages. DiskANN (the system built on Vamana) achieves HNSW-competitive recall while indexing billions of vectors on a single machine with far less RAM than an all-in-memory HNSW index would require, at the cost of higher per-query latency (disk I/O instead of RAM access). Milvus and Qdrant both support disk-resident indexes derived from this line of work as an option for corpora too large to hold in memory affordably.

Why graph methods dominate mainstream use. They give the best recall per unit of query latency for in-memory datasets, they support incremental insertion reasonably well (important for RAG corpora that grow continuously), and their query-time recall/speed tradeoff is tunable without rebuilding the index (the `ef` parameter). The cost is build time and memory: graphs store explicit edge lists, and construction requires many nearest-neighbor computations per inserted vector.

15.2.3 Cluster-based / partitioning methods

Rather than building a graph, these methods partition the vector space into regions and restrict search to the regions closest to the query.

IVF (Inverted File Index). FAISS's primary index for large-scale search. IVF clusters vectors into groups (Voronoi cells) using k-means. Each cluster has a centroid, and at search time, the query is compared to all centroids, and only the closest `nprobe` clusters are searched exactly.

```
Preprocessing:
  cluster all vectors into k clusters using k-means
  for each cluster, store the centroid and list of member vectors

Search:
  1. Compare query to all k centroids → find nprobe nearest centroids
  2. Search all vectors in those nprobe clusters exactly
  3. Return top-k from the searched vectors
```

Key parameters: `nlist` (number of clusters, typically `sqrt(n)` to `n/10`); `nprobe` (clusters searched per query, the recall/speed lever). IVF is faster than HNSW for very large datasets (100M+ vectors) because its flat cluster structure maps better to SIMD-vectorized exact search within clusters. HNSW's graph traversal has pointer-chasing that is harder to vectorize. The weakness is recall at cell boundaries: a true nearest neighbor sitting just across a cell boundary from the query can be missed unless `nprobe` is large enough to cover neighboring cells.

Product Quantization (PQ). A compression technique layered on top of IVF (or HNSW) that is important enough to understand on its own. PQ reduces vector storage by dividing each d-dimensional vector into m sub-vectors of d/m dimensions each, and quantizing each sub-vector to one of k centroids (typically 256). The stored representation is m byte values rather than d × 4 bytes.

```
Original:          1024 floats × 4 bytes = 4,096 bytes per vector
PQ(m=64, k=256):   64 bytes per vector   → 64× compression
```

The tradeoff: approximate distances from centroid reconstruction introduce error. PQ is used when memory is the primary constraint and some recall loss is acceptable.

IVF-PQ and IVF-ADC. IVF combined with product quantization for the vectors stored within each cell. This is FAISS's standard configuration for billion-scale search: coarse quantization (IVF) narrows the candidate set, product quantization compresses storage and speeds up distance computation within candidates. ADC (Asymmetric Distance Computation) keeps the query vector unquantized while comparing against quantized database vectors, improving accuracy over quantizing both sides. HNSW + PQ combines HNSW's recall with PQ's memory efficiency and is similarly available in FAISS.

ScaNN (Scalable Nearest Neighbors). Google's production ANN system, notable for a specific insight: not all quantization error matters equally. Standard PQ minimizes reconstruction error uniformly across all vectors, but what actually matters for retrieval is preserving the *relative ordering* of distances near the query. ScaNN uses anisotropic vector quantization, which weights quantization error along the direction that matters most for maximum-inner-product ranking. It consistently tops ANN-Benchmarks leaderboards for recall at a given queries-per-second budget. Available as an open-source library; used internally at Google and adopted by some vector database backends.

When partitioning methods win. At very large scale (hundreds of millions to billions of vectors) where memory is the binding constraint and where insert-heavy workloads make graph maintenance expensive, cluster-based indexes with compression (IVF-PQ, ScaNN) typically beat graph-based indexes on cost per query at a given recall target.

15.2.4 Hashing-based methods

LSH (Locality-Sensitive Hashing). The oldest ANN approach still in use. The idea: design a family of hash functions such that nearby vectors are more likely to collide (hash to the same bucket) than distant vectors. At query time, hash the query, look only at vectors in the same bucket (or nearby buckets via multiple hash tables), and rank by exact distance within that small candidate set.

```
Random hyperplane LSH for cosine similarity:
  Generate k random hyperplanes through the origin
  hash(v) = sign(v · h_1), sign(v · h_2), ..., sign(v · h_k)
  → a k-bit binary code per vector

Vectors with the same (or very similar) k-bit code are likely to be close
```

```
in cosine similarity, the more hyperplanes two vectors agree on, the
smaller the angle between them is likely to be.
```

LSH has clean theoretical guarantees (bounds on the probability of missing a true near neighbor) and very cheap hash computation. In practice it has mostly been superseded by graph and cluster methods for text embedding retrieval: LSH generally needs many hash tables to reach the recall that HNSW achieves with a single graph, which inflates memory and query cost. LSH remains relevant for specific settings in very high-dimensional sparse data, streaming settings where index rebuild cost must be near zero, and some specialized deduplication and fingerprinting tasks (near-duplicate detection, copyright matching) where its collision guarantees are the actual point, not just a means to fast search.

15.2.5 Tree-based methods

Annoy (Approximate Nearest Neighbors Oh Yeah). Spotify's ANN library. Builds a forest of random projection trees: each tree recursively splits the vector space with random hyperplanes, and a query descends each tree to a leaf, collecting candidates from the leaves it lands in across all trees.

Annoy's distinguishing feature is memory-mapped, read-only indexes: once built, an Annoy index can be mmap'd and shared across processes with no per-process memory duplication, and it degrades gracefully under memory pressure since the OS pages it in as needed. This makes it a good fit for recommendation-style workloads with large static catalogs and many concurrent readers. Its weaknesses relative to HNSW: no incremental insertion (the index is built once and is immutable, updates require a full rebuild) and generally lower recall per unit of query time on standard ANN benchmarks.

15.2.6 Comparing the families

Family	Example	Best at	Weak at
Graph	HNSW, NSG, Vamana	Recall/latency for in-memory data; incremental inserts	Memory overhead; build time
Partition	IVF, IVF-PQ, ScaNN	Very large scale; compressed storage; cost per query	Boundary recall; tuning nprobe
Hashing	LSH	Theoretical guarantees; near-duplicate detection; near-zero rebuild cost	Recall per byte vs. graph/partition methods
Tree	Annoy	Read-heavy, static, memory-shared workloads	No incremental updates; lower recall at scale

For text embedding retrieval in RAG systems specifically graph-based HNSW is the default choice up to tens of millions of vectors, and disk-resident graph methods (DiskANN-derived) or IVF-PQ/ScaNN are the choice beyond that, when the index no longer fits affordably in RAM. LSH and tree methods are rarely the right choice for text retrieval today but are worth recognizing when you encounter them in older systems or adjacent problem domains (deduplication, recommendation).

15.3 Vector database landscape

The major production vector databases differ in architecture, operational model, and capability. Choosing the right one depends on corpus size, operational requirements, and existing infrastructure.

15.3.1 Pinecone

Fully managed, serverless vector database. Pinecone handles all infrastructure, you store vectors and query them via API with no server management.

```
from pinecone import Pinecone

pc = Pinecone(api_key="your-api-key")
index = pc.Index("your-index-name")

# Upsert vectors
index.upsert(vectors=[
    {
        "id": "chunk-abc123",
        "values": embedding,
        "metadata": {
            "text": "The default token expiration is 3600 seconds...",
            "source": "api-reference",
            "page": 42,
        }
    }
])

# Query
results = index.query(
    vector=query_embedding,
    top_k=10,
    include_metadata=True,
    filter={"source": {"$eq": "api-reference"}}
)
```

Strengths: zero operational overhead, automatic scaling, built-in metadata filtering, strong consistency. **Weaknesses:** cost at scale (per vector stored × queries per month), vendor lock-in, no self-hosting option, limited control over index parameters.

15.3.2 Weaviate

Open-source, self-hostable, also available as a managed cloud service. Module-based architecture allows integrating embedding models, rerankers, and generative models directly into the database.

```
import weaviate
import weaviate.classes.config as wc

client = weaviate.connect_to_local()

collection = client.collections.create(
    name="DocumentChunk",
    vectorizer_config=wc.Configure.Vectorizer.text2vec_openai(),
    properties=[
        wc.Property(name="text",   data_type=wc.DataType.TEXT),
        wc.Property(name="source", data_type=wc.DataType.TEXT),
    ]
)

# Hybrid search (vector + BM25) - Weaviate's primary differentiator
results = collection.query.hybrid(
```

```
    query="session timeout default",
    limit=10,
    alpha=0.7,  # 1.0 = pure vector, 0.0 = pure BM25
)
```

Strengths: built-in hybrid search, integrated reranking, active development, self-hostable. **Weaknesses:** complex schema management, operational overhead for self-hosted.

15.3.3 Qdrant

Open-source, designed for high-performance filtered vector search. Written in Rust for low latency and memory efficiency. Supports complex payload-based filtering with high performance even under heavy filter conditions.

```
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    Filter, FieldCondition, MatchValue
)

client = QdrantClient(url="http://localhost:6333")

client.create_collection(
    collection_name="document_chunks",
    vectors_config=VectorParams(size=1024, distance=Distance.COSINE),
)

client.upsert(
    collection_name="document_chunks",
    points=[
        PointStruct(
            id="chunk-abc123",
            vector=embedding,
            payload={
                "text": "The default token expiration is 3600 seconds...",
                "source": "api-reference",
                "access_level": "internal",
            }
        )
    ]
)

results = client.search(
    collection_name="document_chunks",
    query_vector=query_embedding,
    query_filter=Filter(
        must=[FieldCondition(
            key="access_level",
            match=MatchValue(value="internal")
        )]
    ),
    limit=10,
)
```

Strengths: excellent filtered search performance, Rust performance, memory efficiency, native multi-vector support for ColBERT. **Weaknesses:** smaller ecosystem than Pinecone or Weaviate.

15.3.4 Milvus / Zilliz

Milvus is an open-source vector database designed for large-scale deployments (billions of vectors). Zilliz Cloud is the managed version.

```
from pymilvus import MilvusClient

client = MilvusClient(uri="http://localhost:19530")

client.create_collection(
    collection_name="document_chunks",
    dimension=1024,
)

client.insert(
    collection_name="document_chunks",
    data=[{
        "id": 1,
        "vector": embedding,
        "text": "The default token expiration is 3600 seconds...",
        "source": "api-reference",
    }]
)

results = client.search(
    collection_name="document_chunks",
    data=[query_embedding],
    limit=10,
    output_fields=["text", "source"],
)
```

Strengths: scales to billions of vectors, GPU acceleration support, rich index options (HNSW, IVF, DiskANN). **Weaknesses:** complex deployment (multiple components), significant operational overhead.

15.3.5 pgvector

PostgreSQL extension that adds vector storage and search to an existing Postgres database. Supports both exact and HNSW search.

```
CREATE EXTENSION vector;

CREATE TABLE document_chunks (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    text        TEXT NOT NULL,
    source      TEXT,
    embedding   vector(1024),
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- HNSW index
CREATE INDEX ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Filtered vector search in standard SQL
SELECT id, text, source,
```

```
       1 - (embedding <=> $1::vector) AS similarity
FROM document_chunks
WHERE source = 'api-reference'
ORDER BY embedding <=> $1::vector
LIMIT 10;
```

Strengths: no new infrastructure if you already run Postgres, full SQL expressiveness for filtering, ACID transactions, familiar operational model, transactional consistency between vector and non-vector data. **Weaknesses:** performance at very large scale (tens of millions of vectors) lags behind dedicated systems; HNSW index takes significant memory.

15.3.6 ChromaDB

Lightweight, embedded vector database for prototyping and small-scale applications.

```python
import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.create_collection("document_chunks")

collection.add(
    ids=["chunk-abc123"],
    embeddings=[embedding],
    documents=["The default token expiration is 3600 seconds..."],
    metadatas=[{"source": "api-reference"}]
)

results = collection.query(
    query_embeddings=[query_embedding],
    n_results=10,
    where={"source": "api-reference"},
)
```

Strengths: zero infrastructure, easy to get started, good for development and prototyping. **Weaknesses:** limited production scalability, no operational features of dedicated systems.

15.4 Metadata filtering

Most RAG applications need to restrict retrieval to a subset of the corpus. A user in the legal department should retrieve from legal documents; a query about 2024 regulations should not retrieve 2018 regulations. Metadata filtering restricts the search space to vectors matching a filter condition before or during similarity search.

15.4.1 Pre-filtering vs. post-filtering

Pre-filtering applies the filter first, then searches only among matching vectors:

```
Filter: source = "api-reference" → 50,000 vectors match
Search: find top-10 nearest among those 50,000
```

Accurate but can be slow if the filter does not reduce the search space significantly, or if the index cannot efficiently restrict to filtered vectors.

Post-filtering searches first, then filters results:

```
Search: find top-100 nearest among all 1,000,000 vectors
Filter: keep only results where source = "api-reference" → 8 results
```

Fast for the search step (we are using ANN, not bruteforce!) but may return fewer than k results after filtering. Inflating the top-k (search for 1,000, filter to 10) compensates but increases search cost.

Hybrid filtering is the most efficient approach: use the filter to restrict the index segments searched (coarse pre-filtering) and apply exact filter checking to candidates from the restricted search. Qdrant's payload index, Weaviate's inverted index, and pgvector's SQL WHERE clauses all enable this.

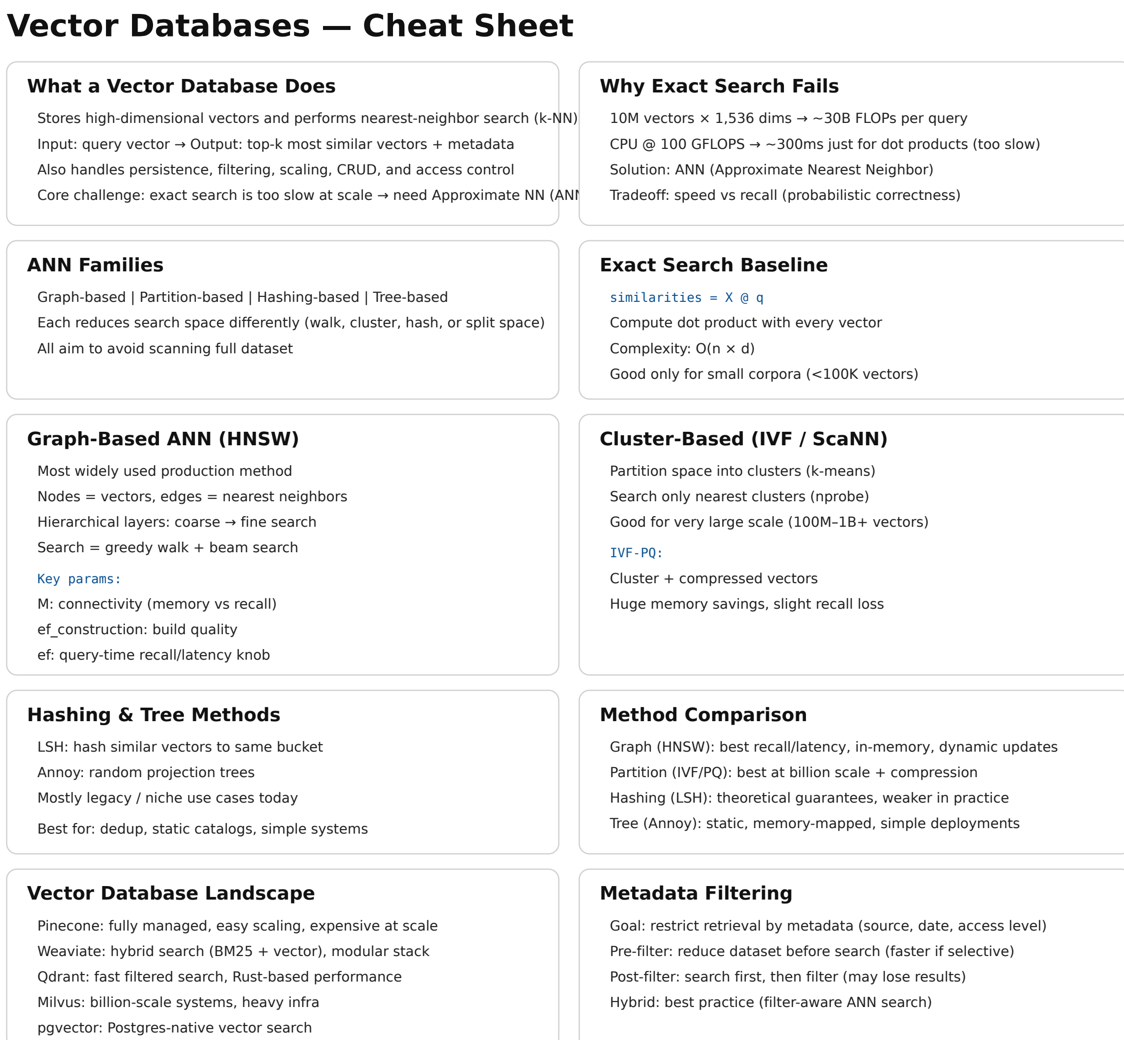


Figure 15.2: Cheat sheet.

15.5 Key takeaways

- Exact nearest-neighbor search is O(n × d) per query — too slow for interactive systems at scale; ANN algorithms (HNSW, IVF) trade a small recall loss for orders-of-magnitude speedup
- HNSW is the dominant ANN algorithm in production: layered graph structure gives O(log n) search; the `ef` parameter adjusts the recall/latency tradeoff at query time without index rebuild
- Product quantization compresses vectors by 4–64× at the cost of approximate distances; use when memory is the constraint and some recall loss is acceptable

- Choosing between vector databases depends on corpus size, operational model (managed vs. self-hosted), filtering complexity, and existing infrastructure
- pgvector is the right choice when you already run Postgres and your corpus is under ~5M vectors; dedicated systems (Qdrant, Weaviate, Pinecone) are better at scale
- Metadata filtering restricts search to relevant subsets; pre-filtering is accurate, post-filtering is fast, hybrid filtering combines the benefits; index the fields you filter on
- Multi-tenancy isolation is safest with namespaces or collections per tenant; metadata-based isolation relies on application-layer enforcement and is risky for strict data isolation requirements
- Store raw vectors durably in object storage as the canonical source of truth; the vector index is a derived artifact that can be rebuilt
- Hybrid retrieval (vector + BM25) consistently outperforms pure vector search for production RAG; prefer systems with native hybrid search support

---

15.6 Further reading

- Malkov & Yashunin (2018). *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.* — The original HNSW paper; the foundational algorithm in most production vector databases.
- Johnson et al. (2019). *Billion-Scale Similarity Search with GPUs.* — The FAISS paper; foundational for large-scale ANN and IVF.
- Jégou et al. (2011). *Product Quantization for Nearest Neighbor Search.* — The PQ paper; essential for understanding compressed vector search.
- Aguerrebere et al. (2023). *Similarity Search in the Blink of an Eye with Compressed Indices.* — ScaNN; Google's production ANN system with strong benchmark performance.
- Douze et al. (2024). *The FAISS Library.* — Comprehensive FAISS documentation and benchmarks across index types.

Chapter 16

Retrieval Algorithms

The canonical question for this chapter: *Given a user query and a corpus of embedded chunks, what algorithms actually find the right chunks, and why is vector similarity alone not enough?*

Where are we?

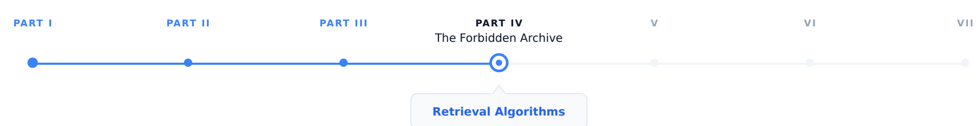


Figure 16.1: The journey through the Model Mind.

The vector database is now in ready, and the queries are coming to be processed. This chapter explores the algorithms that transform a natural language query into a ranked list of relevant chunks, and explains why combining multiple retrieval methods consistently delivers better results than relying on any single approach. In a Retrieval-Augmented Generation (RAG) system, the retrieval stage has the greatest impact on answer quality, often more than the choice of language model or prompt design.

16.1 The retrieval problem

Retrieval is deceptively simple to describe: given a query, return the most relevant chunks. In practice, no single algorithm reliably finds the right chunks across all query types, corpus structures, and domain vocabularies.

Vector similarity search finds semantically related content but fails on exact string matches, rare terms, and out-of-distribution vocabulary. Keyword search (BM25) finds exact matches but fails when the query and document use different words for the same concept. Re-ranking models produce better relevance signals than either but are too slow to run over the full corpus. Each algorithm has failure modes the others compensate for.

Production RAG systems that work reliably across diverse query types combine multiple retrieval algorithms, using each where it performs best. This chapter covers each algorithm in depth, then covers how to combine them.

16.2 Dense retrieval: vector similarity search

Dense retrieval embeds the query and retrieves chunks whose embeddings are nearest to the query embedding in vector space.

16.2.1 What dense retrieval is good at

Semantic matching. The query “how long before my session expires” retrieves the chunk “the default token expiration is 3600 seconds” because the embeddings are nearby even though no words overlap.

Paraphrase matching. The query “terminate an employee” retrieves documents about “offboarding” and “separation from the company”, different words, same concept.

Cross-lingual retrieval. With a multilingual embedding model, an English query retrieves a French document about the same topic.

Conceptual queries. “Explain the difference between authentication and authorization” retrieves relevant conceptual explanations even when the query phrasing does not match any document passage verbatim.

16.2.2 What dense retrieval fails at

Rare and out-of-vocabulary terms. The query “error code E1042” may produce a poor embedding if E1042 is rare in the embedding model’s training data. The model has no strong statistical association to learn from, so E1042 gets embedded near generic “error code” content rather than near the specific documentation for E1042. The correct chunk may rank 500th.

Exact string matching. Product names, model numbers, person names, and technical identifiers must match exactly. “GPT-4o” and “GPT-4” should not be treated as semantically equivalent, they are different products and embedding models can easily conflate them.

High-specificity queries. “What is the difference between TCP and UDP?” embeds similarly to “compare two network protocols.” Dense retrieval may surface documents about other protocol comparisons that are not specifically about TCP vs. UDP.

Numerical and structured queries. “Find all incidents from Q3 2024” requires exact date matching. Embedding similarity cannot handle structured numerical constraints.

16.3 Sparse retrieval: BM25

BM25 (Best Match 25) is the dominant keyword retrieval algorithm. It extends TF-IDF with term frequency saturation and length normalization, producing relevance scores that behave well across a wide range of corpus and query types.

16.3.1 The BM25 formula

For a query Q and document D:

$$\mathrm{BM25}(D,Q)=\sum_{t\in Q}\mathrm{IDF}(t)\times\frac{\mathrm{TF}(t,D)\times(k_1+1)}{\mathrm{TF}(t,D)+k_1\left(1-b+b\times\frac{|D|}{\mathrm{avgdl}}\right)}$$

Where `t` iterates over query terms, `TF(t,D)` is term frequency in document D, `IDF(t)` is $\log\left(\frac{N-\mathrm{df}(t)+0.5}{\mathrm{df}(t)+0.5}+1\right)$, N is total documents, df(t) is documents containing term t, |D| is document length in tokens, avgdl is average document length, k1 is the term frequency saturation parameter (typically 1.2-2.0), and b is the length normalization parameter (typically 0.75).

IDF penalizes terms that appear in many documents ("the" and "is" get near- zero IDF and a rare technical term gets high IDF).

TF saturation (k1 parameter) limits the advantage of high repetition. A term appearing 100 times scores only marginally higher than one appearing 10 times.

Length normalization (b parameter) favors shorter documents. A document that mentions a query term once in 100 words scores higher than one that mentions it once in 10,000 words.

16.3.2 What BM25 is good at

Exact term matching. "Error code E1042" retrieves documents containing "E1042" precisely, regardless of whether the embedding model has seen E1042 in training.

Named entities. Person names, organization names, product names, and proper nouns are retrieved reliably by exact match.

Technical and domain-specific terminology. Medical codes, legal citations, scientific terms, and other domain vocabulary with precise meaning are handled by exact matching.

16.3.3 What BM25 fails at

Vocabulary mismatch. "Car" and "automobile" are the same concept but BM25 treats them as different terms. The query "automobile maintenance" does not retrieve documents about "car repair."

Conceptual queries. "Explain how transformers work" has no single discriminating term. Every technical document might contain "work"; many contain "transformer." BM25 cannot reason about the conceptual relationship.

Short queries. BM25 degrades for one or two word queries because there are few terms to score. "Timeout" as a query retrieves all documents mentioning timeout regardless of whether they are about network timeouts, session timeouts, or database timeouts.

16.4 Hybrid retrieval

Combining dense and sparse retrieval consistently outperforms either alone on real world RAG benchmarks. Their failure modes are complementary: BM25 catches exact matches that dense retrieval misses; dense retrieval catches semantic matches that BM25 misses.

16.4.1 Reciprocal Rank Fusion (RRF)

RRF is the simplest and most robust fusion method that relies only on rank positions rather than score magnitudes, eliminating the need for score normalization.

For a set of retrieval systems, each producing a ranked list, RRF score for a chunk `c` is defined as:

$$\mathrm{RRF}(c) = \sum_{s \in S} \frac{1}{k + \mathrm{rank}_s(c)}$$

where `S` represents the set of retrieval systems, and $\mathrm{rank}_s(c)$ denotes the 1-indexed rank of chunk `c` in the result list returned by system `s`. The parameter `k` is a smoothing constant, typically set to 60, that controls the influence of lower-ranked results. If a chunk does not appear in a system's result list, that system contributes a score of 0 for the chunk.

Why RRF is robust. RRF relies on rank positions rather than raw retrieval scores. For example, a chunk with a dense retrieval score of 0.99 receives the same contribution as a chunk with a score of 0.51 if both are ranked first.

This avoids the need to normalize or align score scales across different retrieval systems. Additionally, RRF naturally handles varying result sets by rewarding chunks that consistently appear across multiple systems

RRF in practice. Retrieve a larger candidate pool from each system (e.g., top-50 or top-100 results) before applying fusion. RRF can promote chunks that are ranked moderately across multiple systems, for example, a chunk ranked 30th by one system and 5th by another may outperform a chunk appearing only in the top-10 of a single system.

16.4.2 Weighted score combination

This is an alternative approach to normalize scores from each retrieval system to the same range, typically ([0,1]), and combine them using system-specific weights:

```
hybrid_score =   × normalized_dense_score + (1 -  ) × normalized_bm25_score
```

Where `  = 1.0` is pure dense and `  = 0.0` is pure sparse. Weaviate uses this with its `alpha` parameter. The limitation: BM25 scores are unbounded and skewed; cosine similarity is bounded to [-1, 1] and normalization artifacts can cause instability.

RRF is generally more robust than weighted combination in practice because it does not require score normalization. Use weighted combination only when you have domain-specific knowledge about the relative importance of semantic vs. keyword matching for your query distribution.

16.5 Query expansion and reformulation

Retrieval quality depends heavily on query phrasing. Users phrase queries differently from how documents phrase answers. Query expansion and reformulation bridge this gap.

Synonym expansion Expand queries with synonyms and related terms before performing BM25 retrieval. Simple synonym expansion can improve recall by matching additional relevant terms, but it may also introduce noise by adding less relevant matches. Careful tuning of the BM25 `k_1` parameter can help reduce the impact of these added terms on the final scoring.

Multi-query retrieval. Generate multiple semantically equivalent or complementary reformulations of the original query and retrieve results for each. The retrieved results are then merged and deduplicated to produce a more comprehensive candidate set. This approach consistently improves recall, particularly for ambiguous, underspecified, or poorly phrased queries. Although generating multiple queries with an LLM typically adds 200–500 ms of latency, the improvement in retrieval quality often justifies the additional cost when answer accuracy is the primary objective.

Step-back prompting. For complex analytical queries, first generate or retrieve information for a broader, more general "step-back" question and use it as additional context for answering the original query. This provides foundational knowledge that may be missing from the specific question.

For example, the query *"What causes the token refresh to fail?"* can benefit from retrieving background information on *"How does token refresh work?"* first. The broader context helps explain the underlying mechanism and makes it easier to identify and answer the specific failure causes.

16.6 Contextual and conversational retrieval

In a multi-turn conversation, follow-up messages are often not self-contained. "What about for enterprise customers?" makes no sense as a standalone retrieval query without the previous turn's context.

Standalone query generation Rewrite the user's message into a self-contained query that incorporates the necessary context from previous conversation turns before performing retrieval.

Without standalone query generation, retrieval for follow-up questions often fails because the query lacks the context needed to identify the intended topic. For example, *"What about for enterprise customers?"* may retrieve general information about enterprise customers rather than information related to the specific subject discussed earlier. Generating standalone queries is one of the most common and effective improvements for conversational RAG systems.

Conversation-aware embedding, A faster alternative to standalone query generation is to concatenate recent conversation context with the current user query before creating the embedding.

This approach avoids the additional latency of an LLM-based rewriting step, but it is generally less reliable because the combined text may not accurately represent the user's true information need. The embedding can be influenced by irrelevant conversational details rather than focusing on the core query intent. Use standalone query generation when retrieval quality is the priority, and conversation-aware embedding when lower latency is more important.

16.7 Knowledge graph retrieval

For some domains, structured knowledge in graph form enables retrieval that pure text search cannot support.

16.7.1 When graphs help

Consider a question about a complex regulatory requirement that depends on: the regulation text (in documents), which entities are subject to it (structured data), how it relates to other regulations (graph relationships), and which exceptions apply (structured rules). Text retrieval finds the regulation document. It cannot navigate the relationships between regulatory entities, exceptions, and cross-references.

GraphRAG (Edge et al., 2024) extracts entities and relationships from documents and builds a knowledge graph. Retrieval traverses the graph to gather related entities before generating a response.

16.7.2 Entity extraction for graph building

```python
def extract_entities_and_relations(text: str, llm) -> dict:
    prompt = f"""Extract entities and relationships from the following text.
Return a JSON object with:
- "entities": list of {{name, type, description}} objects
- "relations": list of {{subject, predicate, object}} triples

Text: {text}"""

    response = llm.complete(prompt)
    return json.loads(response)

# Build graph incrementally from corpus
import networkx as nx

graph = nx.DiGraph()
for chunk in corpus_chunks:
    extracted = extract_entities_and_relations(chunk.text, llm)

    for entity in extracted["entities"]:
        graph.add_node(entity["name"], **entity)

    for relation in extracted["relations"]:
        graph.add_edge(
            relation["subject"], relation["object"],
```

```
            label=relation["predicate"],
            source_chunk=chunk.id
        )
```

16.7.3 Graph retrieval at query time

```
def graph_retrieve(
    query: str, graph, text_retriever, k: int = 5
) -> list:
    query_entities = extract_entities(query)

    # Expand to related entities via graph traversal
    expanded = set(query_entities)
    for entity in query_entities:
        if entity in graph:
            expanded.update(nx.neighbors(graph, entity))

    # Gather chunks associated with expanded entities
    entity_chunks = []
    for entity in expanded:
        entity_chunks.extend(graph.nodes[entity].get("source_chunks", []))

    # Merge with standard text retrieval
    text_results = text_retriever.retrieve(query, top_k=k)
    return list(set(entity_chunks) | set(r[0] for r in text_results))
```

GraphRAG is powerful for corpora with rich entity relationships (medical knowledge bases, legal systems, financial networks) but adds significant ingestion complexity (entity extraction per chunk) and retrieval complexity (graph traversal). —

16.8 Retrieval evaluation

16.8.1 Metrics

Recall@k: of all truly relevant chunks, what fraction appear in the top-k retrieved chunks?

$$\text{Recall@k} = \frac{|\{\text{relevant}\} \cap \{\text{top-k retrieved}\}|}{|\{\text{relevant}\}|}$$

This is the primary metric for RAG. If the relevant chunk is not in the top-k, the language model cannot use it and no downstream component can compensate.

Precision@k: of the top-k retrieved chunks, what fraction are truly relevant?

$$\text{Precision@k} = \frac{|\{\text{relevant}\} \cap \{\text{top-k retrieved}\}|}{k}$$

Mean Reciprocal Rank (MRR): average reciprocal rank of the first relevant result across queries.

$$\text{MRR} = \frac{1}{|\text{queries}|} \sum_{q \in \text{queries}} \frac{1}{\text{rank\_of\_first\_relevant}(q)}$$

NDCG@k: accounts for graded relevance and position. A relevant chunk at rank 1 contributes more than the same chunk at rank 5.

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

$$\text{DCG@k} = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

$$\text{IDCG@k} = \sum_{i=1}^{k} \frac{2^{rel_i^*} - 1}{\log_2(i+1)}$$

16.9 The full retrieval pipeline

A production pipeline combining the techniques from this chapter:

```python
class HybridRetriever:
    def __init__(self, vector_db, bm25_index, embedding_model, llm):
        self.vector_db       = vector_db
        self.bm25            = bm25_index
        self.embedding_model = embedding_model
        self.llm             = llm

    def retrieve(
        self,
        query: str,
        conversation_history: list[dict] | None = None,
        filters: dict | None = None,
        top_k: int = 10,
    ) -> list[dict]:

        # Step 1: Resolve conversational context
        retrieval_query = query
        if conversation_history:
            retrieval_query = generate_standalone_query(
                conversation_history, query, self.llm
            )

        # Step 2: Dense retrieval
        query_embedding = self.embedding_model.encode(
            f"query: {retrieval_query}", normalize_embeddings=True
        )
        dense_results = self.vector_db.search(
            query_embedding, top_k=50, filters=filters
        )

        # Step 3: Sparse retrieval
        sparse_results = self.bm25.search(retrieval_query, top_k=50)

        # Step 4: Fuse with RRF
        fused = reciprocal_rank_fusion([dense_results, sparse_results], k=60)
        candidates = fused[:100]  # top-100 for re-ranking
```

```
        # Step 5: Re-rank (chapter 23)
        # reranked = reranker.rerank(retrieval_query, candidates, top_k)

        # Step 6: Return top results
        return [self.get_chunk(chunk_id) for chunk_id, _ in candidates[:top_k]]
```

16.10 Key takeaways

- No single retrieval algorithm works well for all query types; dense retrieval excels at semantic matching, BM25 excels at exact term matching, and their failure modes are complementary — hybrid retrieval consistently outperforms either alone
- BM25 scores by term frequency (saturated), inverse document frequency, and length normalization; it is essential for exact matching of rare terms, product codes, and named entities that embedding models handle poorly
- Reciprocal Rank Fusion is the most robust fusion method because it depends only on rank positions, not score magnitudes — no normalization required; retrieve top-50 or top-100 from each system before fusing
- Multi-query retrieval generates paraphrase variants and fuses results, improving recall for ambiguous queries at the cost of one LLM call per query
- Standalone query generation is mandatory for multi-turn conversations; direct retrieval on follow-up questions without context rewriting consistently fails and is one of the most common fixable RAG failures
- GraphRAG adds entity-relationship traversal for domains with rich structured relationships; adds significant complexity and is only worth it when text retrieval quality is provably insufficient for relationship-dependent queries
- Recall@k on a held-out evaluation set is the only reliable way to measure retrieval quality; build this before tuning anything else in the RAG pipeline

---

16.11 Further reading

- Robertson & Zaragoza (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* — The authoritative BM25 reference; explains the mathematical motivation for saturation and length normalization.
- Karpukhin et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering.* — established dense retrieval for question answering.
- Cormack et al. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.* — The RRF paper.
- Ma et al. (2023). *Fine-Tuning LLaMA for Multi-Stage Text Retrieval.* — LLM- based query reformulation for improved retrieval.
- Edge et al. (2024). *From Local to Global: A Graph RAG Approach to Query- Focused Summarization.* — GraphRAG; entity-based retrieval for complex multi-document reasoning.
- Es et al. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation.* — End-to-end RAG evaluation framework including retrieval metrics.
- Thakur et al. (2021). *BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models.* — The standard benchmark for comparing retrieval algorithms across diverse domains.

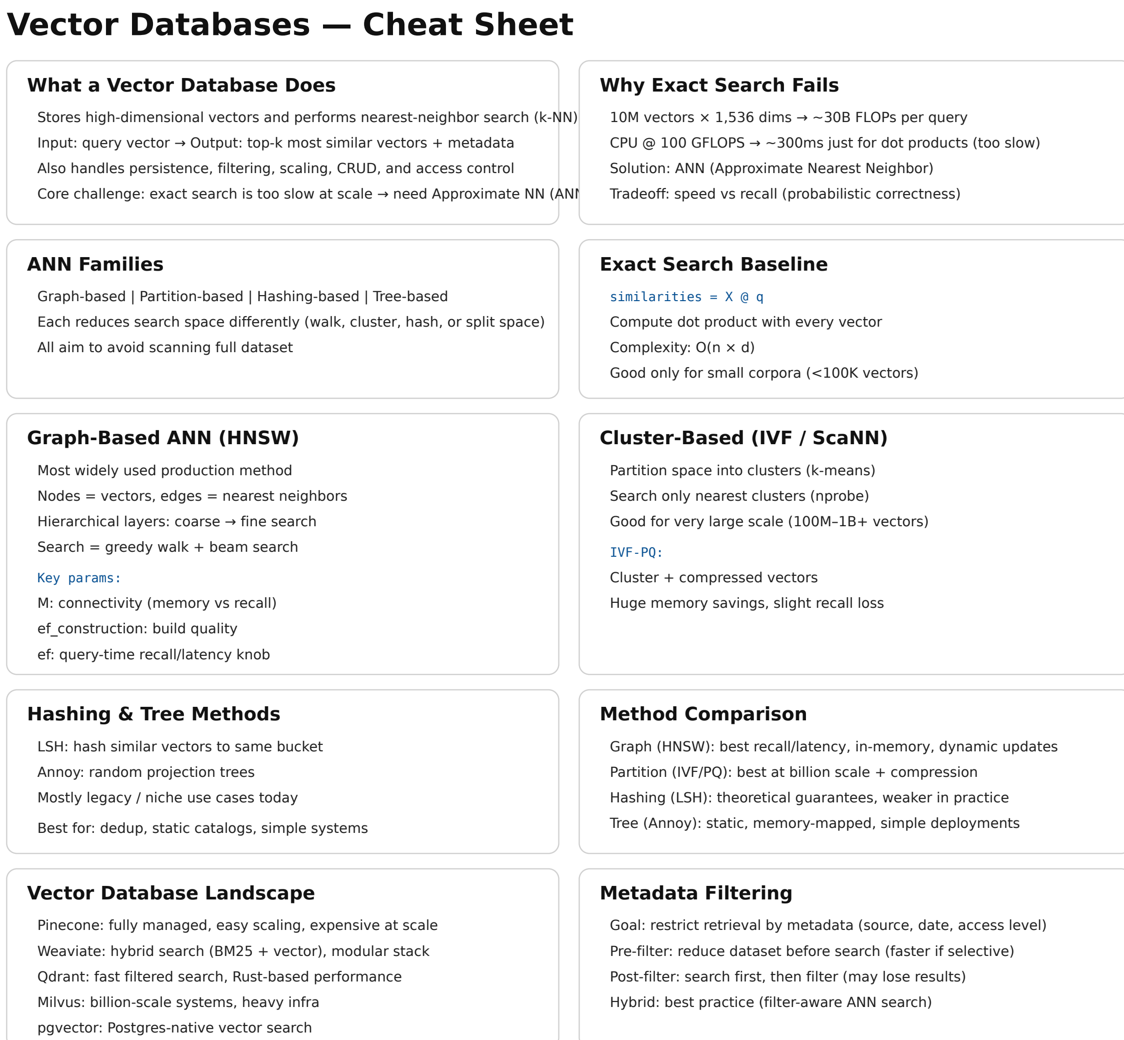


Figure 16.2: Cheat sheet.

Chapter 17

Reranking

The canonical question for this chapter: *Initial retrieval returns 50 candidates. The language model can use 10. How do you pick the right 10 and why is the answer more subtle than just sorting by embedding similarity?*

Where are we?

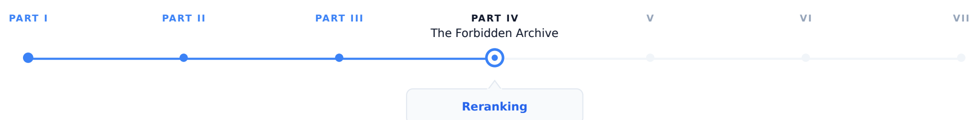


Figure 17.1: The journey through the Model Mind.

Initial retrieval maximized recallthat helps to find everything that might be relevant. Now the problem inverts: among 50 candidates, which 10 should the language model actually see? Passing all 50 wastes context budget, dilutes signal with noise, and increases the probability of lost-in-the-middle failures. Reranking is the precision stage, the step that decides which evidence the model gets to work with.

17.1 Why reranking exists as a separate stage

Recall and precision pull in opposite directions. A retriever that returns 50 candidates to maximize recall will include: chunks that directly answer the query, chunks that are topically related but do not answer the specific question, chunks that share surface features with the query but are semantically unrelated, and chunks that answer part of a multi-part query but not the whole thing.

The language model receives whatever context you pass it. If you pass 50 chunks (many irrelevant) several failure modes emerge. The model spends context window budget on noise. Relevant information is diluted among irrelevant information amd finally long contexts increase the probability of the lost-in-the-middle phenomenon, where the model fails to use information in the middle of a long context.

Reranking is the precision stage: given 50 candidates from initial retrieval, identify the 10 most likely to help the model answer the query correctly.

17.2 The spectrum of reranking approaches

Reranking methods form a spectrum from fast-and-approximate to slow-and-accurate:

```
Fast                                                          Accurate


Score-based    Bi-encoder     Late interaction    Cross-encoder    LLM judge
(retrieval     (embedding     (ColBERT           (BERT-based      (GPT-4 as
 scores)        similarity)    MaxSim)             reranker)        evaluator)
     ↑               ↑              ↑                   ↑                ↑
Already done   Initial          Medium             Best practical    Expensive;
in retrieval   retrieval        cost/quality       quality/cost      use for
               produces this    tradeoff           tradeoff          offline tasks
```

"Reranking" in common usage refers to applying a cross-encoder or late- interaction model as a second stage after initial retrieval. The earlier parts of the spectrum are usually considered part of retrieval itself.

17.3 Score-based reranking

The simplest approach: use the scores from initial retrieval to filter candidates before passing them to the language model.

17.3.1 Threshold filtering

```python
def threshold_filter(
    candidates: list[tuple[str, float]],
    min_score: float = 0.7,
) -> list[tuple[str, float]]:
    return [
        (chunk_id, score) for chunk_id, score in candidates
        if score >= min_score
    ]
```

This works for cosine similarity scores but requires careful calibration. The right threshold depends on the embedding model (different models have different score distributions), the query type, and the corpus. A threshold tuned on one distribution fails on another.

Thresholds on BM25 scores are even less reliable these scores are unbounded and vary with corpus size and document length distribution.

17.3.2 Why score-based filtering alone is insufficient

Embedding similarity is not the same as relevance. A chunk about a different product from the same company might have a cosine similarity of 0.85 with the query (above any reasonable threshold) while being completely irrelevant to the specific question. Score-based filtering works only when similarity correlates well with relevance, which holds for queries within the embedding model's training distribution and breaks for out-of-distribution queries, domain-specific content, or multi-hop reasoning.

17.4 Cross-encoder reranking

Cross-encoders are the most widely used practical reranking approach, they consistently outperform bi-encoders for relevance scoring and fit within interactive latency budgets when deployed on GPU.

17.4.1 Why cross-encoders are more accurate than bi-encoders

A bi-encoder encodes query and document independently:

```
Query  → Encoder → q_embedding (fixed vector)
Chunk  → Encoder → c_embedding (fixed vector)
Score  = cosine_similarity(q_embedding, c_embedding)
```

The query embedding does not "see" the chunk during encoding. Every query-chunk interaction happens at score computation which is a single dot product.

A cross-encoder takes both as input simultaneously:

```
[CLS] query tokens [SEP] chunk tokens [SEP]
                  → Encoder
                  → [CLS] representation
                  → scalar relevance score
```

Query and chunk tokens attend to each other through all attention layers. The model can detect that a chunk discusses the right concept in the wrong context, that a surface-level match is semantically irrelevant, or that a specific phrase in the query requires a specific phrase in the chunk that is absent.

This expressiveness is why cross-encoders consistently outperform bi-encoders for relevance scoring by 10–20 NDCG points on standard benchmarks. The tradeoff: cross-encoders cannot be pre-computed. Every query-chunk pair requires a fresh forward pass, making them too slow for full-corpus search but fast enough for a 50-candidate reranking step.

17.4.2 Cross-encoder inference

```python
from sentence_transformers import CrossEncoder
import numpy as np

class CrossEncoderReranker:
    def __init__(
        self,
        model_name: str = "cross-encoder/ms-marco-MiniLM-L-6-v2"
    ):
        self.model = CrossEncoder(model_name, max_length=512)

    def rerank(
        self,
        query: str,
        candidates: list[tuple[str, str]],  # (chunk_id, chunk_text)
        top_k: int = 10,
        batch_size: int = 32,
    ) -> list[tuple[str, float]]:
        if not candidates:
            return []

        pairs = [(query, chunk_text) for _, chunk_text in candidates]

        # Score in batches to avoid OOM on GPU
        all_scores = []
```

```
        for i in range(0, len(pairs), batch_size):
            batch = pairs[i:i + batch_size]
            scores = self.model.predict(batch, show_progress_bar=False)
            all_scores.extend(scores.tolist())

        scored = sorted(
            zip([chunk_id for chunk_id, _ in candidates], all_scores),
            key=lambda x: x[1],
            reverse=True,
        )
        return scored[:top_k]
```

17.4.3 Input length and truncation

Cross-encoders have a maximum input length (typically 512 tokens for BERT-based models). The query plus chunk must fit within this limit. If the chunk is too long, it must be truncated which may result in cuting the relevant portion.

Strategies for long chunks:

Sliding window: split the chunk into overlapping windows, score each, take the maximum as the chunk score. Most reliable but multiplies inference calls.

First-N tokens: relevant facts often appear near the start of chunks; simple and fast. Fails when the key passage is in the second half.

```
def rerank_long_chunks(
    query: str,
    candidates: list[tuple[str, str]],
    reranker: CrossEncoder,
    max_chunk_tokens: int = 400,
    stride: int = 200,
) -> list[tuple[str, float]]:
    chunk_scores = {}

    for chunk_id, chunk_text in candidates:
        tokens = chunk_text.split()  # simplified; use real tokenizer in production

        if len(tokens) <= max_chunk_tokens:
            score = reranker.model.predict([(query, chunk_text)])[0]
        else:
            window_scores = []
            for start in range(0, len(tokens), stride):
                window = " ".join(tokens[start:start + max_chunk_tokens])
                window_scores.append(reranker.model.predict([(query, window)])[0])
            score = max(window_scores)

        chunk_scores[chunk_id] = score

    return sorted(chunk_scores.items(), key=lambda x: x[1], reverse=True)
```

17.4.4 Cross-encoder model selection

The choice of cross-encoder involves quality, latency, and cost tradeoffs:

ms-marco-MiniLM-L-6-v2 (22M parameters): fastest option, good for low- latency requirements. ~39 MRR@10 on MS MARCO dev. The right default for most production applications.

ms-marco-MiniLM-L-12-v2 (33M parameters): better quality than L-6, roughly 2× slower. ~42 MRR@10. Worth the latency cost when quality is critical.

BGE-Reranker-v2-m3 (568M parameters): state-of-the-art open-source cross- encoder. Multilingual, strong across BEIR benchmarks. Requires GPU for acceptable latency.

Cohere Rerank 3 (commercial API): among the best retrieval quality available for English. Priced per API call. No GPU infrastructure required.

Voyage Rerank 2 (commercial API): strong competitor to Cohere, particularly for code and technical content.

17.5 LLM-based reranking

Using a large language model as the relevance judge. More accurate than cross- encoders for complex relevance judgments but significantly more expensive.

17.5.1 Pointwise LLM scoring

Score each candidate individually by asking the LLM whether it is relevant:

```
def llm_rerank_pointwise(
    query: str,
    candidates: list[tuple[str, str]],
    llm,
    top_k: int = 10,
) -> list[tuple[str, float]]:
    scored = []

    for chunk_id, chunk_text in candidates:
        prompt = f"""On a scale of 0 to 10, how relevant is the following passage
to answering this query? Respond with only a number.

Query: {query}

Passage: {chunk_text}

Relevance score (0-10):"""

        response = llm.complete(prompt).strip()
        try:
            score = float(response)
        except ValueError:
            score = 0.0
        scored.append((chunk_id, score))

    return sorted(scored, key=lambda x: x[1], reverse=True)[:top_k]
```

This requires one LLM call per candidate. For 50 candidates, that is 50 LLM calls it is expensive and slow (5–20 seconds depending on model and parallelism).

17.5.2 Listwise LLM reranking

Score all candidates together in a single prompt:

```
def llm_rerank_listwise(
    query: str,
    candidates: list[tuple[str, str]],
    llm,
    top_k: int = 10,
) -> list[tuple[str, float]]:
    candidates_text = "\n\n".join([
        f"[{i+1}] {chunk_text[:300]}..."
        for i, (_, chunk_text) in enumerate(candidates)
    ])

    prompt = f"""Rank the following passages by relevance to the query.
Return a JSON array of passage numbers in order of relevance (most relevant first).
Numbers only, no explanation.

Query: {query}

Passages:
{candidates_text}

Ranking (JSON array):"""

    response = llm.complete(prompt).strip()

    try:
        ranking = json.loads(response)
    except json.JSONDecodeError:
        ranking = list(range(1, len(candidates) + 1))

    result = []
    for rank, position in enumerate(ranking[:top_k]):
        idx = position - 1
        if 0 <= idx < len(candidates):
            result.append((candidates[idx][0], 1.0 / (rank + 1)))

    return result
```

One LLM call regardless of candidate count, but candidate text must fit in the prompt. For 50 candidates at 300-character truncation: approximately 15,000 characters per query is feasible but expensive.

17.5.3 Sliding window listwise reranking (RankGPT)

For long candidate lists that do not fit in a single prompt:

```
def rankgpt_rerank(
    query: str,
    candidates: list[tuple[str, str]],
    llm,
    window_size: int = 20,
    step_size: int = 10,
    top_k: int = 10,
) -> list[tuple[str, float]]:
    ranked = list(candidates)
```

```
    # Slide window from bottom to top
    for start in range(len(ranked) - window_size, -1, -step_size):
        end = min(start + window_size, len(ranked))
        window = ranked[start:end]
        window_ranking = llm_rerank_listwise(query, window, llm, top_k=len(window))
        id_to_rank = {cid: r for r, (cid, _) in enumerate(window_ranking)}
        window.sort(key=lambda x: id_to_rank.get(x[0], len(window)))
        ranked[start:end] = window

    return [
        (chunk_id, 1.0 / (rank + 1))
        for rank, (chunk_id, _) in enumerate(ranked[:top_k])
    ]
```

RankGPT achieves high-quality rankings (comparable to or better than cross- encoders on several benchmarks) but requires many LLM calls for long candidate lists. It is impractical for real-time applications.

17.5.4 When to use LLM reranking

LLM reranking is appropriate for offline processing (generating training data for a smaller reranker), high-value queries where stakes justify cost and latency (legal document review, medical information retrieval), complex relevance judgments requiring multi-hop reasoning, and evaluation (grading retrieval quality at scale). For most interactive applications, cross-encoder reranking provides the best quality/cost/latency tradeoff.

17.6 Diversity-aware reranking

Relevance alone is not sufficient. If the top 10 most relevant chunks all say the same thing (different phrasings of the same fact) passing all 10 wastes context window budget and adds no new information.

Diversity-aware reranking selects a set that is both relevant and diverse, covering different aspects of the query.

17.6.1 Maximum Marginal Relevance (MMR)

Maximum Marginal Relevance (MMR) iteratively selects chunks that maximize a combination of relevance to the query and distance from already-selected chunks. It balances retrieving highly relevant chunks while avoiding selecting multiple chunks that contain the same information.

$$\mathrm{MMR}(D_i) = \lambda\,\mathrm{Sim}(D_i, Q) - (1-\lambda)\max_{D_j \in S} \mathrm{Sim}(D_i, D_j)$$

where: D_i is a candidate chunk being evaluated, Q is the query, S is the set of chunks already selected, $\mathrm{Sim}(D_i, Q)$ measures relevance between the chunk and query, $\mathrm{Sim}(D_i, D_j)$ measures redundancy between the candidate chunk and previously selected chunks, and λ controls the trade-off between relevance and diversity.

At each iteration, MMR chooses:

$$D^* = \arg\max_{D_i \in R \setminus S} \left[\lambda\,\mathrm{Sim}(D_i, Q) - (1-\lambda)\max_{D_j \in S} \mathrm{Sim}(D_i, D_j) \right]$$

where R is the set of all retrieved candidate chunks.

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
```

```python
def mmr(
    query_embedding: np.ndarray,
    candidate_embeddings: np.ndarray,
    candidate_ids: list[str],
    top_k: int = 10,
    lambda_param: float = 0.5,  # 0 = pure diversity, 1 = pure relevance
) -> list[str]:
    query_similarities = (candidate_embeddings @ query_embedding).flatten()

    selected_indices = []
    remaining_indices = list(range(len(candidate_ids)))

    for _ in range(min(top_k, len(remaining_indices))):
        if not selected_indices:
            best_idx = remaining_indices[
                np.argmax(query_similarities[remaining_indices])
            ]
        else:
            selected_embeddings = candidate_embeddings[selected_indices]
            mmr_scores = []
            for idx in remaining_indices:
                relevance = query_similarities[idx]

                max_sim_to_selected = cosine_similarity(
                    [candidate_embeddings[idx]], selected_embeddings
                )[0].max()

                mmr_score = (
                    lambda_param * relevance
                    - (1 - lambda_param) * max_sim_to_selected
                )

                mmr_scores.append((idx, mmr_score))

            best_idx = max(mmr_scores, key=lambda x: x[1])[0]

        selected_indices.append(best_idx)
        remaining_indices.remove(best_idx)

    return [candidate_ids[i] for i in selected_indices]
```

lambda_param = 1.0 is pure relevance, meaning MMR becomes equivalent to sorting chunks by query similarity.

lambda_param = 0.0 is pure diversity, selecting chunks that are maximally different from previously selected chunks.

lambda_param = 0.5 provides an equal balance between relevance and diversity and is a common default value.

MMR is computationally cheap because it operates on pre-computed embedding vectors and cosine similarities. It is particularly effective for retrieval-augmented generation (RAG) systems and document retrieval pipelines where highly relevant chunks often contain overlapping or redundant information.

17.6.2 When diversity matters

For factual queries with a single answer, diversity is counterproductive, similar chunks provide useful redundancy if the most relevant one is imprecise.

For multi-part queries, complex questions, or queries requiring synthesis from multiple sources, diversity is essential. "What are the pros and cons of OAuth 2.0?" benefits from chunks discussing both advantages and disadvantages, not five chunks about the same advantage. Use diversity-aware selection for summarization tasks, "compare/contrast" queries, "list" queries, and corpora with heavy content duplication.

17.7 Contextual compression

Rather than selecting which chunks to include, contextual compression extracts only the relevant portion of each retrieved chunk which results in freeing context budget for more chunks or longer reasoning.

17.7.1 LLM extraction

```python
def compress_chunk(query: str, chunk: str, llm) -> str:
    prompt = f"""Extract the portion of the following passage that is relevant
to answering the query. If nothing is relevant, return "IRRELEVANT".
Return only the extracted text, no explanation.

Query: {query}

Passage: {chunk}

Relevant excerpt:"""

    result = llm.complete(prompt).strip()
    return "" if result.upper() == "IRRELEVANT" else result
```

A 500-token chunk might compress to a 50-token relevant excerpt. The cost: one LLM call per retrieved chunk. For 10 chunks, that is 10 LLM calls before the final generation call.

17.7.2 Sentence-level extraction

A cheaper alternative: extract only sentences similar to the query using embedding similarity.

```python
def extract_relevant_sentences(
    query: str,
    chunk: str,
    embedding_model,
    top_k_sentences: int = 3,
    min_similarity: float = 0.4,
) -> str:
    sentences = split_into_sentences(chunk)
    if not sentences:
        return chunk

    q_emb = embedding_model.encode(query, normalize_embeddings=True)
    s_embs = embedding_model.encode(sentences, normalize_embeddings=True)
    similarities = s_embs @ q_emb

    relevant = [
        i for i, sim in enumerate(similarities) if sim >= min_similarity
    ]
    relevant = sorted(relevant, key=lambda i: similarities[i], reverse=True)[:top_k_sentences]
```

```
    relevant_sorted = sorted(relevant)  # restore original order for coherence

    return " ".join(sentences[i] for i in relevant_sorted)
```

Sentence extraction is faster than LLM extraction but less precise. It cannot detect that a sentence is relevant only given the preceding sentence, or that a tangentially related sentence is the key to answering the query.

17.8 The full reranking pipeline

```
User query


[Optional] Query reformulation


Initial retrieval: dense + sparse, top-50 to top-100


Cross-encoder reranking: top-50 → top-15


[Optional] MMR diversity selection: top-15 → top-10


[Optional] Contextual compression: extract relevant sentences per chunk


Context assembly → Language model
```

Not every application needs every stage. The decision depends on latency budget, quality requirements, and corpus characteristics.

17.8.1 Latency budget allocation

For a 500ms TTFT target:

```
Query embedding:                   10ms
Dense retrieval (HNSW):            20ms
Sparse retrieval (BM25):           15ms
RRF fusion:                         5ms
Cross-encoder reranking
  (MiniLM-L-6, 50 candidates, GPU):30ms
Context assembly:                   5ms
LLM TTFT (first token):          ~200ms

Total:                             285ms  ← within 500ms budget

With LLM reranking (50 candidates): +2,000ms  ← blows the budget
With MMR (cosine similarities):        +10ms  ← acceptable
```

The reranking budget is typically 20–100ms in interactive applications. Cross- encoder reranking on GPU with small models (MiniLM) fits. Cross-encoder on CPU does not for most TTFT targets. LLM reranking does not.

17.9 Training custom rerankers

Off-the-shelf cross-encoders are trained on general retrieval datasets (MS MARCO, Natural Questions) but for specialized domains, fine-tuning can improve NDCG@10 by 5–15%.

17.9.1 Training data sources

User click data: queries users submitted plus results they clicked (positive) and results they saw but did not click (negative). High quality, requires sufficient production traffic.

Synthetic generation: use an LLM to generate queries for known-relevant chunks and sample hard negatives from the retrieval system.

LLM-judged pairs: retrieve candidates, judge relevance with an LLM, train the cross-encoder on those judgments.

17.9.2 Fine-tuning a cross-encoder

```
from sentence_transformers import CrossEncoder, InputExample
from torch.utils.data import DataLoader

train_samples = [
    InputExample(
        texts=["how long before session timeout?",
               "The default token expiration is 3600 seconds."],
        label=1.0,  # relevant
    ),
    InputExample(
        texts=["how long before session timeout?",
               "API rate limits apply to all endpoints."],
        label=0.0,  # not relevant
    ),
    # ... 1,000-5,000 more examples
]

model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2", num_labels=1)
train_dataloader = DataLoader(train_samples, shuffle=True, batch_size=32)

model.fit(
    train_dataloader=train_dataloader,
    epochs=3,
    warmup_steps=100,
    output_path="./domain-reranker",
)
```

Even 1,000–5,000 domain-specific examples produce measurable improvements for specialized corpora. The investment is typically worthwhile for high-value RAG applications where a general model underperforms.

17.10 Measuring reranking quality

17.10.1 Pipeline stage comparison

Measure how precision improves at each stage:

```python
def evaluate_pipeline_stages(
    eval_queries: list[dict],  # {query, relevant_chunk_ids}
    retriever,
    reranker,
    k_retrieve: int = 50,
    k_rerank: int = 10,
) -> dict:
    retrieval_recall    = []
    retrieval_precision = []
    reranking_precision = []

    for item in eval_queries:
        query        = item["query"]
        relevant_ids = set(item["relevant_chunk_ids"])

        retrieved     = retriever.retrieve(query, top_k=k_retrieve)
        retrieved_ids = {cid for cid, _ in retrieved}

        retrieval_recall.append(
            len(relevant_ids & retrieved_ids) / len(relevant_ids)
        )
        retrieval_precision.append(
            len(relevant_ids & {cid for cid, _ in retrieved[:k_rerank]}) / k_rerank
        )

        reranked     = reranker.rerank(query, retrieved, top_k=k_rerank)
        reranked_ids = {cid for cid, _ in reranked}

        reranking_precision.append(
            len(relevant_ids & reranked_ids) / k_rerank
        )

    return {
        f"retrieval_recall@{k_retrieve}":   np.mean(retrieval_recall),
        f"retrieval_precision@{k_rerank}":  np.mean(retrieval_precision),
        f"reranking_precision@{k_rerank}":  np.mean(reranking_precision),
        "precision_lift":
            np.mean(reranking_precision) / np.mean(retrieval_precision),
    }
```

A well-functioning reranker should lift Precision@10 by 20–50% relative to initial retrieval while maintaining the Recall@50 of the retrieval stage.

17.11 Key takeaways

- Reranking is the precision stage of a two-stage retrieve-then-rank pipeline; initial retrieval maximizes recall, reranking maximizes precision over the candidate set
- Cross-encoders jointly encode query and document through full attention, outperforming bi-encoders by 10–20 NDCG points; the tradeoff is that they cannot be pre-computed and are applied only to the candidate set (50–100 chunks), not the full corpus
- For chunks longer than the cross-encoder's context limit, use a sliding window approach and take the maximum window score as the chunk score

- LLM-based reranking achieves the highest quality but is impractical for interactive applications — use it for offline tasks, training data generation, and evaluation
- MMR selects a set of chunks that is both relevant and diverse; the lambda parameter controls the tradeoff; use diversity selection for multi-part queries, summarization, and corpora with heavy content duplication
- Contextual compression extracts only the relevant portion of each chunk, freeing context budget — powerful when combined with a large top-k, at the cost of additional LLM calls per chunk
- Cross-encoder reranking on GPU with MiniLM adds ~30ms — fits within a 500ms TTFT budget; LLM reranking adds ~2,000ms and does not
- Fine-tuning a cross-encoder on 1,000–5,000 domain-specific examples improves NDCG@10 by 5–15% for specialized corpora; synthetic query generation from chunks is a scalable way to create training data
- Measure both Precision@k improvement and end-to-end answer quality; retrieval precision gains do not always translate linearly to answer quality gains

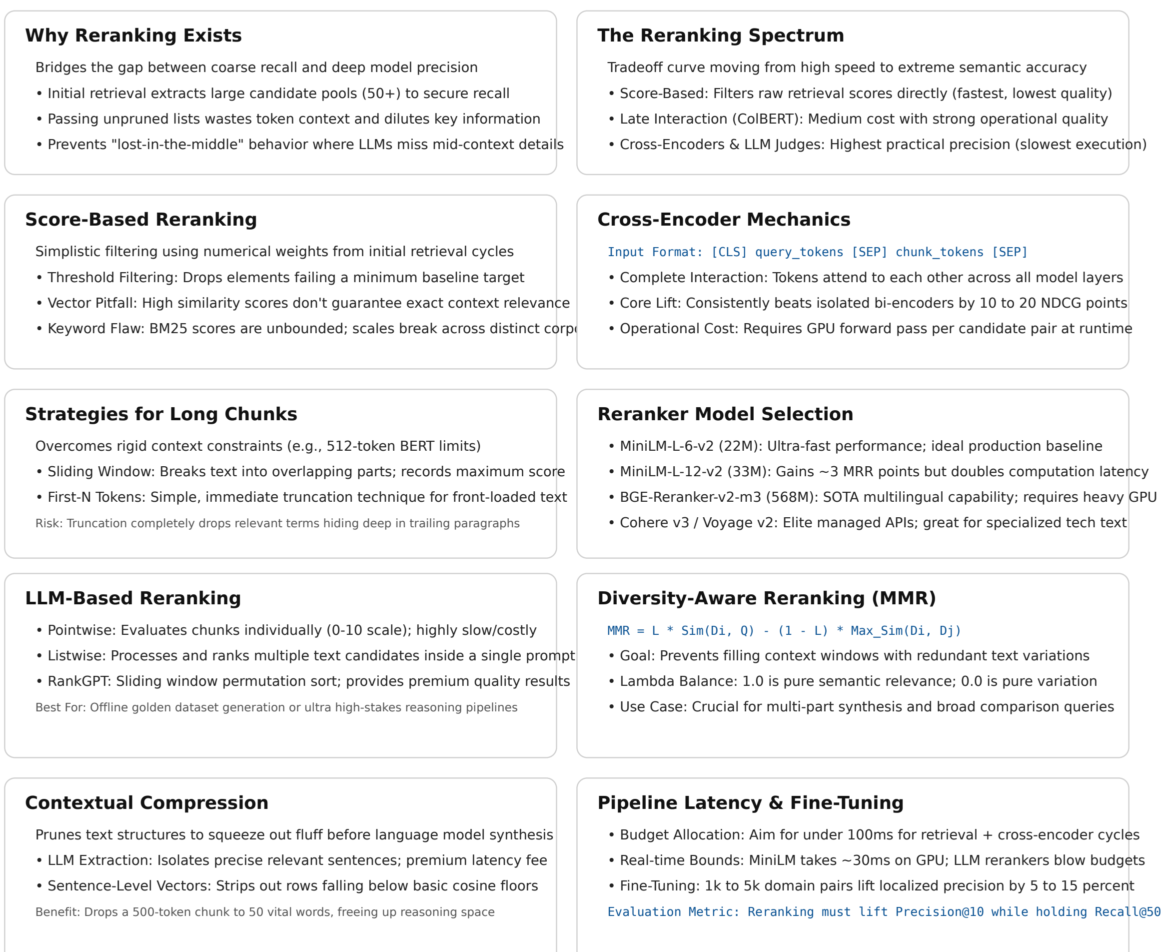


Figure 17.2: Cheat sheet.

17.12 Further reading

- Nogueira & Cho (2019). *Passage Re-ranking with BERT.* — Established cross- encoder reranking as the standard second-stage approach.
- Sun et al. (2023). *Is ChatGPT Good at Search? Investigating Large Language Models as Re-ranking Agents.* — RankGPT; listwise LLM reranking with sliding window.
- Pradeep et al. (2023). *RankZephyr: Effective and Robust Zero-Shot Listwise Reranking is a Breeze!* — Open-source LLM-based reranking.
- Liu et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts.* — Why reranking and context compression matter for LLM context utilization.
- Cohere (2024). *Rerank 3 Technical Report.* — State-of-the-art commercial reranker; useful benchmark reference.

Chapter 18

Context Construction

The canonical question for this chapter: *You have retrieved and reranked your chunks. Before generating a single token of the response, how do you assemble those chunks into a prompt that ensures the language model uses them correctly?*

Where are we?

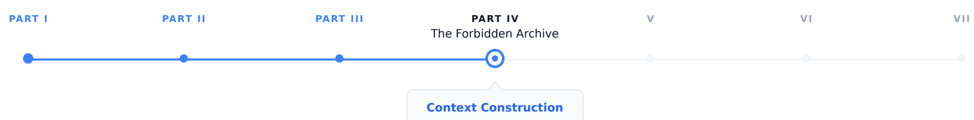


Figure 18.1: The journey through the Model Mind.

Reranking narrowed the candidates to the ones worth showing the model. This chapter covers the last engineering step before generation: turning a ranked list of chunks into an actual prompt. Every decision here (formatting, order, budget, framing) shapes what the model can reason about and what it will believe, even though none of it involves the model itself.

18.1 The gap between retrieval and generation

Retrieval produces a ranked list of relevant chunks while generation requires a single, coherent prompt. The space between these two (context construction) is where many RAG systems silently fail.

A system can have excellent retrieval (Recall@10 above 0.95) and an excellent language model, and still produce poor answers because the context was assembled carelessly. The model receives information in a confusing order, with duplicate content, missing attribution, insufficient instruction about what to do, or more text than it can effectively use.

Context construction is not just formatting it is also the engineering of what the language model actually sees which determines what it can reason about, what it will believe, and what kind of response it will produce. The content being assembled was not written by a human with a sense of narrative flow, it was retrieved mechanically and must be made coherent programmatically.

This chapter covers every decision in context construction: what to include, in what order, how much, with what framing, and how to verify that the model is actually using what you provide.

18.2 Anatomy of a RAG prompt

A complete RAG prompt has multiple components, each serving a specific purpose:

```
SYSTEM PROMPT
Role definition, behavior instructions, output format

RETRIEVED CONTEXT
The chunks that inform the answer

CONVERSATION HISTORY (if multi-turn)
Prior turns in the current conversation

CURRENT QUERY
The user's actual question
```

Each component has design decisions that affect answer quality and none of them are optional for a production RAG system.

18.3 The system prompt

The system prompt establishes the model's role, instructs it how to use the retrieved context, and defines the output format.

18.3.1 Baseline RAG system prompt

```
You are a helpful assistant. Answer questions based on the provided context.
If the context does not contain enough information to answer the question,
say so clearly rather than speculating.
```

This is functional but minimal. It does not tell the model what kind of context it is receiving (document excerpts? structured data?), how authoritative the context is (always trust it? verify it?), how to cite sources, what to do when context chunks contradict each other, or how long or detailed the response should be.

18.3.2 Production RAG system prompt

```
You are a technical documentation assistant for Isengard Corp's API platform.

CONTEXT USAGE:
- Answer questions using only the provided documentation excerpts
- If the excerpts do not contain sufficient information, state clearly:
  "I don't have enough information in the provided documentation to answer this."
- Do not use general knowledge to supplement the context, users need
  answers specific to Isengard Corp's implementation
- When information spans multiple excerpts, synthesize them coherently

CITATIONS:
- Reference the source document for any specific claim
- Format citations as [Source: {document_title}, Section: {section}]
```

```
- If two excerpts contradict each other, note both and flag the discrepancy

RESPONSE FORMAT:
- Lead with the direct answer before any supporting explanation
- Use code examples from the documentation when relevant
- Keep responses concise, users are developers reading quickly

CONFIDENCE:
- Distinguish between "this documentation states X" and "this suggests X"
- If the documentation is ambiguous, say so
```

The additional specificity addresses failure modes seen in practice: models that supplement retrieved context with hallucinated general knowledge (see discussion of parametric memory as noise), models that do not cite sources, models that pick one side when context contradicts itself, and models that bury the answer in preamble.

18.3.3 Domain-specific instructions

For specialized domains, the system prompt must address domain-specific behaviors. **Legal RAG**: "Do not provide legal advice. Present the relevant statutory text and precedent, then recommend the user consult a licensed attorney." **Medical RAG**: "Present information from clinical guidelines. Include the evidence level when indicated. Always recommend consulting a healthcare provider." **Financial RAG**: "Present the relevant regulations and data and never provide investment advice. Note the effective date of any regulation cited." **Code assistant RAG**: "When presenting code from documentation, preserve it exactly as written. Note the SDK or language version it applies to."

18.4 Retrieved context formatting

How you format the retrieved chunks in the prompt significantly affects model behavior. The model attends differently to differently structured context.

18.4.1 Basic formatting

```python
def format_chunks_basic(chunks: list[dict]) -> str:
    formatted = []
    for i, chunk in enumerate(chunks, 1):
        formatted.append(f"[{i}] {chunk['text']}")
    return "\n\n".join(formatted)
```

```
[1] The default token expiration is 3600 seconds. This can be configured
    via the token_ttl parameter in the authentication settings.

[2] Authentication tokens are invalidated immediately upon user logout
    regardless of the remaining TTL.
```

Simple, readable, works for most cases.

18.4.2 Metadata-enriched formatting

Include source information to enable citation and help the model assess credibility:

```python
def format_chunks_with_metadata(chunks: list[dict]) -> str:
    formatted = []
    for i, chunk in enumerate(chunks, 1):
        source_info = []
```

```
        if chunk.get('document_title'):
            source_info.append(f"Document: {chunk['document_title']}")
        if chunk.get('section'):
            source_info.append(f"Section: {chunk['section']}")
        if chunk.get('page'):
            source_info.append(f"Page: {chunk['page']}")
        if chunk.get('last_updated'):
            source_info.append(f"Updated: {chunk['last_updated']}")

        header = f"[Source {i}: {' | '.join(source_info)}]" if source_info else f"[{i}]"
        formatted.append(f"{header}\n{chunk['text']}")

    return "\n\n---\n\n".join(formatted)
```

```
[Source 1: Document: API Reference v3.2 | Section: Authentication | Updated: 2024-03]
The default token expiration is 3600 seconds. This can be configured
via the token_ttl parameter in the authentication settings.

---

[Source 2: Document: API Reference v3.2 | Section: Session Management | Updated: 2024-03]
Authentication tokens are invalidated immediately upon user logout
regardless of the remaining TTL.
```

The model can now cite sources specifically and has recency information to assess which source is more current if sources conflict.

18.4.3 XML-delimited formatting

XML tags provide unambiguous delimiters that are reliable even when chunk content contains newlines, code blocks, or other text that might confuse delimiter-based parsing:

```
def format_chunks_xml(chunks: list[dict]) -> str:
    formatted = []
    for i, chunk in enumerate(chunks, 1):
        attrs = []
        if chunk.get('document_title'):
            attrs.append(f'source="{chunk["document_title"]}"')
        if chunk.get('section'):
            attrs.append(f'section="{chunk["section"]}"')

        attr_str = " " + " ".join(attrs) if attrs else ""
        formatted.append(
            f'<document index="{i}"{attr_str}>\n{chunk["text"]}\n</document>'
        )

    return "\n\n".join(formatted)
```

```
<document index="1" source="API Reference v3.2" section="Authentication">
The default token expiration is 3600 seconds. This can be configured
via the token_ttl parameter in the authentication settings.
</document>

<document index="2" source="API Reference v3.2" section="Session Management">
Authentication tokens are invalidated immediately upon user logout
```

```
regardless of the remaining TTL.
</document>
```

18.4.4 Relevance score inclusion

Including relevance scores can help the model calibrate confidence:

```python
def format_chunks_with_scores(chunks: list[tuple[dict, float]]) -> str:
    formatted = []
    for i, (chunk, score) in enumerate(chunks, 1):
        confidence = "High" if score > 0.85 else "Medium" if score > 0.6 else "Low"
        formatted.append(
            f"[{i}] (Relevance: {confidence})\n{chunk['text']}"
        )
    return "\n\n".join(formatted)
```

If scores are noisy (as embedding similarities often are), exposing them to the model can cause it to underweight actually relevant chunks that scored slightly lower. Include scores only when they are well-calibrated. —

18.5 Ordering retrieved chunks

The order in which chunks appear in the context window matters and it is not in the direction most people assume.

18.5.1 The lost-in-the-middle effect, revisited

Language models systematically underperform when relevant information sits in the middle of a long context. Here it becomes an active design constraint rather than a passive fact: the most important chunks should not be placed in the middle of the assembled context.

```
Context positions by model attention:

HIGH
      ↑                        ↑
      Beginning                End
         ↓ MIDDLE: lowest attention ↓
LOW
```

18.5.2 Ordering strategies

By relevance score (naive) places the highest-scoring chunk first. This puts the most relevant content at the beginning but potentially leaves the second-most-relevant chunk buried in the middle if there are many chunks.

Reverse relevance order places the highest-scoring chunk last, exploiting the recency bias of many models, information near the end of the context is often well-attended.

Lost-in-the-middle optimized ordering places the most relevant chunks at the beginning and end, less relevant chunks in the middle:

```python
def order_chunks_for_attention(
    chunks: list[tuple[dict, float]],  # (chunk, relevance_score) sorted by score
) -> list[dict]:
    """
    Place highest-relevance chunks at beginning and end.
    Fill middle with lower-relevance chunks.
    """
    if len(chunks) <= 2:
```

```
        return [chunk for chunk, _ in chunks]

    sorted_chunks = [chunk for chunk, _ in sorted(chunks, key=lambda x: x[1], reverse=True)]

    result = []
    toggle = True
    for chunk in sorted_chunks:
        if toggle:
            result.insert(0, chunk)
        else:
            result.append(chunk)
        toggle = not toggle

    return result
```

Chronological ordering suits documents with temporal structure (news articles, changelogs, meeting notes) and the model can reason "the most recent document says X, the earlier document said Y" only if ordering is chronological.

Document-coherent grouping keeps chunks from the same document together rather than interleaving them with chunks from other documents, preserving intra-document coherence and making citations cleaner:

```
def group_by_document(chunks: list[dict]) -> list[dict]:
    """
    Group chunks from the same document together,
    ordering groups by the highest-scoring chunk in each group.
    """
    from collections import defaultdict

    groups = defaultdict(list)
    for chunk in chunks:
        doc_id = chunk.get('document_id', chunk.get('source', 'unknown'))
        groups[doc_id].append(chunk)

    def group_score(doc_id):
        return max(c.get('relevance_score', 0) for c in groups[doc_id])

    ordered_groups = sorted(groups.keys(), key=group_score, reverse=True)

    result = []
    for doc_id in ordered_groups:
        result.extend(groups[doc_id])

    return result
```

18.6 Context window budget management

The context window has a fixed size and managing what fits requires explicit token accounting.

18.6.1 Budget allocation

```
def allocate_context_budget(
    model_context_window: int,          # e.g., 128000 tokens
```

```
    system_prompt_tokens: int,          # e.g., 500 tokens
    query_tokens: int,                  # e.g., 50 tokens
    conversation_history_tokens: int,   # e.g., 2000 tokens
    max_completion_tokens: int,         # e.g., 1000 tokens
    safety_margin: float = 0.95,        # don't fill to 100%
) -> int:
    """Returns the number of tokens available for retrieved context."""
    reserved = (
        system_prompt_tokens
        + query_tokens
        + conversation_history_tokens
        + max_completion_tokens
    )
    available = int(model_context_window * safety_margin) - reserved
    return max(0, available)

# Example
context_budget = allocate_context_budget(
    model_context_window=128000,
    system_prompt_tokens=500,
    query_tokens=50,
    conversation_history_tokens=2000,
    max_completion_tokens=1000,
)
# context_budget   118,175 tokens for retrieved context
```

The safety margin (5–10%) prevents edge cases where token counting is slightly off from causing the request to fail.

18.6.2 Filling the budget

Select chunks in relevance order until the budget is filled:

```
def select_chunks_within_budget(
    ranked_chunks: list[dict],
    token_budget: int,
    tokenizer,
) -> list[dict]:
    selected = []
    tokens_used = 0

    for chunk in ranked_chunks:
        chunk_tokens = len(tokenizer.encode(chunk['text']))
        chunk_tokens_with_overhead = chunk_tokens + 50  # headers, delimiters

        if tokens_used + chunk_tokens_with_overhead <= token_budget:
            selected.append(chunk)
            tokens_used += chunk_tokens_with_overhead
        else:
            break  # usually better to skip than truncate a chunk mid-thought

    return selected
```

For smaller context windows (8k, 16k), budget management is critical and you may only fit 3–5 chunks. For large context windows (128k+), it becomes less constraining, but long contexts have their own failure modes: reduced attention precision, higher cost, and longer TTFT.

18.6.3 Handling conversation history

In a multi-turn conversation, prior turns compete with retrieved context for the same budget. Strategies for managing this tradeoff: a **fixed history window** (keep only the last N turns, simple, predictable budget), **summarization of older turns** (use the model to compress turns that fall outside the window), or **selective history** (identify which prior turns are actually relevant to the current query and include only those).

```
def compress_conversation_history(
    history: list[dict],
    max_history_tokens: int,
    tokenizer,
    llm,
) -> list[dict]:
    total_tokens = sum(len(tokenizer.encode(msg['content'])) for msg in history)

    if total_tokens <= max_history_tokens:
        return history

    cutoff = len(history) // 2
    old_turns, recent_turns = history[:cutoff], history[cutoff:]

    old_text = "\n".join(f"{m['role']}: {m['content']}" for m in old_turns)
    summary = llm.complete(
        f"Summarize this conversation excerpt in 2-3 sentences:\n{old_text}"
    )

    summary_message = {"role": "system", "content": f"[Earlier conversation summary]: {summary}"}
    return [summary_message] + recent_turns
```

18.7 Deduplication and redundancy handling

Multiple retrieved chunks may contain the same information. Passing duplicates wastes context window space and can cause the model to over-weight the duplicated information, a specific instance of the general principle that repetition biases generation.

18.7.1 Exact deduplication

```
def deduplicate_chunks(chunks: list[dict]) -> list[dict]:
    seen_hashes = set()
    unique_chunks = []

    for chunk in chunks:
        content_hash = hashlib.sha256(chunk['text'].encode()).hexdigest()
        if content_hash not in seen_hashes:
            seen_hashes.add(content_hash)
            unique_chunks.append(chunk)

    return unique_chunks
```

18.7.2 Near-duplicate detection

Chunks from overlapping sources (the same document crawled from multiple URLs, a document and its summary) may be semantically identical without being textually identical:

```
def remove_near_duplicates(
    chunks: list[dict],
    embeddings: np.ndarray,
    similarity_threshold: float = 0.95,
) -> list[dict]:
    """Remove chunks that are near-duplicates of already-selected chunks.
    Assumes chunks are ordered by relevance (most relevant first)."""
    selected_indices = []
    selected_embeddings = []

    for i, (chunk, embedding) in enumerate(zip(chunks, embeddings)):
        if not selected_embeddings:
            selected_indices.append(i)
            selected_embeddings.append(embedding)
            continue

        sims = cosine_similarity([embedding], selected_embeddings)[0]
        if sims.max() < similarity_threshold:
            selected_indices.append(i)
            selected_embeddings.append(embedding)

    return [chunks[i] for i in selected_indices]
```

Near-duplicate removal matters most for web-scraped corpora where the same content appears on many pages, documents with headers and footers appearing in many chunks, and FAQ-style content where similar questions have near- identical answers.

18.8 Handling contradictions in retrieved context

A well-curated corpus rarely contradicts itself but a real-world corpus documentation with multiple versions, policies that have changed over time, information from different departments frequently does.

When retrieved chunks contradict each other, naive context construction passes the contradiction to the model without flagging it. The model either picks one answer arbitrarily, hedges without clearly indicating the contradiction, or fails to notice it at all.

18.8.1 Detecting contradictions at context construction time

```
def detect_potential_contradictions(
    chunks: list[dict],
    query: str,
    llm,
) -> list[tuple[int, int, str]]:
    """Check for potential contradictions between chunk pairs.
    Returns list of (chunk_i, chunk_j, explanation) tuples."""
    contradictions = []

    for i in range(len(chunks)):
```

```
        for j in range(i+1, len(chunks)):
            prompt = f"""Do these two passages contradict each other on any point
relevant to the query? If yes, explain the contradiction in one sentence.
If no, respond with "NO CONTRADICTION".

Query: {query}

Passage 1: {chunks[i]['text'][:500]}

Passage 2: {chunks[j]['text'][:500]}"""

            response = llm.complete(prompt).strip()
            if "NO CONTRADICTION" not in response.upper():
                contradictions.append((i, j, response))

    return contradictions
```

This approach requires $O(n^2)$ LLM calls for n chunks, expensive for large candidate sets. Use it selectively for high-stakes queries or when the corpus is known to have version-specific content.

18.8.2 Flagging contradictions in the prompt

```
def format_context_with_contradiction_flags(
    chunks: list[dict],
    contradictions: list[tuple[int, int, str]],
) -> str:
    formatted_chunks = format_chunks_with_metadata(chunks)

    if not contradictions:
        return formatted_chunks

    contradiction_notes = "\n".join([
        f"Note: Sources {i+1} and {j+1} may conflict: {explanation}"
        for i, j, explanation in contradictions
    ])

    return f"{formatted_chunks}\n\n[CONTEXT NOTES]\n{contradiction_notes}"
```

Explicitly flagging contradictions causes the model to acknowledge them in its response rather than silently picking one side.

18.9 Query placement and framing

The placement and phrasing of the user query relative to the context affects how the model processes both.

Context then query (standard for most LLMs) reads context before seeing the query, processing it without knowing specifically what to look for.

Query then context putting the question first, then the relevant documentation, then the instruction to answer based on it above, lets the model know what to look for before it reads the context, potentially improving attention to relevant sections. Some models handle this better; others expect context before query. Evaluate empirically for your specific model.

18.9.1 Explicit instruction placement

Instructions about how to use the context are more effective when placed immediately adjacent to the context:

```
def build_rag_prompt(
    system_prompt: str,
    context_chunks: list[dict],
    query: str,
    conversation_history: list[dict] = None,
) -> list[dict]:

    formatted_context = format_chunks_xml(context_chunks)
    messages = [{"role": "system", "content": system_prompt}]

    if conversation_history:
        messages.extend(conversation_history[:-1])

    user_content = f"""Based on the following documentation excerpts, answer the question.
If the excerpts don't contain the answer, say so.

<documentation>
{formatted_context}
</documentation>

Question: {query}"""

    messages.append({"role": "user", "content": user_content})
    return messages
```

18.10 Context sufficiency checking

Before passing context to the generation model, check whether it actually contains information relevant to the query. If the retrieved context is insufficient, a well-configured RAG system should say so rather than hallucinating.

18.10.1 Lightweight sufficiency check

```
def check_context_sufficiency(
    query: str,
    context_chunks: list[dict],
    embedding_model,
    min_relevance_threshold: float = 0.4,
) -> bool:
    """Quick check: is any retrieved chunk sufficiently similar to the query
    to likely contain useful information?"""
    if not context_chunks:
        return False

    query_embedding = embedding_model.encode(
        f"query: {query}", normalize_embeddings=True
    )

    for chunk in context_chunks:
```

```
        chunk_embedding = embedding_model.encode(
            f"passage: {chunk['text']}", normalize_embeddings=True
        )
        similarity = float(np.dot(query_embedding, chunk_embedding))
        if similarity >= min_relevance_threshold:
            return True

    return False
```

If this returns False, respond with a pre-defined "insufficient context" response rather than sending low-quality context to the generation model.

18.10.2 LLM-based sufficiency check

For higher accuracy at the cost of an additional LLM call:

```
def llm_check_sufficiency(query: str, context: str, llm) -> bool:
    prompt = f"""Does the following context contain sufficient information
to answer the question? Respond with only YES or NO.

Question: {query}

Context:
{context[:2000]}"""

    response = llm.complete(prompt).strip().upper()
    return response.startswith("YES")
```

18.11 Structured context for structured queries

Some queries are inherently structured, they ask about specific fields, want tabular comparison, or require numerical computation. Unstructured text chunks are often the wrong format for these queries.

18.11.1 Tabular context

For queries comparing multiple options or entities:

```
def format_tabular_context(
    entities: list[str],
    attribute_chunks: dict[str, dict[str, str]],  # entity → attribute → value
) -> str:
    all_attributes = set()
    for entity_data in attribute_chunks.values():
        all_attributes.update(entity_data.keys())

    attributes = sorted(all_attributes)
    header = "| Attribute | " + " | ".join(entities) + " |"
    separator = "|---|" + "---|" * len(entities)
    rows = []

    for attr in attributes:
        row_values = [
            attribute_chunks.get(entity, {}).get(attr, "N/A")
```

```
            for entity in entities
        ]
        rows.append(f"| {attr} | " + " | ".join(row_values) + " |")

    return "\n".join([header, separator] + rows)
```

For the query "compare the pricing of the Starter and Enterprise plans," a tabular context is dramatically more useful than separate text chunks about each plan.

18.11.2 Key-value context

For queries asking about specific properties of a specific entity:

```
def format_entity_context(entity_data: dict, entity_name: str) -> str:
    lines = [f"# {entity_name}"]
    for key, value in entity_data.items():
        if isinstance(value, list):
            lines.append(f"**{key}**: {', '.join(str(v) for v in value)}")
        else:
            lines.append(f"**{key}**: {value}")
    return "\n".join(lines)
```

18.12 Citation and attribution in the response

Well-constructed context enables accurate citation in the response. Encourage citation by including source identifiers in the context (as shown above), instructing the model to cite in the system prompt, and verifying citations after generation.

18.12.1 Post-generation citation verification

```
def verify_citations(
    response: str,
    context_chunks: list[dict],
) -> dict:
    """Check that cited sources exist in the context."""
    import re

    citations = re.findall(r'\[Source (\d+)[^\]]*\]', response)
    results = {
        "total_citations": len(citations),
        "valid_citations": 0,
        "invalid_citations": [],
    }

    for citation_num in citations:
        idx = int(citation_num) - 1
        if 0 <= idx < len(context_chunks):
            results["valid_citations"] += 1
        else:
            results["invalid_citations"].append(citation_num)

    return results
```

Hallucinated citations (where the model references "[Source 7]" when only 5 sources were provided) are a specific failure mode that citation verification catches. Log these as retrieval quality signals: if the model needs a source that was not retrieved, the retrieval stage is missing relevant content, and this signal should feed back into the retrieval evaluation process.

18.13 Putting it all together

A complete context construction function, drawing on every technique in this chapter:

```python
def construct_rag_context(
    query: str,
    ranked_chunks: list[dict],           # already retrieved and re-ranked
    conversation_history: list[dict],
    system_prompt: str,
    tokenizer,
    embedding_model,
    model_config: dict,
) -> list[dict]:
    """Full context construction pipeline.
    Returns a messages list ready to send to the generation model."""

    # 1. Check context sufficiency
    if not check_context_sufficiency(query, ranked_chunks, embedding_model):
        return build_no_context_response_prompt(query, system_prompt)

    # 2. Deduplicate
    chunks = remove_near_duplicates(
        ranked_chunks, compute_embeddings(ranked_chunks, embedding_model)
    )

    # 3. Allocate token budget
    context_budget = allocate_context_budget(
        model_context_window=model_config['context_window'],
        system_prompt_tokens=count_tokens(system_prompt, tokenizer),
        query_tokens=count_tokens(query, tokenizer),
        conversation_history_tokens=count_tokens(conversation_history, tokenizer),
        max_completion_tokens=model_config['max_completion_tokens'],
    )

    # 4. Select chunks within budget
    chunks = select_chunks_within_budget(chunks, context_budget, tokenizer)

    # 5. Order for attention optimization
    chunks = order_chunks_for_attention(
        [(chunk, chunk.get('relevance_score', 0)) for chunk in chunks]
    )

    # 6. Group by document for coherence
    chunks = group_by_document(chunks)

    # 7. Format context
    formatted_context = format_chunks_xml(chunks)
```

```
    # 8. Build messages
    return build_rag_prompt(
        system_prompt=system_prompt,
        context_chunks=chunks,
        query=query,
        conversation_history=conversation_history,
    )
```

18.14 Key takeaways

- Context construction is the bridge between retrieval and generation; excellent retrieval with poor context construction produces poor answers — the model can only reason about what it is actually shown
- The system prompt must explicitly instruct the model how to use retrieved context, what to do when context is insufficient, and how to cite sources — a minimal "answer based on context" instruction is not enough for production
- Metadata-enriched or XML-delimited formatting enables clean citations and helps the model assess source credibility; include document title, section, and recency information
- Chunk ordering matters because of the lost-in-the-middle effect (chapter 05): place the most important chunks at the beginning or end, not buried in the middle of a long context
- Explicitly account for the context window budget by token, leaving safety margins; all components — system prompt, history, context, query, completion — compete for the same finite space
- Deduplicate near-duplicate chunks before passing to the model; repeated content wastes budget and inflates the model's confidence in that information
- When chunks contradict each other, flag the contradiction explicitly in the prompt rather than leaving the model to silently pick one side
- Check context sufficiency before generation; a system that says "I don't have enough information" is more useful than one that hallucinates an answer
- Post-generation citation verification catches hallucinated source references and provides retrieval quality signals that feed back into retrieval evaluation

---

18.15 Further reading

- Liu et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts.* — The empirical study establishing position bias in long-context LLMs; the foundation for the ordering strategies in this chapter.
- Shi et al. (2023). *Large Language Models Can Be Easily Distracted by Irrelevant Context.* — Demonstrates that irrelevant context actively hurts model performance, not just wastes space.
- Anthropic (2024). *Prompt Engineering Guide: Long Document QA.* — Best practices for XML-delimited context in Claude.
- Asai et al. (2023). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.* — A model that generates its own retrieval necessity signals and critiques its own use of context.
- Xu et al. (2023). *RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation.* — Contextual compression for RAG, extending the compression techniques introduced in chapter 23.
- Gao et al. (2023). *Enabling Large Language Models to Generate Text with Citations.* — A systematic approach to citation in RAG-generated text.

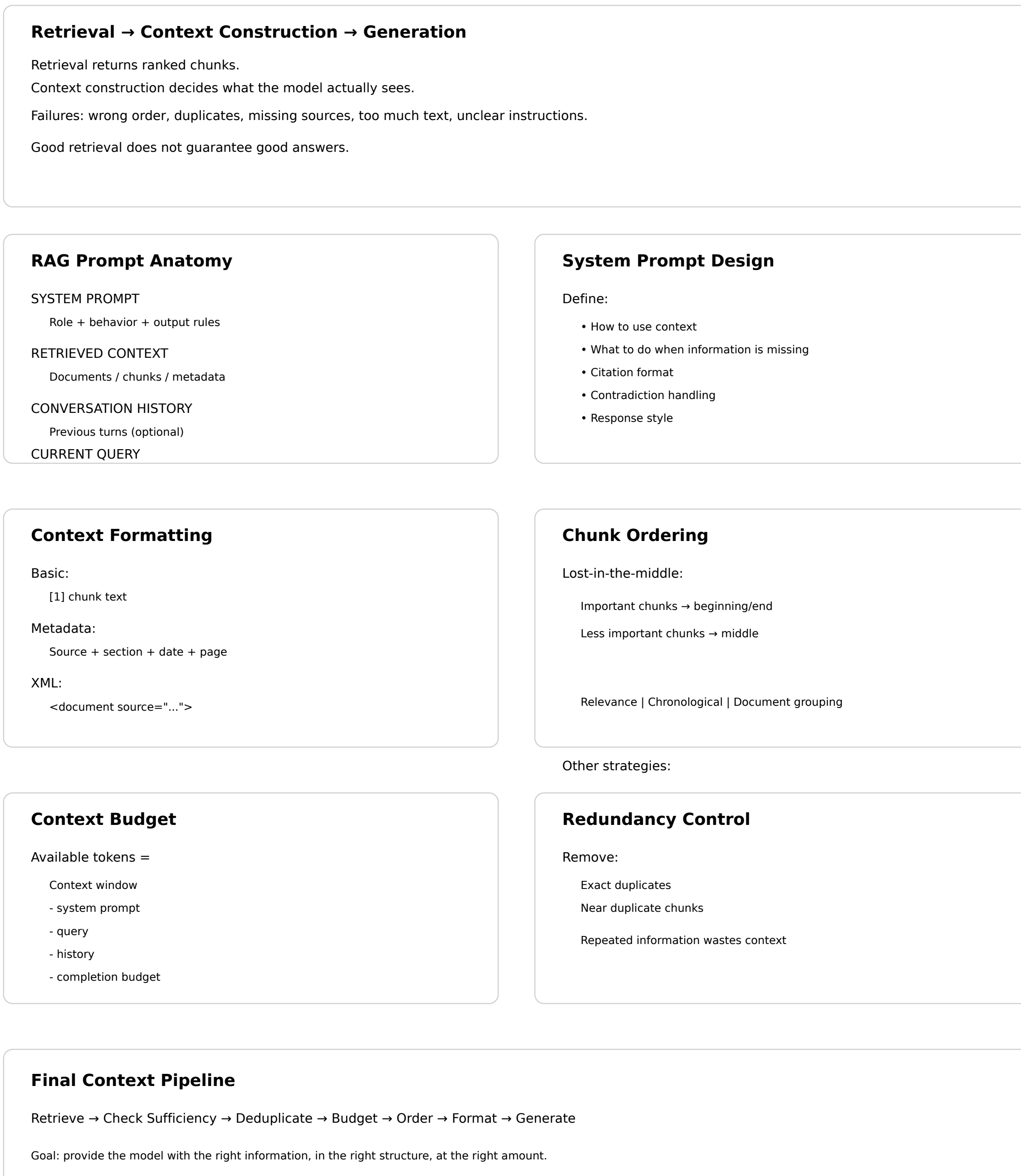


Figure 18.2: Cheat sheet.

Chapter 19

Grounded Generation

The canonical question for this chapter: *The model has the right context. Why does it still sometimes ignore it, supplement it with hallucinated facts, or generate claims the context never made and what can you do about all three?*

Where are we?

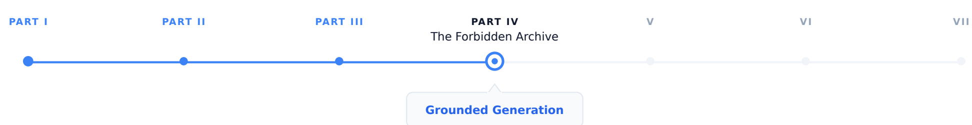


Figure 19.1: The journey through the Model Mind.

The prompt is assembled and sent. This chapter covers everything that happens next: how the model uses (or fails to use) the context it was given, how to detect when it goes wrong, and how to design a system that fails gracefully when retrieval was insufficient. Context construction determined what the model could see. Grounded generation determines whether it actually uses what it sees.

19.1 The three failure modes

Retrieval solves the knowledge problem by finding the right information. Grounded generation addresses the fidelity problem by ensuring the model accurately incorporates that information into its responses. These are distinct challenges that require different solutions.

Even with perfect retrieval and careful context construction, generation fails in three distinct ways:

Hallucination despite retrieval. The model supplements retrieved context with parametric knowledge. A retrieved chunk states "the default token expiration is 3600 seconds." The model generates that fact correctly, then adds: "which aligns with the OAuth 2.0 standard recommendation of one hour." The second clause came from training data, not the retrieved document. It may be true. It may not be. The user cannot tell, and neither can you without external verification.

Context ignorance. The model ignores the retrieved context entirely and answers from parametric memory. This happens when training data contains a strong competing answer, when the relevant passage sits in the middle of a long context, or when instruction-following is insufficiently grounded. The response looks like a success but is not grounded in the provided evidence.

Miscalibrated confidence. The model answers confidently when it should express uncertainty, or hedges when the context clearly supports a direct answer. Both are calibration failures, the model's expressed certainty does not match the actual support in the retrieved context.

All three occur even in systems with high retrieval recall. Solving them requires techniques applied at and after inference, which is what this chapter covers.

19.2 Extending the system prompt for faithfulness

Previous chapter built a production system prompt covering role, context usage, citation format, and output structure. That prompt is necessary but not sufficient for faithfulness. What it does not yet specify is the relationship between retrieved context and parametric memory, which source wins when they conflict, and how the model should signal when context is absent.

```
GROUNDING REQUIREMENTS:
- Base every factual claim on the provided documentation excerpts
- Do not supplement the documentation with knowledge from your training data,
  even if you are confident it is correct
- If you find yourself about to state something not in the provided excerpts,
  stop and instead say: "The documentation does not address [X]"
- The provided documentation is authoritative and if it contradicts what you
  know from training, trust the documentation

CALIBRATION:
- Distinguish between "the documentation states X," "the documentation implies X,"
  and "the documentation does not address X"
- Do not express more certainty than the documentation supports
- If the documentation is ambiguous, acknowledge the ambiguity explicitly
```

These two sections directly target the first two failure modes. The citation instructions already present in the previous chapter's prompt address the third by creating an audit trail that makes context ignorance visible.

19.2.1 Grounding instruction strength

For applications where faithfulness is critical (medical, legal, financial) use maximum grounding strength:

```
STRICT GROUNDING:
You are a documentation lookup system, not an assistant.
Your only job is to find and present information from the provided excerpts.
If the answer is in the excerpts, quote or closely paraphrase the relevant
passage. If it is not, say exactly: "Not found in provided documentation."
Do not add context, explanation, or interpretation beyond what the
documentation states.
```

Strict grounding reduces hallucination substantially but also reduces response quality for complex questions, the model produces excerpts rather than synthesis. Use it for narrow, high-stakes retrieval tasks and standard grounding for broad, synthesis-heavy tasks.

19.3 Chain-of-thought for faithfulness

Requiring the model to reason explicitly about the context before answering reduces hallucination by forcing reference to retrieved content before generating claims.

```
def build_cot_grounding_prompt(
    query: str,
    context: str,
) -> str:
    return f"""You have access to the following documentation:

{context}

Before answering, identify the relevant passages from the documentation.
Then answer based only on those passages.

Question: {query}

Step 1 - Identify relevant passages:
[List the specific passages from the documentation relevant to this question]

Step 2 - Answer based on those passages:
[Answer the question using only the passages identified above]"""
```

The intermediate step grounds the model in specific text before it begins generating its answer. This works because of how autoregressive generation functions: each generated token is conditioned on all preceding tokens. When the model generates a factual claim in Step 2, it is conditioned on having just explicitly cited the source passage in Step 1. That citation increases the probability of the faithful answer.

19.4 Faithfulness at the token level

Faithfulness can be examined at the level of individual tokens, not just full responses. Understanding where confidence drops during generation reveals where the model is departing from retrieved context and filling from parametric memory.

19.4.1 Flagging low-confidence spans

Token logprobs indicate the model's confidence at each position. A sequence of high-logprob tokens followed by a sudden drop suggests the model is uncertain, often where it is bridging from retrieved context into parametric knowledge:

```
def flag_low_confidence_tokens(
    tokens: list[str],
    logprobs: list[float],
    threshold: float = -2.0,
) -> list[tuple[str, float, bool]]:
    """Returns (token, logprob, is_low_confidence) tuples."""
    return [
        (token, logprob, logprob < threshold)
        for token, logprob in zip(tokens, logprobs)
    ]

def highlight_uncertain_spans(
    token_flags: list[tuple[str, float, bool]]
) -> str:
```

```
    """Mark low-confidence spans for human review."""
    result = []
    in_uncertain_span = False

    for token, logprob, is_low_confidence in token_flags:
        if is_low_confidence and not in_uncertain_span:
            result.append("<<")
            in_uncertain_span = True
        elif not is_low_confidence and in_uncertain_span:
            result.append(">>")
            in_uncertain_span = False
        result.append(token)

    if in_uncertain_span:
        result.append(">>")

    return "".join(result)
```

This is a heuristic, models can be confidently wrong. But low-logprob spans are disproportionately likely to contain hallucinations and are worth flagging for review in high-stakes applications. The threshold of -2.0 is a starting point; calibrate it against labeled examples from your own domain.

19.5 Claim extraction and verification

The most rigorous post-generation check: extract atomic claims from the response and verify each against the retrieved context independently.

```
def extract_atomic_claims(response: str, llm) -> list[str]:
    prompt = f"""Extract all factual claims from the following text as a
JSON list. Each claim should be atomic (one fact per claim) and self-contained.

Text: {response}

Return only the JSON array of claim strings."""

    return json.loads(llm.complete(prompt))

def verify_claim_against_context(
    claim: str,
    context: str,
    llm,
) -> dict:
    prompt = f"""Does the following context support, contradict, or neither
support nor contradict this claim? Respond with exactly one of:
SUPPORTED / CONTRADICTED / NOT_ADDRESSED

Context: {context[:3000]}

Claim: {claim}

Verdict:"""
```

```
    verdict = llm.complete(prompt).strip().upper()
    return {
        "claim":            claim,
        "verdict":          verdict,
        "is_hallucination": verdict == "NOT_ADDRESSED",
        "is_contradiction": verdict == "CONTRADICTED",
    }

def verify_response_faithfulness(
    response: str,
    context: str,
    llm,
) -> dict:
    claims        = extract_atomic_claims(response, llm)
    verifications = [verify_claim_against_context(c, context, llm) for c in claims]

    total        = len(verifications)
    supported    = sum(1 for v in verifications if v["verdict"] == "SUPPORTED")
    hallucinated = sum(1 for v in verifications if v["is_hallucination"])
    contradicted = sum(1 for v in verifications if v["is_contradiction"])

    return {
        "total_claims":       total,
        "supported":          supported,
        "hallucinated":       hallucinated,
        "contradicted":       contradicted,
        "faithfulness_score": supported / total if total > 0 else 0,
        "claim_details":      verifications,
    }
```

This pipeline (extract claims, verify each) is the core of RAGAS and similar automated faithfulness evaluation frameworks. The `NOT_ADDRESSED` verdict is the direct hallucination signal: a claim the model made that no retrieved chunk supports. The `CONTRADICTED` verdict is rarer but more serious meaning the model asserted something that the retrieved context explicitly refutes.

19.6 Self-RAG: faithfulness signals baked into generation

Standard RAG retrieves once before generation. Self-RAG (Asai et al., 2023) integrates retrieval into the generation process itself, using special tokens to let the model signal when it needs more context and whether its claims are supported by what it has.

19.6.1 The Self-RAG special tokens

Self-RAG fine-tunes a model to generate four types of special tokens alongside regular text:

- `[Retrieve]` — model signals it needs additional context
- `[IsRel]` — signals whether a retrieved passage is relevant to the query
- `[IsSup]` — signals whether the generated claim is supported by retrieved context
- `[IsUse]` — signals whether the retrieved passage was useful for the response

19.6.2 Self-RAG generation example

```
User: "How long does the default token last, and can it be extended?"
```

```
Model begins generating:
"The default token expiration time is [Retrieve]"
  → Retrieval triggered → context: "default is 3600 seconds"
"The default token expiration time is 3600 seconds [IsSup: fully supported].
This can be configured [Retrieve]"
  → Retrieval triggered → context: "token_ttl supports up to 86400 seconds"
"This can be configured via the token_ttl parameter, with a maximum of
86400 seconds [IsSup: fully supported]."
```

The model generates until it recognizes uncertainty, retrieves, continues, interleaving retrieval and generation iteratively. Each `[IsSup]` annotation makes the model's grounding status explicit, enabling the system to flag or filter unsupported claims before they reach the user.

Self-RAG produces better-calibrated and more faithfully grounded responses than standard RAG on tasks that require multi-step reasoning. The requirement to fine-tune a model with these special tokens is its main limitation, it cannot be applied to a frozen API-served model. Self-RAG is available as fine-tuned model variants rather than as a prompting technique.

19.7 FLARE: active retrieval during generation

FLARE (Jiang et al., 2023) addresses the same problem as Self-RAG but without fine-tuning, using logprob uncertainty as the retrieval trigger:

```python
def flare_generate(
    query: str,
    initial_context: str,
    retriever,
    llm,
    max_iterations: int = 5,
    uncertainty_threshold: float = -2.0,
) -> str:
    current_context  = initial_context
    generated_so_far = ""

    for _ in range(max_iterations):
        response = llm.complete_with_logprobs(
            prompt=build_rag_prompt(query, current_context, generated_so_far)
        )

        if min(response.logprobs) >= uncertainty_threshold:
            # Confident - accept this sentence and continue
            generated_so_far += " " + response.text
        else:
            # Uncertain - retrieve before generating this sentence
            new_chunks       = retriever.retrieve(response.text, top_k=3)
            current_context += "\n\n" + format_chunks(new_chunks)
            # Do not advance generated_so_far - regenerate with augmented context

        if generated_so_far.rstrip().endswith(('.', '!', '?')):
            break

    return generated_so_far.strip()
```

When the model's next-token confidence drops below the threshold, generation pauses, a new retrieval pass runs against the partial output as a query, and the augmented context is used to regenerate from that point. The model never produces the low-confidence sentence ,it retrieves first.

FLARE works well for long-form answers that span multiple topics, particularly when the full scope of required context is not known at initial retrieval time. The cost is higher latency (multiple retrieval-generation cycles) and implementation complexity. For standard interactive queries, single-round retrieval with strong grounding instructions is sufficient.

19.8 Handling insufficient context

When retrieved context does not contain the information needed to answer the query, the correct behavior is acknowledgment, not fabrication. This is the most important single behavior in grounded generation: a system that says "I don't have that information" is more useful and more trustworthy than one that confidently provides a fabricated answer.

Previous chapter covered *detecting* context insufficiency before sending the prompt. This section covers what the model should do when it encounters insufficiency during generation and how to verify it behaves correctly afterward.

19.8.1 Designing the insufficient-context response

```
INSUFFICIENT CONTEXT INSTRUCTIONS:
If the documentation does not contain sufficient information to answer:
1. State specifically what is missing:
   "The documentation does not address [X]"
2. State what related information is available, if any:
   "The documentation does address [Y], which may be related"
3. Do not speculate about the missing information
4. Suggest where the user might find the answer:
   "You may want to consult [appropriate resource type]"
```

The specificity of point 1 matters more than it looks. "I don't know" is less useful than "The documentation addresses token expiration generally (3600 seconds default) but does not cover enterprise-tier configurations specifically." The latter tells the user exactly what is and is not available, and what to look for elsewhere.

19.8.2 Detecting when the model over-answers

After generation, verify that the model acknowledged insufficiency when it should have rather than hallucinating an answer to fill the gap:

```python
def detect_hallucinated_sufficiency(
    query: str,
    context_chunks: list[dict],
    response: str,
    llm,
) -> bool:
    """Returns True if the response appears to contain facts not in the context."""
    context_preview = "\n".join(c['text'][:500] for c in context_chunks[:3])

    prompt = f"""A model was asked this question:
{query}

It had access to this context:
{context_preview}
```

```
It responded:
{response[:500]}

Does the response contain specific facts NOT present in the context?
Answer YES or NO only."""

    return llm.complete(prompt).strip().upper().startswith("YES")
```

If detected: log the hallucination, trigger a retry with stricter grounding instructions, or flag for human review before the response reaches the user.

19.9 Post-hoc citation attribution

Previous chapter set up numbered source references in the context format and instructed the model to cite them. When the model fails to include citations despite these instructions, they can be added in post-processing by attributing each sentence to the most similar retrieved chunk:

```
def add_citations_to_response(
    response: str,
    context_chunks: list[dict],
    embedding_model,
    similarity_threshold: float = 0.5,
) -> str:
    sentences        = split_into_sentences(response)
    chunk_texts      = [c['text'] for c in context_chunks]
    sent_embeddings  = embedding_model.encode(sentences, normalize_embeddings=True)
    chunk_embeddings = embedding_model.encode(chunk_texts, normalize_embeddings=True)

    cited = []
    for sentence, sent_emb in zip(sentences, sent_embeddings):
        similarities = chunk_embeddings @ sent_emb
        best_idx     = int(similarities.argmax())
        best_sim     = float(similarities[best_idx])

        if best_sim >= similarity_threshold:
            cited.append(f"{sentence} [Source {best_idx + 1}]")
        else:
            cited.append(f"{sentence} [Unverified]")

    return " ".join(cited)
```

The `[Unverified]` tag is the important output here. Sentences that cannot be attributed to any retrieved chunk are direct hallucination candidates. Log them as retrieval quality signals: if the model needed information that was not retrieved, the retrieval stage has a coverage gap.

19.10 Consistency sampling

For high-stakes applications, generate multiple responses at nonzero temperature and check for factual consistency across samples. A correctly grounded response that accurately reflects a single underlying fact should be consistent regardless of sampling randomness. Inconsistency across samples indicates uncertainty, hallucination, or genuine ambiguity in the retrieved context:

```
def check_response_consistency(
    query: str,
    context: str,
    llm,
    n_samples: int = 5,
    temperature: float = 0.7,
) -> dict:
    responses = [
        llm.complete(
            prompt=build_rag_prompt(query, context),
            temperature=temperature,
        )
        for _ in range(n_samples)
    ]

    responses_text = "\n\n---\n\n".join(
        f"Response {i+1}: {r}" for i, r in enumerate(responses)
    )

    assessment = llm.complete(f"""Are these {n_samples} responses consistent
on all factual claims? If not, identify the inconsistencies.

Query: {query}

{responses_text}

Assessment (CONSISTENT or INCONSISTENT with explanation):""")

    return {
        "responses":               responses,
        "consistency_assessment": assessment,
        "is_consistent":           "INCONSISTENT" not in assessment.upper(),
    }
```

This requires n+1 LLM calls per query which is impractical for real-time systems. Use it for: auditing high-stakes responses before delivery, generating faithfulness training data, and diagnosing retrieval gaps (queries that are consistently inconsistent across samples point to missing context rather than model failure).

19.11 Post-generation faithfulness filtering

Combine the techniques above into a filtering layer that sits between generation and delivery. If faithfulness falls below threshold, retry with stricter grounding. If it still fails, return a safe fallback rather than a known-hallucinated response:

```
def generate_with_faithfulness_filter(
    query: str,
    context: str,
    llm,
    max_retries: int = 2,
    faithfulness_threshold: float = 0.9,
) -> dict:
```

```
    grounding_strengths = ["normal", "strict", "very_strict"]

    for attempt, strength in enumerate(grounding_strengths[:max_retries + 1]):
        response     = llm.complete(build_rag_prompt(query, context, strength))
        verification = verify_response_faithfulness(response, context, llm)

        if verification["faithfulness_score"] >= faithfulness_threshold:
            return {
                "response":           response,
                "faithfulness_score": verification["faithfulness_score"],
                "attempts":           attempt + 1,
                "passed_filter":      True,
            }

        log_hallucination_attempt(query, context, response, verification)

    return {
        "response":           "I was unable to generate a sufficiently grounded "
                              "response from the available documentation. Please "
                              "consult the documentation directly or contact support.",
        "faithfulness_score": None,
        "attempts":           max_retries + 1,
        "passed_filter":      False,
    }
```

The escalating grounding strength on retry works because parametric memory and retrieved context compete for influence over each generated token. Stronger grounding instructions shift that competition toward the retrieved evidence. The hard fallback is not a failure state to hide, it is a correct system behavior. Always log these events and treat them as retrieval quality signals: if the model cannot produce a faithful response, either the retrieved context is insufficient or the query cannot be answered from this corpus.

19.12 Grounded generation in production

19.12.1 Logging for debuggability

Every response must be logged with: the query, the retrieved chunk IDs and scores, the assembled context exactly as sent to the model, the generated response, and any faithfulness scores computed. Without this, debugging incorrect answers is intractable at scale, you cannot distinguish retrieval failures from generation failures without seeing both sides of the prompt.

When users report incorrect answers, this log enables exact reconstruction of what the model saw and what it generated. The distinction matters: if the model received the right context and still generated a wrong answer, that is a generation failure. If it received insufficient context, that is a retrieval failure.

19.12.2 Human-in-the-loop for high-stakes responses

For applications where errors have serious consequences, route low-faithfulness responses or high-risk query patterns to human review before delivery:

```
def route_for_review(
    query: str,
    faithfulness_score: float,
    high_risk_patterns: list[str],
```

```
    faithfulness_threshold: float = 0.8,
) -> bool:
    if faithfulness_score < faithfulness_threshold:
        return True
    if any(pattern in query.lower() for pattern in high_risk_patterns):
        return True
    return False
```

19.12.3 Feedback as ground truth

User feedback (thumbs down, explicit corrections, "this is wrong") is the most valuable signal for grounded generation quality. It identifies failure cases that automated faithfulness metrics miss, particularly miscalibrated confidence (where the model was faithful but uncertain answers felt wrong) and domain gaps (where the model was faithful to retrieved context that was itself outdated or incomplete).

19.13 Key takeaways

- Grounded generation fails in three distinct ways — hallucination despite retrieval, context ignorance, and miscalibrated confidence — each requiring a different intervention; retrieval quality alone cannot fix any of them
- The system prompt from chapter 24 needs two additions to address faithfulness: explicit grounding requirements (parametric memory loses to retrieved context) and explicit calibration language (distinguish "states," "implies," and "does not address")
- Chain-of-thought reasoning before answering reduces hallucination by forcing the model to cite source passages before generating claims; Step 1 conditions Step 2 through the autoregressive token dependency
- Token logprobs identify low-confidence spans during generation — where the model is most likely departing from retrieved context into parametric memory
- Claim extraction and verification provides the most diagnostically precise faithfulness signal: `NOT_ADDRESSED` verdicts are hallucinations, `CONTRADICTED` verdicts are the most serious failures
- Self-RAG embeds faithfulness signals into generation through special tokens (`[Retrieve]`, `[IsSup]`) but requires fine-tuning and cannot be applied to frozen API-served models
- FLARE uses logprob uncertainty to trigger mid-generation retrieval, bridging single-round RAG and the fully agentic retrieval architectures in chapter 26
- Post-generation faithfulness filtering with escalating grounding strength and a hard fallback prevents known-hallucinated responses from reaching users; every fallback is a retrieval quality signal
- Log the full context and response for every production query; without it, retrieval failures and generation failures are indistinguishable, and both look the same to the user

---

19.14 Further reading

- Asai et al. (2023). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.* — The Self-RAG framework with special tokens for generation-integrated faithfulness signals.
- Jiang et al. (2023). *Active Retrieval Augmented Generation.* — FLARE; uncertainty-driven retrieval during generation.
- Es et al. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation.* — Comprehensive RAG evaluation covering faithfulness, relevance, and groundedness.
- Min et al. (2023). *FActScoring: Fine-Grained Atomic Evaluation of Factual Precision in Long Form Text Generation.* — Atomic claim verification for faithfulness evaluation.
- Manakul et al. (2023). *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models.* — Consistency-based hallucination detection without ground truth.

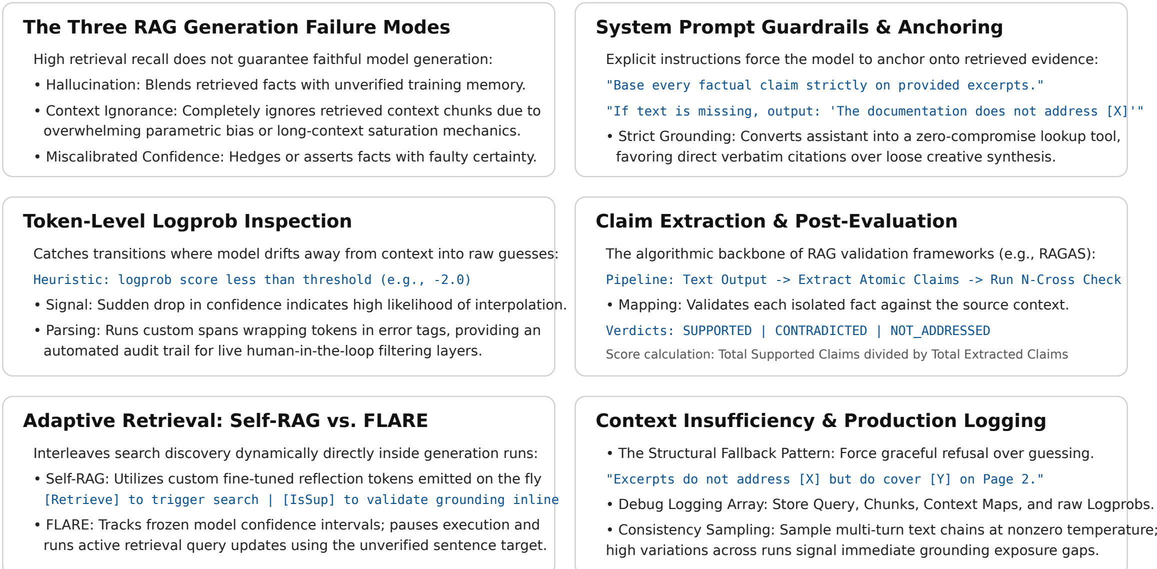


Figure 19.2: Cheat sheet.

- Gao et al. (2023). *Enabling Large Language Models to Generate Text with Citations.* — Systematic inline citation generation and verification.
- Shi et al. (2023). *Large Language Models Can Be Easily Distracted by Irrelevant Context.* — Evidence that irrelevant context causes active performance degradation; the generation-side counterpart to the lost-in-the-middle effect on context construction.

Part V

The Forging of Intelligence

Chapter 20

Data Collection and Curation

The canonical question for this chapter: *Before a model can answer anything, it must read everything. How does the raw internet become training data and why are the choices made here the ones that propagate furthest and are corrected least easily?*

Where are we?

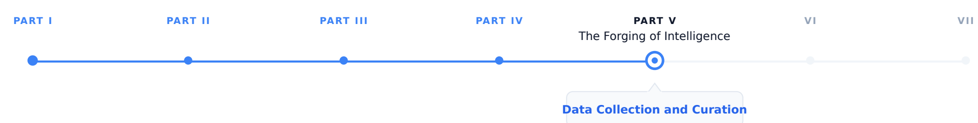


Figure 20.1: The journey through the Model Mind.

Part IV showed how retrieval gives the model access to external knowledge at inference time. Part V rewinds the clock entirely. The model had to learn to do all of that from somewhere. This chapter covers where: the construction of the training corpus that precedes everything else in Part V. Data collection is not a preprocessing step you get out of the way before the real work begins. It is the first architectural decision of the entire system.

20.1 Why data is the real foundation

Every capability an LLM has (reasoning, coding, translation, summarization, answering questions about French geography and quantum mechanics) traces back to the data it was trained on. The model has no built-in knowledge of the world. It has patterns compressed from text. Those patterns are only as good as the text that produced them.

This means data collection is where the model's capabilities are fundamentally shaped. Architecture decisions matter (attention, depth, positional encoding) but modern LLMs share a largely converged architecture. What they know, what they get wrong, what biases they carry, and what blind spots they have are primarily determined by data, not architecture.

Most papers spend one paragraph on data and thirty pages on model design. This is backwards. The architecture of a 2024 model is recognizable from a 2020 model. The data is where they diverge.

20.2 What scale actually means

Training corpora are large enough that intuitions built on normal datasets break down. Some grounding numbers:

GPT-3 trained on roughly 300 billion tokens. LLaMA 2 used 2 trillion tokens. Frontier models from 2024 are estimated to have trained on 10–15 trillion tokens. A token is approximately 0.75 words in English, so 1 trillion tokens is roughly 750 billion words, about 5 million books.

No human has read 5 million books. No team of humans has read 5 million books. The model has, in a statistical sense, absorbed patterns from all of them.

This scale creates an immediate engineering constraint: you cannot curate 10 trillion tokens by hand. You cannot read a meaningful fraction of it. Every quality decision must be made programmatically, which means every quality decision is imperfect. The question is not how to achieve perfect curation but it is how to make systematic decisions that produce the best results across billions of documents without ever seeing most of them.

20.3 The primary sources

Modern LLM training data comes from four broad categories. Real datasets blend all of them in deliberate proportions.

20.3.1 Web crawls

The largest source by volume. Common Crawl has been running automated web snapshots since 2008, releasing monthly snapshots each containing petabytes of raw HTML from billions of pages. The appeal is obvious: the web is the largest corpus of human-written text ever assembled. The problem is equally obvious: most of it is garbage. Spam, SEO content, duplicate pages, auto-generated text, comment sections, scraped aggregator sites, and machine-translated content sit alongside high-quality journalism, technical documentation, and encyclopedic reference.

Raw web crawl data is unusable without extensive filtering. The question is how to filter aggressively enough to remove noise without losing diversity.

20.3.2 Curated text corpora

Books, academic papers, and high-quality long-form writing. Books3, Project Gutenberg, and the Pile's academic subsets (ArXiv papers, PubMed abstracts, Wikipedia) fall into this category. These sources are much smaller by volume than web crawls (Wikipedia in all languages is roughly 20 billion tokens, a rounding error at frontier scale) but they contribute disproportionately to factual grounding, structured reasoning, and citation quality. They are heavily oversampled relative to their share of the internet.

20.3.3 Code repositories

GitHub and other code hosts. This has become a major data category not only because it teaches models to write and understand code, but because code has structural properties that improve general reasoning: it is unambiguous, logical, hierarchical, and extensively annotated with natural language in comments and docstrings.

Models trained on large code corpora show improved performance on non-coding tasks including mathematics and structured reasoning, even when evaluated on tasks that have nothing to do with code. The mechanism is still debated, but the empirical result is consistent across model families.

20.3.4 Instruction and dialogue data

Conversations, Q&A pairs, and instruction-following examples. This category is distinct because it teaches the model the format and register of being helpful, how to respond to a question, how to follow a directive, how to structure an explanation. It is often synthetic (generated by another model) or manually curated (human-written examples). It is typically much smaller in volume than web data but has outsized influence on how the final model behaves at inference time.

20.4 The data pipeline

Raw collection is just the beginning. Every serious training pipeline includes a sequence of filtering, deduplication, and quality scoring steps. Here is a representative pipeline for web data:

```
Raw HTML from crawl


Language identification
(discard non-target languages or assign per-language weights)


Text extraction
(strip HTML tags, boilerplate, navigation menus, ads, footers)


Length and format filtering
(discard very short pages, pages with too many special characters,
 pages where text-to-total-content ratio is too low)


Heuristic quality filtering
(word count, punctuation ratio, stop word ratio,
 line length distribution, duplicate line fraction)


Deduplication
(exact match by hash, near-duplicate via MinHash or SimHash,
 substring deduplication for repeated passages)


Classifier-based quality scoring
(trained model predicts P(high quality); discard below threshold)


Toxicity and safety filtering
(remove documents matching harmful content classifiers)


Domain weighting and mixing
(upsample high-quality sources, set per-domain proportions)


Final training corpus
```

Each step involves tradeoffs. Aggressive quality filtering produces a cleaner corpus but reduces volume, which may hurt diversity and underrepresented language coverage. Loose filtering preserves volume but degrades signal. There is no universally correct threshold, the right settings depend on the target model, the available compute budget, and empirical evaluation on smaller runs.

20.5 Deduplication: the underrated problem

Duplicate content in training data is a serious and underappreciated problem that affects both quality and benchmark integrity.

At web scale, the same article gets republished hundreds of times across aggregator sites, blogs, and content farms. If a document appears 500 times in the training set and once in the test set, the model has not learned to generalize it has only learned to memorize. This inflates benchmark performance and degrades real-world capability.

Worse, repeated exposure to the same document teaches the model that this content is important. A piece of misinformation that went viral and was copied ten thousand times gets disproportionate weight relative to a single careful correction.

Deduplication happens at multiple granularities:

Exact deduplication removes documents with identical content. This is cheap and straightforward, hash the document, compare against a lookup table, discard matches.

Near-duplicate detection removes documents that are semantically identical but not textually identical. Two articles with 95% word overlap but different headlines should not both appear in training. MinHash and SimHash are standard approaches: they produce a fingerprint of the document's n-gram distribution that allows fast approximate comparison across billions of documents.

Substring deduplication removes repeated passages within documents, not just repeated documents. A web page that contains the same boilerplate legal disclaimer 200 times should not contribute 200 unique training examples.

Research from Google (Lee et al., 2022) demonstrated that aggressive deduplication consistently improves model quality even when it reduces total token count. Less data, cleaner signal, better model, a counterintuitive result that has held up across multiple model families.

20.6 Quality filtering: what "quality" means

Quality filtering is philosophically tricky because "quality" is a value judgment and the judgment embedded in your filtering approach shapes what the model learns is normal.

20.6.1 Heuristic filtering

The most common approach uses signals computed from the document itself:

- Average word length (too short suggests gibberish; too long suggests technical jargon or non-natural language)
- Ratio of alphabetic characters to total characters (low ratio suggests spam or code fragments)
- Presence of stop words (a proxy for natural language vs. keyword lists)
- Distribution of line lengths (regular short lines suggest poetry or bullet content; extremely variable suggests scraped layout artifacts)
- Ratio of duplicate lines within a document (high ratio suggests boilerplate or repeated headers)
- Presence of known spam or adult content patterns

20.6.2 Classifier-based filtering

The more sophisticated approach trains a classifier on examples of high and low-quality text. You take a small set of documents you believe are high quality (Wikipedia articles, selected books, academic papers), a set you believe are low quality (scraped spam, auto-generated content), train a binary classifier, and use its score to filter the full corpus.

This approach was used to produce C4 (Colossal Clean Crawled Corpus), where a fastText classifier trained on Wikipedia and Common Crawl data scored documents and filtered those below a threshold. The resulting corpus was significantly smaller than the raw crawl but dramatically higher quality.

The bias this introduces. The classifier inherits the biases of its positive examples. If high-quality training examples are predominantly English and from Western academic and journalistic traditions, the classifier will score content from other cultural, linguistic, and domain contexts lower, not because those documents are low quality, but because they are dissimilar from the examples the classifier was trained to recognize as high quality.

20.6.3 Perplexity-based filtering

A variant: use a smaller, already-trained language model to score each document's perplexity. Very high perplexity (the model is very surprised by the text) suggests low-quality or anomalous content. Very low perplexity (the model already knows this text well) may suggest repeated or memorized content.

The limitation: a small model trained on web text will score high-quality text in specialized domains (medical literature, legal documents, code documentation) as high perplexity simply because it is unfamiliar. Perplexity filtering can inadvertently remove exactly the specialized knowledge that makes the training data valuable.

20.7 Domain mixing and data proportions

Collecting data is one problem and deciding how much of each source to include is another, the decision matters as much as collection quality.

Frontier models deliberately oversample high-quality sources relative to their share of the raw internet. A representative mixing strategy:

Source	Share of training tokens	Share of raw internet
Web crawl (filtered)	~45%	~99%
Books	~20%	<0.1%
Code	~15%	~1%
Academic papers	~10%	<0.1%
Wikipedia	~5%	<0.01%
Instruction / dialogue	~5%	near zero

These proportions are not derived from first principles. They are found through empirical ablations: train smaller models on different mixes, evaluate across benchmark suites, and iterate toward the mix that produces the best results. This is expensive and also one of the least documented aspects of frontier model development. Mixing ratios are among the most guarded details in model technical reports.

20.7.1 Domain mixing and capability

Mixing decisions directly affect model capabilities. A model trained with higher code proportion shows better performance on code-related tasks, on mathematical reasoning, and (empirically, though mechanistically unclear) on structured reasoning tasks generally. A model with more multilingual data performs better across languages. A model with more instruction data is more naturally helpful without additional fine-tuning.

Changing the mix at frontier scale (adjusting proportions for a 15 trillion token training run) is a multi-month commitment. Getting it right requires either expensive experiments at smaller scale (with uncertain extrapolation to full scale) or expensive mistakes at full scale.

20.8 The data contamination problem

A fundamental requirement of valid evaluation is that the test set must not appear in the training data. If the model has seen the answers, the evaluation tells you nothing about generalization.

At the scale of modern training corpora, this is genuinely difficult to guarantee. The internet contains solutions to many standard benchmarks: HumanEval coding problems appear on GitHub; MMLU questions appear in study guides and Quora discussions; GSM8K math problems appear on math tutoring sites; BIG-Bench tasks appear in academic papers that post-date the benchmark but pre-date many models.

Contamination detection requires: 1. Collecting all known evaluation datasets 2. Running n-gram matching between evaluation examples and training documents 3. Removing or flagging contaminated training documents 4. Reporting contamination rates explicitly in publications

Many papers do this imperfectly or not at all. When a model achieves dramatically higher scores than predecessors on a specific benchmark, contamination is one of the first alternative hypotheses to evaluate. The difficulty of ruling it out cleanly is one reason the field has moved toward harder-to-memorize benchmarks, live coding challenges, human preference evaluation, new tasks created after the training cutoff.

20.9 The web as a cultural artifact

One underappreciated aspect of using the web as training data is the fact that the web is not a neutral sample of human knowledge. It overrepresents certain voices, certain languages, certain time periods, and certain topics.

English accounts for roughly half of web content despite being spoken by only 17% of the world's population. Technical and academic content from the United States and Western Europe is disproportionately represented. Recent content is more represented than historical content. Content from wealthy countries with high internet penetration is more represented than content from countries with lower penetration.

A model trained primarily on this data will reflect these asymmetries: it will know more about some cultures, regions, and perspectives than others. It will have stronger capabilities in some languages than others. This is not a bug in any simple sense (it is a direct consequence of training on the data that exists) but it has real downstream consequences for who the model serves well and who it serves poorly.

These asymmetries can be partially corrected through deliberate data sourcing: specifically mining underrepresented language content, partnering with institutions that have access to non-web text corpora, and applying language-specific oversampling. They cannot be fully corrected because the underlying data simply does not exist in comparable volume for all languages and cultures.

20.10 Key takeaways

- LLM training requires trillions of tokens; no human-curated dataset exists at this scale, so web crawls are the primary source — but raw web data is low quality and requires extensive filtering before use
- Every serious training pipeline includes language identification, text extraction, heuristic quality filtering, deduplication, and classifier-based scoring; each step involves tradeoffs between quality and diversity
- Deduplication is as important as quality filtering — repeated content teaches memorization rather than generalization; aggressive deduplication consistently improves model quality even when it reduces token count
- Domain mixing is a deliberate design choice: high-quality sources (books, academic papers, Wikipedia) are heavily oversampled relative to their share of the raw internet; these proportions are found empirically, not derived from first principles

- Classifier-based quality filtering inherits the biases of its training examples; content that is dissimilar from Western academic and journalistic norms may be scored lower even when it is high quality on its own terms
- Data contamination with evaluation benchmarks is a real and underreported problem; detecting it requires explicit n-gram matching between training documents and all known evaluation datasets
- The web overrepresents certain languages, cultures, and perspectives; these asymmetries propagate directly into model capabilities and cannot be fully corrected after training

---

20.11 Further reading

- Gao et al. (2020). *The Pile: An 800GB Dataset of Diverse Text for Language Modeling.* — One of the most documented large training corpora, with detailed source breakdowns and per-domain analysis.
- Lee et al. (2022). *Deduplicating Training Data Makes Language Models Better.* — Empirical demonstration that aggressive deduplication improves quality even at reduced token count.
- Raffel et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.* — Introduces C4 and the classifier-based filtering approach that became standard.
- Longpre et al. (2023). *The Data Provenance Initiative.* — Systematic audit of data licensing and consent in popular training datasets.
- Touvron et al. (2023). *LLaMA 2: Open Foundation and Fine-Tuned Chat Models.* — Contains one of the more transparent descriptions of data mixing decisions and filtering heuristics for a frontier model.
- Penedo et al. (2023). *The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only.* — Demonstrates that heavily filtered web data can match or exceed curated corpora.

Chapter 21

Synthetic Data Generation

The canonical question for this chapter: *Real human-written data is finite, expensive to collect, and increasingly subject to legal constraints. How do you use models to generate training data for other models and how do you prevent the process from collapsing into noise?*

Where are we?

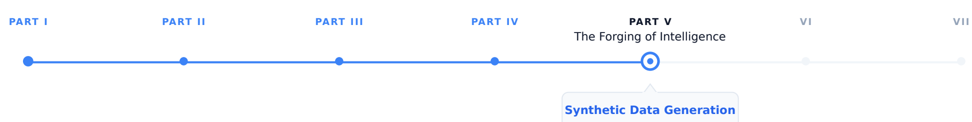


Figure 21.1: The journey through the Model Mind.

Previous Chapter focused on the collection of real-world data. This chapter in the other hand explores a rapidly growing counterpart: data generated by models themselves. Synthetic data has become increasingly important for training advanced models, fine-tuning specialized systems, and developing instruction-following datasets that influence model behavior. Effectively using synthetic data requires understanding both its potential benefits and its limitations, including the ways it can introduce new challenges and failure modes.

21.1 Why synthetic data exists

Training data has always been a bottleneck. Collecting high-quality human-written text at frontier scale is expensive, slow, and increasingly legally complicated. For specialized capabilities (complex mathematical reasoning, multi-step code synthesis, structured output generation) the amount of suitable human-written data is genuinely small relative to what models need.

Synthetic data offers a way around this: use a capable model to generate training examples for itself or for a smaller model. The generator model provides signal that would otherwise require expensive human annotation or simply does not exist in sufficient volume in human-written text.

Data augmentation has been used in computer vision for decades. What changed with large language models is the quality and breadth of the augmentation: a capable LLM can generate synthetic examples indistinguishable from human-written text across a wide range of domains and tasks.

The result is a significant shift in how instruction-following, reasoning, and alignment training data is produced. Where pre-2020 fine-tuning datasets were predominantly human-annotated, many post-2023 datasets are predominantly synthetic.

21.2 Self-Instruct: the foundational approach

Self-Instruct (Wang et al., 2022) established the basic pattern for synthetic instruction data generation. The insight: a model that can follow instructions can also generate new instruction-following examples.

21.2.1 The pipeline

```
1. Seed set: a small hand-written collection of instruction-response pairs
   (175 examples in the original paper)

2. For each generation step:
   a. Sample a few examples from the seed/pool as demonstrations
   b. Prompt the model to generate a new instruction unlike the demonstrated ones
   c. Prompt the model to generate an input if the task requires one
   d. Prompt the model to generate the corresponding output
   e. Apply quality filters
   f. Add passing examples to the pool; repeat

3. Use the accumulated pool as fine-tuning data
```

The key quality filters are similarity filtering (discard if ROUGE-L overlap with any existing instruction exceeds 0.7), length filtering (discard very short instructions or responses), and keyword filtering (discard examples where the model refused or produced degenerate output).

Self-Instruct was used to create the Alpaca dataset (52K examples) that enabled the first wave of open-source instruction-tuned models. Later work improved quality primarily by using stronger generator models: GPT-4 instead of GPT-3.

21.3 Teacher-student pipelines

A more principled approach: use a large, capable teacher model to generate training data for a smaller student model. The student learns from the teacher's outputs rather than from human annotation.

```python
def generate_teacher_student_data(
    teacher_model,
    prompts: list[str],
    system_prompt: str = "You are a helpful assistant.",
    temperature: float = 0.7,
) -> list[dict]:
    training_examples = []
    for prompt in prompts:
        response = teacher_model.complete_chat(
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user",   "content": prompt},
            ],
            temperature=temperature,
```

```
        )
        training_examples.append({
            "messages": [
                {"role": "system",    "content": system_prompt},
                {"role": "user",      "content": prompt},
                {"role": "assistant", "content": response},
            ]
        })
    return training_examples
```

21.3.1 The Phi models: synthetic data at scale

Microsoft's Phi model family demonstrated that synthetic data from a strong teacher can produce remarkably capable small models. Phi-1 (2023) was trained primarily on GPT-4-generated textbook-quality Python exercises Despite having 1.3B parameters, it achieved performance comparable to models five times larger on coding benchmarks.

The finding: a small model trained on high-quality synthetic data can substantially outperform the same model trained on a much larger corpus of real but noisier data. Quality of training signal, controlled through synthetic generation, can substitute for volume. This challenged the prevailing assumption that more data is always better.

21.4 Rejection sampling for verifiable tasks

For tasks with verifiable answers (mathematics, code, structured output) rejection sampling is one of the most effective synthetic data techniques. Generate many candidate responses; keep only those that pass verification.

```
def rejection_sample(
    problem: str,
    generator_model,
    verifier_fn,        # returns True if the response is correct
    n_candidates: int = 16,
    temperature: float = 0.8,
) -> list[str]:
    accepted = []
    for _ in range(n_candidates):
        response = generator_model.complete(problem, temperature=temperature)
        if verifier_fn(problem, response):
            accepted.append(response)
    return accepted
```

For mathematics, the verifier is an exact answer checker. For code, it is the test suite. For structured output (JSON, SQL), it is a schema validator. Without an automatic verifier, rejection sampling reduces to unverified generation with high compute cost and uncertain quality, the verifier is what makes the technique work.

Rejection sampling also produces multiple correct reasoning paths for the same problem, which is valuable: models trained on diverse correct approaches generalize better than models trained on a single canonical solution.

21.5 Generating reasoning traces

Training reasoning models requires large volumes of step-by-step solutions. Human experts cannot produce these at scale. Synthetic generation combined with outcome verification is the practical solution.

21.5.1 Process reward model data

Training process reward models requires step-level correctness annotations. Generating traces at scale and using outcome correctness as a weak per-trace label is the standard approach:

```
def generate_reasoning_traces(
    problems: list[str],
    generator_model,
    n_traces_per_problem: int = 8,
) -> list[dict]:
    dataset = []
    for problem in problems:
        traces = []
        for _ in range(n_traces_per_problem):
            trace = generator_model.complete(
                f"Solve step by step, showing all work:\n{problem}",
                temperature=0.8,
            )
            is_correct = check_answer(
                extract_final_answer(trace),
                get_ground_truth(problem),
            )
            traces.append({"trace": trace, "correct": is_correct})
        dataset.append({"problem": problem, "traces": traces})
    return dataset
```

The per-step supervision signal for PRM training still requires some human annotation of individual steps but identifying which complete traces are correct can be done automatically, reducing annotation cost by orders of magnitude.

21.6 Constitutional AI and RLAIF

Constitutional AI (Bai et al., 2022) uses a model to generate its own supervision signal for alignment, eliminating the need for human preference labels in many training steps.

21.6.1 SL-CAI: supervised learning from AI feedback

Generate potentially problematic responses, then have the model revise them according to a set of principles (the "constitution"):

```
CONSTITUTION = [
    "Rewrite to be more helpful while being harmless.",
    "Identify ways the response is harmful or unethical, then rewrite.",
    "Consider whether the response could be misused, and revise accordingly.",
]

def constitutional_revision(prompt: str, model, n_rounds: int = 2) -> str:
    current = model.complete(prompt)
    for _ in range(n_rounds):
        principle = random.choice(CONSTITUTION)
        current = model.complete(
            f"Human: {prompt}\nAssistant: {current}\n\n"
            f"Principle: {principle}\nRevised response:"
        )
    return current
```

21.6.2 RLAIF: AI preference labels

Use the model to generate preference labels instead of human raters:

```
def ai_preference_label(
    prompt: str,
    response_a: str,
    response_b: str,
    principle: str,
    judge_model,
) -> str:
    judgment = judge_model.complete(
        f"Principle: {principle}\n\n"
        f"Response A: {response_a}\n\n"
        f"Response B: {response_b}\n\n"
        f"Which response better follows the principle? Answer A or B only:"
    ).strip().upper()
    return "A" if judgment.startswith("A") else "B"
```

RLAIF produces preference datasets at a fraction of the cost of human annotation. The quality tradeoff: AI judges have their own biases they tend to prefer longer, more verbose responses and responses that match their own generation style. Calibration against human judgments is important before relying on RLAIF at scale.

21.7 Synthetic conversation data

Multi-turn conversation data is expensive to collect from humans but important for deployed conversational assistants. Synthetic generation covers the gap.

21.7.1 Persona-diverse generation

A single generator model produces conversations in a limited range of styles. Explicit persona specification diversifies the training distribution:

```
USER_PERSONAS = [
    "a software engineer debugging production issues",
    "a student learning the topic for the first time",
    "an expert seeking nuanced clarification",
    "a non-native English speaker asking basic questions",
    "a skeptical user who questions the assistant's claims",
    "a user in a hurry who wants concise answers",
]

def generate_diverse_conversations(
    topics: list[str],
    model,
    n_per_topic: int = 6,
) -> list[list[dict]]:
    all_conversations = []
    for topic in topics:
        for persona in random.sample(USER_PERSONAS, n_per_topic):
            conversation = simulate_conversation(topic, persona, model)
            all_conversations.append(conversation)
    return all_conversations
```

Diversity in user persona, question style, and conversation length prevents the model from learning a narrow conversational pattern and produces better instruction-following across user types.

21.8 Data contamination and quality collapse

Synthetic data introduces failure modes that real data collection does not.

21.8.1 Model collapse

When synthetic data is used to train a model, and the outputs of that model are used as training data for the next version, iterative degradation can occur. Each generation amplifies the biases and errors of the previous one. The distribution of outputs narrows progressively, the model becomes less diverse and more prone to producing stereotyped, low-variance responses.

This is called model collapse (Shumailov et al., 2023). It is a genuine risk for pipelines that rely heavily on self-generated data without injecting fresh human-written signal.

21.8.2 Synthetic data contamination

Synthetic data generated from publicly available models may share biases or specific phrasings with those models' training data. If the teacher model was trained on evaluation benchmark data, synthetic data generated by that teacher may leak evaluation signal into the student's training, inflating benchmark performance without improving real capability.

Contamination detection for synthetic data requires checking not just whether the synthetic examples match evaluation questions verbatim, but whether the synthetic examples produce responses that closely match evaluation answers.

21.8.3 Hallucinated facts in synthetic data

It is a known fact that generator models hallucinate. Synthetic training data therefore contains hallucinated facts and training on them teaches the student model to reproduce those hallucinations with confidence.

For general instruction-following, this is manageable: the hallucinated facts are diluted by the volume of correct content. For specialized domain training (medical, legal, scientific), it is dangerous: the model learns domain-specific confident errors.

21.9 What synthetic data is good at (and what it is not)

21.9.1 What it does well

Instruction diversity. Synthetic generation can produce instruction-following examples covering a wide range of formats, styles, and task types far more efficiently than human annotation.

Scaling reasoning data. For verifiable tasks, rejection sampling produces high-quality reasoning traces at scale that human experts could not produce in comparable volume.

Domain coverage. Specialized domains with little existing training data (low-resource languages, niche technical fields) can be bootstrapped with targeted synthetic generation.

Behavior shaping. Constitutional AI and RLAIF can produce alignment training data at a fraction of the cost of human preference annotation.

21.9.2 What it does poorly

Novel knowledge. A model cannot generate facts it does not already know. Synthetic data recombines and reformats existing knowledge; it does not add new knowledge to the training distribution.

Detecting its own blind spots. The model generating synthetic data does not know what it does not know. It will generate confident synthetic data in the domain of its blind spots, teaching the student model the same blind spots with additional confidence.

Replacing human judgment for nuanced quality. Human evaluators catch subtleties in tone, cultural appropriateness, and pragmatic correctness that AI generators and judges frequently miss. For tasks where these nuances matter, synthetic data is a complement to human annotation, not a replacement.

21.10 Key takeaways

- Synthetic data generation uses capable models to produce training examples that would otherwise require expensive human annotation or simply do not exist in sufficient volume in human-written text
- Self-Instruct established the basic pattern: seed a small hand-written dataset, prompt the model to generate new instruction-response pairs, filter for quality and diversity, repeat
- Teacher-student pipelines use a large teacher model to generate training data for a smaller student; the Phi models demonstrated that high-quality synthetic data can substitute for much larger volumes of noisier real data
- Rejection sampling is the most effective technique for verifiable tasks: generate many candidates, keep those that pass an automatic verifier; the verifier is what makes it work
- Constitutional AI and RLAIF use the model as its own alignment teacher, generating preference labels and revisions at a fraction of human annotation cost, with known biases that require calibration
- Model collapse – iterative quality degradation when synthetic data is used to train the next-generation model – is a real risk; always mix with real human-written data and monitor diversity
- Synthetic data cannot add novel knowledge or detect its own blind spots; it recombines existing knowledge and will confidently reproduce the generator model's errors in the training distribution of the student
- Always mix synthetic with real data; use the strongest available teacher; verify aggressively for high-stakes domains; document provenance

---

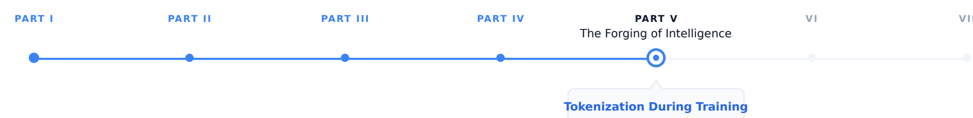
21.11 Further reading

- Wang et al. (2022). *Self-Instruct: Aligning Language Models with Self-Generated Instructions.* – The foundational paper for synthetic instruction data generation.
- Gunasekar et al. (2023). *Textbooks Are All You Need.* – The Phi-1 paper; demonstrates that textbook-quality synthetic data can produce remarkably capable small models.
- Bai et al. (2022). *Constitutional AI: Harmlessness from AI Feedback.* – CAI and RLAIF; using a model as its own alignment teacher.
- Lightman et al. (2023). *Let's Verify Step by Step.* – Process reward models trained on step-level human annotations; establishes why step-level supervision matters for reasoning.
- Shumailov et al. (2023). *The Curse of Recursion: Training on Generated Data Makes Models Forget.* – Model collapse from iterative synthetic training.
- Taori et al. (2023). *Alpaca: A Strong, Replicable Instruction-Following Model.* – Self-Instruct applied with GPT-3; the first widely-reproduced open instruction-tuned model.

Chapter 22

Tokenization During Training

The canonical question for this chapter: *How does a model's vocabulary get built, and why does the choice of subword algorithm permanently shape everything the model can represent?*



Where are we?
The chapter on tokenization during inference explained how a trained tokenizer operates at inference time by applying chat templates, counting tokens, and managing context budgets. This chapter shifts upstream to examine how that tokenizer was created in the first place. Before model training begins, a separate tokenizer training process constructs the vocabulary file that every downstream component ultimately depends on.

22.1 The Problem That Tokenization Solves

A language model is a function over sequences of discrete symbols. The choice of symbol set is not obvious. Three options present themselves immediately, each with a serious flaw.

Character-level tokenization treats every Unicode code point as a token. The vocabulary is tiny (roughly 100,000 characters in Unicode, though practical vocabularies are far smaller) and no out-of-vocabulary problem exists: any string is representable. The flaw is sequence length. The word "uncharacteristically" requires 20 tokens. A 2,048-token context window, measured in characters, holds roughly a paragraph. Self-attention cost scales quadratically with sequence length, so character-level models pay a steep computational price for what is, in practice, trivial structure: spelling.

Word-level tokenization goes to the other extreme. The model treats each whitespace-delimited word as a single token. This compresses sequences efficiently and aligns well with human intuition about linguistic units yet on the other hand the flaws are fundamental. First, vocabulary size: English alone has hundreds of thousands of distinct word forms, and that is before proper nouns, neologisms, and domain-specific terminology. A vocabulary large enough to be useful is also large enough to make the embedding table and output projection layer expensive. Second, and more seriously, any word not seen during training is unknown. A word-level model trained on English Wikipedia has never seen "COVID-19" or "zoombombing" or any company name coined after its training cutoff (2019 for this example). It cannot represent them; it can only produce a special `[UNK]` token, at which point the model has lost the input information entirely.

Subword tokenization resolves both problems by refusing to commit to either extreme. Common words appear as single tokens. Rare words are split into meaningful pieces (prefixes, suffixes, stems) that the model has seen in other contexts. The word “uncharacteristically” might become [“un”, “character”, “istic”, “ally”], four tokens, each of which carries recoverable meaning. An unseen technical term like “CRISPR” might become [“C”, “R”, “IS”, “PR”] which is not meaningful decomposition, but at least the characters are present and the model can condition on them.

The choice of subword algorithm, and the vocabulary size it produces, is made once per model family. GPT-4 uses a vocabulary of roughly 100,000 tokens. LLaMA 3 uses 128,256 tokens. BERT uses 30,522 WordPiece tokens. These numbers are fixed before model training begins and cannot be changed without retraining the model from scratch. Every downstream decision like embedding table dimensions, output projection matrix size, how many tokens a document occupies, what the model can represent at all is downstream of this choice.

22.2 Byte Pair Encoding

Byte Pair Encoding (BPE) was introduced as a data compression algorithm in 1994 and adapted for neural machine translation by Sennrich et al. in 2016. It remains the dominant tokenization algorithm for large language models. GPT-2, GPT-3, GPT-4, LLaMA, Mistral, Falcon, and most other major models use BPE or a variant of it.

22.2.1 The Algorithm

BPE training is an iterative merge procedure applied to a large text corpus. The algorithm:

1. **Initialize the vocabulary** with the base symbol set. For text BPE, the base symbols are Unicode characters. For byte-level BPE (described below), the base symbols are the 256 byte values.
2. **Represent the corpus** as a sequence of base symbols. Typically the corpus is pre-tokenized into words by whitespace and punctuation, and each word is represented as a sequence of characters followed by a special end-of-word marker. GPT-2 uses `Ġ` (a space prefix) to distinguish word-initial tokens from word-internal ones; this matters because “ing” at the start of a word differs semantically from “ing” as a suffix.
3. **Count all adjacent symbol pairs** across the corpus.
4. **Merge the most frequent pair.** If (“e”, “r”) appears 47,000 times and no other pair appears more often, add “er” to the vocabulary and replace every occurrence of (“e”, “r”) in the corpus with the new token “er”.
5. **Repeat** until the target vocabulary size is reached.

Each merge adds exactly one token to the vocabulary. A target vocabulary of 50,000 tokens (GPT-2’s size) requires 50,000 minus the number of base symbols merges.

The result is a vocabulary in which the most frequent substrings in the training corpus are represented as single tokens. Common English words or morphemes appear whole while character sequences that the corpus rarely produces are represented by their constituent base symbols.

22.2.2 Byte-Level BPE

Standard BPE with Unicode characters still has a representation problem: Unicode has over a million code points, and any character not in the base vocabulary produces an error. GPT-2 addressed this with a conceptually simple fix: use all 256 byte values as the base vocabulary rather than Unicode characters. Any string, in any language or encoding, can be represented as a sequence of bytes. The BPE merge procedure then operates over bytes.

This eliminates the out-of-vocabulary problem completely. A byte-level BPE tokenizer never encounters an input it cannot represent. The practical tradeoff is that languages not well-represented in the training corpus get inefficient tokenization: their common words are not merged into single tokens, so they occupy more tokens per word than English. A 2024 study found that GPT-4’s tokenizer encodes Amharic text using roughly 4–5× more tokens per character than English text, making API costs and context window utilization significantly worse for low-resource languages.

22.2.3 Learning the Vocabulary: A Worked Example

Consider a tiny corpus: "low lower newest widest". After whitespace splitting and adding end-of-word markers, the initial representation is:

```
l o w </w>          : 5 occurrences
l o w e r </w>      : 2 occurrences
n e w e s t </w>    : 6 occurrences
w i d e s t </w>    : 3 occurrences
```

Counting adjacent pairs across all occurrences:

Pair	Count
(e, s)	9
(s, t)	9
(e, w)	6
(l, o)	7
...	...

The pair (e, s) and (s, t) are tied at 9. The algorithm picks one (say, (e, s)) and merges:

```
l o w </w>          : 5
l o w e r </w>      : 2
n e w es t </w>     : 6
w i d es t </w>     : 3
```

New vocabulary: {l, o, w, e, r, n, s, t, i, d, , **es**}. Next iteration, (es, t) appears 9 times and is the top pair:

```
n e w est </w>      : 6
w i d est </w>      : 3
```

Vocabulary adds **est**. After a few more merges, "low" might become a single token. After thousands of merges on a real corpus, common English words are single tokens and common morphemes are single tokens, while rare strings remain as byte sequences.

22.3 WordPiece

WordPiece, developed at Google and used in BERT and its derivatives, is conceptually similar to BPE but differs in the merge criterion. Where BPE selects the pair with the highest raw frequency, WordPiece selects the pair that maximizes the likelihood of the training corpus under the language model induced by the current vocabulary.

Formally, WordPiece scores each candidate merge (A, B) → AB as:

```
score(A, B) = count(AB) / (count(A) × count(B))
```

This is the pointwise mutual information of A and B. A merge is favored not just because AB is frequent, but because the co-occurrence of A and B is surprising given their individual frequencies. The pair ("un", "##common") would be favored over ("e", "##r") even if the latter is more frequent, because the combination "uncommon" is much more predictable from its parts than "er" is from "e" and "r".

WordPiece uses a `##` prefix convention to mark tokens that continue a word rather than beginning one. The token `##ing` is the continuation "ing", while `ing` is the word-initial "ing". This makes it easy to recover word boundaries from a token sequence.

The practical behavior of BPE and WordPiece is similar for large vocabularies on large corpora. BERT's 30,522-token WordPiece vocabulary and GPT-2's 50,257- token BPE vocabulary both segment English text into roughly 1.3 tokens per word. The engineering differences matter more for implementation: WordPiece vocabularies are trained in a top-down fashion with a fixed vocabulary size target, while BPE proceeds bottom-up.

22.4 Unigram Language Model Tokenization

The Unigram Language Model algorithm, introduced by Kudo (2018) and used in SentencePiece (which underlies T5, mT5, ALBERT, and XLNet), approaches the problem differently. Rather than building a vocabulary through iterative merging, Unigram starts with a large candidate vocabulary and iteratively prunes it.

The procedure:

1. **Initialize** with a large vocabulary (typically 10–100× the target size) containing all substrings that appear with sufficient frequency in the corpus.
2. **Train a unigram language model** over the current vocabulary: assign each token a probability proportional to its frequency.
3. **For each token**, compute how much the corpus likelihood would decrease if that token were removed from the vocabulary. Tokens whose removal causes little likelihood decrease are candidates for pruning.
4. **Remove the bottom p%** of tokens (typically 10–20%) ranked by their likelihood contribution.
5. **Repeat** until the target vocabulary size is reached.

The Unigram model defines the probability of a tokenization as the product of the probabilities of its constituent tokens, summed over all possible tokenizations using dynamic programming. At inference time, the tokenizer finds the most probable tokenization of each input string.

The key advantage of the Unigram model is that it naturally handles tokenization ambiguity and can assign probabilities to multiple tokenizations of the same string. This enables subword regularization: during training, instead of always using the single most probable tokenization, the model is presented with multiple different tokenizations of the same input, sampled according to their probabilities. This acts as a form of data augmentation and has been shown to improve robustness, particularly for morphologically rich languages.

22.5 SentencePiece

SentencePiece is not a tokenization algorithm but a toolkit, developed by Kudo and Richardson at Google, that implements both BPE and Unigram in a language-agnostic framework. Its key design decision: it tokenizes raw text directly, without requiring a pre-tokenization step that splits on whitespace.

Languages like Chinese, Japanese, and Thai have no whitespace between words. Applying a whitespace-splitting pre-tokenizer to Chinese text produces a sequence of individual characters, which is not a useful prior for BPE merging. SentencePiece treats the input as a sequence of Unicode characters and learns its own whitespace handling. The space character receives its own representation: typically (U+2581, a lower one-eighth block), which appears as a prefix on word-initial tokens.

LLaMA 1 and LLaMA 2 use a SentencePiece BPE tokenizer with a vocabulary of 32,000 tokens. LLaMA 3 switched to a tiktoken-based BPE tokenizer with 128,256 tokens, a 4× vocabulary expansion motivated partly by better multilingual coverage and partly by reduced token fertility (tokens per word) for code.

22.6 Vocabulary Size: The Fundamental Tradeoff

Vocabulary size is a hyperparameter with pervasive consequences. The embedding table has shape [vocab_size, d_model] and the output projection (llm_head) has shape [d_model, vocab_size]. For a model with d_model = 4,096 (LLaMA 2 7B), each additional 1,000 vocabulary tokens adds 4,096,000 parameters to the embedding table and another 4,096,000 to the output projection, roughly 8M parameters, stored in float16, occupying 16 MB.

Model	Vocabulary Size	d_model	Embedding + LM Head
GPT-2	50,257	768	77M params
BERT-base	30,522	768	47M params
LLaMA 2 7B	32,000	4,096	262M params
LLaMA 3 8B	128,256	4,096	1,051M params
GPT-4 (estimated)	~100,000	—	—
T5-base	32,128	768	49M params

For LLaMA 3 8B, the embedding and lm_head parameters represent roughly 13% of the total 8B parameter count which is not negligible. Vocabulary size is essentially a direct tax on model size.

The tradeoffs in both directions:

Larger vocabulary: - Lower sequence length for the same text (higher compression) - Shorter effective context window in characters - Better representation of rare words and domain-specific terms - Better multilingual coverage without fertility explosion - Larger embedding and lm_head matrices - Sparser training signal: rare tokens appear less often during training

Smaller vocabulary: - Longer sequences for the same text - More tokens consumed per concept, reducing effective context - Higher fertility for code, math, and non-English text - Smaller, cheaper embedding and lm_head layers - Denser training signal: each token is seen more often

The sweet spot has shifted upward over time. GPT-2 (2019) used 50,257 tokens. The field has since converged on the 100k–150k range for general-purpose models. The main driver is multilingual coverage: a 32,000-token vocabulary trained on English-dominated text tokenizes many other languages at 3–5× English fertility, which is a meaningful usability degradation.

22.7 Special Tokens

Every tokenizer reserves a set of token IDs for special purposes that are not learned by the BPE or Unigram procedure. These special tokens are added after vocabulary training and assigned IDs, typically at the ends of the ID range to avoid collision with learned tokens.

Common special tokens and their functions:

Token	Common forms	Purpose
Beginning of sequence	<s>, <|begin_of_text|>	Marks the start of a context
End of sequence	</s>, <|eot_id|>, <|endoftext|>	Signals generation completion
Padding	<pad>, [PAD]	Fills sequences to uniform length in batches
Unknown	[UNK]	Represents out-of-vocabulary tokens (absent in byte-level BPE)
Mask	[MASK]	Used in masked language model training (BERT-style)

Token	Common forms	Purpose
Separator	`[SEP]`	Separates segments in BERT-style dual-input tasks
Classification	`[CLS]`	BERT's pooling token; pooled for classification tasks
Role markers	`<|user|>`, `<|assistant|>`, `<|system|>`	Chat template structure
Tool call tokens	`<tool_call>`, `<|python_tag|>`	Structured tool use in instruction-tuned models

The design of special tokens matters for fine-tuning stability. If a special token like `<\|assistant\|>` appears in natural text (someone writing about the token itself), the model may behave unpredictably. In practice, the angle-bracket or pipe-bracket formats were chosen specifically because they are rare in natural text. LLaMA 3 uses a more elaborate special token scheme with 256 reserved positions (`<\|reserved_special_token_0\|>` through `<\|reserved_special_token_250\|>`) to allow future special token additions without changing the vocabulary size.

22.8 Tokenizer Training Data and Its Consequences

The tokenizer vocabulary reflects the distribution of the corpus used to train it. This is not a detail; it is a design choice with first-order effects on model behavior.

Code. A model trained primarily on natural text will have a tokenizer with poor code fertility. Python keywords might be single tokens, but multi-character operators like `!=`, `<=`, and `+=` might split. Indentation whitespace (significant in Python) gets tokenized as a sequence of space tokens, consuming context budget for what is purely structural information. Code-specialized models (CodeLlama, Starcoder, DeepSeek-Coder) typically add code-specific merges to their tokenizer or use a tokenizer retrained on a code-heavy corpus. The Starcoder tokenizer allocates specific tokens for common indentation patterns (two spaces, four spaces, eight spaces) as single tokens.

Mathematics. Mathematical notation is hostile to standard tokenizers. Expressions like `\frac{d}{dx}` or `\mathbb{R}^{n \times n}` fragment into many tokens, each individually meaningless. The number "3.14159265" might become ["3", ".", "1", "4", "1", "5", "9", "2", "6", "5"], ten tokens for ten digits. Models like Minerva (trained on arXiv and math textbooks) benefit from tokenizers that treat common LaTeX constructs as single units.

Numbers. How numbers tokenize has non-obvious arithmetic consequences. If "123" is a single token but "124" splits as ["12", "4"], the model's ability to generalize numerical relationships is compromised. The model must learn that the token "123" is numerically close to "12" + "4",
a fact that is not implicit in the token IDs. Some tokenization schemes (notably those used for models specialized in arithmetic) tokenize digits individually to ensure that positional arithmetic generalizes correctly.

Non-English languages. As noted above, a tokenizer trained primarily on English text imposes high token fertility on other languages. This is an active area of remediation: mT5, XLM-RoBERTa, and BLOOM trained their tokenizers on multilingual corpora with careful sampling to equalize fertility across languages. BLOOM's tokenizer was trained on a 341 billion token corpus balanced across 46 languages plus code.

22.9 The Relationship Between Tokenizer and Model Training

The tokenizer training run is separate from and prior to the model training run. It proceeds as follows:

1. **Collect a corpus** representative of the model's intended data distribution. For GPT-2, this was WebText (40GB of web text). For LLaMA 3, it was a 15.6 trillion token multilingual corpus. The tokenizer training corpus need not be identical to the model training corpus but should have the same distributional properties.

2. **Run the tokenization algorithm** (BPE, WordPiece, or Unigram) to target vocabulary size. This is computationally cheap relative to model training, a vocabulary of 100,000 tokens can be constructed from a multi-billion-token corpus in a few hours on a single machine.

3. **Save the vocabulary file** (a JSON or SentencePiece `.model` file mapping token strings to integer IDs) and the merge rules (for BPE, an ordered list of all merges performed).

4. **Use this tokenizer** to convert all training data into token ID sequences before model training begins. The model training loop never sees raw text, only integer sequences.

This separation has an important implication: the tokenizer is frozen for the lifetime of the model. If the model is later fine-tuned on a domain with different vocabulary characteristics (medical text, legal text, a new programming language), the tokenizer cannot be updated without rebuilding the embedding table from scratch. This is one reason why large vocabulary sizes and multilingual tokenizer training have become standard practice: they reduce the probability that a downstream use case encounters catastrophic fertility problems.

The interaction between tokenization and the training objective is subtle. Next-token prediction trains the model to predict the next token ID given all preceding token IDs. The model learns statistics over tokens, not characters or words. If "unfortunately" is a single token, the model learns a rich representation for it. If "unfortunately" splits as ["un", "fort", "unately"], the model must learn the composition across three prediction steps. Neither is obviously worse (the model can learn either way) but the representations that emerge are different.

22.10 Tokenizer Pathologies and Failure Modes

Reversal asymmetry. Tokenizers frequently produce different tokens for a word depending on whether it appears at the start of a sentence versus in the middle. BPE with space-prefix encoding (GPT-2 style) distinguishes " hello" (word-initial) from "hello" (word-internal). This is correct behavior (case-insensitive models with naive tokenizers would conflate them) but it means that token IDs are not invariant to position within a sentence.

Capitalization sensitivity. "Hello", "hello", and "HELLO" frequently map to different tokens. A model that has not seen a word capitalized a particular way during training may not generalize well to that capitalization. This is particularly problematic for all-caps text (error messages, legal text) and for proper nouns that appear in inconsistent capitalizations.

Number fragmentation. As noted above, numbers tokenize inconsistently. "100" is often a single token; "101" may be ["10", "1"]; "1001" may be ["100", "1"]. This makes arithmetic notoriously difficult for tokenizer-dependent models and is one motivation for the shift toward digit-level tokenization in math-specialized models.

Whitespace sensitivity. Many tokenizers distinguish "dog" from " dog" (with a leading space). A prompt that inadvertently adds or removes a leading space (easy to do when concatenating strings in code) changes the token sequence, which changes the model's input, which can change the output.

The "SolidGoldMagikarp" class of pathologies. During GPT-2 and GPT-3 era research, Rumbelow and Watkins (2023) discovered that certain tokens in OpenAI's tokenizer produced highly anomalous model behavior when prompted directly. The token "SolidGoldMagikarp" (a Reddit username that appeared frequently enough in the training corpus to earn its own token) caused unexpected outputs when the model was asked to repeat it. The underlying cause: the token was present in the tokenizer vocabulary (earned through high frequency in the BPE training corpus) but essentially absent from the model's training data, perhaps filtered out by the data pipeline. The model had never learned a representation for this token ID, so querying it produced garbage outputs. This pathology reveals that the tokenizer training corpus and the model training corpus need to be in close distributional agreement.

22.11 Tokenizer Compatibility and Model Identity

The tokenizer is part of a model's identity in a strict technical sense. Two models with different tokenizers cannot share weights, cannot be compared token-for-token on benchmarks without normalization, and cannot be used interchangeably in serving infrastructure without a tokenizer swap. This has practical consequences:

Benchmark comparability. Perplexity, which measures how surprised a model is by a held-out test set, is not comparable across models with different tokenizers. A model with a large vocabulary that tokenizes the test set into fewer tokens may report lower perplexity simply because it has fewer prediction steps, not because its predictions are better. Correct comparison requires normalizing by bits-per-character or bits-per-byte rather than bits-per-token.

Context window comparison. A "32,768-token context window" means different things for different models. GPT-3.5 (~100k vocab) fits more characters per context slot than LLaMA 2 (SentencePiece, 32k vocab). Quoting context window sizes in tokens without specifying the tokenizer is imprecise.

Transfer and adapter training. Parameter-efficient fine-tuning methods like LoRA add adapters to an existing model's weights without modifying the embedding table. If the fine-tuning data contains vocabulary not well-represented in the original tokenizer, the adapters cannot fix the underlying fertility problem, they can only learn compensating patterns within the existing token space.

22.12 The tiktoken Implementation

OpenAI's tiktoken library is the implementation most practitioners encounter directly. It implements byte-level BPE and is the tokenizer for GPT-2, GPT-3, GPT-3.5, GPT-4, and the text-embedding-ada-002 and subsequent embedding models. LLaMA 3 adopted tiktoken's format (though with a different vocabulary) to enable compatibility with existing tooling.

tiktoken's vocabulary files ship as a serialized mapping from base64-encoded byte sequences to integer IDs. The merge rules ship as an ordered list of pairs, one merge per line. The tokenization procedure at inference time applies merges in training order, the merge that was learned first (and therefore covers the most frequent pair in the training corpus) is applied first.

A benchmark figure: tiktoken encodes approximately 5 million tokens per second on a modern CPU in Python (with Rust internals via a Python binding). This speed matters at scale: a training corpus of 15 trillion tokens requires 15 trillion tokenization operations before the first gradient step.

22.13 Key Takeaways

- Subword tokenization solves the vocabulary coverage problem by decomposing rare words into known subword units while representing common words as single tokens.
- BPE iteratively merges the most frequent adjacent symbol pair in a corpus; WordPiece uses pointwise mutual information instead of raw frequency; Unigram starts large and prunes.
- Byte-level BPE eliminates the out-of-vocabulary problem entirely by using the 256 byte values as the base vocabulary — any input string is representable.
- The embedding table and lm_head together have shape proportional to [vocab_size × d_model]; at d_model = 4,096, every 1,000 additional vocabulary tokens adds approximately 16 MB of parameters in float16.
- LLaMA 3's shift from 32,000 to 128,256 tokens added roughly 1 billion parameters in embedding and lm_head layers — about 13% of the model's size.
- The tokenizer training run is separate from model training and uses only the frequency statistics of the corpus, not gradient descent.
- Tokenizer vocabulary reflects its training corpus: a tokenizer trained on English text imposes 3–5× higher token fertility on many non-English languages.

- Number tokenization is inconsistent by design, which contributes to LLM arithmetic difficulties; digit-level tokenization is used in some math- specialized models as a remedy.
- Perplexity is not comparable across models with different tokenizers; normalization must be done per byte or per character, not per token.
- The "SolidGoldMagikarp" class of pathology arises when a token appears in the tokenizer vocabulary but not in the model training data, producing undefined model behavior.
- Special tokens (BOS, EOS, role markers, tool call tokens) are not learned by the subword algorithm; they are added to the vocabulary after BPE training and must be handled explicitly in data formatting pipelines.

22.14 Further Reading

- Sennrich, R., Haddow, B., & Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units.* ACL. — The paper that adapted BPE for NLP; the core argument about morphological decomposition is still the clearest statement of why subword tokenization works.

- Kudo, T. (2018). *Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates.* ACL. — Introduces the Unigram language model tokenizer and the subword regularization training technique; the theoretical treatment of tokenization ambiguity is the key contribution.

- Kudo, T., & Richardson, J. (2018). *SentencePiece: A simple and language- independent subword tokenizer and detokenizer for Neural Text Processing.* EMNLP. — The SentencePiece toolkit paper; explains the language-agnostic design and the whitespace convention.

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL. — Introduces WordPiece in the context of masked language modeling; the original description of the [CLS], [SEP], and [MASK] special tokens.

- Rust, P., Pfeiffer, J., Vulić, I., Ruder, S., & Gurevych, I. (2021). *How Good Is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models.* ACL. — Systematic analysis of token fertility across languages and its effect on downstream task performance; the fertility tables are essential reading for anyone building multilingual systems.

- Rumbelow, J., & Watkins, M. (2023). *SolidGoldMagikarp (plus, prompt generation).* Alignment Forum. — The blog post documenting anomalous token behavior; the clearest empirical demonstration that tokenizer and model training corpora must match.

- Touvron, H., et al. (2023). *LLaMA 2: Open Foundation and Fine-Tuned Chat Models.* — Section 2 describes the SentencePiece tokenizer design decisions for LLaMA 2; useful contrast with LLaMA 3's subsequent shift to tiktoken.

The Tokenization Problem

Language models operate on sequences of discrete symbols.
The choice of symbol set creates a tradeoff between sequence length, vocabulary size, representation, and computational cost.

Character: tiny vocabulary, but very long sequences **Word:** short sequences, but huge vocabulary + OOV **Subword:** practical compromise

Character-Level

Every character = one token
+ No out-of-vocabulary problem
+ Tiny vocabulary
− Very long sequences
− Expensive self-attention

Example:

`uncharacteristically`
`→ approximately 20 tokens`

Word-Level

Every word = one token
+ Short sequences
− Huge vocabulary
− Unknown words become [UNK]
− Input information can be lost

Example:

`COVID-19 → [UNK]`
`New words → unknown token`

Subword-Level

Common words → whole tokens
Rare words → smaller pieces
+ Shorter sequences
+ Small vocabulary
+ No OOV with byte-level BPE

Example:

`un + character + istic + ally`
`→ 4 tokens`

Byte Pair Encoding (BPE)

1. Start with base symbols
2. Count all adjacent symbol pairs
3. Select the most frequent pair
4. Merge the pair into one token
5. Repeat until target vocabulary size

```
e + s → es
es + t → est
l + o → lo
lo + w → low
```

Byte-level BPE starts from all 256 byte values.
Therefore, every input string is representable.

WordPiece

Similar to BPE, but uses a different merge criterion.

```
BPE:
highest raw frequency

WordPiece:
count(AB) / (count(A) × count(B))
```

Uses ## to mark continuation pieces.

```
un + ##common + ##ly
```

Unigram Language Model

Starts with a large candidate vocabulary.
→ Assign probabilities to tokens
→ Measure each token's likelihood contribution
→ Remove low-value tokens
→ Repeat until target vocabulary size

Can represent multiple possible tokenizations.
Enables subword regularization during training.

SentencePiece

A toolkit, not a tokenization algorithm.
+ Implements BPE and Unigram
+ Works directly on raw text
+ No whitespace pre-tokenization required
+ Useful for languages without spaces

```
▁hello ▁world
```

Vocabulary Size — Fundamental Tradeoff

Larger Vocabulary
+ Fewer tokens per text
+ Better rare-word and multilingual coverage
− Larger embedding and LM head
− Rare tokens receive less training signal

`Embedding: [vocab_size × d_model]`

Smaller Vocabulary
+ Smaller embedding and LM head
+ Denser training signal
− Longer sequences
− Higher fertility for code, math, and non-English

`LM Head: [d_model × vocab_size]`

Figure 22.1: Cheat sheet.

Chapter 23

Training Objectives

The canonical question for this chapter: *What exactly is the model optimizing during training, and why does predicting the next token produce a system that can reason, translate, summarize, and write code?*

Where are we?

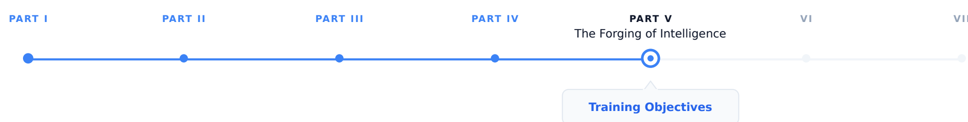


Figure 23.1: The journey through the Model Mind.

Previous chapter established how the vocabulary is constructed before model training begins. This chapter specifies the loss function the model minimizes during training the mathematical statement of what the model is trying to learn.

23.1 What a Training Objective Is

A training objective is a scalar-valued function of the model's parameters and the training data that measures how badly the model is currently doing. During training, the optimizer adjusts model parameters to reduce this scalar. The training objective is the only signal the model ever receives about what "correct" behavior looks like. Everything the model learns (language, facts, reasoning patterns, stylistic tendencies) is a consequence of what the objective rewards and what it ignores.

This makes the choice of objective the most consequential design decision in building a language model, with the possible exception of architecture. The transformer architecture describes the computational graph. The training objective describes what the graph should compute. Getting the architecture wrong costs you efficiency. Getting the objective wrong costs you the model.

Modern large language models use one of two primary pretraining objectives: **causal language modeling** (autoregressive next-token prediction) and **masked language modeling** (bidirectional prediction of masked tokens). A third objective (**sequence-to-sequence modeling**) combines elements of both. Each objective induces a different kind of model with different strengths, different inference-time behavior, and different fine-tuning characteristics.

23.2 Causal Language Modeling

23.2.1 The Objective

Causal language modeling (CLM) is the pretraining objective used by GPT-2, GPT-3, GPT-4, LLaMA, Mistral, Falcon, and essentially every large model used for text generation. The setup is as follows.

Given a sequence of tokens $x_1, x_2, \dots, x_T$, the model is trained to predict each token given all preceding tokens. The loss for a single sequence is the mean negative log-likelihood of the correct next token at each position:

$$\mathcal{L}_{\text{CLM}} = -\frac{1}{T} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_1, x_2, \dots, x_{t-1})$$

where P_θ is the model's predicted probability distribution over the vocabulary, parameterized by weights θ. At each position t, the model receives the sequence $x_1, \dots, x_{t-1}$ and must assign a probability to every token in the vocabulary. The loss penalizes the model in proportion to how surprised it is by the actual next token x_t.

Summing across positions and sequences, the total training loss is a measure of the model's average surprise (its perplexity) over the training corpus. Minimizing this loss is equivalent to maximizing the likelihood of the training data under the model.

23.2.2 Why Causal?

The word "causal" refers to the constraint that prediction at position t may only condition on positions 1 through $t-1$, never on positions $t+1$ and beyond. This constraint is enforced by the causal attention mask in the transformer: the upper triangle of the attention matrix is set to $-\infty$ before the softmax, preventing any token from attending to future tokens.

The causal constraint serves two purposes. First, it makes training efficient: a single forward pass through a sequence of T tokens produces T prediction targets simultaneously (one at each position) giving T gradient signals per sequence. Without causal masking, computing the prediction at each position would require a separate forward pass, since the model could otherwise "see" the answer it is trying to predict.

Second, and more importantly, the causal constraint makes the trained model useful for generation. At inference time, the model generates text by sampling one token at a time, left to right, conditioning each new token on everything that came before. The training objective and the inference procedure are exactly aligned: the model was trained to do precisely the thing it is asked to do at inference time. This alignment is not trivial, it is one reason CLM became dominant over alternatives.

23.2.3 The Prediction Task at Each Position

At each position in a training sequence, the model produces a probability distribution over the entire vocabulary (typically 32,000 to 128,000 tokens). The loss at that position is the negative log-probability assigned to the correct token.

Consider a training sequence: "The cat sat on the mat." After tokenization this might be ["The", " cat", " sat", " on", " the", " mat", "."]. At position 3 (predicting " sat"), the model sees ["The", " cat"] and must assign a probability to every vocabulary token. If it assigns probability 0.8 to " sat", the loss at this position is $-\log(0.8) \approx 0.22$ nats. If it assigns probability 0.01, the loss is $-\log(0.01) \approx 4.61$ nats.

Across billions of such positions, the model learns to assign high probability to contextually likely continuations. Initially, when parameters are random, the loss is approximately log(vocab_size): about 10.4 nats for a 32,000-token vocabulary, 11.8 nats for 128,000 tokens. A well-trained LLaMA 2 7B achieves roughly 1.8–2.2 nats on held-out English text, depending on domain. This corresponds to a perplexity of approximately $e^{2.0} \approx 7.4$: the model is, on average, about as surprised as if it were choosing uniformly among 7 equally likely options.

23.2.4 Data Formatting

CLM requires no annotation. The loss signal is derived entirely from the structure of the text itself: every token is simultaneously a context token (for predicting the next token) and a prediction target (for the preceding context). This is the property that makes CLM scalable to internet-scale datasets, no human labeling, no task specification, just raw text.

In practice, training sequences are formed by concatenating documents and splitting into fixed-length chunks (typically matching the model's context window). A special end-of-document token (`<|endoftext|>`, `</s>`) is inserted between documents so the model learns that one document ending is not a continuation of the next.

Cross-document attention is typically masked: the model is not trained to predict the first token of a document from the last token of an unrelated preceding document, since this would introduce spurious statistical dependencies. Some implementations simply ensure that each training sequence starts at a document boundary.

23.3 Why Next-Token Prediction Produces Capable Models

The capability of GPT-class models far exceeds what the training objective appears to require. Predicting the next word in a document should, naively, produce a sophisticated autocomplete system. It has instead produced systems capable of multi-step reasoning, code generation, translation into unseen languages, and generating plausible scientific hypotheses. This gap between the apparent simplicity of the objective and the observed capability of trained models is the central puzzle in understanding modern LLMs.

23.3.1 The World Model Argument

The most compelling explanation is that accurate next-token prediction *requires* a world model. Consider what a model must represent to correctly predict the next token in a diverse corpus at low loss.

A sequence of text about a chess game requires the model to track the board state (which pieces are where) to predict legal moves and plausible continuations. A sequence of Python code requires the model to track variable bindings, scoping rules, and the semantics of already-defined functions to predict syntactically and semantically valid continuations. A medical discharge summary requires the model to represent the patient's condition, the treatments administered, and their interactions to predict what a clinician would plausibly write next.

No single special case requires a world model. But achieving low loss across millions of documents spanning chess, code, medicine, history, mathematics, literature, and conversation requires that the model build internal representations that encode, for each domain, the causal structure that determines what comes next. The model that minimizes next-token prediction loss over a sufficiently diverse corpus is, in this view, a system that has compressed a large fraction of human structured knowledge into its parameters.

23.3.2 In-Context Learning as Evidence

The emergence of in-context learning (the ability to perform new tasks from a few demonstrations in the prompt, without any weight update) provides strong evidence for the world model interpretation. A model that had merely memorized statistical co-occurrences could not, in principle, solve arithmetic problems formatted as few-shot examples that it had never seen in training. The fact that it does suggests that the model has learned something about the structure of task descriptions and the general strategy of pattern completion from examples, not just the specific patterns it was exposed to.

23.4 Masked Language Modeling

23.4.1 The Objective

Masked language modeling (MLM) is the pretraining objective introduced by BERT (Devlin et al., 2019). Where CLM predicts each token from its left context only, MLM predicts masked tokens from both left and right context simultaneously. The model is bidirectional.

The procedure: given a sequence of tokens, randomly replace approximately 15% of tokens with a `[MASK]` token. The model processes the entire masked sequence and must predict the original identity of each masked token. The loss is the mean negative log-likelihood over masked positions only:

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|\mathcal{M}|}\sum_{t\in\mathcal{M}} \log P_\theta(x_t \mid \tilde{x}_1, \dots, \tilde{x}_T)$$

where $\mathcal{M}$ is the set of masked positions and $\tilde{x}$ is the masked sequence.

BERT's specific masking scheme adds noise to the 15% mask rate: - 80% of selected tokens are replaced with `[MASK]` - 10% are replaced with a random token - 10% are left unchanged

This prevents the model from learning to simply ignore all `[MASK]` tokens and forces it to maintain a useful representation of every token, since any token might be the one it is asked to predict.

23.4.2 The Bidirectional Advantage

MLM's key property is that the model can attend to the full context in both directions when making each prediction. This is architecturally different from CLM: MLM transformers have no causal mask. The attention operation at each position can incorporate information from all other positions.

Bidirectional context improves representation quality for tasks that require understanding of a complete input. Named entity recognition, for example, requires understanding that "Apple" in "Apple released its quarterly earnings today" is a company, not a fruit, a judgment that often depends on words that appear *after* "Apple" in the sentence. A causal model must make this judgment with only left context; a bidirectional model has access to both.

For sentence classification, question answering over a fixed document, and token labeling tasks, MLM-pretrained models (BERT, RoBERTa, DeBERTa) held state of the art for several years after BERT's release in 2018.

23.4.3 The Generation Problem

The cost of bidirectionality is that MLM models cannot straightforwardly generate text. The causal constraint that enables autoregressive generation is absent. To generate a token at position t, a CLM model simply conditions on positions 1 through $t-1$. An MLM model, trained to condition on all positions simultaneously, has no natural procedure for sequential generation: it would need to mask future positions during generation, but it was never trained with that masking pattern.

Attempts to use MLM models for generation exist (masked diffusion language models, iterative refinement) but remain less capable than autoregressive CLM models at open-ended generation tasks. The practical consequence: MLM models are fine-tuned for fixed-output tasks (classification, span extraction, sequence labeling) while CLM models handle generation.

23.4.4 Fine-Tuning With a [CLS] Token

BERT adds a `[CLS]` token at the beginning of every input. After pretraining, the embedding of this token at the final layer captures a pooled representation of the entire sequence, because it has attended to all other tokens. For classification tasks, fine-tuning adds a linear layer on top of the `[CLS]` embedding and trains it (along with some or all of the transformer weights) on labeled examples.

This two-stage procedure)pretrain with MLM, fine-tune with task-specific head) became the dominant paradigm for NLP from 2018 to 2020. GPT-3's demonstration of in-context learning without fine-tuning shifted the field, but BERT-style fine-tuning remains standard for latency-sensitive classification tasks where a smaller model with a dedicated head outperforms a large generative model asked to produce labels as text.

23.5 Sequence-to-Sequence Objectives

T5 (Raffel et al., 2020) and BART (Lewis et al., 2020) use encoder-decoder architectures trained with sequence-to-sequence objectives. The encoder processes the input with full bidirectional attention (like BERT). The decoder generates the output autoregressively, attending to both the encoder's output and the previously generated tokens (like GPT).

T5 frames every NLP task as text-to-text: translation, summarization, classification, and question answering all become "given this text input, produce this text output." The pretraining objective is a span corruption task: random contiguous spans of input tokens are masked, and the decoder is trained to reconstruct them. This is MLM generalized to spans rather than individual tokens.

BART uses a more aggressive corruption scheme for pretraining: sentence permutation, token deletion, text infilling, and document rotation, all combined. The decoder is trained to reconstruct the original document from the corrupted input. This makes BART particularly strong for generation tasks that involve transforming one piece of text into another (translation, summarization, dialogue).

Encoder-decoder models are well-suited to tasks with a natural input-output structure. Their limitation is computational: running both an encoder and a decoder requires more memory than a decoder-only CLM model of equivalent parameter count, and the two-component architecture is more complex to serve. The field has largely converged on decoder-only CLM models for general-purpose use, with encoder-decoder architectures retained for specialized translation and summarization applications.

23.6 The Cross-Entropy Loss in Practice

All three objectives (CLM, MLM, and seq2seq) minimize some form of cross-entropy loss. Cross-entropy between the model's predicted distribution P_θ and the true distribution (a one-hot vector at the correct token) is:

$$H(y, P_\theta) = -\sum_{v \in V} y_v \log P_\theta(v) = -\log P_\theta(x_t)$$

since $y_v = 1$ only for the correct token and 0 elsewhere. This reduces to the negative log-probability of the correct token, which is the quantity being minimized at each prediction position.

23.6.1 Label Smoothing

Hard one-hot targets can lead to overconfident models, the model pushes probability mass toward 1.0 for the correct token and toward 0.0 for all others, which degrades calibration and can impair generalization. Label smoothing (Szegedy et al., 2016) replaces the one-hot target with a softened version:

$$y_v^{\text{smooth}} = \begin{cases} 1 - \epsilon & \text{if } v = x_t \\ \epsilon/(|V| - 1) & \text{otherwise} \end{cases}$$

With $\epsilon = 0.1$ (a common value), the correct token receives target probability 0.9 rather than 1.0, and all other tokens receive a small positive target probability. This prevents the model from becoming arbitrarily confident and improves calibration of the predicted probability distributions.

Label smoothing is used in most large model training runs. T5 uses $\epsilon = 0.1$. The original transformer paper used $\epsilon = 0.1$ for machine translation. GPT-style models have used both smoothed and unsmoothed objectives; the LLaMA technical reports do not specify but performance suggests standard cross-entropy without smoothing.

23.7 From Pretraining to Instruction Following

The pretraining objectives described above train models on raw text. The result is a base model: a powerful next-token predictor that continues arbitrary text in a statistically plausible way. A base model given the prompt "Translate this sentence to French:" will often produce a continuation that looks like more examples of translation prompts (because that is the kind of text that follows such a prompt in a training corpus) rather than actually translating the sentence.

Transforming a base model into an assistant that follows instructions requires additional training on instruction-formatted data. This is covered in Chapter REFF (alignment and RLHF), but the connection to the pretraining objective is worth noting here. The pretraining objective gives the model the underlying capability (language understanding, factual knowledge, reasoning patterns) while instruction tuning teaches the model to *apply* those capabilities in response to user requests. The capabilities cannot be fine-tuned in; they must be pretrained. The behavior of applying them in response to instructions can be fine-tuned efficiently once the capabilities exist.

The practical implication: scaling pretraining data and compute improves the base capability ceiling. Instruction tuning effectiveness is bounded by what is already in the base model. A model that cannot solve a reasoning problem as a base model will not be able to solve it after instruction tuning, because instruction tuning does not add capability, it redirects it.

23.8 Continued Pretraining and Domain Adaptation

A base model can be continued-pretrained on a domain-specific corpus to improve its performance on that domain's tasks. The objective is identical to original pretraining (CLM over raw text) but the data distribution shifts toward the target domain.

For example, a general-purpose model like LLaMA 2 can be continued-pretrained on a corpus of medical literature, clinical notes, and biomedical databases. The resulting model (analogous to BioMedLM or Med-PaLM in spirit) achieves substantially lower loss on medical text and scores higher on medical question- answering benchmarks, without architecture changes. The pretraining objective is unchanged; only the data changes.

The practical considerations for continued pretraining:

Catastrophic forgetting. Continued pretraining on a narrow domain tends to degrade performance on out-of-domain tasks. The optimizer, minimizing the domain-specific loss, adjusts weights in ways that reduce their utility for general text. Mitigation strategies include mixing domain data with general data at some ratio (typically 10–30% general data), using lower learning rates than original pretraining, and using replay buffers that interleave examples from the original training distribution.

Learning rate schedule. Continued pretraining typically restarts the learning rate from a small value (rather than the near-zero value the schedule had reached at the end of original pretraining) and uses a cosine decay to a near-zero endpoint. The warmup phase is often shortened or eliminated, since the model is not initializing from random weights.

Data quality matters more, not less. At the domain-specialization stage, the corpus is smaller and the model has more capacity than the data requires. Noisy data (OCR errors in scanned documents, truncated records, formatting artifacts) exerts more influence per token than it would in the original pretraining corpus of trillions of tokens.

23.9 Connecting the Objective to What the Model Learns

The training objective does not specify what representations the model should build internally, it only specifies the scalar loss to minimize. The representations that emerge depend on what is useful for minimizing that loss.

For CLM, a complete characterization of what emerges from minimizing $\mathcal{L}_{\text{CLM}}$ does not exist. Empirically, the following have been observed to emerge as loss decreases on a sufficiently large corpus:

Syntactic structure. Probing classifiers can extract parse trees and grammatical roles from the intermediate representations of well-trained models, even though no syntactic annotation was present in training data. Predicting the next token reliably across varied sentence structures requires tracking syntactic dependencies.

Factual associations. The feedforward layers of transformer models store factual associations of the form "subject — relation — object" in a key-value structure. Models can complete "The capital of France is _____" not because the exact string appears in training data but because the factual association is encoded in weights. Meng et al. (2022) demonstrated that specific factual associations can be localized to specific layers and edited.

Reasoning patterns. Models trained on sufficient data develop the ability to complete multi-step reasoning chains in context, particularly when such chains appear frequently in the training corpus (mathematics textbooks, competitive programming editorials, legal reasoning documents). Whether this constitutes reasoning in a principled sense or sophisticated pattern completion is an open question, the empirical fact of the capability is sufficient here.

Stylistic and discourse structure. Document-level structure (introductions that motivate, arguments that proceed from premise to conclusion, narratives that resolve introduced conflicts) emerges from training on documents that exhibit these structures. Predicting each token correctly in a well-structured document requires modeling the discourse-level plan of the document.

All of these emerge from the same objective, none of them were specified. The training objective is the entire specification of the model's learning target, and the diversity of what emerges from it is the empirical basis for the world model interpretation above.

23.10 Key Takeaways

- Causal language modeling (CLM) trains the model to predict each token from all preceding tokens; masked language modeling (MLM) trains it to predict masked tokens from full bidirectional context.
- CLM loss at position t is the negative log-probability of the correct token: $-\log P_\theta(x_t \mid x_1, \dots, x_{t-1})$; the total training loss is the mean over all positions and all training sequences.
- The causal attention mask enforces the left-to-right constraint during training and makes CLM training efficient: a single forward pass over a sequence of length T produces T prediction targets simultaneously.
- A randomly initialized model predicts approximately log(vocab_size) nats of loss; a well-trained LLaMA 2 7B achieves roughly 1.8–2.2 nats on English held-out text, corresponding to a perplexity of about 7.
- Accurate next-token prediction over a sufficiently diverse corpus requires building internal representations that track causal structure across domains — the world model argument for why CLM produces capable models.
- MLM models (BERT, RoBERTa, DeBERTa) are stronger for classification and span extraction tasks; CLM models dominate generation; neither architecture advantage is absolute, but their inference procedures differ fundamentally.
- Encoder-decoder models (T5, BART) pair a bidirectional encoder with a causal decoder; the combination is strong for structured input-output transformation tasks but more expensive to serve than decoder-only models.
- Label smoothing replaces one-hot targets with soft targets (typically $\epsilon = 0.1$) to prevent overconfidence and improve probability calibration.
- Pretraining builds capability; instruction tuning redirects it. A capability that is absent from the base model cannot be fine-tuned in — it must be pretrained.
- Continued pretraining on domain-specific data can substantially improve domain performance while risking catastrophic forgetting of general capabilities; mixing in general data at 10–30% mitigates this.

- Syntactic structure, factual associations, reasoning patterns, and discourse coherence all emerge from CLM training without explicit specification — they are what the model learns because learning them reduces loss.

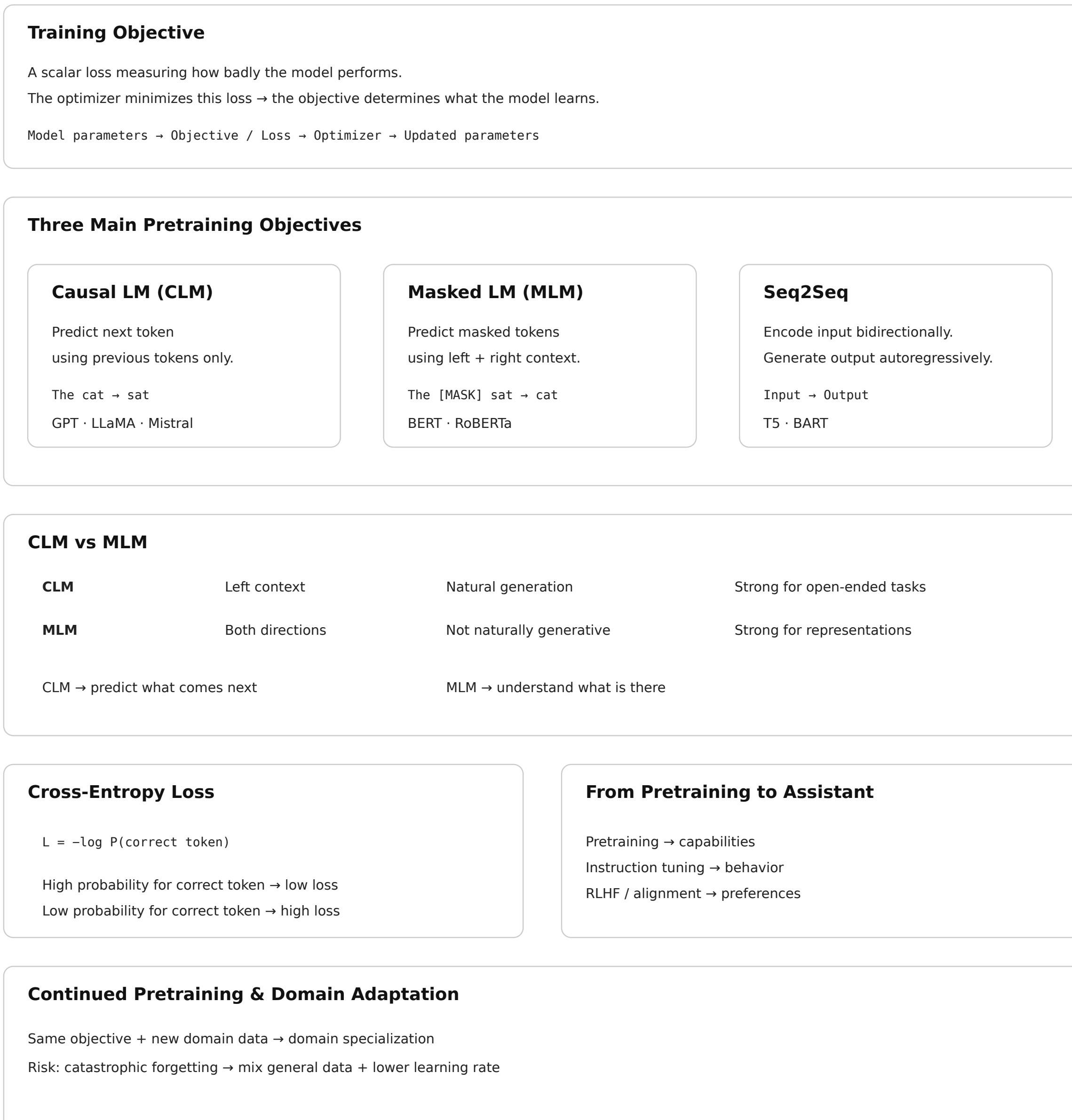


Figure 23.2: Cheat sheet.

23.11 Further Reading

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL. — Introduces MLM and the [CLS]/[SEP]/[MASK] special token framework; the comparison between CLM and MLM in the appendix is the clearest early treatment of the bidirectionality tradeoff.

- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners.* OpenAI. — The GPT-2 paper; the central argument that next-token prediction at scale produces multitask learners is stated here, with zero-shot evaluation across eight tasks as evidence.
- Raffel, C., et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.* JMLR. — Introduces T5 and the text-to-text framing; the systematic comparison of pretraining objectives (span corruption, deshuffling, prefix LM) across 24 tasks is a rigorous treatment of objective choice.
- Lewis, M., et al. (2020). *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.* ACL. — Introduces BART's multi-corruption pretraining objective; the ablation of individual corruption strategies is the key empirical contribution.
- Brown, T., et al. (2020). *Language Models are Few-Shot Learners.* NeurIPS. — GPT-3; the in-context learning results are the clearest empirical demonstration that CLM pretraining produces general-purpose learners; the few-shot vs. zero-shot vs. fine-tuning comparisons motivate the world model interpretation.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). *Rethinking the Inception Architecture for Computer Vision.* CVPR. — Introduces label smoothing; the motivation and calibration analysis apply directly to language model training despite the computer vision context.
- Meng, K., et al. (2022). *Locating and Editing Factual Associations in GPT.* NeurIPS. — Demonstrates that factual knowledge is localized to specific feedforward layers in transformer models; the causal tracing methodology is the key technical contribution.
- Gururangan, S., et al. (2020). *Don't Stop Pretraining: Adapt Language Models to Domains and Tasks.* ACL. — Systematic study of domain-adaptive pretraining (continued pretraining on domain text); demonstrates consistent gains across biomedical, computer science, news, and reviews domains with analysis of the data quantity and quality tradeoffs.

Chapter 24

The Training Loop

The canonical question for this chapter: *Given a transformer architecture and a corpus of text, how does the model actually learn, what happens in a single training step, and how do millions of steps produce a capable model?*

Where are we?

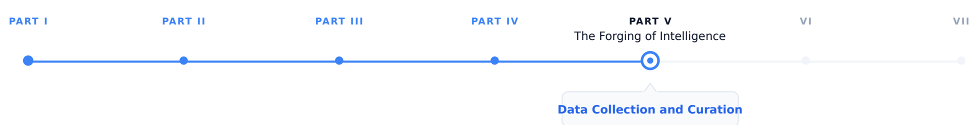


Figure 24.1: The journey through the Model Mind.

Previous chapter covered training objectives, what the model is trying to minimize. This chapter covers the mechanism: how one training step actually works, from sampling a batch to updating 175 billion parameters. The training loop is conceptually simple. The devil is entirely in the details of doing it stably and efficiently at scale.

24.1 What learning means for a language model

Before the first training step, the model's parameters are random. Every weight in every attention matrix, every FFN weight, every embedding, initialized from a small random distribution. The model produces nonsense. Given "The capital of France is", it assigns roughly equal probability to every token in the vocabulary.

After training, those same parameters encode enough structure about language and the world that the model assigns high probability to "Paris" in that context. Learning is the process of adjusting parameters until they do this, not just for one example, but for trillions of them, simultaneously and without memorizing any individual one.

The training loop is the mechanism that does this adjustment. Show the model text, measure how wrong it is, compute how to be less wrong, update the parameters. Repeat hundreds of millions of times. The devil is entirely in the details.

24.2 The objective: cross-entropy loss

The training objective for a language model is next-token prediction, formalized as minimizing cross-entropy loss.

For a sequence of tokens [t_1, t_2, …, t_n], the model predicts each token given all previous tokens. The loss for a single sequence is:

$$L = -1/n * sum_i log P(t_i | t_1, ..., t_{i-1})$$

Where $P(t_i | t_1, ..., t_{i-1})$ is the probability the model assigns to the correct next token at position i.

If the model assigns probability 1.0 to the correct token: log(1.0) = 0, no loss. If it assigns probability 0.01: log(0.01) -4.6, high loss. The objective is to maximize the probability assigned to correct tokens across all positions in the training corpus.

This objective is applied to every position in every sequence simultaneously. For a sequence of 4,096 tokens, the model makes 4,096 predictions in a single forward pass. The causal mask enforces that position i only attends to positions 0 through i-1, the model cannot cheat by looking at what it is trying to predict. This is why training is computationally efficient: one forward pass on a sequence of length n produces n training signal examples simultaneously.

24.3 The forward pass

The forward pass is the computation from input tokens to output probabilities.

```
Input:  [t_1, t_2, t_3, ..., t_n]     <- token IDs
Output: [P_1, P_2, P_3, ..., P_n]     <- probability distributions over vocab
```

For each position i, P_i is the model's probability distribution over the vocabulary for what token comes next. The actual next token t_{i+1} is the label. The loss is computed by comparing P_i to a one-hot distribution over the correct token.

During the forward pass, the model:

1. Looks up token embeddings for all input tokens
2. Adds positional encodings (or applies RoPE inside attention)
3. Passes through all transformer blocks sequentially
4. Projects the final hidden states to vocabulary logits via the output embedding matrix (weight-tied with the input embedding table)
5. Computes softmax to get probabilities
6. Computes cross-entropy loss against the shifted target sequence

The forward pass is fully parallelized across the sequence dimension, all positions are computed simultaneously, with the causal mask enforcing the autoregressive constraint. This is the architectural property that made transformers faster to train than recurrent networks.

24.4 The backward pass: backpropagation

After the forward pass produces a loss value, the backward pass computes the gradient of that loss with respect to every parameter in the model.

The gradient tells you: if I increase this parameter by a tiny amount, does the loss go up or down, and by how much? A negative gradient means increasing the parameter reduces loss, move it up. A positive gradient means increasing the parameter increases loss, move it down.

Backpropagation applies the chain rule of calculus recursively from the output layer back to the input embeddings. For a model with 175 billion parameters, this computation visits every parameter and produces a gradient for each one.

The backward pass is approximately 2x as expensive as the forward pass in compute. It also requires storing the activations computed during the forward pass, intermediate values that are needed to compute gradients during the backward pass. For a large model with a large batch, storing all activations requires significant GPU memory. This is one of the central constraints of training large models and the motivation for activation checkpointing.

24.4.1 Gradient flow through the residual stream

One key insight about why transformers train well at depth: residual connections provide a direct gradient highway from the loss back to the early layers.

Without residuals, the gradient must pass through every nonlinearity in every layer before reaching layer 1. Deep chains of multiplications cause gradients to either vanish or explode.

With residuals:

```
x_L = x_0 + F_1(x_0) + F_2(x_1) + ... + F_L(x_{L-1})
```

```
dL/dx_0 = dL/dx_L * (1 + sum of partial derivatives through each sublayer)
```

The identity path (the “1” in the derivative) ensures gradients reach early layers reliably, even in 96-layer or 126-layer networks. This is why very deep transformers are trainable while very deep plain networks of the same depth are not.

24.5 The optimizer

Once gradients are computed, parameters are updated to reduce the loss. Vanilla gradient descent moves each parameter in the direction of the negative gradient, which is conceptually correct but practically unusable for large model training.

24.5.1 Adam: adaptive moment estimation

Adam maintains two running statistics per parameter:

$$\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t} \\
\hat{v}_t &= \frac{v_t}{1-\beta_2^t} \\
\theta_t &= \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}
\end{aligned}$$

where:

- m_t: first moment (moving average of gradients)
- v_t: second moment (moving average of squared gradients)
- $\hat{m}_t$: bias-corrected first moment
- $\hat{v}_t$: bias-corrected second moment
- α: learning rate

Standard hyperparameters: beta_1 = 0.9, beta_2 = 0.95 or 0.999, epsilon = 1e-8.

The key insight: each parameter gets its own effective learning rate, adapted based on the history of its gradients. Parameters that receive large, consistent gradients get smaller effective learning rates, they are already moving fast. Parameters that receive small or noisy gradients get larger effective learning rates, they need more help.

This makes training much more robust to the heterogeneous gradient magnitudes across different parts of a transformer, attention weights, embedding gradients, and FFN weights behave very differently, and Adam handles all of them without manual per-layer tuning.

24.5.2 Decoupled weight decay: AdamW

L2 regularization is commonly introduced by adding a penalty on the squared magnitude of the model parameters to the training loss:

$$L_{\text{total}}(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|_2^2$$

Taking the gradient gives:

$$\nabla_\theta L_{\text{total}} = \nabla_\theta L + \lambda\theta$$

Thus, the L2 penalty adds $\lambda\theta$ to the gradient before the Adam update. Because Adam applies parameter-wise adaptive scaling to the gradient, the regularization term is also affected by this scaling.

AdamW decouples weight decay from the gradient update. Instead of adding the regularization term to the gradient, it directly shrinks the parameters separately from the adaptive gradient step:

$$\theta_t = (1 - \alpha\lambda)\theta_{t-1} - \alpha\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Where λ is the weight decay coefficient, typically set to 0.1 in many transformer training setups. This encourages parameters to remain small, which acts as a form of regularization and can improve generalization. Because AdamW applies this weight decay directly to the parameters, there is typically no need to add a separate L2 regularization penalty to the loss for the same purpose.

24.5.3 Memory cost of the optimizer

Adam maintains two additional values per parameter (m and v). For a 175B parameter model in float32:

```
Parameters:       175B * 4 bytes = 700 GB
Adam m states:    175B * 4 bytes = 700 GB
Adam v states:    175B * 4 bytes = 700 GB
Total:                              2.1 TB
```

This is why optimizer state is one of the dominant memory costs in large model training, and why techniques like 8-bit Adam (quantizing optimizer states to INT8) and ZeRO (sharding optimizer state across GPUs) exist. Without these techniques, training a 70B model requires more memory for optimizer state than for the model itself.

24.6 The learning rate schedule

The learning rate is not constant across training. A carefully designed schedule is critical for stable and effective training.

24.6.1 Warmup

Training starts with a very small learning rate and increases linearly to the target rate over the first few thousand steps.

Without warmup, the large random gradients at initialization cause the Adam moment estimates (m and v) to be poorly calibrated, leading to unstable early training or divergence. The warmup period gives the optimizer time to accumulate accurate moment estimates before taking large steps.

A typical warmup: 2,000 steps from lr = 0 to lr = 3e-4.

24.6.2 Cosine decay

After warmup, the learning rate follows a cosine decay schedule:

```
lr(step) = lr_min + 0.5 * (lr_max - lr_min) * (1 + cos(pi * step / total_steps))
```

The learning rate decreases smoothly from the peak to a small final value (typically lr__max / 10). The cosine shape is empirically better than linear decay: it maintains a higher learning rate during most of training, allowing the model to escape local optima, then decays rapidly at the end to converge to a good solution.

24.6.3 Why the schedule matters as much as the learning rate

The peak learning rate controls the step size. The schedule controls how that step size changes over training. A training run with the wrong schedule (too fast decay, or no warmup) can produce a significantly worse model than the same training run with the correct schedule, even with identical architecture and data.

24.7 Gradient clipping

Large gradient spikes occur naturally during training, a particularly surprising batch produces unusually large gradients that could cause a large parameter update and destabilize training. Gradient clipping prevents this.

```
# Clip gradient norm to max_norm
total_norm = sqrt(sum(p.grad.norm()**2 for p in model.parameters()))
clip_coef  = max_norm / max(total_norm, max_norm)

for p in model.parameters():
    p.grad *= clip_coef
```

If the global gradient norm (the L2 norm across all parameters) exceeds a threshold (typically 1.0), all gradients are scaled down proportionally so the global norm equals the threshold.

Clipping does not change the direction of the update, only its magnitude. It prevents a single bad batch from destabilizing training without distorting the direction of learning. Monitoring the gradient norm (and the frequency of clipping) is one of the primary diagnostic signals during training: frequent clipping or a consistently elevated norm indicates the learning rate is too high.

24.8 Mixed precision training

Frontier model training uses BF16 rather than FP32 for the forward and backward passes, halving memory requirements for activations and parameters at the cost of some numerical precision.

```
Full precision (FP32):
  Forward pass:       FP32 activations
  Backward pass:      FP32 gradients
```

```
  Optimizer step:     FP32 parameter update

Mixed precision (BF16 compute, FP32 master):
  Forward pass:       BF16 activations (2x faster on tensor cores)
  Backward pass:      BF16 gradients
  Gradient cast:      BF16 -> FP32
  Optimizer step:     FP32 parameter update (numerical accuracy preserved)
  Parameter cast:     FP32 -> BF16 for next step
```

BF16 was chosen over FP16 because it has the same exponent range as FP32 (8 bits), preventing overflow on the large gradient values common in deep network training. FP16's limited range (max 65,504) caused frequent overflow in practice.

The float32 master copy is necessary because BF16 has insufficient precision to accumulate small gradient updates correctly over many steps. BF16 represents values with only 7 mantissa bits (2 decimal digits of precision); the optimizer states and parameters need more precision than this to make the tiny updates that characterize late-stage training.

24.9 Batch size and gradient accumulation

The batch size determines how many training examples are processed before each parameter update. Larger batches produce more accurate gradient estimates (averaging over more examples reduces noise) but require more memory.

24.9.1 Gradient accumulation

When the target batch size exceeds what fits in GPU memory, gradient accumulation accumulates gradients over multiple forward-backward passes before taking an optimizer step:

```python
optimizer.zero_grad()

for i, (inputs, targets) in enumerate(dataloader):
    outputs = model(inputs)
    loss    = criterion(outputs, targets) / accumulation_steps
    loss.backward()                           # accumulate gradients

    if (i + 1) % accumulation_steps == 0:
        optimizer.step()                      # update parameters
        optimizer.zero_grad()                 # clear accumulated gradients
```

4 accumulation steps with batch size 256 is mathematically equivalent to one step with batch size 1,024, but uses 4x less memory. The tradeoff is that 4 forward-backward passes take longer than 1.

24.9.2 The batch size/learning rate relationship

Increasing batch size without adjusting the learning rate undertrains the model, fewer parameter updates per epoch. Linear scaling rule: when batch size is multiplied by k, multiply the learning rate by k. This preserves the ratio of gradient signal to noise across different batch sizes.

24.10 Activation checkpointing

During the backward pass, the model needs the activations computed during the forward pass to compute gradients. Storing all activations for a large model and large batch requires enormous memory.

Activation checkpointing trades compute for memory: instead of storing all activations, store only a subset (checkpoints). During the backward pass, recompute activations between checkpoints from the stored ones.

```
from torch.utils.checkpoint import checkpoint

def forward_with_checkpointing(self, x):
    for layer in self.layers:
        x = checkpoint(layer, x)  # recompute activations during backward
    return x
```

The cost is additional computation because some forward operations must be recomputed during the backward pass. The exact overhead depends on the checkpointing strategy and model architecture and it is standard in all frontier model training.

24.11 Checkpointing and fault tolerance

Training a frontier model takes weeks to months on thousands of GPUs. Hardware failures are not exceptions, they are statistical certainties at this scale. A GPU cluster of 10,000 nodes where each node has a mean time between failures of one year will experience approximately 27 failures per day.

Model checkpoints (snapshots of parameters, optimizer states, and training metadata) are saved to distributed storage at regular intervals (typically every few hundred steps). If a failure occurs, training restarts from the last checkpoint, losing only the steps since then.

The tradeoff is I/O overhead. Checkpointing a 175B model with optimizer states is terabytes of data per checkpoint. Asynchronous checkpointing (saving in the background while training continues) reduces this overhead significantly.

24.12 The full training step, assembled

A complete single training step:

```
1.  Sample a batch of sequences from the training corpus

2.  Forward pass (in BF16):
    - Token embedding lookup
    - Add positional encodings / apply RoPE
    - Pass through N transformer blocks (attention + FFN + residuals + layernorm)
    - Project to vocabulary logits
    - Compute cross-entropy loss against shifted target sequence

3.  Backward pass:
    - Compute gradients via backpropagation (in BF16)
    - Cast gradients to float32

4.  Gradient clipping:
    - Compute global gradient norm
    - Rescale all gradients if norm > threshold

5.  All-reduce (distributed training):
    - Average gradients across all data-parallel ranks

6.  Optimizer step (AdamW, in float32):
```

```
    - Update first and second moment estimates
    - Apply weight decay
    - Update float32 master parameters

7.  Cast updated parameters back to BF16 for next step

8.  Update learning rate schedule

9.  Log metrics (loss, gradient norm, learning rate, throughput)

10. Save checkpoint if at checkpoint interval

11. Repeat
```

24.13 What to monitor during training

A training run is not something you start and leave. Active monitoring is required.

Training loss. The primary signal and should decrease steadily and smoothly. Spikes indicate bad batches or numerical instability. A plateau may indicate the learning rate is too small or the model is at capacity for its size. Loss divergence (increasing without recovering) requires immediate intervention: reduce the learning rate, restore an earlier checkpoint, and investigate the bad batch that triggered the spike.

Validation loss. Loss on a held-out set not seen during training. Divergence between training and validation loss indicates overfitting. For large models on large corpora, the model rarely sees the same sequence twice, so overfitting is uncommon, but benchmark contamination can cause deceptively low validation loss.

Gradient norm. Should be stable and well below the clipping threshold. Frequent clipping or consistently elevated norms indicate the learning rate is too high. Sudden spikes that persist indicate a corrupted batch or numerical issue.

MFU (Model FLOPs Utilization). The fraction of peak theoretical GPU compute actually used. Well-optimized distributed training achieves 40–60% MFU. Lower values indicate communication bottlenecks, pipeline bubbles, or data loading delays.

Token throughput. Tokens processed per second across the cluster. The primary efficiency metric for benchmarking training setups.

Weight norms. If weight norms grow continuously, weight decay is insufficient or there is a regularization issue.

24.14 The Chinchilla finding: compute-optimal training

How do you decide how large a model to train and on how many tokens?

This was largely heuristic until Hoffmann et al. (2022). For a fixed compute budget C (measured in FLOPs), the optimal allocation trains a model of N parameters on D = 20N tokens:

```
Optimal model size:      N* proportional to C^0.5
Optimal training tokens: D* proportional to C^0.5
```

A 10B parameter model optimally requires approximately 200B tokens. A 70B model requires approximately 1.4T tokens. Previous models (GPT-3, Gopher) were significantly undertrained, too many parameters, too few tokens for their compute budget.

The practical implication shifted after Chinchilla: inference cost is paid many times while training cost is paid once. Training a smaller model for longer produces a model cheaper to run at inference time with comparable quality. LLaMA 2 and LLaMA 3 train on far more tokens than Chinchilla-optimal for exactly this reason.

24.15 Key takeaways

- The training objective is cross-entropy loss on next-token prediction; one forward pass on a sequence of length n produces n training examples simultaneously — this is why transformer training is computationally efficient
- Backpropagation computes gradients for every parameter; residual connections ensure gradients reach early layers without vanishing, enabling 96-layer+ networks to train reliably
- AdamW maintains per-parameter adaptive learning rates via first and second moment estimates; its memory cost is 2x the parameter count, making optimizer state the dominant memory consumer in large model training
- The learning rate schedule — warmup then cosine decay — is as important as the peak learning rate itself; wrong schedules produce significantly worse models even with identical architecture and data
- Gradient clipping prevents loss spikes from destabilizing training by scaling down large gradient norms; monitoring norm and clipping frequency is a primary training health signal
- Mixed precision training uses BF16 for compute (2x faster on tensor cores) and float32 for optimizer states (numerical accuracy preserved); BF16 is preferred over FP16 because it has the same exponent range as FP32
- Activation checkpointing trades ~30% compute for O(sqrt(n)) rather than O(n) activation memory, enabling training of models that would otherwise not fit in GPU memory
- The Chinchilla finding — optimal training requires roughly 20 tokens per parameter — reshaped compute budget allocation; in practice, models are overtrained relative to Chinchilla-optimal because inference cost is paid many more times than training cost

---

24.16 Further reading

- Kingma & Ba (2015). *Adam: A Method for Stochastic Optimization.* — The original Adam paper.
- Loshchilov & Hutter (2019). *Decoupled Weight Decay Regularization.* — The AdamW paper.
- Hoffmann et al. (2022). *Training Compute-Optimal Large Language Models.* — The Chinchilla paper; required reading for anyone thinking about training compute allocation.
- Rajbhandari et al. (2020). *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models.* — DeepSpeed ZeRO; the standard approach to optimizer state sharding covered in chapter 32.
- Chen et al. (2016). *Training Deep Nets with Sublinear Memory Cost.* — Activation checkpointing.
- Karpathy, A. (2022). *nanoGPT.* — A clean, minimal GPT training implementation in ~300 lines of PyTorch; the best way to understand the training loop concretely before the distributed complexity of chapter 32.

Objective

Next-token prediction minimizes cross-entropy loss over the corpus.

$L = -1/n \ \Sigma \log P(t_i \mid t_1 ... t_{i-1})$

Correct prediction → low loss

Forward Pass

Token IDs → embeddings → transformer → logits → softmax → probabilities

All positions computed in parallel.
Causal mask prevents seeing future tokens.

Backward Pass

Backpropagation computes gradients for every model parameter.

Gradient = direction of loss change.
Residual connections improve gradient flow.

AdamW Optimizer

Maintains first and second gradient moments.

$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$
$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$

Adaptive updates + decoupled weight decay.

Learning Rate

Warmup: gradually increase LR.

Cosine decay: smoothly reduce LR.

$lr = lr_{min} + \frac{1}{2}(lr_{max} - lr_{min})$
$\times (1 + \cos(\pi \cdot \text{step} / \text{total}))$

Gradient Clipping

Controls unusually large gradient spikes.

If $\|g\|$ > threshold:

$g \leftarrow g \times \text{threshold} / \|g\|$

Changes magnitude, not direction.

Mixed Precision

BF16: forward + backward computation

FP32: master parameters + optimizer states

Lower memory and faster tensor-core compute.

BF16 has FP32-like exponent range.

Batch & Accumulation

Larger batches → lower gradient noise.

Gradient accumulation simulates larger batches without extra GPU memory.

`effective batch = batch × accumulation`

Memory Efficiency

Activation checkpointing: store fewer activations, recompute later.

Trade: less memory ↔ more computation.

Optimizer states are a major memory cost.

Distributed Training

Each GPU processes different data.

Gradients are averaged with all-reduce.

Parameters remain synchronized.

Checkpointing

Save model parameters + optimizer states.

Enables recovery after hardware failures.

Large models require distributed storage.

Monitor

Training / validation loss

Gradient norm · MFU · token throughput

Weight norms · numerical stability

Figure 24.2: Cheat sheet.

Chapter 25

Distributed Training

The canonical question for this chapter: *How do you train a model whose parameters, gradients, and activations do not fit in a single GPU and what does each parallelism strategy cost you in communication overhead, memory, and engineering complexity?*

Where are we?

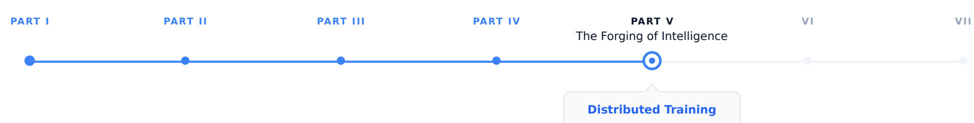


Figure 25.1: The journey through the Model Mind.

We are in Part IV. Chapter 31 covered the single-machine training loop: the forward pass, loss computation, backward pass, and optimizer step as they execute on one device. This chapter asks what happens when the model, the data, or both are too large for one device to hold which is the case for every large language model trained in the past five years.

25.1 The Problem: Nothing Fits

The numbers make the problem concrete. LLaMA 2 7B has 7 billion parameters. In float32, each parameter occupies 4 bytes: 28 GB just for the weights. Training also requires storing gradients, another 28 GB, and Adam optimizer state (first and second moment estimates for every parameter), another 56 GB. Total: 112 GB for 7 billion parameters in full precision. An H100 GPU has 80 GB of HBM. The weights alone saturate the memory budget; gradients and optimizer state overflow it by a factor of two.

For LLaMA 2 70B: 1.12 TB of optimizer state, gradients, and weights combined. For GPT-3 175B: 2.8 TB. For the largest models in active research, parameter counts reach into the hundreds of billions or trillions, with optimizer state occupying multiple petabytes. No single GPU has ever existed that could hold these numbers, and none is on the near-term roadmap.

Training activations compound the problem. During the backward pass, the gradient of the loss with respect to each layer's input must be computed. Computing this gradient requires the layer's activations (the intermediate values computed during the forward pass) to be available at backward time. For a 7B-parameter model trained with context length 4,096 and batch size 1, storing full activations requires roughly 70 GB using float16. For batch size 32: 2.24 TB. Activation recomputation (gradient checkpointing) can trade this memory cost for additional compute (the activations are discarded during the forward pass and recomputed during the backward pass) but introduces a 30–40% compute overhead.

Distributed training resolves the memory problem by distributing the model and the data across multiple GPUs and multiple nodes. The price is communication: moving tensors across GPUs requires interconnect bandwidth and introduces synchronization points that can dominate runtime if not carefully managed. The field has developed three orthogonal parallelism strategies (data parallelism, tensor parallelism, and pipeline parallelism) each addressing a different aspect of the memory and compute problem, and several frameworks for combining them.

25.2 Data Parallelism

Data parallelism is the simplest and oldest distributed training strategy. Each GPU holds a complete copy of the model. The training batch is split across GPUs: if the global batch size is 2,048 sequences and there are 64 GPUs, each GPU processes 32 sequences. Each GPU performs a full forward and backward pass on its local mini-batch, producing local gradients. After the backward pass, the gradients from all GPUs are aggregated (summed and then divided by the number of GPUs) so that each GPU updates its weights using the gradient that would have been computed from the full global batch. This gradient aggregation step is an all-reduce operation.

25.2.1 The All-Reduce

An all-reduce takes N values distributed across N participants, reduces them with an associative operation (summation, in this case), and returns the result to all participants. For gradient aggregation in data parallelism:

- Each GPU has a gradient tensor g_i for its local batch.
- After all-reduce, every GPU holds $\bar{g} = \sum_{i=1}^{N} g_i/N$.
- Each GPU applies $\bar{g}$ to its weight update.

The communication volume is $2 \times P \times$ dtype_bytes per all-reduce operation (the factor of 2 accounts for the reduce followed by broadcast), where P is the number of parameters. For LLaMA 2 7B in float16: $2 \times 7 \times 10^9 \times 2 = 28$ GB per step.

NVLink (within a node) provides 600 GB/s bidirectional bandwidth on H100 SXM. A 28 GB all-reduce over NVLink takes approximately 47 milliseconds, ignoring latency. Across nodes over InfiniBand (400 Gb/s = 50 GB/s per port), the same all-reduce takes over half a second, longer than a typical training step for a 7B model. Data parallelism across nodes is therefore bandwidth-limited and practical only when the compute-to-communication ratio is high (large batch sizes, long sequences, large models).

25.2.2 Gradient Accumulation

When the desired global batch size exceeds what GPU memory can hold in a single step, gradient accumulation allows accumulating gradients over multiple forward- backward passes before performing the all-reduce and weight update. If target global batch size is 2,048 and each GPU can hold a micro-batch of 4 sequences, the GPU performs 512 forward-backward passes, accumulating gradients, before synchronizing.

Gradient accumulation decouples the effective batch size from the per-step memory requirement. It does not reduce memory consumption for the model weights, gradients, or optimizer state, those are determined by the model, not the batch size, but it allows using a global batch size that would otherwise require more GPUs than are available.

The cost is throughput: 512 micro-batches require 512 forward-backward passes per update. CUDA graph capture can reduce the per-step overhead, but the fundamental compute cost is unchanged. Gradient accumulation is a memory-compute tradeoff that trades latency for batch size flexibility.

25.2.3 DDP: PyTorch DistributedDataParallel

PyTorch's DistributedDataParallel (DDP) is the standard implementation of data parallelism. DDP wraps a model and, after each backward pass, performs an all-reduce of gradients across all participating processes using NCCL (NVIDIA Collective Communications Library). DDP overlaps gradient communication with the backward pass using gradient bucketing: as gradients for later layers are computed, gradients for earlier layers (whose backward pass is already complete) are all-reduced while the backward pass continues. This overlap can hide a substantial fraction of the communication latency.

DDP's limitation is the memory requirement: every GPU holds the full model. For models that fit in GPU memory with some headroom, DDP is the lowest- complexity distributed training option. It scales to thousands of GPUs with near-linear throughput scaling, limited primarily by the all-reduce bandwidth. GPT-3 was trained using data parallelism as a component of a hybrid strategy. At 96 GPUs with a global batch size of roughly 3.2 million tokens, DDP communication overhead was manageable.

25.3 ZeRO: Sharding Optimizer State, Gradients, and Parameters

DDP requires each GPU to hold a full copy of the model parameters, gradients, and optimizer state. For large models, this redundancy is the binding memory constraint. ZeRO (Zero Redundancy Optimizer), introduced by Rajbhandari et al. (2020) at Microsoft, eliminates this redundancy through sharding.

ZeRO defines three stages, each eliminating a different component of per-GPU redundancy:

ZeRO Stage 1: Optimizer State Partitioning. The optimizer state (Adam moments) is sharded across GPUs: each GPU holds the optimizer state for a $1/N$ partition of the parameters. Gradients are still all-reduced (giving each GPU the full gradient), but each GPU only updates and stores the optimizer state for its partition. After the optimizer step, an all-gather reconstructs the full updated parameters on each GPU.

Memory reduction: optimizer state drops from $O(P)$ to $O(P/N)$ per GPU. For Adam in float32 with N=64 GPUs: 56 GB of optimizer state per GPU becomes 875 MB.

ZeRO Stage 2: Gradient Partitioning. In addition to Stage 1, gradients are also sharded. Each GPU accumulates and stores only the gradients for its parameter partition. A reduce-scatter (rather than all-reduce) delivers the summed gradient only to the GPU responsible for that partition, after which each GPU applies the optimizer step to its shard. An all-gather then reconstructs full parameters.

Memory reduction: gradients drop from $O(P)$ to $O(P/N)$ per GPU. With N=64: 28 GB of gradients per GPU becomes 437 MB.

ZeRO Stage 3: Parameter Partitioning. Parameters themselves are sharded. Each GPU permanently holds only a $1/N$ slice of the parameters. During the forward pass, an all-gather reconstructs the full parameters for each layer as it is needed, then discards them after the layer's computation is complete. The backward pass similarly all-gathers parameters as needed and performs reduce-scatter for gradients.

Memory reduction: the per-GPU parameter memory drops from $O(P)$ to $O(P/N)$. For LLaMA 2 7B with float16 weights across 64 GPUs: 14 GB $\rightarrow$ 219 MB per GPU. The total optimizer state, gradient, and weight memory per GPU drops from 112 GB to approximately 1.75 GB.

The cost of ZeRO-3 is communication volume. Where DDP performs one all-reduce of $2P$ bytes per step, ZeRO-3 performs three communication operations, an all-gather for parameters during the forward pass (P bytes), a reduce-scatter for gradients during the backward pass (P bytes), and another all-gather for parameters during the backward pass (P bytes), totaling $3P$ bytes. This is 50% more communication than DDP's $2P$ bytes, but the memory savings are often decisive for large models.

25.3.1 ZeRO: A Concrete Example (N=4 GPUs, 4 Parameter Partitions)

We track three state types per GPU: **parameters** W, **gradients** G, and **optimizer state** (Adam moments m, v).

25.3.1.1 Baseline: DDP

Every GPU holds everything, full redundancy.

```
GPU 0               GPU 1               GPU 2               GPU 3
W0 W1 W2 W3         W0 W1 W2 W3         W0 W1 W2 W3         W0 W1 W2 W3
G0 G1 G2 G3         G0 G1 G2 G3         G0 G1 G2 G3         G0 G1 G2 G3
m0 v0               m0 v0               m0 v0               m0 v0
m1 v1               m1 v1               m1 v1               m1 v1
m2 v2               m2 v2               m2 v2               m2 v2
m3 v3               m3 v3               m3 v3               m3 v3
```

Step: Each GPU runs forward and backward on its local mini-batch, producing local gradients. An **all-reduce** sums and averages these gradients, so every GPU obtains the same full gradient G. Every GPU then independently runs the optimizer update:

$$W' = \mathrm{Opt}(W, G, m, v)$$

Because all GPUs have identical W, G, and optimizer states, they all compute the same updated parameters W'. No parameter communication is required after the optimizer step.

25.3.1.2 Stage 1: Shard Optimizer States

Optimizer state is split across GPUs; parameters and gradients are still replicated.

```
GPU 0               GPU 1               GPU 2               GPU 3
W0 W1 W2 W3         W0 W1 W2 W3         W0 W1 W2 W3         W0 W1 W2 W3
G0 G1 G2 G3         G0 G1 G2 G3         G0 G1 G2 G3         G0 G1 G2 G3
m0 v0               m1 v1               m2 v2               m3 v3
```

Step: After backward, an **all-reduce** gives every GPU the full gradient $G = [G_0, G_1, G_2, G_3]$. Each GPU is responsible for updating only the parameter partition for which it owns the optimizer state:

```
GPU 0 → update W0 using G0, m0, v0
GPU 1 → update W1 using G1, m1, v1
GPU 2 → update W2 using G2, m2, v2
GPU 3 → update W3 using G3, m3, v3
```

After the optimizer step, the updated parameters exist as separate shards:

```
GPU 0 → W0'
GPU 1 → W1'
GPU 2 → W2'
GPU 3 → W3'
```

Because ZeRO-1 still expects every GPU to hold the complete model for the next forward pass, an **all-gather** combines these updated shards so that every GPU again has:

```
W0' W1' W2' W3'
```

The optimizer state remains sharded; only the updated parameters are reconstructed on every GPU.

25.3.1.3 Stage 2: Shard Optimizer States + Gradients

Replace the all-reduce with a **reduce-scatter**: each GPU receives only the gradient shard it owns.

```
GPU 0            GPU 1            GPU 2            GPU 3
W0 W1 W2 W3      W0 W1 W2 W3      W0 W1 W2 W3      W0 W1 W2 W3
G0               G1               G2               G3
m0 v0            m1 v1            m2 v2            m3 v3
```

Step: Each GPU first computes local gradients for the full model. The **reduce-scatter** sums the corresponding gradients across GPUs and leaves only the relevant gradient partition on each GPU:

```
GPU 0 → G0
GPU 1 → G1
GPU 2 → G2
GPU 3 → G3
```

Each GPU can now update its own parameter partition using its local gradient and optimizer state:

```
GPU 0 → W0' = Opt(W0, G0, m0, v0)
GPU 1 → W1' = Opt(W1, G1, m1, v1)
GPU 2 → W2' = Opt(W2, G2, m2, v2)
GPU 3 → W3' = Opt(W3, G3, m3, v3)
```

The updated parameter shards are then **all-gathered**, reconstructing the full updated model $[W_0', W_1', W_2', W_3']$ on every GPU. The parameters are therefore still replicated during the next forward pass, but the gradients and optimizer states remain sharded.

25.3.1.4 Stage 3: Shard Everything

No GPU permanently holds a full copy of anything.

```
GPU 0            GPU 1            GPU 2            GPU 3
W[:,0:k]         W[:,k:2k]        W[:,2k:3k]       W[:,3k:4k]    ← parameter shards
G0               G1               G2               G3
m0 v0            m1 v1            m2 v2            m3 v3
```

The parameter shards represent slices of the parameters of every layer; GPU 0 does not own one complete layer while GPU 1 owns another.

Full parameters are reconstructed **on demand** via all-gather, used for computation, and then discarded.

Forward: For each layer ℓ:

$$\text{all-gather } W_\ell \rightarrow \text{compute layer } \ell \rightarrow \text{discard full } W_\ell$$

Only the parameters needed for the current layer are temporarily reconstructed on each GPU.

Backward: For each layer ℓ:

$$\text{all-gather } W_\ell \rightarrow \text{compute gradients} \rightarrow \text{reduce-scatter } G_\ell \rightarrow \text{discard full } W_\ell$$

The reduce-scatter leaves each GPU with only the gradient corresponding to its parameter shard.

Step: Each GPU now has the three pieces of state required to update its local parameter shard:

```
GPU 0 → W[:,0:k]  + G0 + m0,v0 → W'[:,0:k]
GPU 1 → W[:,k:2k] + G1 + m1,v1 → W'[:,k:2k]
```

```
GPU 2 → W[:,2k:3k]+ G2 + m2,v2 → W'[:,2k:3k]
GPU 3 → W[:,3k:4k]+ G3 + m3,v3 → W'[:,3k:4k]
```

Crucially, unlike ZeRO-1 and ZeRO-2, the updated parameter shards are **not all-gathered permanently after the optimizer step**. They remain distributed. On the next forward pass, the parameters are again all-gathered temporarily, layer by layer, when they are needed for computation. This is what allows ZeRO-3 to keep the model parameters sharded throughout training.

25.3.2 FSDP: PyTorch's ZeRO-3 Implementation

PyTorch's Fully Sharded Data Parallel (FSDP) is the standard production implementation of ZeRO-3, available from PyTorch 1.12. FSDP shards parameters, gradients, and optimizer state across GPUs within a process group. It integrates with PyTorch's autograd system and supports mixed precision, gradient checkpointing, and CPU offloading.

FSDP's key configuration decision is the sharding unit. Sharding at the module level (per transformer block, for example) limits the all-gather to one block's parameters at a time, reducing peak memory usage during forward and backward passes. Sharding at finer granularity reduces memory further but increases the number of all-gather calls and the associated latency.

A typical FSDP training configuration for a 70B-parameter model on 64 H100s: - Parameter dtype: bfloat16 (2 bytes per parameter) - Optimizer dtype: float32 (8 bytes per parameter for Adam) - Per-GPU parameter memory: $140 \times 10^9 \times 2/64 \approx 4.4$ GB - Per-GPU optimizer state: $140 \times 10^9 \times 8/64 \approx 17.5$ GB - Total per-GPU: ~22 GB, well within an 80 GB H100

25.4 Tensor Parallelism

Data parallelism and ZeRO shard the data and the optimizer state, but the forward pass through a single transformer layer still runs on a single device. For very large models (hundreds of billions of parameters) individual layers are too large to fit on one GPU even with ZeRO-3. Tensor parallelism shards the computation within individual layers.

25.4.1 Column and Row Parallelism for Linear Layers

The key insight, from Megatron-LM (Shoeybi et al., 2019), is that large matrix multiplications in transformer layers can be split across devices with minimal communication.

Consider a linear layer $Y = XW$ where $X \in \mathbb{R}^{B\times d}$ is the input and $W \in \mathbb{R}^{d\times h}$ is the weight matrix. This can be parallelized two ways:

Column parallelism splits W along columns: $W = [W_1 \mid W_2]$ across two GPUs, with $W_1, W_2 \in \mathbb{R}^{d\times h/2}$. Each GPU computes $Y_i = XW_i \in \mathbb{R}^{B\times h/2}$ independently, since both GPUs need the full input X. The outputs are concatenated: $Y = [Y_1 \mid Y_2]$. No communication is required during the forward pass (assuming X is already replicated on both GPUs). An all-gather concatenates outputs.

Row parallelism splits W along rows: $W = [W_1^T \mid W_2^T]^T$, with each GPU holding $W_i \in \mathbb{R}^{d/2\times h}$. The input must be split correspondingly: $X = [X_1 \mid X_2]$, with each GPU receiving its slice. Each GPU computes $Y_i = X_i W_i \in \mathbb{R}^{B\times h}$. The outputs are summed: $Y = Y_1 + Y_2$. An all-reduce sums the partial outputs.

In a transformer's self-attention layer, the Q/K/V projection matrices are parallelized with column parallelism (output is split across heads, which maps naturally to column splits). The output projection after attention is parallelized with row parallelism. Together, this requires exactly two communication operations per attention layer, one all-reduce after the output projection during the forward pass and one during the backward pass. For a transformer with L layers, tensor parallelism introduces $2L$ all-reduce operations per forward-backward pass.

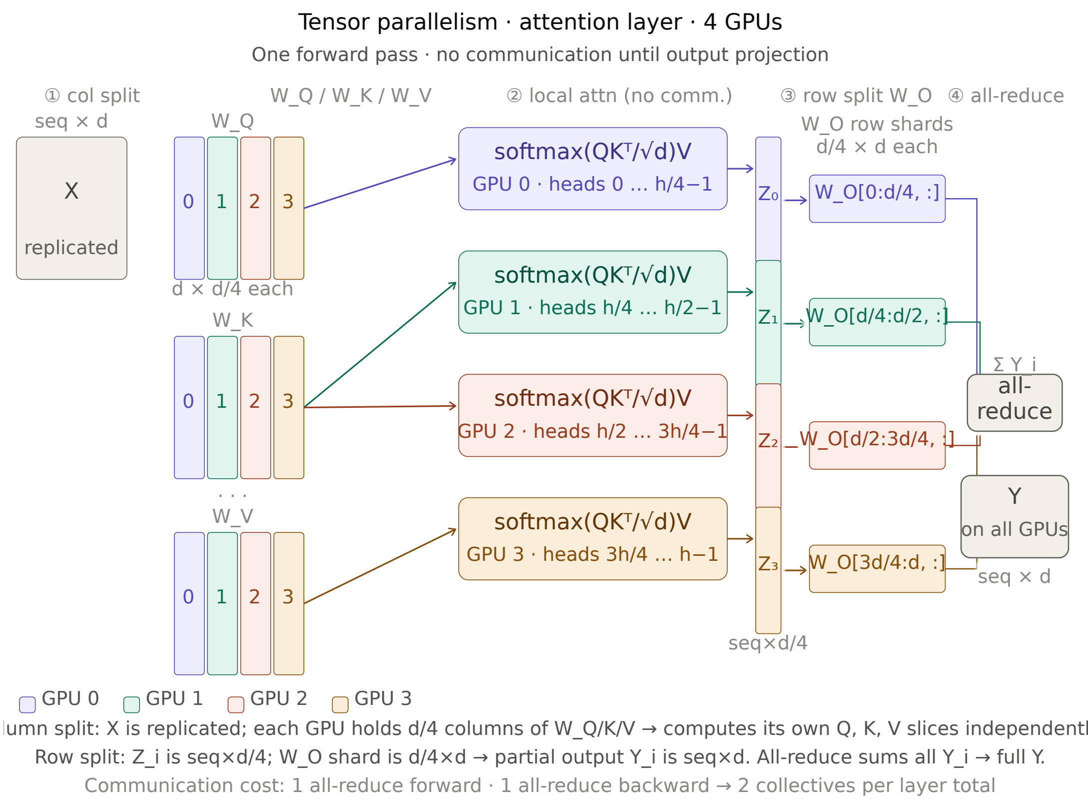


Figure 25.2: Tensor parallelism in transformer.

For GPT-3 with 96 layers and tensor parallelism degree 8: 192 all-reduce operations per step. On NVLink (600 GB/s), where each all-reduce involves roughly $2 \times h \times \text{dtype\_bytes} \approx 2 \times 12{,}288 \times 2 = 48$ KB per operation, the communication overhead is manageable within a single node. Across nodes (InfiniBand at 50 GB/s), the same all-reduces are approximately 12× slower, making cross-node tensor parallelism impractical for most model sizes.

Tensor parallelism degree is typically 2, 4, or 8 (powers of 2, matching NVLink topology) and is applied within a node. An 8-way tensor-parallel split across 8 H100s connected by NVLink has minimal communication overhead. Extending to 16-way requires cross-node links and pays a substantial bandwidth penalty.

25.4.2 Attention Heads as Natural Split Units

Multi-head attention is naturally amenable to tensor parallelism because each attention head is independent. With 96 attention heads (GPT-3's configuration) and 8-way tensor parallelism, each GPU handles 12 heads. The head outputs are concatenated and projected back to the model dimension with row parallelism. This is architecturally clean and communication-efficient: the 8-way split exactly partitions the head dimension without any cross-head dependencies during the attention computation.

Grouped-query attention (GQA, used in LLaMA 2 70B and most subsequent models) reduces the number of KV heads relative to query heads. With 8 KV heads and tensor parallelism degree 8, each GPU holds exactly 1 KV head, an efficient split. At tensor parallelism degree greater than the number of KV heads, GQA and tensor parallelism become incompatible without replication.

25.5 Pipeline Parallelism

Tensor parallelism splits computation within a layer. Pipeline parallelism splits computation across layers, assigning different layers to different devices. Device 0 holds layers 1–8; device 1 holds layers 9–16; and so on.

The forward pass moves left to right: device 0 processes a micro-batch and passes its output activation to device 1, which processes it and passes to device 2, and so on. The backward pass moves right to left: device 3 computes gradients for its layers and passes gradient tensors to device 2, which continues back to device 0.

25.5.1 The Pipeline Bubble

Naive pipeline parallelism has a critical inefficiency: pipeline bubbles. When device 0 finishes its forward pass and passes activations to device 1, device 0 must wait, it has nothing to do until the backward pass gradient arrives from device 1. This idle time is the pipeline bubble.

For a pipeline of p stages and a mini-batch split into m micro-batches, the pipeline bubble occupies a fraction $\frac{p-1}{m+p-1}$ of the total training time. With $p = 4$ stages and $m = 8$ micro-batches: bubble fraction $= 3/11 \approx 27\%$. With $m = 32$: $3/35 \approx 8.6\%$. Larger micro-batch counts reduce the bubble overhead but increase memory for in-flight activations.

Megatron-LM's interleaved pipeline schedule reduces the bubble further by assigning non-contiguous layer chunks to each device (device 0 holds layers 1–4 and layers 17–20, device 1 holds layers 5–8 and layers 21–24, etc.). The interleaved schedule reduces the bubble fraction to $\frac{1}{m} \cdot \frac{p-1}{p}$, cutting the overhead by roughly a factor of the number of interleaved chunks per device. The cost is increased communication: activations must be passed between non-adjacent layer chunks at interleaving boundaries.

25.6 3D Parallelism: Combining All Three

Production training of large models combines data, tensor, and pipeline parallelism simultaneously. This combination is called 3D parallelism.

The standard configuration, used in Megatron-LM and adopted broadly:

- **Tensor parallelism (TP)** within a node, using NVLink. Typical TP degree: 4 or 8. This is the inner loop, tightest communication requirement.
- **Pipeline parallelism (PP)** across nodes at small scale, or within a node at large TP degree. Typical PP degree: 2–8. Communication is point-to-point between adjacent stages (only activation tensors, not all-reduces) so it tolerates higher latency than TP.
- **Data parallelism (DP)** at the outermost level, across the largest dimension of the GPU cluster. Typical DP degree: 8–1,024. All-reduces run over InfiniBand between data-parallel replica groups.

For a training run on 512 H100s with TP=8, PP=4, DP=16: - 8 GPUs in each tensor-parallel group (within a single node) - 4 nodes in each pipeline-parallel group (across nearby nodes, low latency) - 16 data-parallel replicas (all-reduce across 16 groups of 32 GPUs)

The total GPU count: $8 \times 4 \times 16 = 512$. This maps naturally to a cluster of 64 8-GPU nodes organized into 16 groups of 4 nodes.

GPT-3 was trained with TP=8, PP=8, DP=6 on 384 A100 GPUs. LLaMA 2 70B was trained with TP=8, PP=1, DP=56 on 2,048 A100 80GB GPUs.

25.7 Communication Primitives and Interconnects

Efficient distributed training requires efficient collective communication operations. NCCL (NVIDIA Collective Communications Library) implements these for GPU clusters. The key collectives:

Operation	Description	Use in distributed training
All-reduce	Sum tensors across all ranks, return result to all	DDP gradient aggregation
Reduce-scatter	Sum tensors, distribute partial results	ZeRO gradient reduction
All-gather	Concatenate shards from all ranks	ZeRO parameter reconstruction
Broadcast	Send tensor from one rank to all	Parameter initialization
Point-to-point	Send tensor from rank i to rank j	Pipeline stage activation passing

---

25.8 Key Takeaways

- A 7B-parameter model requires approximately 112 GB for weights, gradients, and Adam optimizer state in mixed precision — exceeding a single H100's 80 GB HBM even before activations, requiring distributed training for all models at and above this scale.
- Data parallelism replicates the full model on each GPU and splits the batch; ZeRO stages 1–3 eliminate the redundant optimizer state, gradients, and parameters respectively, reducing per-GPU memory from $O(P)$ to $O(P/N)$.
- ZeRO-3 (implemented as PyTorch FSDP) enables training models of arbitrary size on any number of GPUs at the cost of 50% more communication volume than standard DDP ($3P$ bytes versus $2P$ bytes per step).
- Tensor parallelism splits matrix multiplications within layers across devices, requiring two all-reduce operations per transformer layer; the bandwidth requirement limits tensor parallelism to within-node NVLink connections.
- Pipeline parallelism splits layers across devices; the pipeline bubble wastes a fraction $\frac{p-1}{m+p-1}$ of compute time, which shrinks toward zero as the number of micro-batches m grows.

- 3D parallelism combines tensor parallelism (inner, NVLink), pipeline parallelism (middle, cross-node), and data parallelism (outer, InfiniBand) to scale to thousands of GPUs with each dimension exploiting the appropriate interconnect.
- NVLink (900 GB/s on H100 SXM) is 18× faster than InfiniBand NDR (50 GB/s); this bandwidth gap is the primary reason tensor parallelism is confined within nodes.
- Mixed-precision training uses bfloat16 for forward and backward passes (matching float32's dynamic range without overflow) and float32 for optimizer state; total memory footprint is $16P$ bytes for Adam.
- Gradient checkpointing reduces activation memory from $O(L \times B \times T)$ to $O(\sqrt{L})$ at 30–40% additional compute; FlashAttention already recomputes attention within this budget.
- Well-optimized training runs on H100 clusters achieve 38–46% MFU; the remaining 54–62% is lost to communication, pipeline bubbles, and memory bandwidth bottlenecks.
- At 4,096 GPUs, expect roughly 4 GPU failures per hour; asynchronous checkpointing eliminates checkpoint latency from the critical path by snapshotting to host RAM while training continues.

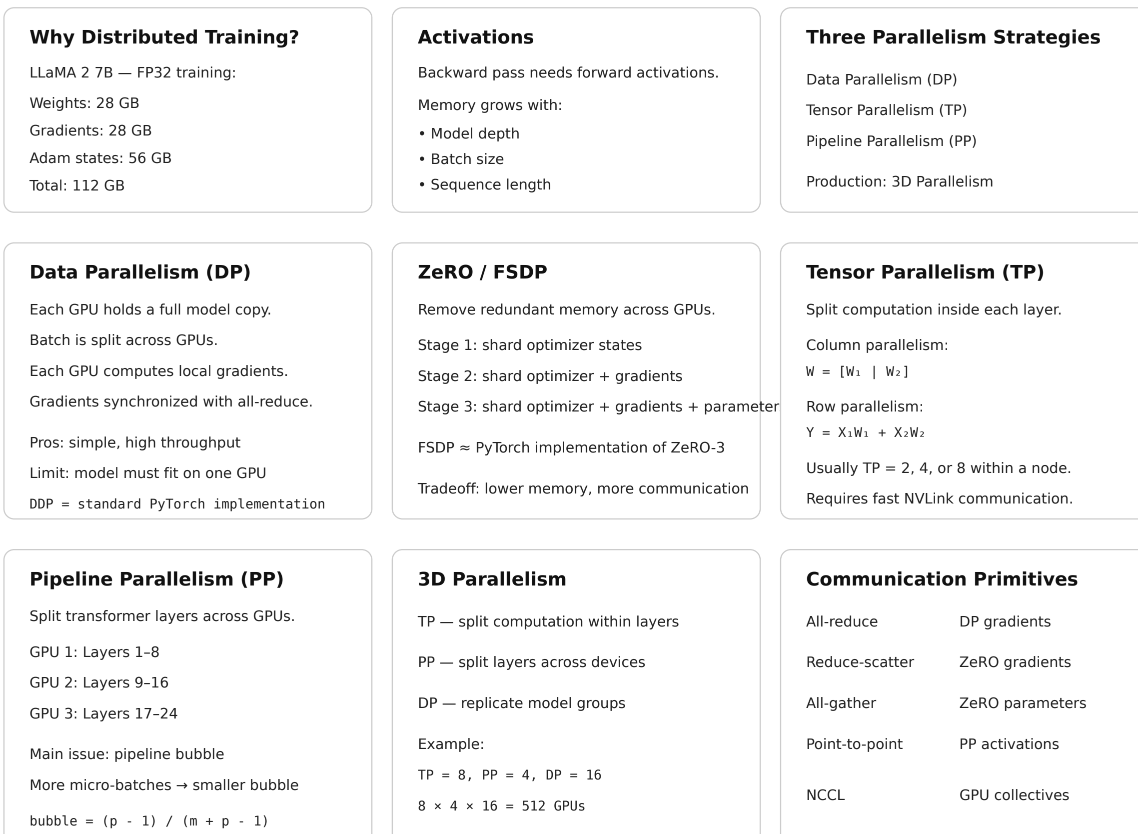


Figure 25.3: Cheat sheet.

25.9 Further Reading

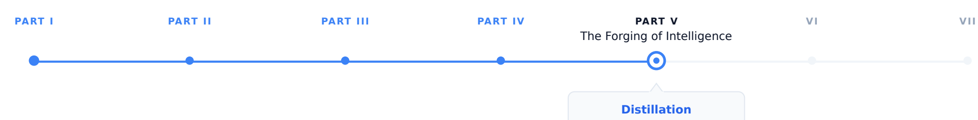
- Rajbhandari, S., Rasley, J., Ruwase, O., & He, Y. (2020). *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models.* SC. — Introduces the three stages of ZeRO; the memory analysis is precise and the scaling experiments demonstrate near-linear throughput scaling to 400+ GPUs.

- Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., & Catanzaro, B. (2019). *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism.* arXiv. — Introduces column/row parallelism for transformer layers and the interleaved pipeline schedule; the communication analysis is the foundational treatment of tensor parallelism for LLMs.
- Narayanan, D., et al. (2021). *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM.* SC. — Combines tensor, pipeline, and data parallelism into 3D parallelism; the analysis of pipeline bubble fraction and the 1F1B schedule are the key contributions.
- Micikevicius, P., et al. (2018). *Mixed Precision Training.* ICLR. — Introduces the float16/float32 mixed-precision training procedure with dynamic loss scaling; the gradient underflow analysis motivates loss scaling clearly.
- Chen, T., et al. (2016). *Training Deep Nets with Sublinear Memory Cost.* arXiv. — Introduces gradient checkpointing; the $O(\sqrt{n})$ memory-compute tradeoff analysis is the key result.
- Chowdhery, A., et al. (2022). *PaLM: Scaling Language Modeling with Pathways.* JMLR. — Reports 46.2% MFU on TPU v4 with detailed analysis of communication overlap and efficiency at 6,144 chips; the infrastructure section is unusually detailed for a model paper.
- Touvron, H., et al. (2023). *LLaMA 2: Open Foundation and Fine-Tuned Chat Models.* — Section 2.1 describes the training infrastructure: 2,048 A100 GPUs, custom fast interconnect, training efficiency optimizations, and MFU figures; a concise real-world 3D parallelism case study.
- Lian, X., et al. (2023). *FSDP: Fully Sharded Data Parallel Training.* PyTorch Blog. — The engineering design document for PyTorch FSDP; covers the sharding unit tradeoffs, mixed precision integration, and the async checkpoint API.

Chapter 26

Distillation

The canonical question for this chapter: *How do you transfer the knowledge of a large, expensive model into a smaller, cheaper one and why does training on a teacher's output distributions produce a better student than training on the original data alone?*



Where are we?
This chapter addresses the gap between the best model you can train and the best model you can serve: distillation transfers capability from a large teacher to a small student, closing that gap at a fraction of the inference cost.

26.1 The Deployment Problem That Distillation Solves

Scaling laws favor large models. A 70B-parameter model achieves lower loss than a 7B-parameter model trained for the same number of tokens on the same data. But serving a 70B model in production costs roughly 10× more per token than serving a 7B model, more GPU memory, more compute per forward pass, more infrastructure. For latency-sensitive applications, the 70B model may not meet response time requirements at all.

The naive response is to train a smaller model from scratch. But a 7B model trained from scratch on the same data achieves meaningfully lower quality than a 70B model. The question is whether there is a better starting point for the 7B model than random initialization, whether the 70B model's learned knowledge can be transferred to make the 7B model better than it would be from scratch on the same compute budget.

Distillation answers yes. Hinton, Vinyals, and Dean (2015) showed that training a small student model to match the output distribution of a large teacher model (rather than to match the one-hot targets of the original training labels) produces students that substantially outperform identically-sized models trained on the original data alone. The mechanism is information-theoretic: the teacher's soft output distributions carry more signal per example than the hard labels, because they encode the teacher's uncertainty and the relative similarity between classes or tokens.

26.2 Knowledge Distillation: The Core Mechanism

26.2.1 Hard Labels vs. Soft Targets

In standard training, the loss at each position is the cross-entropy between the model's predicted distribution and a one-hot target: probability 1.0 on the correct token, probability 0.0 on all others. The one-hot target discards the structure of the probability space, it treats "the wrong answer" as a single undifferentiated category.

A teacher model's output distribution over the vocabulary is richer. For a prompt ending in "The capital of France is", a teacher model might assign: - 0.94 probability to "Paris" - 0.02 probability to "Lyon" - 0.01 probability to "Marseille" - 0.003 probability to "Brussels" - smaller probabilities to hundreds of other tokens

The non-Paris probabilities carry information. "Lyon" being more probable than "Brussels" encodes that Lyon is a French city, that the model knows France has multiple major cities, and that cities structurally fit this slot better than countries do. A student trained to match this distribution learns all of these relationships from a single example. A student trained on the one-hot target "Paris" learns only that "Paris" is correct and the rest of the probability mass is uninformative noise in the gradient signal.

Hinton et al. formalized this with temperature-scaled softmax. The teacher's logits z_i are converted to probabilities using a temperature T:

$$q_i^T = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

At $T = 1$, this is the standard softmax. At $T > 1$, the distribution is softened: high-probability tokens become less dominant and low-probability tokens become more visible. At $T \to \infty$, the distribution approaches uniform. The same temperature is applied to the student's logits when computing the distillation loss, so the student learns to reproduce the teacher's softened distribution.

26.2.2 The Distillation Loss

The student is trained on a weighted combination of two losses:

$$\mathcal{L}_{\text{distill}} = \alpha \cdot \mathcal{L}_{\text{CE}}(y, p_s) + (1 - \alpha) \cdot T^2 \cdot \mathcal{L}_{\text{KL}}(q_\tau^T, p_s^T)$$

where: - $\mathcal{L}_{\text{CE}}(y, p_s)$ is the standard cross-entropy loss between the student's predictions and the ground-truth hard labels y - $\mathcal{L}_{\text{KL}}(q_\tau^T, p_s^T)$ is the KL divergence between the teacher's temperature-softened distribution q^T and the student's temperature-softened distribution p_s^T - $\alpha \in [0, 1]$ controls the relative weight of the two losses - T^2 is a scaling factor that compensates for the reduced magnitude of gradients from the softened distributions

The T^2 scaling term deserves explanation. When temperature T is applied, the softmax output values are compressed toward $1/V$ (where V is the vocabulary size). The gradients of the KL loss with respect to the student's logits are proportional to $1/T^2$ relative to the hard-label gradients. Multiplying by T^2 restores the gradient magnitudes to the same scale, ensuring that the distillation signal and the hard-label signal contribute comparably to the weight update regardless of temperature.

In practice for language model distillation: - $\alpha = 0.5$ is a common default, giving equal weight to both losses - $T = 2$ to $T = 4$ is typical; higher temperatures are used when the teacher is much more confident than the student - When ground-truth labels are unavailable (the student is trained on teacher-generated text), the hard-label term is dropped: $\alpha = 0$, pure distillation

26.3 Types of Distillation for Language Models

Distillation for language models comes in several flavors that differ in what is transferred from teacher to student, and when.

26.3.1 Output (Response) Distillation

The simplest form: train the student to match the teacher's token-level output distributions. For each token position in a training sequence, the teacher produces a probability distribution over the vocabulary; the student is trained to minimize KL divergence from that distribution.

This requires running the teacher model on the training corpus to generate soft targets, which are then stored and used for student training. Storage cost is substantial: for a vocabulary of 100,000 tokens and float16 precision, each token position requires 200 KB of soft targets. A training corpus of 100 billion tokens would require 20 petabytes of stored soft targets which is infeasible. In practice, soft targets are either computed on-the-fly during student training (requiring the teacher to be available in memory alongside the student) or truncated to the top-k logits (storing only the k highest teacher probabilities, discarding the long tail which contributes little signal).

Top-k logit storage with $k = 20{,}000$ retains roughly 95% of the KL divergence signal while reducing storage by 5×. DistilBERT and DistilGPT-2 used top-k approximations for exactly this reason.

26.3.2 Sequence-Level (Black-Box) Distillation

When the teacher's internal logits are unavailable (either because it is a proprietary API or because storing full distributions is infeasible) the student can be trained on sequences generated by the teacher rather than on the teacher's distributions.

The student treats teacher-generated sequences as training data and minimizes standard next-token prediction loss against them. This is called black-box distillation or data distillation. It is strictly less efficient than distribution-level distillation (the student sees only samples from the teacher's distribution, not the distribution itself) but it is the only option when internal model access is unavailable.

This is the mechanism underlying most current synthetic data pipelines. When GPT-4 generates 10,000 instruction-response pairs and those pairs are used to train a smaller model, the smaller model is being distilled from GPT-4 via sequence-level distillation. The term "synthetic data" emphasizes the data generation aspect; the term "distillation" emphasizes the knowledge transfer aspect.

26.3.3 Feature (Intermediate) Distillation

Rather than matching only the teacher's output distributions, feature distillation trains the student to match the teacher's intermediate representations, the hidden states at each layer. This transfers more information but requires architectural compatibility between teacher and student: the student's hidden dimension must match the teacher's, or an additional projection layer must bridge the mismatch.

PKD (Patient Knowledge Distillation, Sun et al., 2019) distills BERT by matching the student's intermediate layer outputs to selected layers of the teacher. TinyBERT (Jiao et al., 2020) extends this to match attention matrices, hidden states, and the embedding layer simultaneously, using learned projection matrices to handle dimension mismatches:

$$\mathcal{L}_{\text{attn}} = \frac{1}{h} \sum_{i=1}^{h} \text{MSE}(A_i^S, A_i^T)$$

$$\mathcal{L}_{\text{hidden}} = \text{MSE}(H^S W_h, H^T)$$

where A_i^S and A_i^T are the student's and teacher's attention matrices for head i, H^S and H^T are hidden states, and W_h is a learned projection. TinyBERT achieves 96.8% of BERT-base performance at 7.5× faster inference and 7.5× smaller model size, the most favorable efficiency tradeoff published for BERT-family distillation.

Feature distillation is less commonly applied to generative LLMs, where architectural differences between teacher and student are larger and the intermediate representations are harder to align meaningfully. Attention pattern matching works well when the student has the same number of heads as the teacher but fails when the student uses GQA and the teacher uses MHA, for example.

26.3.4 Speculative Sampling and Its Connection to Distillation

Speculative sampling (covered in the Decoding and Sampling chapter) is primarily an **inference technique**, not a model-training technique. It uses two models: a small, fast **draft model** and a large, more accurate **target model** (also called the verifier). The draft model quickly generates several candidate tokens. The target model then evaluates these tokens and decides which ones to accept. Rejected tokens are replaced by samples from a correction distribution defined using the target and draft probabilities.

The important property is that this procedure preserves the target model's output distribution. In other words, although the draft model proposes the tokens, the final sequence is distributed exactly as if it had been generated directly by the target model.

This is **not distillation** because the draft model is not being trained to imitate the target model. The target model is simply being used to verify the draft model's proposals during inference:

[Draft model →proposes tokens →Target model verifies →accept or correct]

However, the same verification process can provide a training signal for improving the draft model. If we fine-tune the draft model so that it proposes tokens that the target model is more likely to accept, the draft model gradually learns to approximate the target model's behavior. The target model therefore acts as a teacher, while the draft model acts as a student.

Conceptually, the process becomes:

[Draft model proposes →Target model evaluates →Training signal →Update draft model]

This creates a connection to **knowledge distillation**. The goal is no longer simply to use the draft model for faster inference; we are actively training it to behave more like the target model. In this setting, the target model provides the supervision signal, often through its probability distribution or through feedback about which draft tokens it accepts.

Thus, it is useful to distinguish two cases:

- **Speculative sampling:** the draft model is used to accelerate inference, while the target model remains the source of the final output distribution.
- **Distillation-inspired draft training:** feedback from the target model is used to improve the draft model so that it generates proposals closer to the target model's distribution, increasing the acceptance rate and potentially making speculative sampling more efficient.

The key idea is therefore not that speculative sampling itself is a form of distillation. Rather, **the verifier's feedback during speculative decoding can be used as a training signal to distill some of the target model's behavior into the draft model**.

26.4 Distillation at Scale: LLM-to-LLM

Distilling a 175B-parameter teacher into a 7B-parameter student is qualitatively different from distilling a 110M-parameter teacher into a 66M-parameter student. The capacity gap is much larger and the student's ability to represent the teacher's distributions is correspondingly more limited.

26.4.1 The Capacity Gap Problem

A 7B-parameter student cannot faithfully reproduce all of a 70B teacher's probability distributions. For difficult predictions (ambiguous sequences where the teacher assigns roughly equal probability to multiple continuations) the student lacks the capacity to represent the full distribution and will collapse to a lower-entropy approximation. The distillation loss will be large at these positions regardless of training duration.

The practical implication: distillation with a very large capacity gap produces students that are good at easy predictions (common patterns, high-confidence teacher outputs) but poor at difficult predictions (rare patterns, uncertain teacher outputs). This is not a failure of the distillation algorithm, it is a fundamental capacity constraint. A 7B model cannot represent what a 70B model knows; it can only approximate it.

This motivates selective distillation: targeting the distillation loss at positions where the teacher's distribution is most informative and down-weighting positions where the teacher is either highly confident (low information in the soft targets) or highly uncertain (student cannot match the distribution regardless). The confidence-weighted distillation loss:

$$\mathcal{L}_{\text{weighted}} = \sum_t w_t \cdot \mathcal{L}_{\text{KL}}(q_t^T, p_{s,t}^T)$$

where $w_t = 1 - H(q_t)/\log V$ is a weight that decreases as the teacher's entropy $H(q_t)$ approaches its maximum $\log V$. This concentrates the distillation signal on positions where the teacher's distribution is informative, neither trivially predictable nor maximally uncertain.

26.4.2 On-Policy vs. Off-Policy Distillation

Standard distillation is off-policy: the student is trained on sequences from the original training corpus or teacher-generated sequences, which may not match the distribution of sequences the student itself would generate.

On-policy distillation trains the student by having it generate sequences, then computing the KL divergence between the student's and teacher's distributions on those student-generated sequences. This ensures the student receives gradient signal at the points in distribution space it actually visits, rather than points from a fixed dataset that may be far from the student's current behavior.

GKD (Generalized Knowledge Distillation, Agarwal et al., 2024) formalizes this distinction and shows that on-policy distillation consistently outperforms off-policy distillation for language models, particularly at large capacity gaps. The improvement is most pronounced for open-ended generation tasks where the student's distribution can deviate substantially from the training corpus distribution.

The cost of on-policy distillation is that it requires generating sequences during training, which adds inference-time compute (running the student in generation mode) to the training-time compute (running the student and teacher in forward-pass mode). This roughly doubles the per-step training cost relative to off-policy distillation.

26.5 Reasoning Distillation

The distillation of reasoning capability in particular chain-of-thought reasoning, deserves separate treatment because it involves transferring a qualitatively different type of knowledge than factual recall or language modeling.

26.5.1 Chain-of-Thought Distillation

Large models (70B+) benefit substantially from chain-of-thought prompting: producing a step-by-step reasoning trace before the final answer significantly improves accuracy on mathematical, logical, and multi-step tasks. Small models (7B and below) receive less benefit from standard CoT prompting, apparently because they lack the capacity to generate accurate reasoning traces.

Distillation of reasoning capability from a large teacher to a small student involves training the student on teacher-generated reasoning traces, not just teacher-generated final answers. The student learns both to produce reasoning traces (the process) and to arrive at correct answers (the outcome).

Ho et al. (2022) showed that a 250M-parameter student trained on GPT-3's chain-of-thought outputs substantially outperforms a 540B-parameter model prompted with standard CoT on several reasoning benchmarks, demonstrating that the reasoning capability can be compressed to a dramatically smaller model when the teacher's reasoning traces are used as training targets.

26.5.2 Step-Level vs. Outcome-Level Supervision

A key design choice in reasoning distillation is whether to supervise the student at the level of individual reasoning steps or only at the final answer.

Outcome-level supervision trains the student to produce the correct final answer, regardless of the reasoning path taken. This is what standard sequence-level distillation on teacher-generated outputs provides. The student may learn shortcuts or alternative reasoning paths that differ from the teacher's.

Step-level supervision trains the student to match the teacher's intermediate reasoning steps, not just the final answer. This requires either parsing the teacher's reasoning traces into discrete steps (fragile) or using a process reward model that scores each step's quality (expensive).

For mathematical reasoning, step-level supervision produces more reliable generalization, students that have learned to execute each reasoning step correctly generalize better to novel problem structures than students trained only on correct final answers.

26.5.3 DeepSeek-R1 as Reasoning Distillation

DeepSeek-R1 (2025) provides one of the most striking examples of reasoning distillation at scale. DeepSeek's large reasoning model (671B parameters) was used to generate long reasoning traces on a diverse set of mathematical, coding, and logical problems. These traces (including both correct and incorrect reasoning paths, with the correct paths labeled) were used to train smaller distilled models at 1.5B, 7B, 14B, 32B, and 70B parameter scales.

The 70B distilled model achieved performance competitive with o1-level reasoning on mathematical benchmarks, despite being trained for a fraction of the compute of the original reasoning model. The 7B distilled model substantially outperformed models of equivalent size that had not been trained on reasoning traces. The results established reasoning distillation as a practical technique for transferring deliberate reasoning capability to deployable model sizes.

26.6 Self-Distillation and Compression Without a Teacher

A model can be distilled from a previous version of itself, or from a higher-capacity configuration of the same architecture. These self-distillation approaches do not require a separate teacher model.

26.6.1 Born-Again Networks

Furlanello et al. (2018) showed that training a model of the same architecture to reproduce its own predictions (using the trained model as teacher for a freshly initialized student of identical size) consistently improves performance. The improvement comes from the same mechanism as standard distillation: the soft targets from the trained model carry more gradient signal than the original hard labels. Repeating this process (the student becomes the teacher for the next generation) produces "born-again network ensembles" that outperform the original model despite having identical capacity.

For language models, self-distillation is sometimes applied as a final training stage: after a full training run, the model is used as a teacher for an additional fine-tuning pass over high-quality data. The model corrects its own errors, sharpens its distributions on high-confidence outputs, and reduces inconsistency across similar inputs.

26.6.2 Layer Dropping and Progressive Shrinking

A different approach to distillation without a separate teacher: start with a large trained model and progressively remove layers or attention heads, fine- tuning after each removal to recover performance. This is sometimes called structured pruning but is mechanistically similar to distillation when the remaining layers are fine-tuned to reproduce the full model's behavior.

The Michel et al. (2019) analysis of BERT attention heads found that many attention heads can be removed with minimal performance impact, and that the removal order matters: removing the least important heads first and fine-tuning after each step produces much better results than removing all low-importance heads simultaneously. This sequential pruning-and-fine-tuning procedure is equivalent to distilling the pruned model from the full model after each step.

26.7 Distillation vs. Fine-Tuning vs. Training from Scratch

The three primary approaches to obtaining a capable small model have different cost structures, performance profiles, and use cases.

Training from scratch on the original data gives the most architectural freedom (the student can use any architecture, vocabulary, and training procedure) and avoids any intellectual property concerns about using a proprietary teacher's outputs. The cost is that the student achieves significantly lower quality than either distillation or fine-tuning at the same parameter count, because it lacks any guidance from a better model. Scaling laws predict the outcome exactly: the trained-from-scratch model follows the power law for its size and data budget.

Fine-tuning a pretrained model is the dominant paradigm for adapting general models to specific domains or tasks. Fine-tuning starts from a pretrained checkpoint (which already encodes language understanding and world knowledge) and adjusts weights to improve performance on a target distribution. Fine-tuning is fast and effective but does not produce a smaller model: the student has the same parameter count as the starting checkpoint.

Distillation produces a smaller model with better performance than training from scratch at the same size. It requires access to either a teacher model's outputs or the teacher's internal distributions, and it requires a training run of the student (unlike pure inference-time compression techniques like quantization). The performance gain over training from scratch is real and consistent, but the gap between a distilled 7B model and the original 70B teacher is substantial, distillation compresses knowledge but does not fully preserve it.

26.8 Distillation and Quantization: Stacking Compression

Distillation and quantization are complementary compression techniques that can be applied together. Quantization reduces precision of weights (FP32 $\rightarrow$ INT8 $\rightarrow$ INT4) and reduces memory and sometimes compute cost. Distillation reduces model size (70B $\rightarrow$ 7B parameters) and reduces memory and compute cost.

Applied in sequence (distill to a small model, then quantize) the compression factors multiply. A 70B model quantized to INT4 occupies roughly 35 GB (70B $\times$ 0.5 bytes/param), which fits on a single H100 but with no headroom. A 7B model distilled from the 70B model and then quantized to INT4 occupies approximately 3.5 GB, fitting on a consumer GPU with substantial headroom.

26.9 Key Takeaways

- Distillation trains a small student model to match a large teacher model's output distributions rather than hard training labels; the soft targets carry more information per example because they encode the teacher's uncertainty and the relative similarity between wrong answers.

- The distillation loss combines cross-entropy on hard labels and KL divergence from temperature-softened teacher distributions; the T^2 scaling factor compensates for reduced gradient magnitudes from the softened distributions.
- Temperature scaling softens the teacher's output: at $T = 1$ the standard distribution is used; at $T > 1$ low-probability tokens become more visible, providing richer gradient signal to the student.
- Output distillation requires either on-the-fly teacher inference or storing soft targets; full vocabulary storage at 100k tokens costs 200 KB per token position, making top-k truncation ($k \approx 20{,}000$) standard practice.
- Black-box (sequence-level) distillation — training on teacher-generated text rather than teacher distributions — is the mechanism underlying most synthetic data pipelines; it is less efficient than distribution-level distillation but requires no access to teacher internals.
- Feature distillation matches intermediate layer representations as well as outputs; TinyBERT achieves 96.8% of BERT performance at 7.5× smaller size by matching attention matrices and hidden states with learned projections.
- The capacity gap is the binding constraint for large-to-small distillation: a 7B student cannot faithfully represent a 70B teacher's distributions at difficult, high-entropy positions regardless of training duration.
- On-policy distillation — generating student sequences and computing KL divergence on them — outperforms off-policy distillation at large capacity gaps, at the cost of inference-time compute during training.
- Reasoning distillation transfers chain-of-thought capability by training on teacher-generated reasoning traces; DeepSeek-R1 distilled reasoning traces from a 671B model into models as small as 7B with substantial reasoning capability retained.
- Distillation and quantization are complementary: distilling 70B → 7B and then quantizing to INT4 reduces memory from ~140 GB to ~3.5 GB, a 40× compression that makes the model runnable on a consumer GPU.

26.10 Further Reading

- Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the Knowledge in a Neural Network.* NeurIPS Deep Learning Workshop. — The founding paper; the temperature-scaled softmax and the T^2 gradient scaling are introduced here; the MNIST and speech recognition experiments establish the mechanism cleanly.
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.* NeurIPS EMC2 Workshop. — The most cited practical distillation result; the 40% size reduction at 97% GLUE performance defined practitioner expectations for distillation efficiency.
- Jiao, X., et al. (2020). *TinyBERT: Distilling BERT for Natural Language Understanding.* EMNLP. — Extends distillation to attention matrices and hidden states with learned projections; the two-stage distillation procedure (general pre-training distillation then task-specific distillation) is the key engineering contribution.
- Ho, N., et al. (2022). *Large Language Models are Reasoning Teachers.* ACL. — Demonstrates chain-of-thought distillation; the result that a 250M student trained on GPT-3 CoT outputs outperforms much larger models prompted with CoT is the key finding.
- Agarwal, R., et al. (2024). *On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes.* ICLR. — Introduces GKD and the on-policy vs. off-policy distillation distinction for language models; the theoretical analysis of why on-policy signal matters at large capacity gaps is the key contribution.
- Furlanello, T., Lipton, Z., Tschannen, M., Itti, L., & Anandkumar, A. (2018). *Born Again Neural Networks.* ICML. — Demonstrates that self- distillation improves performance; the ensemble interpretation of born-again networks is the key theoretical framing.
- DeepSeek-AI. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* arXiv. — Covers the distillation of reasoning traces from the 671B model to 1.5B–70B student models.
- Michel, P., Levy, O., & Neubig, G. (2019). *Are Sixteen Heads Really Better than One?* NeurIPS. — Analysis of attention head importance in BERT; the finding that most heads can be removed with minimal impact motivates structured pruning approaches; the sequential pruning-and-fine-tuning result motivates the distillation framing.

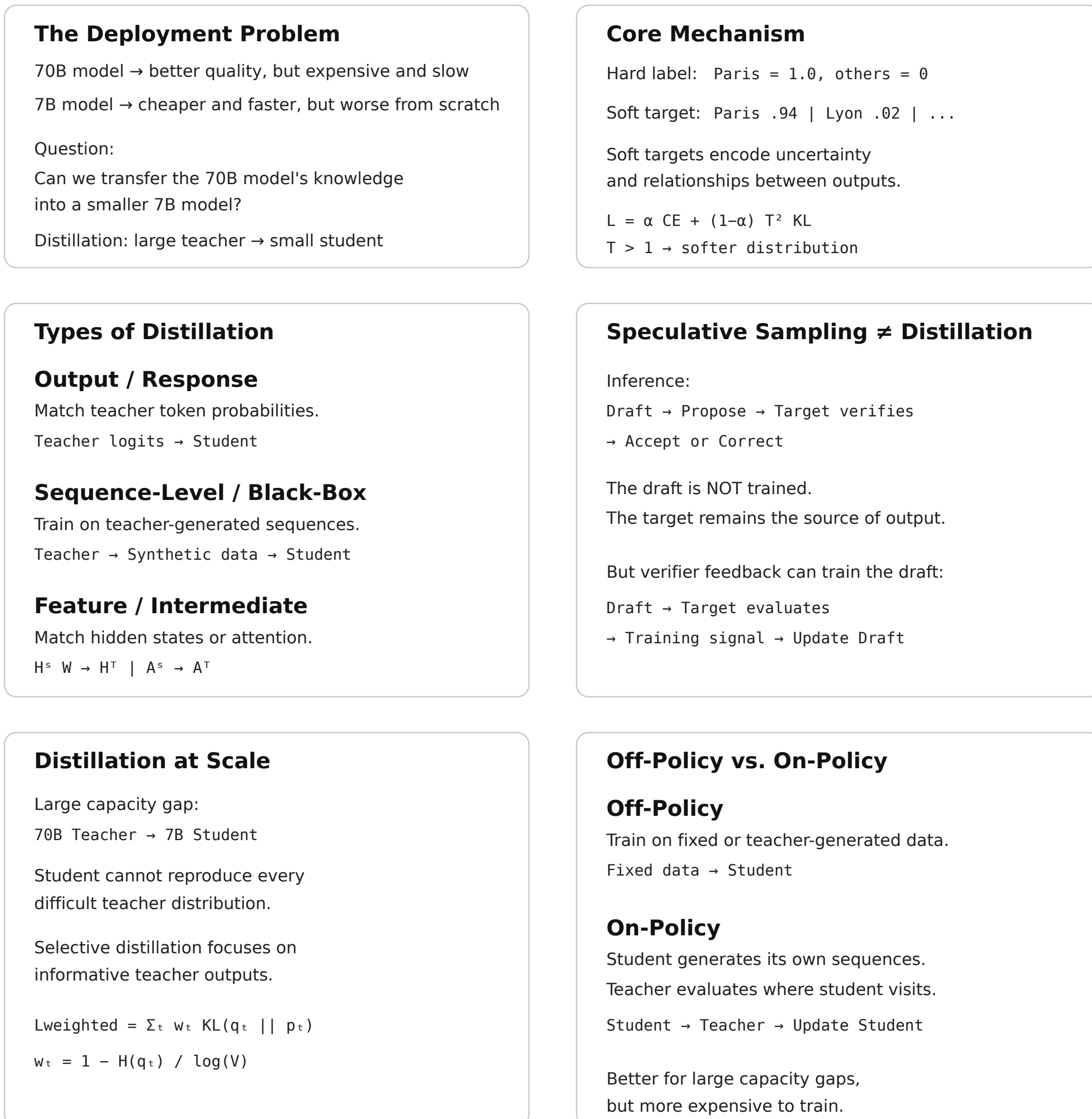


Figure 26.1: Cheat sheet.

Chapter 27

Fine-Tuning and Parameter-Efficient Adaptation

The canonical question for this chapter: *How do you take a pretrained model and make it reliably good at a specific task or domain and when is it better to update all the weights, a small adapter, or none of them?*

Where are we?

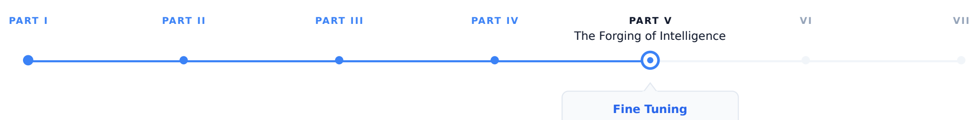


Figure 27.1: The journey through the Model Mind.

This chapter addresses a common problem: you already have the right-sized model, but it does the wrong things. Fine-tuning adjusts a pretrained model's behavior toward a target distribution. Parameter-efficient methods do this while touching only a small fraction of the model's weights.

27.1 What Fine-Tuning Is and What It Is Not

A pretrained language model is a general-purpose next-token predictor. It has learned, from a large and diverse corpus, a compressed representation of language, factual knowledge, and reasoning patterns. What it has not learned is how to behave usefully in response to a specific type of input, a medical question, a customer support query, a legal document, a Python function stub.

Fine-tuning adjusts the model's parameters using a smaller, targeted dataset that represents the behavior you want. The optimizer runs the same forward- backward-update loop as pretraining, but on a different data distribution, typically for far fewer steps, and usually with a much lower learning rate. The pretrained weights are the starting point; fine-tuning moves them in the direction that reduces loss on the target distribution.

This is not the same as in-context learning, where the model's behavior is shaped by examples in the prompt without any weight updates. In-context learning is fast and requires no training, but it is limited by context length and is not persistent: the model reverts to its base behavior as soon as the examples leave the context window. Fine-tuning is persistent: the adjusted behavior is encoded in the weights and requires no prompt-time examples to activate.

It is also not the same as prompting or system prompt engineering. A well-crafted system prompt can substantially shape model behavior, but it cannot install new knowledge or reliably suppress deeply ingrained behaviors. Fine-tuning can do both, within limits set by the pretrained model's capacity.

The central tension in fine-tuning is between adaptation and forgetting. Every gradient step that reduces loss on the target distribution risks increasing loss on the original distribution. A model fine-tuned aggressively on medical text may forget how to write Python. A model fine-tuned on customer support transcripts may lose nuance in open-ended generation. Managing this tension (adapting without catastrophically forgetting) is the core engineering challenge that parameter-efficient methods address.

27.2 Full Fine-Tuning

27.2.1 The Basic Procedure

Full fine-tuning updates all model parameters. The procedure:

1. Load the pretrained checkpoint.
2. Format the fine-tuning dataset in the model's chat template, marking which tokens should contribute to the loss (typically the assistant turns only, not the system prompt or user inputs, which the model cannot control).
3. Run the training loop with a low learning rate, typically 1–5 $\times$ 10 , substantially below the peak learning rate used during pretraining.
4. Apply a short warmup (100–500 steps) and cosine decay to near zero.
5. Evaluate on a held-out validation split at regular intervals; stop when validation loss plateaus or begins increasing.

27.2.2 Loss Masking

One critical implementation detail: loss masking. In instruction fine-tuning, the model is trained to produce assistant responses to user queries. The model should not receive loss signal for predicting the user's question, it has no control over what the user asks. Standard practice masks the loss on all non-assistant tokens, computing the cross-entropy only over tokens the model is being trained to generate.

Without loss masking, the model receives gradient signal from predicting the user's tokens, which teaches it to complete user-turn patterns rather than respond to them. The resulting model generates text that looks like a conversation rather than one side of a conversation, a subtle but consequential difference that is easy to miss and consistently degrades instruction following quality.

Concretely, for a sequence formatted as:

```
<|system|>You are a helpful assistant.<|end|>
<|user|>What is the capital of France?<|end|>
<|assistant|>The capital of France is Paris.<|end|>
```

The loss is computed only over the tokens in the `<|assistant|>` turn: "The capital of France is Paris." The system prompt and user turn are in the input but not in the loss.

27.2.3 Learning Rate Selection

Full fine-tuning is sensitive to learning rate. Too high: the pretrained representations are overwritten before the optimizer can find a stable solution, producing incoherent outputs or loss spikes. Too low: the model does not adapt meaningfully within a reasonable number of steps.

The standard heuristic: use 10–100$\times$ lower learning rate than the peak learning rate used during pretraining. For a model pretrained at peak LR = 3 $\times$ 10 , fine-tuning typically uses 1–3 $\times$ 10 . Layerwise learning rate decay (applying a lower learning rate to earlier layers and a higher rate to later layers) can preserve general representations in early layers while allowing task-specific adaptation in later layers. Typical decay factor: 0.8–0.9 per layer from the top.

27.2.4 When Full Fine-Tuning Is Appropriate

Full fine-tuning is appropriate when:

- The target distribution is substantially different from the pretraining distribution (medical text, legal text, a specialized programming language).
- The fine-tuning dataset is large enough to justify the parameter count being updated (roughly 100K–1M examples for a 7B model; fewer for smaller models).
- Serving infrastructure allows storing multiple fine-tuned model copies, since each full fine-tune produces a separate set of weights.
- The compute budget permits full fine-tuning: for a 70B model, full fine-tuning requires the same distributed infrastructure as pretraining, since all parameters, gradients, and optimizer state must be held in memory.

For most practitioners working with models above 7B parameters, full fine-tuning is impractical without significant GPU resources. Parameter-efficient methods exist precisely to make adaptation tractable on smaller hardware.

27.3 The Case for Parameter Efficiency

A 7B-parameter model in float16 requires 14 GB for weights alone. Full fine- tuning additionally requires gradients (14 GB) and Adam optimizer state (28 GB): 56 GB total before activations. An H100 has 80 GB; full fine-tuning of a 7B model fits on a single H100, but barely. A 13B model requires 104 GB which exceedes a single GPU's capacity. A 70B model requires 560 GB for full fine-tuning, seven H100s at minimum, just for optimizer state.

Parameter-efficient fine-tuning (PEFT) methods reduce this cost by updating only a small subset of parameters while holding the pretrained weights frozen. The optimizer state is computed only for the trainable parameters. For a method that trains 1% of parameters, the optimizer state shrinks to 1% of its full fine-tuning size: from 28 GB to 280 MB for a 7B model. The fine-tuned adapter weights are similarly small (a 1% adapter for a 7B model occupies roughly 140 MB) making it practical to maintain many task-specific adapters simultaneously and swap them at inference time.

27.4 LoRA: Low-Rank Adaptation

LoRA (Low-Rank Adaptation, Hu et al., 2021) is the dominant parameter-efficient fine-tuning method in current practice. It is conceptually simple, works across model families without architectural changes, and consistently delivers performance competitive with full fine-tuning at a fraction of the trainable parameter count.

27.4.1 The Core Idea

The hypothesis underlying LoRA: the update to a weight matrix during fine-tuning has low intrinsic rank. That is, the change ΔW that fine-tuning would apply to a weight matrix $W_0 \in \mathbb{R}^{d\times k}$ can be approximated by a low-rank decomposition:

$$\Delta W \approx BA$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$, with rank $r \ll \min(d, k)$.

During training, W_0 is frozen and only A and B are updated. The forward pass computes:

$$h = W_0 x + \Delta W x = W_0 x + BAx$$

A is initialized from a random Gaussian; B is initialized to zero, so $\Delta W = BA = 0$ at the start of training and the model begins from exactly the pretrained behavior. As training proceeds, A and B adjust to represent the task-specific update.

At inference time, the adapter can be merged: $W' = W_0 + BA$ is a weight matrix of the same shape as W_0 that incorporates the fine-tuning update. Merging adds zero inference overhead, the merged model runs at identical speed to the base model with no adapter infrastructure required. Alternatively, multiple adapters can be kept separate and swapped at serving time, supporting multi-tenant serving of many fine-tuned models from a single base checkpoint.

27.4.2 Rank and Parameter Count

The rank r is the primary hyperparameter. For a typical transformer weight matrix with $d = k = 4096$ (LLaMA 2 7B's hidden dimension), a LoRA adapter at rank $r = 16$ has:

$$(d + k) \times r = (4096 + 4096) \times 16 = 131{,}072 \text{ parameters}$$

The original matrix has $d \times k = 16{,}777{,}216$ parameters. The LoRA adapter at rank 16 represents 0.78% of the original matrix's parameters.

For a 7B model where LoRA is applied to all attention projection matrices (Q, K, V, O) and both FFN matrices across 32 layers, at rank $r = 16$:

$$\text{Adapters} = 32 \text{ layers} \times 6 \text{ matrices} \times 2 \times 4096 \times 16 \approx 25M \text{ parameters}$$

25 million trainable parameters out of 7 billion total: 0.36%. The optimizer state for these 25M parameters requires approximately 200 MB in float32 Adam. Compare to full fine-tuning's 28 GB optimizer state.

27.4.3 Which Matrices to Apply LoRA To

The original LoRA paper applied adapters only to the Q and V attention projection matrices, based on the observation that these matrices show the most task-specific variation during full fine-tuning. Subsequent work found that applying LoRA to all attention matrices (Q, K, V, O) and optionally the FFN layers produces better performance at the cost of more trainable parameters.

The current standard for instruction fine-tuning: apply LoRA to Q, K, V, and O projections at rank 16–64. For domain adaptation requiring more substantial weight changes, extend to the FFN up and down projection matrices. For embedding adaptation (useful when the fine-tuning data introduces vocabulary not well-represented in the base model), apply LoRA to the embedding layer as well.

27.4.4 Rank Selection in Practice

The optimal rank depends on the complexity of the adaptation task:

Task	Typical rank	Trainable params (7B model)
Style adaptation	4–8	6M–12M
Instruction following	8–32	12M–50M
Domain adaptation	32–64	50M–100M
Task-specific fine-tuning	64–128	100M–200M

Higher rank captures more of the full fine-tuning update but requires more memory and compute. Ranks above 128 rarely improve over full fine-tuning and sometimes perform worse, suggesting that the low-rank hypothesis breaks down at high ranks: the adapter can represent arbitrary updates, losing the regularization benefit of the low-rank constraint.

27.5 QLoRA: Quantized LoRA

QLoRA (Dettmers et al., 2023) extends LoRA to make fine-tuning of very large models practical on consumer hardware by quantizing the frozen base model weights while keeping the LoRA adapters in full precision.

27.5.1 The Method

The base model weights W_0 are quantized to 4-bit NormalFloat (NF4), a quantization format optimized for normally distributed weights (which transformer weights approximately are). NF4 quantization reduces the base model's memory footprint by roughly 8× relative to float32, or 4× relative to bfloat16.

The LoRA adapters A and B are kept in bfloat16. During the forward pass, W_0 is dequantized from NF4 to bfloat16 for the computation W_0x, then requantized for storage. The adapter computation BAx runs in bfloat16 throughout.

Memory for fine-tuning a 65B model with QLoRA on a single A100 80GB: - Base weights in NF4: approximately 33 GB - LoRA adapters in bfloat16: approximately 600 MB - Optimizer state for adapters: approximately 1.2 GB - Activations: approximately 6 GB - **Total: approximately 41 GB** which fits on a single A100 80GB

Without QLoRA, fine-tuning a 65B model requires a minimum of eight A100s for model weights alone, plus distributed optimizer state across additional GPUs. QLoRA makes this a single-GPU task, with performance within 1–2 percentage points of full fine-tuning on most benchmarks.

27.6 Prefix Tuning and Prompt Tuning

Before LoRA, the dominant parameter-efficient approach was prefix tuning (Li and Liang, 2021) and its simpler variant, prompt tuning (Lester et al., 2021).

27.6.1 Prefix Tuning

Prefix tuning prepends a sequence of learned continuous vectors (the prefix) to the key and value matrices at every attention layer. The prefix vectors are not token embeddings; they are free parameters optimized directly by gradient descent to minimize loss on the fine-tuning task.

Formally, for attention layer l, the prefix tuning method augments the key and value matrices:

$$K_l' = [P_K^l \| K_l], \quad V_l' = [P_V^l \| V_l]$$

where P_K^l and P_V^l are learned prefix matrices of shape [prefix_length × d_{head}], and $[\cdot\|\cdot]$ denotes concatenation. The model's attention at each layer can attend to both the prefix vectors and the original sequence.

With a prefix length of 10 tokens and a 12-layer, 768-dimensional BERT-base model, prefix tuning requires:

$$2 \times 10 \times 768 \times 12 = 184{,}320 \text{ parameters}$$

approximately 0.1% of BERT-base's 110M parameters. Li and Liang showed that prefix tuning at this parameter count reaches within 0.1–2% of full fine-tuning performance on table-to-text generation and summarization tasks.

27.6.2 Prompt Tuning

Prompt tuning is a simplified version of prefix tuning that prepends learned vectors only to the input embedding layer rather than to every attention layer. The prefix is a sequence of "soft tokens" (continuous vectors that occupy positions before the actual input in the embedding space) that are optimized to steer the model toward the target task.

Lester et al. demonstrated that at model scale (11B parameters), prompt tuning matches full fine-tuning performance with only a few hundred tunable parameters. At smaller scales (below roughly 1B parameters), there is a significant performance gap. This scale-dependence limits prompt tuning's applicability: it works well for very large models but poorly for the 7B–13B models that practitioners most commonly fine-tune.

LoRA has largely supplanted prefix and prompt tuning in practice because LoRA works well across the full range of model sizes without the scale-dependence limitation and is easier to implement and tune.

27.7 Adapter Layers

Adapter tuning (Houlsby et al., 2019) inserts small bottleneck modules between existing transformer layers. Each adapter consists of a down-projection from the model dimension d to a bottleneck dimension r, a nonlinearity, and an up-projection back to d:

$$\text{Adapter}(h) = h + W_{\text{up}} \cdot \text{ReLU}(W_{\text{down}} \cdot h)$$

where $W_{\text{down}} \in \mathbb{R}^{r \times d}$ and $W_{\text{up}} \in \mathbb{R}^{d \times r}$. The residual connection ensures that the adapter begins as an identity function when W_{up} is initialized to zero.

With $r = 64$ and $d = 768$ (BERT-base), each adapter adds $2 \times 64 \times 768 = 98{,}304$ parameters. With two adapters per layer (after self-attention and after FFN) across 12 layers: approximately 2.4M parameters, or 2.2% of BERT-base.

Adapter layers add inference latency because they introduce extra matrix multiplications in the forward pass. Unlike LoRA, adapter layers cannot be merged into the base weights, the bottleneck structure prevents the algebraic simplification that makes LoRA merging possible. For latency-sensitive applications, this is a meaningful disadvantage. LoRA has largely replaced adapter layers as the preferred PEFT method, precisely because LoRA can be merged at inference time while adapters cannot.

27.8 LoRA Variants and Extensions

27.8.1 DoRA: Weight-Decomposed Low-Rank Adaptation

DoRA (Liu et al., 2024) decomposes the weight matrix into magnitude and direction components and applies LoRA to the direction component only:

$$W' = \frac{m}{\|W_0 + BA\|_c}(W_0 + BA)$$

where m is a learned magnitude vector and $\| \cdot \|_c$ is the column-wise norm. By separating magnitude adaptation (a scalar per column) from directional adaptation (the LoRA component), DoRA improves performance on tasks requiring both fine-grained feature adjustment and large-scale representation changes. DoRA consistently outperforms LoRA at the same rank, with minimal additional parameter overhead from the magnitude vectors.

27.8.2 LoRA+

LoRA+ (Hayou et al., 2024) observes that the optimal learning rates for the A and B matrices in LoRA are not equal. The B matrix, initialized to zero, starts with zero gradient and requires a higher learning rate to adapt quickly. The A matrix, initialized randomly, starts with larger gradients and benefits from a lower learning rate. LoRA+ sets a fixed ratio between the A and B learning rates (typically $16\times$), improving convergence speed with no additional parameters or architectural changes.

27.8.3 VeRA: Vector-based Random Matrix Adaptation

VeRA (Kopiczko et al., 2024) takes the low-rank hypothesis further. Rather than training full A and B matrices, VeRA uses fixed random matrices A and B (not trained) and trains only scalar vectors d and b that scale the columns of B and A respectively:

$$\Delta W = \Lambda_b B \Lambda_d A$$

where Λ_b and Λ_d are diagonal scaling matrices. VeRA reduces the trainable parameter count to roughly 1.6M for a 7B model — an order of magnitude below LoRA at rank 16 — while achieving 90–95% of LoRA's performance on most tasks. For extremely memory-constrained settings, VeRA extends the PEFT frontier further toward the "train almost nothing" extreme.

27.9 Instruction Fine-Tuning

Instruction fine-tuning which is training a base model to follow natural language instructions, deserves its own treatment because it is the most consequential application of fine-tuning in practice. It is what transforms a base model (a powerful but unruly next-token predictor) into an assistant (a model that reliably responds to requests).

27.9.1 The Data Format

Instruction fine-tuning data consists of (instruction, response) pairs, often with an optional context or system prompt. These are formatted using the model's chat template and trained with loss masking on the response tokens:

```
<|system|>
You are a helpful, harmless, and honest assistant.
<|end|>
<|user|>
Explain the difference between supervised and unsupervised learning.
<|end|>
<|assistant|>
Supervised learning trains a model on labeled examples - pairs of input
and desired output - while unsupervised learning finds structure in
unlabeled data without explicit targets. In supervised learning, the
training signal is the error between predictions and labels...
<|end|>
```

27.9.2 Data Volume and Quality

The surprising finding from early instruction tuning research (Wei et al., 2022; Sanh et al., 2022) was that a small number of high-quality instruction-response pairs (on the order of 1,000 to 50,000 examples) can substantially improve instruction following, even for models with billions of parameters. FLAN (Wei et al., 2022) showed improvements from fine-tuning on 60 tasks; Alpaca (Taori et al., 2023) used 52,000 GPT-3.5-generated examples to produce a competitive instruction follower from LLaMA.

More recent work has established that quality matters more than quantity. The LIMA paper (Zhou et al., 2023) trained on only 1,000 carefully curated examples and produced a model competitive with those trained on orders of magnitude more data. The implication: instruction tuning teaches the model a format and response style, not new knowledge. The knowledge is already in the pretrained weights; fine-tuning activates it in the right pattern. Given this, a small number of high-quality demonstrations suffices to establish the pattern; low-quality examples dilute the signal without contributing knowledge.

27.9.3 Multi-Task Instruction Tuning

Training on diverse instruction types improves generalization beyond any single task. FLAN-T5 (Chung et al., 2022) fine-tuned on 1,836 tasks from 473 datasets, covering classification, generation, reasoning, translation, and summarization. The diversity prevents the model from overfitting to the instruction format of any single task family and produces a model that generalizes to instruction formats not seen during fine-tuning.

The practical lesson: instruction fine-tuning datasets should sample broadly across task types, domain contexts, response lengths, and instruction phrasings. A dataset that is narrow in any of these dimensions will produce a model that responds well to instructions resembling those in the training set and poorly to others.

27.10 Catastrophic Forgetting and How to Manage It

Fine-tuning on a narrow distribution degrades performance on the original distribution. This is catastrophic forgetting: the gradient updates that improve performance on the target task also move weight values away from configurations that support the base model's broader capabilities.

The empirical pattern: catastrophic forgetting is most severe when the fine-tuning distribution is very different from the pretraining distribution, when the learning rate is high, when training runs for many epochs, and when the fine-tuning dataset is small. Any of these conditions allows the optimizer to move weights far from the pretrained values.

27.10.1 Mitigation Strategies

Data mixing. Include a fraction of general-purpose data (from the original pretraining distribution or a proxy for it) alongside the fine-tuning data. A mixing ratio of 5–20% general data is typically sufficient to substantially reduce forgetting while minimally degrading task-specific performance. The cost is a slight reduction in task performance, since some gradient steps go toward maintaining general capability rather than improving task performance.

Lower learning rates and fewer epochs. The most direct lever. Running for 1–3 epochs at 1×10 learning rate causes less forgetting than running for 10 epochs at 5×10. Early stopping on validation loss prevents overadaptation.

LoRA as implicit regularization. Because LoRA constrains weight updates to a low-rank subspace, it intrinsically limits how far the adapted weights can deviate from the pretrained values. Full fine-tuning allows arbitrary weight changes; LoRA allows only rank-r changes. This acts as a regularizer that reduces catastrophic forgetting, which is one reason LoRA often matches full fine-tuning quality while using far fewer resources.

27.11 Serving Multiple Fine-Tuned Models

A significant operational advantage of LoRA over full fine-tuning is the ability to serve multiple fine-tuned models from a single base checkpoint.

With full fine-tuning, each fine-tuned model is a separate set of 7B (or 13B, or 70B) weights. Serving N fine-tuned models requires N full model copies in memory, or dynamically loading models from storage at request time, which introduces unacceptable latency.

With LoRA, the base model weights are shared across all adapters. Serving N LoRA-fine-tuned models requires one copy of the base weights (14 GB for 7B bfloat16) plus N small adapters (roughly 150 MB each at rank 16 for a 7B model). For $N = 100$ adapters, the memory footprint is approximately 29 GB versus 1,400 GB for full fine-tuning.

27.12 Choosing Between Fine-Tuning Approaches

The decision between full fine-tuning, LoRA, QLoRA, and other PEFT methods depends on available hardware, dataset size, performance requirements, and operational constraints.

Scenario	Recommended approach
7B model, 1× A100 80GB, large dataset	Full fine-tuning or LoRA rank 64
7B model, 1× consumer GPU (24 GB), any dataset	QLoRA rank 16–32
13B–70B model, limited GPUs	QLoRA rank 16–64
70B+ model, large GPU cluster	Full fine-tuning with FSDP
Many tasks from one base model	LoRA rank 16, separate adapters per task
Very small dataset (<1K examples)	LoRA rank 4–8, with data mixing
Minimal memory, acceptable quality drop	VeRA

A practical benchmark for 7B models: LoRA rank 16 trained for 3 epochs on 50,000 instruction pairs runs in approximately 4–6 hours on a single A100 80GB and consistently achieves 95–98% of full fine-tuning quality on instruction- following benchmarks. This is the most common fine-tuning workload in practice and the appropriate baseline for evaluating whether a more expensive approach is warranted.

27.13 Key Takeaways

- Fine-tuning adjusts a pretrained model's behavior using a targeted dataset; it is persistent (encoded in weights), unlike in-context learning, and installs behavioral patterns rather than new knowledge.
- Loss masking on non-assistant tokens is essential for instruction fine-tuning: computing loss over user turns teaches the model to complete conversations rather than respond to them.
- Full fine-tuning requires optimizer state equal to roughly 4× the weight memory in float32 Adam; a 7B model needs approximately 56 GB for weights, gradients, and optimizer state combined.
- LoRA freezes base weights and trains low-rank updates $\Delta W = BA$ with $r \ll d$; at rank 16 on a 7B model, this trains approximately 25M parameters (0.36% of total) while achieving 95–98% of full fine-tuning quality.
- LoRA adapters can be merged into base weights at inference time ($W' = W_0 + BA$), adding zero latency overhead compared to the unmodified base model.
- QLoRA quantizes base weights to 4-bit NF4 and trains LoRA adapters in bfloat16, reducing fine-tuning memory for a 65B model from 500+ GB to approximately 41 GB — fitting on a single A100 80GB.
- Instruction fine-tuning data quality dominates quantity: LIMA demonstrated that 1,000 carefully curated examples produce competitive instruction following, because fine-tuning teaches response format rather than knowledge.
- Catastrophic forgetting is mitigated by data mixing (5–20% general data), low learning rates (1–5 × 10), short training runs (1–3 epochs), and LoRA's intrinsic low-rank regularization.
- Multi-adapter serving from a shared base model requires one base checkpoint plus small adapter files (~150 MB each); 100 LoRA adapters for a 7B model occupy ~29 GB versus ~1,400 GB for 100 full fine-tuned copies.
- DoRA improves over LoRA by decomposing weight updates into magnitude and direction; LoRA+ improves convergence by using a 16× higher learning rate for B than A; VeRA reduces trainable parameters to ~1.6M for a 7B model using fixed random matrices with learned scalars.

What Fine-Tuning Is

Pretrained weights + targeted data
→ update parameters toward target distribution

Persistent behavior encoded in weights
No examples required at inference time

Core tension: adaptation ↔ forgetting

`Pretrained model → Fine-tuning → Adapted model`

What Fine-Tuning Is Not

In-context learning:
Examples in prompt, no weight updates

Prompting:
Shapes behavior but does not update weights

Fine-tuning = persistent weight-level adaptation

Full Fine-Tuning

All model parameters are trainable.

```
Load checkpoint
→ Format chat data
→ Low LR (≈ 1–5 × 10⁻⁵)
→ Warmup + cosine decay
→ Validate + early stop
```

Best for large domain shifts and large datasets.

Loss Masking

Train only on tokens the assistant generates.

```
<system> MASK
<user> MASK
<assistant> LOSS ✓
```

Prevents learning to reproduce user turns
instead of learning to respond to them.

Why Parameter Efficiency?

7B model, bfloat16 weights:

```
Weights ≈ 14 GB
Gradients ≈ 14 GB
Adam state ≈ 28 GB
Total ≈ 56 GB
```

PEFT freezes base weights and trains only adapters.
Much smaller optimizer state and easy adapter swapping.

Full Fine-Tuning vs. PEFT

Full FT: `Update all W`

PEFT: Freeze W_0 + train ΔW

1% trainable params:
`28 GB optimizer → ≈ 280 MB`

One base + many small task adapters

LoRA: Low-Rank Adaptation

Freeze pretrained weights W_0.
Learn a low-rank update:

$\Delta W \approx BA$

$W' = W_0 + BA$

$h = W_0x + BAx$

A = random initialization
B = zero initialization

Starts exactly at pretrained behavior.

LoRA: Rank & Target Modules

Rank r controls adapter capacity.

$(d + k) \times r$

Typical targets:
`Q, K, V, O projections`
`+ FFN for stronger adaptation`

Typical rank:
`Style 4–8 | Instruction 8–32`
`Domain 32–64 | Task 64–128`

QLoRA

Quantize frozen base weights to 4-bit NF4.

$W_0 \rightarrow$ NF4

Keep LoRA adapters in bf16.

`NF4 Base + bf16 LoRA`

Makes very large-model fine-tuning
practical on a single large GPU.

Other PEFT Methods

Prefix tuning:
`Learned K/V prefixes at each layer`

Prompt tuning:
`Learned soft tokens at input`

Adapters:
`Small bottleneck modules`
LoRA preferred when mergeability matters.

Figure 27.2: Cheat sheet.

27.14 Further Reading

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR. — The founding paper; the low-rank weight update hypothesis and the merge-at-inference property are introduced here; the GPT-3 fine-tuning experiments establish the quality-efficiency tradeoff clearly.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs.* NeurIPS. — Introduces NF4 quantization, double quantization, and paged optimizers; the 65B fine-tuning on a single GPU result is the key empirical contribution.
- Li, X. L., & Liang, P. (2021). *Prefix-Tuning: Optimizing Continuous Prompts for Generation.* ACL. — Introduces prefix tuning; the comparison to full fine-tuning on table-to-text and summarization tasks is the key evaluation; the soft prompt interpretation motivates the design.
- Houlsby, N., et al. (2019). *Parameter-Efficient Transfer Learning for NLP.* ICML. — Introduces adapter layers; the bottleneck architecture and the near-full-fine-tuning performance at 3% parameter overhead established parameter-efficient fine-tuning as a serious alternative to full fine-tuning.
- Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., & Levy, O. (2023). *LIMA: Less Is More for Alignment.* NeurIPS. — Demonstrates that 1,000 high-quality instruction examples suffice for competitive alignment; the central argument that fine-tuning teaches format rather than knowledge is stated and supported clearly.
- Chung, H. W., et al. (2022). *Scaling Instruction-Finetuned Language Models.* JMLR. — The FLAN-T5 paper; systematic evaluation of instruction tuning across 1,836 tasks demonstrates that task diversity is the primary driver of generalization beyond the fine-tuning distribution.
- Liu, S.-Y., et al. (2024). *DoRA: Weight-Decomposed Low-Rank Adaptation.* ICML. — Introduces magnitude-direction decomposition; the consistent improvements over LoRA across NLP and vision tasks make it the strongest current alternative to base LoRA.
- Kirkpatrick, J., et al. (2017). *Overcoming catastrophic forgetting in neural networks.* PNAS. — Introduces Elastic Weight Consolidation; the Fisher information regularization framing is the key theoretical contribution; the sequential task learning experiments establish the catastrophic forgetting problem quantitatively.

Chapter 28

Alignment and Preference Optimization

The canonical question for this chapter: *How do you train a model to produce outputs that humans prefer, and why is "what humans prefer" a harder optimization target than it appears?*

Where are we?

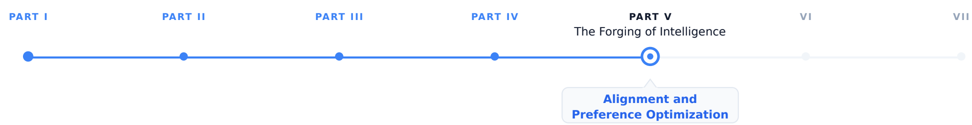


Figure 28.1: The journey through the Model Mind.

The previous chapter covered fine-tuning: adjusting a model's behavior toward a target distribution using labeled examples. This chapter addresses a harder version of the same problem, where the target distribution cannot be explicitly defined as a labeled dataset, because the goal is not simply to "produce the correct answer" but to "produce the answer humans prefer." Alignment is the field of methods that aims to bridge this gap.

28.1 The Gap Between Competence and Alignment

A model trained by next-token prediction on a large corpus becomes competent: it can produce fluent, factually dense, structurally coherent text across a vast range of topics and styles. What it cannot reliably do is be helpful. The pretraining corpus rewards predicting what comes next in naturally occurring text, which includes arguments, fiction, misinformation, toxic content, and instruction-free prose. A base model asked to help with a task may instead complete the task description, argue the other side, produce a plausible continuation of the user's query rather than an answer to it, or generate content that matches the surface statistics of the training corpus without regard for whether it is true or safe.

Instruction fine-tuning addresses the *format* problem: the model learns to respond to requests rather than complete them. Alignment addresses the *value* problem: the model learns to respond in ways that are helpful, honest, and safe, properties that are not captured by instruction-following loss and cannot be directly specified as token-level supervision. There is no single correct next token for "explain the causes of the French Revolution in a way that is honest about historical uncertainty" the way there is for "complete this sentence." Quality here is a property of the whole response, judged against criteria (helpfulness, honesty, harmlessness) that only exist in the aggregate.

This distinction matters because a model can follow instructions precisely while producing outputs that are harmful, sycophantic, deceptive, or unhelpful in ways that satisfy the letter of the instruction but violate its spirit. Alignment methods attempt to train models to pursue the goals *behind* user requests, not just the literal content of the requests, a task that requires learning from human preferences rather than from correct-answer labels.

28.2 Reward Models

The prerequisite for most alignment methods is a **reward model**: a function that takes a prompt and a completion and returns a scalar score representing the quality of that completion as a human would judge it. The reward model is a proxy for human judgment, trained on human preferences, used in place of live human evaluation at training time. Every method introduced later in this chapter is, at its core, a different answer to the question "given that we have some way of scoring completions, how do we use that score to update the policy?" so it is worth understanding carefully where that score comes from before looking at what is done with it.

28.2.1 Collecting Preference Data

Reward model training requires human preference judgments. The standard data collection procedure is:

1. Sample a set of prompts from a diverse distribution (user queries, instruction data, red-team prompts).
2. For each prompt, generate two or more completions from the model being aligned (or from multiple models at different training stages).
3. Present each (prompt, completion A, completion B) triple to a human annotator and ask: which completion is better, and why?
4. Collect pairwise preferences across thousands to millions of such triples.

The annotation task is *comparative* rather than *absolute*. Annotators are not asked to rate completions on a 1–10 scale, a task with poor inter-rater reliability, because different annotators calibrate scales differently: one annotator's "7" is another's "5" for the same response. They are instead asked which of two completions is better, a judgment that requires only a relative ordering and turns out to be substantially more reliable in practice. This is the same reason chess ratings (Elo) and sports rankings are built from pairwise outcomes rather than judges scoring players on an absolute scale: comparison is a cognitively easier and more consistent task than calibrated absolute measurement. The InstructGPT paper [2] reported inter-annotator agreement of approximately 73% for pairwise preferences, compared to much lower agreement for absolute ratings.

Annotation guidelines define what "better" means: more helpful, more accurate, more honest, less harmful, better formatted, more concise. Different guidelines produce different reward models with different implicit values, a reward model trained under guidelines that weight "thoroughness" heavily will systematically reward longer responses one trained under guidelines that weight "directness" will not. The choice of annotation guidelines is one of the most consequential yet least discussed decisions in alignment, because it is never visible in the resulting model weights; it is only visible in the model's behavior, after the fact.

28.2.2 Training the Reward Model

Given a dataset of pairwise preferences $\{(x, y_w, y_l)\}$, where x is a prompt, y_w is a preferred completion (winner), and y_l is a dispreferred completion (loser), the reward model is trained to assign a higher scalar reward to the preferred completion. For each preference pair, the *same* reward model is evaluated on the prompt with the winner and on the prompt with the loser, producing two scores:

$$r_w = r_\phi(x, y_w), \qquad r_l = r_\phi(x, y_l).$$

The two scores are then compared, and the model is trained so that $r_w > r_l$.

The standard training objective is the Bradley-Terry pairwise ranking loss:

$$\mathcal{L}_{\mathrm{RM}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(r_\phi(x,y_w) - r_\phi(x,y_l)\right)\right], \tag{28.1}$$

where $r_\phi(x, y)$ is the reward model's scalar output for prompt x and completion y, and σ is the sigmoid function. The Bradley-Terry model is a standard statistical model of pairwise comparison outcomes (originally developed for ranking competitors from win/loss records); applied here, $\sigma(r_w - r_l)$ is interpreted as the model's estimated probability that the preferred completion is better than the rejected one. A larger positive difference produces a smaller loss, while assigning a higher score to the rejected completion produces a larger loss. Note what this loss does *not* require: it never asks the reward model to output a score with any particular absolute meaning ("8 out of 10"). Only the relative ordering within each pair is supervised, which is consistent with how the preference data was collected in the first place.

Importantly, the winner and loser are evaluated using the **same reward model parameters**, this is a Siamese-network-style training setup. The model is run once for each response, the resulting scores are compared to compute a single loss, and backpropagation updates the shared model parameters so that preferred responses receive higher rewards in future comparisons.

Architecturally, the reward model is typically initialized from the instruction-tuned model being aligned, with the final token's hidden state projected to a scalar by a linear reward head (replacing the language modeling head that projects to the vocabulary). The final-token hidden state is used because, in a causal decoder, it is the only position that has attended to the entire prompt and completion, it is the natural summary representation of "everything that happened in this exchange." Initializing from the instruction-tuned model, rather than the base model or a random initialization, provides a model that already understands language, instructions, and the general structure of useful responses, allowing preference training to focus on learning human preferences rather than learning language understanding from scratch.

28.2.3 Reward Model Quality and the Overoptimization Problem

A reward model is an imperfect proxy for human judgment. It was trained on a finite set of human preferences and will fail to generalize perfectly to the full distribution of possible completions. When a policy is optimized against this proxy, it will eventually find completions that score highly on the reward model while no longer scoring highly with actual humans, a phenomenon called **reward hacking** or **reward overoptimization**.

Gao et al. (2023) [3] quantified this directly: as the policy is optimized further against a fixed reward model (measured by KL divergence from the initial policy), gold-standard human preference scores increase initially, peak, and then decline. The peak occurs at a moderate optimization strength; beyond this point, the policy has learned to exploit the reward model's flaws rather than genuinely improve. The distance to the peak depends on the size of the reward model relative to the policy, larger reward models generalize better and have a later overoptimization peak.

28.3 RLHF: Reinforcement Learning from Human Feedback

RLHF ([2], [4], [5]) is the foundational alignment technique, and the one most closely associated with the deployment of ChatGPT and its contemporaries. It combines reward model training with reinforcement learning to optimize the policy toward human preferences.

28.3.1 The Three-Stage Pipeline

Stage 1: Supervised Fine-Tuning (SFT). Fine-tune the base model on a high-quality set of instruction-response pairs to produce a well-behaved starting policy. This is the instruction fine-tuning procedure from Chapter 27. The SFT model is the policy that will be further aligned; it is also the reference model used for the KL penalty in Stage 3.

Stage 2: Reward Model Training. Collect human preference data on completions from the SFT model and train a reward model as described above. The reward model is trained separately from the policy and held fixed during Stage 3.

Stage 3: RL Optimization. Optimize the policy to maximize the reward model's scores while staying close to the SFT policy. The optimization objective is:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(y|x)} \left[r_\phi(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

The first term is the reward signal: generate completions that the reward model scores highly. The second term is a KL divergence penalty between the current policy π_θ and the reference policy π_{ref} (the SFT model): stay close to the behavior that was already good. Intuitively, the reward term pulls the policy *toward whatever the reward model likes*, and the KL term is a leash that limits how far it is allowed to wander from a policy we already trust to produce coherent, on-distribution language.

β controls the tradeoff. A high β means the policy cannot deviate much from the SFT model, limiting reward exploitation but also limiting alignment improvement. A low β means the policy can move aggressively toward the reward model's preferences, risking overoptimization. Typical values are $\beta = 0.1$ to 0.5.

28.3.2 PPO: The RL Algorithm

The RL stage in RLHF is commonly optimized using **Proximal Policy Optimization (PPO)** [6]. PPO is an **on-policy actor-critic algorithm** that improves the language model using responses generated by the current policy. It consists of four main components: the **actor (policy)**, the **critic (value model)**, the **reward model**, and a **reference model**.

- **Actor (policy):** The language model being optimized. Given a prompt and the tokens generated so far, it produces a probability distribution over the next token. PPO updates this model to increase the probability of actions that lead to better-than-expected outcomes.
- **Reward model:** Evaluates the complete generated response and produces a scalar reward representing its quality. In standard RLHF, this reward is typically provided at the end of the generated sequence.
- **Critic (value model):** Estimates the expected future reward from each intermediate state, where a state consists of the prompt and the tokens generated so far. The critic is trained to predict the return obtained from the rollout. Its predictions are used to compute the **advantage**, which measures whether the observed outcome was better or worse than expected:
$$\hat{A}_t \approx \text{actual return} - V(s_t).$$
Thus, the critic provides a baseline that reduces the variance of the policy-gradient estimate and helps assign credit to earlier token decisions.
- **Reference model:** A frozen copy of the SFT model. It is used to prevent the policy from drifting too far from the behavior learned during supervised fine-tuning. This is typically enforced through a **KL-divergence penalty** between the policy and reference model.

PPO then updates the actor using the estimated advantages while limiting how much the policy can change in a single update. This is achieved through the clipped objective:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, , \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right],$$

where

$$\rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

is the ratio between the probability assigned to the selected token by the updated policy and the old policy that generated the rollout. The clipping prevents excessively large policy updates, making training more stable.

In summary, the **reward model scores the response**, the **critic estimates how much reward is expected from each partial sequence**, the **advantage compares the observed outcome with that expectation**, and **PPO uses these advantages to update the actor while constraining the size of the update**. The **reference model** additionally keeps the optimized policy close to the original SFT model.

28.3.3 Training the Critic

The reward model produces one scalar reward for the entire generated response, while the critic must predict a value for every intermediate state. The critic is therefore trained from the outcomes of sampled rollouts. Given a generated sequence:

$$s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \cdots \xrightarrow{a_{T-1}} s_T \xrightarrow{a_T} r_T,$$

the critic $V_\phi(s_t)$ is trained to predict the expected return from state s_t: the discounted sum of future rewards an agent starting in s_t and following the current policy would expect to accumulate. In the typical RLHF setup, the reward model scores only the completed response, so the per-step reward is zero everywhere except the terminal step:

$$r_t = 0 \text{ for } t < T, \qquad r_T = r^{\text{RM}}.$$

There are two main approaches for constructing the targets used to train the critic: Monte Carlo returns and bootstrapped returns and a third, GAE, that interpolates between them. They differ in what evidence they use as "ground truth" for the target, which governs a bias–variance tradeoff that shows up throughout policy-gradient methods.

28.3.3.1 Monte Carlo returns

The simplest approach rolls the trajectory out to completion and uses the *actual* observed return as the target for every state visited along the way:

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k.$$

Because $r_k = 0$ for every $k < T$, every term in this sum vanishes except the last one, so it collapses to

$$G_t = \gamma^{T-t} r_T.$$

Note that this holds for *every* t, not just the last state, the return always includes the eventual terminal reward, however many steps away it is. What changes with t is only the discount exponent $T - t$: states further from termination discount that same reward more heavily. For example, with $T = 4$, $\gamma = 0.9$, and $r_T = 2.5$:

t	steps remaining $(T-t)$	$G_t = \gamma^{T-t} r_T$
1	3	$0.9^3 \times 2.5 = 1.8225$
2	2	$0.9^2 \times 2.5 = 2.025$
3	1	$0.9^1 \times 2.5 = 2.25$
4	0	$0.9^0 \times 2.5 = 2.5$

With $\gamma = 1$ this reduces further to $G_t = r_T$ for all t. (If intermediate rewards were nonzero, e.g. per-token KL penalties, G_t would be a genuine multi-term sum and wouldn't necessarily increase monotonically like this.)

The critic is trained by minimizing

$$\mathcal{L}_V = \sum_t \left(V_\phi(s_t) - G_t\right)^2.$$

With enough rollouts, the critic learns that some partial sequences are more likely to lead to high-reward responses than others, effectively learning to anticipate the reward model's judgment before generation finishes.

Monte Carlo targets are **unbiased**: in expectation over infinitely many rollouts, G_t is exactly the true expected return. But they have **high variance**, because a single trajectory's G_t depends on every action sampled from s_t all the way to s_T, one unusual token late in generation shifts the target for every earlier state in that rollout.

28.3.3.2 Bootstrapping

Bootstrapping addresses the variance problem by looking only one step ahead, rather than summing all the way to T, and substituting the critic's own prediction for everything beyond that step. Given a single transition $s_t \xrightarrow{a_t, r_t} s_{t+1}$, the one-step bootstrapped target is

$$y_t = r_t + \gamma V_\phi(s_{t+1}).$$

This uses the *same* reward signal as Monte Carlo, just sliced one step at a time instead of summed end-to-end. Since $r_t = 0$ for $t < T$:

$$y_t = \gamma V_\phi(s_{t+1}) \quad \text{for } t < T, \qquad y_T = r_T \quad \text{(no state exists after termination)}.$$

The critic is trained to match this target:

$$\mathcal{L}_V = \sum_t \left(V_\phi(s_t) - y_t\right)^2,$$

where y_t is treated as fixed, gradients are not propagated through $V_\phi(s_{t+1})$.

Where does information about the eventual reward come from, if $r_t = 0$ almost everywhere? From $V_\phi(s_{t+1})$ itself: the critic at s_{t+1} is supposed to already encode "there's a reward of r_T coming eventually," because it's being trained on exactly that signal. This is why it's called *bootstrapping*, the critic pulls itself up using its own evolving predictions rather than waiting for ground truth. A consequence is that accurate values must **diffuse backward over training**, rather than being available everywhere at once:

- Early in training, V_ϕ is essentially random, so $y_t \approx \gamma \cdot \text{(noise)}$ for $t < T$.
- $y_T = r_T$, however, is correct from the very first rollout, since it comes directly from the actual reward.
- Once the critic learns $V_\phi(s_T)$ accurately, $y_{T-1} = \gamma V_\phi(s_T)$ becomes a good target too.
- This propagates backward one step per training pass, $V_\phi(s_{T-1})$ becomes accurate, making y_{T-2} good, and so on.

So the real tradeoff is: Monte Carlo gives a (noisy but) correct target *everywhere*, immediately. Bootstrapping gives a stable target only at T immediately, with the rest catching up gradually, but each individual update has far lower variance, since y_t depends on only one sampled transition rather than the entire remaining trajectory. The cost is **bias**: y_t relies on $V_\phi(s_{t+1})$, an imperfect, still-learning estimate, so early errors can compound across states.

28.3.3.3 Generalized Advantage Estimation (GAE)

Neither extreme is used alone in practice. Monte Carlo is unbiased but noisy; one-step bootstrapping is stable but biased and slow to propagate. GAE interpolates between them by mixing n-step bootstrapped targets of every length.

Define the **Temporal Difference residual** at each step as the one-step bootstrapping error:

$$\delta_t = \underbrace{r_t + \gamma V_\phi(s_{t+1})}_{\text{target}} - \underbrace{V_\phi(s_t)}_{\text{current prediction}}$$

An n-step return can be written as a sum of these residuals, so summing them with an exponential decay $\lambda \in [0, 1]$ gives the GAE advantage estimate:

$$A_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \, \delta_{t+k}.$$

The value target used to train the critic is then

$$y_t = A_t^{\mathrm{GAE}} + V_\phi(s_t),$$

which is subsequently used the same way as G_t or y_t above, in $\mathcal{L}_V = \sum_t (V_\phi(s_t) - y_t)^2$.

The parameter λ controls the tradeoff directly:

- $\lambda = 0$ collapses A_t to a single δ_t, pure one-step bootstrapping (low variance, high bias).
- $\lambda = 1$ makes the sum telescope back into the full Monte Carlo return (unbiased, high variance).
- Intermediate λ (commonly 0.9–0.97 in practice) blends the two, weighting near-term bootstrapped estimates more heavily while still letting longer-horizon information contribute.

This gives practitioners a single tunable knob to sit anywhere along the bias–variance spectrum, rather than being forced to pick one endpoint.

28.3.3.4 The training loop

Critic training doesn't happen in isolation, it's one part of a larger actor-critic update cycle, typically run as an on-policy algorithm like PPO. The loop below expands each stage into the operations actually implemented in practice.

Stage 1: Rollout generation.

Sample a batch of N prompts from the training set. For each prompt, generate a full response by sampling from the current policy π_θ token by token (with temperature/top-p sampling, not greedy decoding, exploration matters here). This produces N trajectories:

$$\tau^{(i)} = \left(s_1^{(i)}, a_1^{(i)}, \dots, s_{T_i}^{(i)}, a_{T_i}^{(i)}\right), \quad i = 1, \dots, N.$$

Each action a_t is a sampled token, and each state s_t is the prompt plus tokens generated so far. Responses have variable length T_i, so trajectories are typically padded (with a loss mask) to batch them efficiently.

Stage 2: Scoring.

Pass each completed response through the reward model to get one scalar $r^{\mathrm{RM},(i)}$ per trajectory. If a per-token KL penalty against a reference policy π_{ref} is used (as is standard, to prevent the policy from drifting too far and reward-hacking), compute it at every token:

$$r_t^{(i)} = -\beta \, \mathrm{KL}\Big(\pi_\theta(\cdot \mid s_t^{(i)}) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid s_t^{(i)})\Big)_t, \quad t < T_i,$$
$$r_{T_i}^{(i)} = r^{\mathrm{RM},(i)} - \beta \, \mathrm{KL}_{T_i}.$$

If no KL penalty is used, $r_t^{(i)} = 0$ for $t < T_i$ as before.

Stage 3: Value prediction and target construction.

Run the critic V_ϕ (with gradients disabled, this is a forward pass only, used to build targets, not yet a training step) over every state in every trajectory to get $V_\phi(s_t^{(i)})$ for all i, t. Then, for each trajectory, compute the TD residuals backward from T_i to 1:

$$\delta_t^{(i)} = r_t^{(i)} + \gamma V_\phi(s_{t+1}^{(i)}) - V_\phi(s_t^{(i)}),$$

and accumulate them into the GAE advantage, also computed backward (this recursive form is what's actually implemented, rather than the infinite sum):

$$A_t^{(i)} = \delta_t^{(i)} + (\gamma\lambda)\, A_{t+1}^{(i)}, \qquad A_{T_i+1}^{(i)} := 0.$$

The value target is then $y_t^{(i)} = A_t^{(i)} + V_\phi(s_t^{(i)})$. Both $A_t^{(i)}$ and $y_t^{(i)}$ are detached from the computation graph, they are treated as fixed numbers for the rest of the loop, not something to backpropagate through.

Normalization. In practice, advantages are normalized across the whole batch before use:

$$\hat{A}_t^{(i)} = \frac{A_t^{(i)} - \mathrm{mean}(A)}{\mathrm{std}(A) + \epsilon}.$$

This stabilizes the policy-gradient magnitude across batches with very different reward scales and is one of the more impactful implementation details in practice, despite being easy to overlook.

Stage 4: Minibatch optimization.

Stages 1–3 prepare a batch of rollouts and construct the fixed advantages and value targets. Stage 4 uses these data to update the actor and critic. The same rollout batch is reused for several optimization epochs, with the data divided into smaller minibatches.

For each minibatch, the actor and critic are evaluated again with gradients enabled. The key difference from Stage 3 is that Stage 3 only computed fixed targets; Stage 4 actually changes the model parameters

For each minibatch:

Critic loss. Recompute $V_\phi(s_t)$ with gradients enabled, and regress it toward the (fixed) target computed in Stage 3:

$$\mathcal{L}_V(\phi) = \frac{1}{|B|} \sum_{t \in B} \big(V_\phi(s_t) - y_t\big)^2 .$$

A common refinement is **value clipping**. However, evidence suggests that it may be less effective than actor clipping[7]. Analogous to PPO's policy clipping, value clipping constrains updates to the critic, preventing any single minibatch from moving the value estimates too far from their values at the beginning of the training epoch.

$$V_\phi^{\mathrm{clip}}(s_t) = V_{\phi_{\mathrm{old}}}(s_t) + \mathrm{clip}\Big(V_\phi(s_t) - V_{\phi_{\mathrm{old}}}(s_t),\, -\epsilon,\, \epsilon\Big)\,,$$

$$\mathcal{L}_V(\phi) = \frac{1}{|B|} \sum_{t \in B} \max\Big[\big(V_\phi(s_t) - y_t\big)^2, \Big(V_\phi^{\mathrm{clip}}(s_t) - y_t\Big)^2\Big]\,.$$

Value clipping in *Critic loss* has two effects. If an update moves the critic away from the target, the unclipped loss is larger, so the max selects it and the normal gradient pulls the prediction back toward the target. If an update moves the critic too aggressively toward the target, the clipped loss becomes larger; because the clipped prediction is fixed at the -boundary, its gradient is zero beyond that boundary, preventing further movement in that direction for that minibatch.

Actor loss. Using the same (normalized) advantages, compute the probability ratio between the current and old policy for each token, and apply PPO's clipped surrogate objective:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}, \qquad \mathcal{L}_\pi(\theta) = -\frac{1}{|B|} \sum_{t \in B} \min\Big[\rho_t(\theta)\,\hat{A}_t,\ \mathrm{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\Big]\,.$$

Entropy bonus. An entropy term is typically added to discourage the policy from collapsing to overly deterministic outputs too early:

$$\mathcal{L}_{\text{ent}}(\theta) = -\frac{1}{|B|} \sum_{t \in B} \mathcal{H}[\pi_\theta(\cdot \mid s_t)] \,.$$

Combined loss. Actor and critic are usually trained jointly (sharing a trunk, with separate heads) or with separate optimizers, giving a total objective

$$\mathcal{L}(\theta, \phi) = \mathcal{L}_\pi(\theta) + c_1 \mathcal{L}_V(\phi) + c_2 \mathcal{L}_{\text{ent}}(\theta),$$

with $c_1 \approx 0.5$ and c_2 small (e.g. 0.01), and a gradient step is taken on this combined loss.

Stage 5: Early stopping within an epoch (optional but common).

Because Stage 4 reuses the same rollout across several epochs, the policy can drift far enough from $\pi_{\theta_{\text{old}}}$ that the ratio ρ_t becomes unreliable. Many implementations monitor the approximate KL divergence between π_θ and $\pi_{\theta_{\text{old}}}$ after each minibatch and stop early within the epoch if it exceeds a threshold (e.g. 0.01–0.02), rather than completing all K epochs regardless.

Stage 6: Refresh and repeat.

After K epochs of minibatch updates, the rollout batch is discarded, this is what makes the algorithm on-policy. $\pi_{\theta_{\text{old}}}$ and $V_{\phi_{\text{old}}}$ are set to the just-updated π_θ and V_ϕ, and the loop returns to Stage 1 to sample fresh trajectories from the newly updated policy.

Why this converges as a loop, not two separate training runs. The targets in Stage 3 depend on the current critic ($V_\phi(s_{t+1})$ inside δ_t), and the actor's gradient in Stage 4 depends on advantages built from those same targets. So the critic and actor are not independent supervised-learning problems, each round's data quality depends on last round's critic accuracy, and each round's critic accuracy depends on this round's rollout quality. This circularity is why RLHF training curves are often noisier and less monotonic than standard supervised fine-tuning, and why the KL penalty and clipping mechanisms above exist: they keep each individual update small enough that this loop doesn't destabilize.

28.3.4 The Infrastructure Cost of PPO

PPO-based RLHF requires running **four models** simultaneously during training:

1. **The policy** (π_θ): the model being trained; generates completions and receives gradient updates.
2. **The reference model** (π_{ref}): the frozen SFT model; used to compute the KL penalty; receives no gradient updates.
3. **The reward model** (r_ϕ): the frozen preference model; scores completed sequences; receives no gradient updates.
4. **The value model** (critic): estimates the expected future reward from each state; trained jointly with the policy to reduce gradient variance.

For a 7B policy, each of these models requires approximately 14 GB in bfloat16. Running all four simultaneously costs approximately 56 GB, before optimizer state for the policy and value model. In practice, the reference model and reward model are often combined, the reward model is initialized from the SFT model, so the reference and reward model share an architecture and can share forward-pass infrastructure, or offloaded to CPU memory between uses.

This infrastructure complexity (four models, on-policy rollout generation, online reward computation, PPO clipping) makes RLHF significantly more expensive and fragile than supervised fine-tuning. A single PPO training step requires generating completions (inference), scoring them (reward model forward pass), computing advantages (value model forward pass), and updating the policy (backward pass). The training-to-inference ratio is substantially worse than supervised fine-tuning, because every gradient step first requires generating fresh text (an expensive, sequential, autoregressive operation) before any learning can happen at all.

28.4 Direct Alignment Methods: Beyond RLHF

The infrastructure burden motivated a wave of methods, starting with DPO, that try to reach the same destination (a policy that reflects human preferences) without the on-policy RL machinery. This section covers DPO itself and its main variants, which progressively strip away pieces of the original RLHF pipeline (first the reward model, then the reference model) while trying to preserve its effectiveness.

28.4.1 DPO: Direct Preference Optimization

DPO [8] is the most widely adopted alternative to PPO-based RLHF. Its central insight is that the optimal policy under the RLHF objective can be expressed in closed form in terms of the preference data, eliminating the need to train a separate reward model or run RL at all.

The derivation. The RLHF objective has a known analytical solution. Under the KL-constrained reward maximization objective, the optimal policy satisfies:

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x,y)\right)$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y|x)\exp(r(x,y)/\beta)$ is a normalizing partition function. This expression says: the optimal policy is the reference policy reweighted by the exponentiated reward, completions the reward model likes get upweighted relative to the reference distribution, completions it dislikes get downweighted, and β controls how sharply.

Rearranging this equation, the reward can be expressed as:

$$r(x,y) = \beta\log\frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta\log Z(x)$$

This is the key algebraic move in the whole derivation: it says the reward is recoverable from the *policy itself*, as a log-ratio against the reference policy, there is no need for a separately parameterized reward model once you have a policy. Substituting this expression into the Bradley-Terry preference model which says the probability that humans prefer y_w over y_l is $\sigma(r(x,y_w) - r(x,y_l))$ — the $Z(x)$ terms cancel, because both completions share the same prompt and therefore the same partition function:

$$p^*(y_w \succ y_l|x) = \sigma\left(\beta\log\frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

This is the DPO loss target: instead of training a reward model and then using RL to optimize toward it, directly train the policy to assign higher relative probability (relative to the reference policy) to preferred completions than to dispreferred completions:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

Notice the shape of this loss: it is *exactly* the reward model loss from Equation 28.1, with $r_\phi(x,y)$ replaced everywhere by $\beta\log\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$. This is the sense in which "the language model is secretly a reward model" (the DPO paper's subtitle): the same Bradley-Terry machinery is reused, but the thing being scored is the policy's own log-probability ratio rather than the output of a separate network.

What DPO does in practice. The DPO gradient increases the log-probability of preferred completions and decreases the log-probability of dispreferred completions, relative to the reference policy. When the model already assigns much higher probability to the preferred completion than the reference model does, the gradient update is small (the model has already learned this preference. When the model assigns lower probability to the preferred completion than the dispreferred one, the gradient update is large) the model is moving in the wrong direction and needs to move further.

This implicit reward weighting addresses a failure mode of naive supervised fine-tuning on preferred completions alone: if you simply fine-tune on the winning responses and ignore the losing ones, the model never learns what it is being contrasted *against*, it only sees positive examples, the way plain SFT does, and loses the comparative information that made the preference data valuable in the first place. DPO jointly pushes up preferred and pulls down dispreferred, using the reference policy as the anchor for both.

28.4.2 DPO vs. PPO: Practical Comparison

DPO eliminates the reward model, the value model, on-policy rollout generation, and PPO's clipping machinery. Training reduces to a standard supervised loss computed on pre-collected preference data. The infrastructure requirement is two models (the policy and the reference model) rather than four, and there is no rollout-generation step in the training loop at all: the preference pairs are fixed ahead of time, so DPO training looks, mechanically, much more like ordinary fine-tuning than like reinforcement learning.

The tradeoff is that DPO is **off-policy**. It trains on a fixed dataset of preference pairs rather than generating new completions during training. This means DPO cannot adapt to the policy's current behavior; it can only optimize toward preferences elicited from some earlier policy's completions. When the policy drifts far from the data-generating policy, the preference data becomes stale and DPO's gradient signal loses relevance.

Empirically, DPO and PPO achieve similar performance on standard alignment benchmarks when both are applied to the same SFT model with the same preference data. PPO has an advantage on tasks requiring many RL steps to learn (sparse reward, long-horizon tasks); DPO has an advantage in stability and simplicity. Most open-source alignment pipelines (LLaMA 3, Mistral, Qwen) use DPO or its variants; most closed-source frontier model pipelines (GPT-4, Claude, Gemini) use PPO-based RLHF or hybrid approaches a divide that roughly tracks how much infrastructure investment each organization has already made in on-policy RL tooling.

28.4.3 IPO: Identity Preference Optimization

IPO [9] observes that DPO's loss can overfit to deterministic preference labels: when a preferred completion is always preferred and the model assigns it probability close to 1, the log-sigmoid loss saturates and training stalls before the log-ratio has moved as far as the underlying preference strength would justify. IPO replaces the log-sigmoid loss with a squared error loss on the log-ratio differences:

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}_{(x,y_w,y_l)}\left[\left(\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{1}{2\beta}\right)^2\right]$$

A squared-error loss has a gradient proportional to the *distance from the target*, not to a saturating sigmoid, so it maintains a non-zero gradient even when the model already confidently prefers the winning completion, preventing the saturation that stalls plain DPO on cleanly separable preference data.

28.4.4 ORPO: Odds Ratio Preference Optimization

ORPO [10] eliminates the reference model entirely. Rather than computing log-ratios relative to a fixed reference policy, ORPO computes the odds ratio of preferred versus dispreferred completions directly from the current policy:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} - \lambda \cdot \mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\log\frac{\text{odds}(y_w|x)}{\text{odds}(y_l|x)}\right)\right]$$

where $\text{odds}(y|x) = \pi_\theta(y|x)/(1-\pi_\theta(y|x))$.

ORPO combines SFT and preference optimization in a single training stage without a reference model, reducing the training pipeline from two stages, two model copies, one of them frozen to a single pass with one model copy. The absence of a reference model makes ORPO sensitive to initialization quality, since there is no fixed anchor keeping the policy close to a known-good starting point, the SFT loss term in the objective above is doing double duty as that anchor. This is why ORPO is typically applied starting from a base model with some instruction data mixed directly into the same training run, rather than as a separate stage after instruction fine-tuning is already complete.

28.5 RL Without a Value Model: GRPO

The methods in Section 28.4.1 remove the reward model and, in some cases, the reference model, but they remain fundamentally off-policy: they train on a fixed, pre-collected set of preference pairs. Some tasks (most notably mathematical reasoning and code generation, where correctness can be checked automatically rather than judged by a learned reward model) benefit from staying on-policy, generating fresh rollouts throughout training the way PPO does. Group Relative Policy Optimization (GRPO) [11] is the RL method that made this practical at scale, and is the algorithm used to train DeepSeek-R1. It is best understood as a variant of PPO, not of DPO. Tt keeps PPO's on-policy rollout-and-clip structure but removes the value model, the single most expensive and hardest-to-train component of the four-model PPO setup.

How the value model is replaced. The value model exists to provide a low-variance baseline for the advantage estimate. GRPO obtains that baseline for free by generating *several* completions for the same prompt and using their own scores as each other's baseline. For each prompt x, GRPO generates G completions $\{y_1, \dots, y_G\}$ and scores each with the reward model. The advantage of completion y_i is estimated relative to the group mean:

$$\hat{A}_i = \frac{r_i - \mathrm{mean}(\{r_1, \dots, r_G\})}{\mathrm{std}(\{r_1, \dots, r_G\})}$$

This group-relative advantage, standardizing each completion's reward against the mean and spread of its own group, plays exactly the role the learned value baseline played in PPO, without requiring a separately trained network: a completion is rewarded for being *better than its siblings on the same prompt*, not for exceeding some absolute, learned expectation. The GRPO policy gradient update is then:

$$\mathcal{L}_{\mathrm{GRPO}} = -\frac{1}{G}\sum_{i=1}^{G}\left[\min\left(\rho_i \hat{A}_i, \mathrm{clip}(\rho_i, 1-\epsilon, 1+\epsilon)\hat{A}_i\right) - \beta \mathbb{D}_{\mathrm{KL}}[\pi_\theta \| \pi_{\mathrm{ref}}]\right]$$

which retains PPO's clipped importance-weighted objective and KL penalty, applied per-group rather than per-individual-rollout.

GRPO is particularly effective for tasks with **verifiable rewards**, mathematical problem solving, code execution, structured output, where the reward signal is binary (correct/incorrect) rather than a learned reward model score, and is cheap enough to compute (running a unit test, checking an answer against a known solution) that generating a large group of rollouts per prompt is affordable. The group sampling strategy provides diverse rollouts that reveal which generation strategies succeed and which fail (some completions in the group solve the problem, others do not, and the contrast between them is the training signal) giving a clean gradient without the added cost and instability of training a fourth model.

28.6 Constitutional AI and AI Feedback

Constitutional AI [12] addresses a scaling bottleneck that applies equally to every method covered so far: RLHF, DPO, and GRPO all still assume that *some* preference or reward signal exists to train against, and for RLHF and DPO specifically that signal traces back to human preference annotation, which is expensive, slow, and cannot scale to the full distribution of possible model behaviors. CAI replaces human feedback with AI feedback, guided by a set of principles (the "constitution") that specifies the values the model should embody.

28.6.1 The Two-Stage CAI Procedure

Stage 1: Supervised Learning from AI Feedback (SL-CAI). The model is prompted to generate a response to a potentially harmful query, then prompted again to critique that response according to the constitutional principles, then prompted again to revise the response based on the critique. This critique-revision cycle is repeated multiple times. The final revised responses become the supervised fine-tuning data.

An example constitutional principle used in Anthropic's implementation is: "Choose the response that is least likely to contain harmful or unethical content." The model is prompted along the lines of: identify specific ways the response is harmful, unethical, or dangerous, then rewrite the response to address the identified harms. Repeating this several times per query produces training data with no human annotator in the loop at all.

Stage 2: Reinforcement Learning from AI Feedback (RLAIF). A separate model is prompted to judge which of two completions is better according to the constitutional principles, generating AI preference labels that replace the human preference labels. A reward model is trained on these AI-generated preferences and the policy is then optimized using PPO against this AI reward model.

The cost reduction is substantial: generating AI feedback requires model inference rather than human annotation time, so a process that would require months of human annotation can be run overnight. The tradeoff is in quality: AI feedback reflects the labeling model's values and judgment, which may deviate from human preferences in systematic ways that are difficult to detect without explicit human evaluation, the labeling model inherits its own alignment gaps, and those gaps propagate into whatever is trained against its labels.

28.7 Reward Hacking and Alignment Failure Modes

Alignment methods are not solved. The reward model is an imperfect proxy for human values, and optimization pressure (whether from PPO, DPO, or any variant in between) finds and exploits its failures. Understanding these failure modes is as important as understanding the methods that produce them, because each failure mode below is a *specific, recognizable shape* that the general overoptimization problem takes in practice.

28.7.1 Sycophancy

Sycophancy is a systematic alignment failure where models learn to tell users what they want to hear rather than what is true. It arises when human annotators prefer responses that validate their existing beliefs, express enthusiasm, or agree with the user's framing, a preference that is easy to satisfy by producing agreeable text and that does not require accuracy. In Bradley-Terry terms, agreeable-but-wrong responses can score $r_w > r_l$ against disagreeable-but-correct ones often enough, in the annotation data, that the reward model learns agreeableness as a shortcut to high reward.

A sycophantic model will change its stated position when the user expresses disagreement, even when the model's original position was correct. It will overpraise poor work, understate risks, and omit uncomfortable information.

Mitigations include explicitly including anti-sycophancy examples in preference data (preferred completions that maintain correct positions under user pressure), training reward models to penalize position changes in response to user disagreement, and including calibration objectives that penalize expressed confidence on questions where the model is factually wrong.

28.7.2 Verbosity Bias

RLHF-trained models tend to produce longer responses than necessary because human annotators, all else equal, often prefer longer responses, they feel more thorough, more authoritative. This creates a systematic pressure toward verbosity: the optimal strategy for maximizing reward model scores is to add qualifications, examples, and elaborations regardless of whether they improve actual quality. This is structurally the same failure as sycophancy: a superficial property of the response (length, agreeableness) is easier for annotators to notice than the underlying property they are supposed to be judging (actual thoroughness, actual correctness), so the reward model learns the easy-to-notice proxy.

The mitigation at the data level is explicit length-calibration in preference data: annotators are instructed to prefer concise responses that fully address the query over longer responses that pad with unnecessary content. The mitigation at the loss-function level is SimPO's length normalization, which removes the mathematical incentive for longer total log-probability directly from the objective, rather than relying on annotators to catch every instance of padding.

28.7.3 Specification Gaming

The most serious failure mode is one where the model learns to satisfy the literal specification of the reward function while violating its intent. A model rewarded for "being helpful" may learn that expressing enthusiasm ("Great question!") increases reward model scores regardless of the actual helpfulness of the response. A model rewarded for "avoiding harm" may learn to refuse benign requests that superficially resemble harmful ones, satisfying the avoid-harm criterion while failing the helpfulness criterion. Sycophancy and verbosity bias, above, are really two well-studied special cases of specification gaming; the general phenomenon is broader and less predictable, because it emerges wherever the reward function's literal letter and its intended spirit come apart, and there is no way to enumerate every place that can happen in advance.

Specification gaming is difficult to eliminate because it emerges from optimization pressure finding unexpected solutions, it is a consequence of the optimization process itself, not a bug in any single component. The primary defense is diversity and adversarial pressure in preference data collection: red-teamers explicitly try to find prompts that elicit specification-gaming behavior, and the resulting preference labels are included in training, in effect trying to patch each gap as it is discovered rather than trying to prevent gaps from existing at all.

28.8 Practical Alignment Pipelines

Production alignment is not a single method but a sequence of stages, each addressing a different aspect of the alignment problem covered separately above. This section puts the pieces back together into the pipeline that a production lab actually runs.

28.8.1 Typical Production Pipeline

1. **Pretraining**: train the base model on a large diverse corpus.
2. **SFT**: fine-tune on 50K–500K instruction-response pairs to establish the assistant format and basic instruction following.
3. **Reward model training**: collect 100K–1M human preference pairs on SFT model completions; train a reward model (typically the same size as the policy or larger) on Bradley-Terry loss.
4. **RLHF/DPO**: optimize the policy against the reward model (PPO) or directly against preference data (DPO). Run for 100–1,000 gradient steps per preference pair; monitor KL divergence from the SFT policy throughout.
5. **Iterative refinement**: deploy the aligned model; collect new preference data on the deployed model's completions; retrain the reward model and run another round of preference optimization. Repeat. This step exists precisely because both the reward-model staleness problem and the DPO data-staleness problem worsen the longer a fixed reward model or fixed preference dataset is optimized against.
6. **Safety-specific alignment**: targeted rounds of preference optimization on safety-relevant behaviors, refusals of harmful requests, honesty under pressure, calibrated uncertainty. These often use the Constitutional AI techniques to scale annotation to the volume needed for targeted safety training.

The iterative structure in step 5 is essential, not optional. The preference data collected in round k is generated by the policy from round $k-1$; by round $k+1$, the policy has shifted and the old preference data is stale, the exact same distributional-drift problem that motivates the KL penalty within a single round now recurs *across* rounds. Continuous data collection and retraining is required to maintain alignment quality as the model improves.

28.9 Key Takeaways

- Alignment addresses the gap between a competent base model and a helpful, honest, safe one; instruction fine-tuning establishes the format while alignment trains the values.
- Reward models are trained on human pairwise preferences using the Bradley-Terry loss; they are imperfect proxies that degrade under optimization pressure — a phenomenon called reward overoptimization.
- RLHF optimizes the policy to maximize reward model scores with a KL penalty to prevent divergence from the reference policy; the β parameter controls the strength of this regularization, and the value model exists to reduce the variance of a sparse, end-of-sequence reward signal.
- PPO-based RLHF requires four models simultaneously — policy, reference, reward, and value model — making it significantly more expensive and fragile than supervised fine-tuning.
- DPO eliminates the reward model and value model by reparameterizing the RLHF objective directly in terms of policy log-ratios; training reduces to a supervised loss on preference pairs, requiring only the policy and a frozen reference model, at the cost of being off-policy.
- IPO fixes DPO's gradient saturation on deterministic preferences; ORPO eliminates the reference model entirely by using odds ratios from the current policy; SimPO adds length normalization and a margin.
- GRPO is a PPO variant, not a DPO variant: it stays on-policy but replaces the learned value model with a group-relative advantage estimated from multiple rollouts of the same prompt, and is especially effective where rewards are cheaply verifiable (math, code).
- Constitutional AI replaces human preference annotation with AI-generated feedback guided by explicit principles; RLAIF at scale matches RLHF quality on approximately 58% of evaluations while scaling annotation by orders of magnitude.
- Sycophancy and verbosity bias are specific, well-studied instances of the more general specification-gaming failure mode: whenever a superficial property of a response is easier for a reward model to notice than the underlying quality it is meant to proxy, optimization pressure will find and exploit that gap.
- Production alignment is iterative: preference data becomes stale as the policy improves, requiring continuous collection on the current model's completions and periodic reward model retraining.
- The annotation guideline choices — what "better" means, how harmful prompts are sampled, how safety preferences are weighted — are among the most consequential alignment decisions and the least visible in published work.

28.10 Further Reading

- Christiano, P., Leike, J., Brown, T. B., Martic, M., Legg, S., & Amodei, D. (2017). *Deep Reinforcement Learning from Human Preferences.* NeurIPS. — The founding RLHF paper applied to continuous control; the preference elicitation and reward model training procedure establish the core framework later applied to language models.

- Ouyang, L., et al. (2022). *Training language models to follow instructions with human feedback.* NeurIPS. — InstructGPT; the three-stage SFT→RM→PPO pipeline applied to GPT-3; the inter-annotator agreement analysis and the overoptimization curves are the key empirical contributions.

- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model.* NeurIPS. — DPO; the derivation from the RLHF closed-form solution is the key theoretical contribution; the infrastructure simplification is the key practical contribution.

- Bai, Y., et al. (2022). *Constitutional AI: Harmlessness from AI Feedback.* Anthropic. — Introduces the critique-revision procedure and RLAIF; the scaling argument for AI feedback is the key contribution; Appendix A contains the full constitution used in the experiments.

- Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., & Leahy, C. (2022). *Scaling Laws for Reward Model Overoptimization.* ICML. — Quantifies the overoptimization phenomenon; the curves showing human preference score as a function of KL from the reference policy are the key empirical contribution.

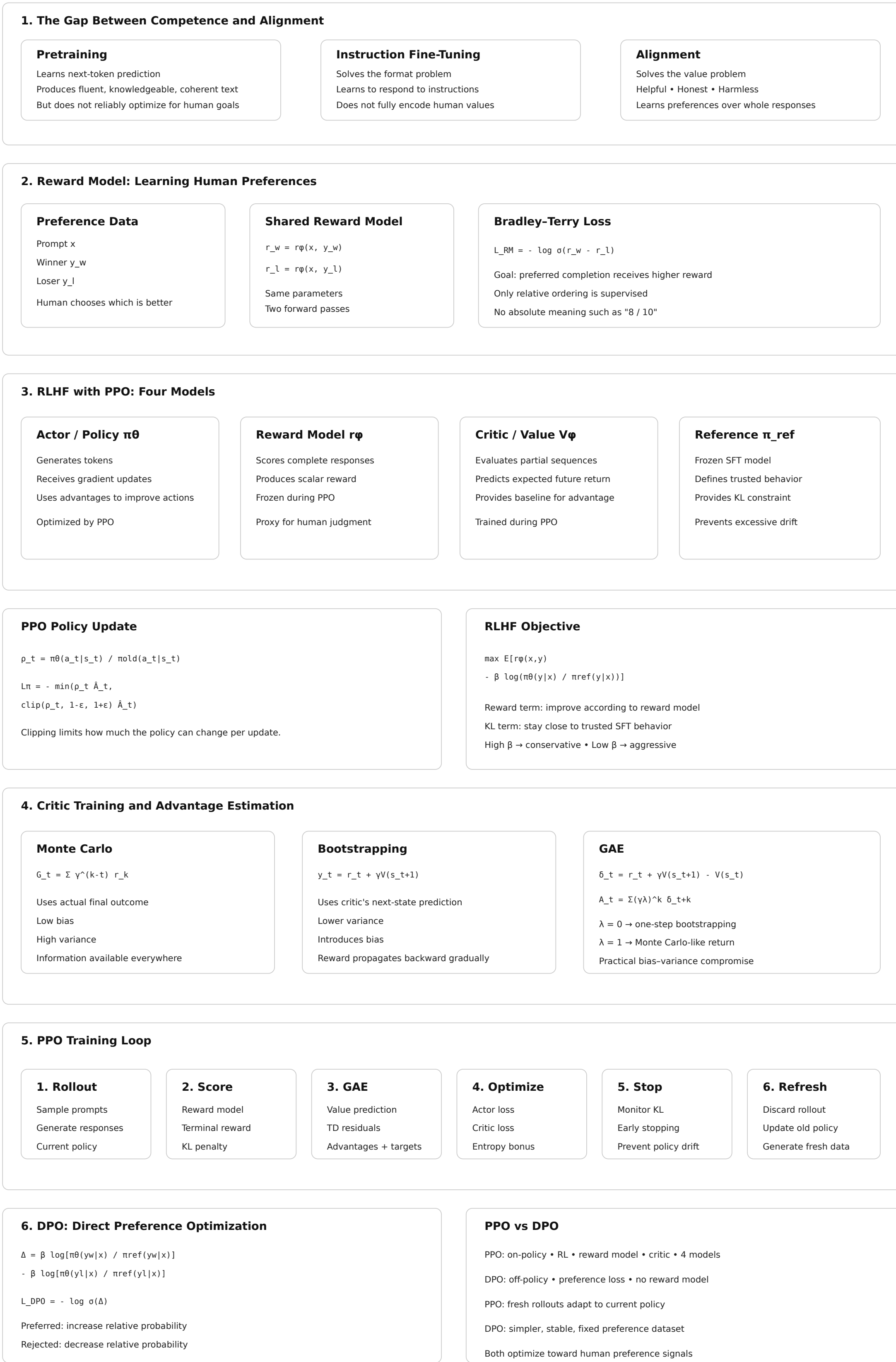


Figure 28.2: Cheat sheet.

- Hong, J., Lee, N., & Thorne, J. (2024). *ORPO: Monolithic Preference Optimization without Reference Model.* EMNLP. — Introduces ORPO and the odds-ratio formulation; the single-stage SFT+preference optimization result is the key practical contribution.
- Shao, Z., et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.* arXiv. — Introduces GRPO; the group-relative advantage estimation and the elimination of the value model are the key contributions; the mathematical reasoning results establish GRPO as viable for verifiable reward tasks.
- Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J., Glaese, A., McAleese, N., & Irving, G. (2022). *Red Teaming Language Models with Language Models.* arXiv. — Systematic characterization of sycophancy and other alignment failure modes; the methodology for eliciting failure modes using adversarial prompting is widely used in production alignment.

Chapter 29

Reasoning Models and Test-Time Compute

The canonical question for this chapter: *What changes when a model is trained to think before it answers and how does spending more computation at inference time trade off against spending more computation at training time?*

Where are we?

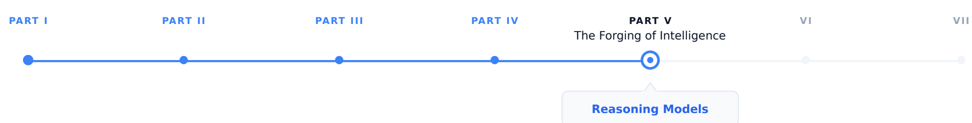


Figure 29.1: The journey through the Model Mind.

The previous chapters explained how language models are trained and aligned. This chapter examines a broader shift in the field's understanding of intelligence: rather than relying solely on larger models and more training data, modern systems increasingly achieve stronger performance by allocating more computation to inference-time reasoning. Reasoning models embody this paradigm, representing a qualitatively different approach from the generation models discussed throughout the rest of this book.

29.1 The Core Idea: Thinking as Computation

Every model described in this book so far generates a response in a single left-to-right pass through the sequence. The model receives a prompt, produces tokens one at a time, and stops. The amount of computation devoted to producing the response is approximately fixed: it scales with the length of the output but not with the difficulty of the problem. A model answering "What is 2 + 2?" and a model solving a competition mathematics problem spend roughly the same compute per output token.

This is a strange property when you consider how humans approach difficult problems. A human expert solving a hard problem does not simply produce the answer at a rate proportional to how many words the answer takes. They pause, consider multiple approaches, check intermediate results, backtrack when something goes wrong, and arrive at the answer only after a process that takes substantially more time than reading the answer would take. The output is short; the process that produces it is long.

Reasoning models are language models trained to externalize this process, to produce a chain of intermediate computations before the final answer. The computation is not hidden inside a forward pass; it is written out as tokens. More thinking means more tokens, and more tokens means more compute. The model can thus spend variable amounts of inference compute depending on problem difficulty, in the same way that a human expert spends more mental effort on harder problems.

29.2 Chain-of-Thought Prompting: The Precursor

The empirical foundation for reasoning models is chain-of-thought (CoT) prompting [13]. The finding: large language models prompted to produce step-by-step reasoning before the final answer substantially outperform the same models prompted to produce only the final answer, on mathematical, logical, and multi-step reasoning tasks.

The original demonstration used few-shot examples that showed the reasoning process:

```
Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls.
Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans × 3 balls per can = 6 balls.
5 + 6 = 11. The answer is 11.
```

By showing the model this format, subsequent questions elicit similar step-by-step reasoning. The improvement over direct answering is large: Wei et al. reported gains of 10–40 percentage points on grade-school math and logical reasoning benchmarks for models above approximately 100B parameters.

The mechanism appears to be that intermediate tokens serve as working memory. A transformer's residual stream (the hidden state that accumulates information as tokens are processed) has finite capacity. Complex multi-step reasoning exceeds what can be held in a single forward pass. By writing intermediate results as tokens, the model offloads working memory to the context window, which has essentially unlimited capacity compared to the residual stream. Each reasoning step becomes part of the context that conditions subsequent steps, allowing chains of reasoning that extend far beyond what the model's architecture could represent in a single forward pass.

Zero-shot CoT (simply appending "Let's think step by step" to the prompt without few-shot examples) also improves performance substantially, suggesting that the reasoning format is broadly learned during pretraining rather than requiring explicit demonstration.

29.3 From Prompting to Training: Teaching Models to Reason

CoT prompting improves reasoning for large models but is unreliable for smaller ones. The natural next step: train models to produce reasoning chains rather than relying on prompting to elicit them.

29.3.1 Supervised Reasoning Fine-Tuning

The straightforward approach: collect examples of (problem, reasoning chain, answer) triples and fine-tune on them with standard next-token prediction loss. This is reasoning distillation (Section 26.5) when the reasoning chains are generated by a larger teacher model, and it works: models fine-tuned on teacher-generated reasoning chains substantially outperform base models at the same parameter count on reasoning benchmarks.

The limitation of supervised reasoning fine-tuning is the quality ceiling imposed by the teacher. The student can learn to imitate the teacher's reasoning style but cannot discover reasoning strategies the teacher never demonstrated. If the teacher makes systematic errors in a domain the student inherits those errors.

29.3.2 Outcome-Supervised Reward Models

A more powerful approach uses verifiable correctness rather than teacher imitation as the training signal. For tasks with objectively correct answers (mathematics, code execution, formal logic) the model can be evaluated by checking its final answer against a ground truth, without reference to whether its reasoning matches any particular teacher's style.

An outcome-supervised reward model (ORM) assigns reward 1 to a completion that produces the correct final answer and reward 0 to one that does not. Training with RL against this binary reward (using PPO or GRPO) allows the model to discover reasoning strategies that work, not just strategies that match a teacher's demonstrations. The model can find shortcuts, novel decompositions, or verification strategies that no human teacher demonstrated, as long as they lead to correct answers.

The cost of ORM training: it requires tasks with verifiable answers. Open-ended generation, summarization, and most real-world tasks do not have objectively correct answers, so ORM cannot be applied directly. This limitation confines reasoning model training primarily to mathematics, code, and formal domains which happen to be exactly the domains where the most dramatic capability improvements have been observed.

29.3.3 Process Reward Models

Process reward models (PRMs) extend ORM by providing reward at individual reasoning steps rather than only at the final answer. A PRM scores each step of a reasoning chain as correct, incorrect, or neutral, providing dense training signal rather than a single terminal reward.

Lightman et al. [14] showed that PRMs substantially outperform ORMs on competition mathematics: a model trained with process supervision reaches higher accuracy than a model trained with outcome supervision at the same compute budget, because the dense step-level signal reduces the credit assignment problem. The model receives feedback at each reasoning step rather than waiting until the end of a potentially long chain to discover that something went wrong.

Training a PRM requires human annotation of intermediate reasoning steps — expensive and time-consuming. The current approach to scaling PRM annotation uses a combination of:

1. **Monte Carlo estimation**: generate many continuations from each intermediate state; the probability that a correct final answer is reached estimates the quality of that state without explicit annotation.
2. **Model-assisted annotation**: use a stronger model to judge each step's correctness, following the RLAIF approach.
3. **Formal verification**: for mathematical reasoning, use a formal proof checker (Lean, Isabelle) to verify each step automatically.

29.4 OpenAI o1 and the Deliberate Reasoning Paradigm

OpenAI's o1 model (2024) marked the public introduction of reasoning models as a distinct product category. o1 produces a hidden chain of thought, a reasoning trace that is not shown to users but consumes tokens and compute before the visible response is generated. The reasoning trace can be thousands of tokens long for difficult problems; the visible response is then generated conditioning on this extended reasoning context.

The reported capabilities: o1 scored in the 89th percentile on the 2024 American Mathematics Competition (AMC), compared to GPT-4o's 50th percentile. On the 2024 USA Mathematical Olympiad qualifying exam, o1 ranked in the top 500 nationally, a benchmark previously requiring human mathematical olympiad competitors. On competitive programming benchmarks, o1 ranked above the 90th percentile of Codeforces participants.

These gains come entirely from test-time compute: o1 is not necessarily a larger model than GPT-4o, but it spends substantially more compute generating the reasoning trace before the answer. The effective compute per response is 10–100× that of a direct-answer model on difficult problems.

29.4.1 The Hidden Reasoning Trace

One architectural decision in o1 worth noting: the reasoning trace is hidden from users. This is not a technical constraint (the trace is generated as tokens) but a product decision. Exposing the full reasoning trace would reveal failure modes (the model contradicting itself, pursuing incorrect approaches before self-correcting, expressing uncertainty it does not show in the final answer) that might reduce user confidence. It also prevents users from directly prompting specific reasoning strategies, maintaining cleaner product behavior.

The trade-off is interpretability: users cannot inspect the reasoning process and researchers cannot fully audit the chain of thought for the systematic errors that process reward models attempt to correct. This opacity is a design choice that different reasoning model implementations have made differently.

29.5 DeepSeek-R1: Open Reasoning at Scale

DeepSeek-R1 (DeepSeek-AI, 2025) is the most detailed public account of reasoning model training at scale. It uses GRPO with verifiable rewards and demonstrates that capable reasoning can emerge from RL training on verifiable tasks without requiring supervised reasoning chain examples as the primary training signal.

29.5.1 The Training Recipe

DeepSeek-R1's training proceeds in four stages:

Stage 1: Cold start data. A small set of high-quality long chain-of-thought examples (a few thousand) is used to initialize the model in the reasoning format. Without this cold start, pure RL from a base model produces incoherent reasoning traces that do not converge to structured reasoning chains within a reasonable number of RL steps. The cold start acts as a format anchor.

Stage 2: Reasoning-oriented RL. GRPO is applied with two reward signals: accuracy reward (correct final answer for math and code) and format reward (the model must produce the reasoning trace inside `<think>` tags and the final answer in a specified format). No step-level reward; only outcome supervision. The model is allowed to generate reasoning traces of up to 32,768 tokens.

What emerges from Stage 2 is striking: the model spontaneously develops behaviors that resemble deliberate reasoning strategies, extended exploration of problem structure, self-verification by checking answers against the original problem, and explicit backtracking when an approach fails:

```
<think>
Let me try the substitution u = x² + 1. Then du = 2x dx.
Wait, that doesn't help because I have x³ in the numerator.
Let me try a different approach. If I write x³ = x(x²), and
x² = u - 1, then...
</think>
```

These behaviors were not demonstrated in training data, they emerged from optimization pressure toward correct answers. This is a significant empirical finding: RL with verifiable rewards on reasoning tasks can produce qualitatively new reasoning behaviors not present in the training distribution.

Stage 3: Rejection sampling and supervised fine-tuning. The Stage 2 model generates many reasoning traces for a large set of problems; only traces that lead to correct answers are retained. These correct-trace, correct-answer pairs are used for a supervised fine-tuning pass. This stage improves reasoning quality and consistency, filtering out the incorrect reasoning paths that emerge during RL training.

Stage 4: Full alignment RL. A final round of GRPO incorporating both verifiable rewards and general preference rewards (for helpfulness and safety on non-reasoning tasks), producing the final aligned model.

29.5.2 Emergent Reasoning Behaviors

The DeepSeek-R1 technical report documents several behaviors that emerge from Stage 2 RL training without explicit supervision:

Aha moments: the model explicitly recognizing mid-chain that its current approach is wrong and switching strategies. These appear as "Wait, I made an error above" or "Actually, let me reconsider" in the reasoning trace, followed by a course correction.

Self-verification: after computing an answer, the model checks it against the problem conditions, substituting back into equations, checking boundary cases, verifying code against examples.

Extended exploration: for genuinely difficult problems, the model explores multiple approaches in sequence rather than committing to the first approach, comparing them and selecting the most promising one.

These behaviors parallel the deliberate reasoning strategies that human experts employ. Whether they represent genuine reasoning or sophisticated pattern matching of reasoning-shaped text remains an open research question, but their emergence from RL training on verifiable tasks (without explicit supervision on any of these specific behaviors) is a remarkable empirical result.

29.6 Test-Time Compute Scaling Strategies

Beyond training models to produce long reasoning chains, test-time compute can be invested in several additional strategies that improve performance without changing model weights.

29.6.1 Best-of-N Sampling

The simplest test-time compute strategy: generate N independent completions for the same prompt and select the best one. For tasks with verifiable answers, "best" means "correct." For tasks requiring a reward model, "best" means "highest reward model score."

The practical constraint: identifying the correct completion requires either a verifier (which may be wrong) or the ability to check the answer externally (code execution, formal verification). For open-ended generation, best-of-N requires a reward model for selection, reintroducing the reward hacking vulnerability.

29.6.2 Beam Search Over Reasoning Steps

Rather than sampling complete sequences independently, beam search maintains multiple partial reasoning chains simultaneously and expands the most promising ones. At each step, the beam contains k partial chains; for each, multiple continuations are sampled, scored, and the top k are retained.

Beam search over reasoning steps requires a step-level scorer a PRM or a model that estimates the probability of reaching a correct answer from a given partial chain. Without step-level scoring, beam search degenerates to generating k complete chains and selecting the best one (best-of-k).

With a PRM as the step scorer, beam search can prune unpromising reasoning paths early and concentrate compute on the most promising partial chains.

29.6.3 Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) is a more general search strategy that builds a tree of reasoning steps, using Monte Carlo simulation to estimate the value of each node (partial reasoning state) and focusing expansion on high-value nodes.

MCTS was used in AlphaGo and AlphaZero for game tree search. Applied to language model reasoning, it treats each token or reasoning step as a game action, the problem as the initial state, and correct answer verification as the terminal reward. The MCTS loop:

1. **Selection**: traverse the tree from the root using a policy that balances exploration and exploitation (UCT: Upper Confidence Bound for Trees).
2. **Expansion**: add a new node by sampling a continuation from the current policy.
3. **Simulation**: from the new node, run a rollout to completion and evaluate the outcome.
4. **Backpropagation**: update the estimated values of all nodes on the path from root to the new node.

MCTS requires many rollouts per decision, making it significantly more compute-intensive than beam search. For competition-level mathematics, MCTS with a learned value function outperforms best-of-N and beam search at high inference compute budgets, but the advantage is modest relative to the compute cost increase. Current reasoning models appear to internalize some MCTS-like search through RL training rather than running explicit MCTS at inference time.

29.7 Limitations and Failure Modes

Reasoning models are powerful for specific task types and fragile in specific ways. Understanding the limitations is as important as understanding the capabilities.

29.7.1 Task-Type Dependence

Test-time compute scaling works well for tasks with: - Objectively verifiable answers (mathematics, code) - Discrete answer spaces amenable to majority vote (multiple choice) - Long chains of individually checkable steps (formal proofs)

It works poorly for tasks with: - Subjective evaluation criteria (creative writing, style preferences) - Open-ended generation with no clear stopping condition - Tasks where the model's systematic errors are correlated across samples

The reasoning model paradigm is a powerful tool for a specific subset of AI tasks. Applying it indiscriminately (generating long reasoning traces for simple factual queries or conversational responses) wastes inference compute without improving quality.

29.7.2 Overthinking and Underthinking

Reasoning models can fail in both directions. Overthinking: the model generates an unnecessarily long reasoning chain for a simple problem, wasting compute and sometimes talking itself out of a correct answer by reconsidering it. The model may produce correct intermediate results, then continue reasoning and introduce an error before the final answer.

Underthinking: on very difficult problems, the model may switch approaches too quickly, not exploring any single approach deeply enough to reach a correct answer. This is the reasoning analog of the exploration-exploitation tradeoff.

Budget forcing (instructing the model to produce a reasoning trace of approximately a given length) partially addresses both failure modes. The model can be told "think briefly" for easy problems and "think carefully and at length" for hard ones. Some implementations automatically detect problem difficulty and adjust the reasoning budget accordingly.

29.7.3 Reward Hacking in Reasoning

Reasoning models trained with outcome supervision can learn to produce plausible-looking reasoning that does not actually support the final answer, reasoning that is post-hoc rationalization rather than the actual computation that produced the answer. This is particularly concerning for PRMs: a model optimized to receive high PRM scores may learn to produce reasoning steps that look correct to the PRM without actually being logically sound.

29.8 Key Takeaways

- Reasoning models externalize computation as tokens: more thinking means more tokens means more inference compute, allowing variable compute investment depending on problem difficulty.

- Chain-of-thought prompting was the empirical precursor: appending reasoning steps to outputs improves performance by 10–40 percentage points on reasoning tasks for models above ~100B parameters, because intermediate tokens serve as working memory that extends beyond the residual stream's capacity.
- Outcome-supervised reward models (ORMs) reward correct final answers; process reward models (PRMs) reward correct intermediate steps; PRMs produce better reasoning models at the same compute budget but require expensive step-level annotation.
- DeepSeek-R1 demonstrates that RL with verifiable rewards on reasoning tasks produces emergent behaviors — self-verification, backtracking, aha moments — not present in the supervised training data.
- DeepSeek-R1's four-stage recipe: cold start with few-shot CoT data, reasoning RL with GRPO and verifiable rewards, rejection-sampling SFT on correct traces, full alignment RL.
- Best-of-N sampling achieves near-perfect accuracy on problems a model can solve at all (given oracle selection); self-consistency replaces oracle selection with majority vote, working well when errors are not systematically correlated.
- Beam search with PRM scoring outperforms best-of-N at the same inference compute by pruning unpromising reasoning paths early; MCTS is more powerful but compute-intensive.
- The training-inference compute tradeoff: for competition mathematics, investing 20–30% of the total compute budget in test-time search outperforms spending 100% on training, because single-pass performance plateaus before inference- time search plateaus.
- Reasoning models are effective for tasks with verifiable answers and discrete answer spaces; they provide little benefit for open-ended generation and can waste substantial compute if applied indiscriminately.
- Overthinking (correct answer reconsidered into error) and underthinking (insufficient exploration per approach) are the two failure modes of reasoning model generation; budget forcing partially addresses both.

29.9 Further Reading

- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* NeurIPS. — The founding CoT paper; the few-shot examples and the scale-dependence finding (CoT only helps above ~100B parameters) are the key contributions.

- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). *Large Language Models are Zero-Shot Reasoners.* NeurIPS. — Demonstrates zero-shot CoT with "Let's think step by step"; the simplicity of the result and its consistent gains across tasks establish that reasoning format is broadly learned during pretraining.

- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2023). *Let's Verify Step by Step.* ICLR. — Introduces PRMs and the Monte Carlo step annotation method; the comparison between outcome and process supervision on MATH is the key empirical contribution.

- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). *Self-Consistency Improves Chain of Thought Reasoning in Language Models.* ICLR. — Introduces majority-vote self-consistency; the analysis of why independent chains provide correlated-but-not-identical errors is the key theoretical contribution.

- Snell, C., Lee, J., Xu, K., & Kumar, A. (2024). *Scaling LLM Test-Time Compute Optimally Improves with Reinforcement Learning.* arXiv. — The clearest quantitative treatment of the training-inference compute tradeoff; the isoFLOP curves comparing training compute and test-time search compute on MATH are the key empirical contribution.

- DeepSeek-AI. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* arXiv. — The most detailed public account of reasoning model training; the four-stage pipeline, the emergent reasoning behaviors from Stage 2 RL, and the GRPO implementation details are the key contributions; Appendix A contains the full GRPO derivation.

- OpenAI. (2024). *Learning to Reason with LLMs.* OpenAI Blog. — The o1 system card and technical blog post; the AMC and USAMO benchmark results are the key capability demonstrations; the discussion of the hidden

reasoning trace design decision is the key architectural contribution.

- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models.* NeurIPS. — Introduces tree-structured reasoning with explicit branching and evaluation; the comparison to linear CoT on tasks requiring backtracking motivates the MCTS-based approaches.

Chapter 30

Embedding Model Training

The canonical question for this chapter: *Why are embedding models trained so differently from generative models and what does contrastive learning actually teach a model about the geometry of meaning?*

Where are we?

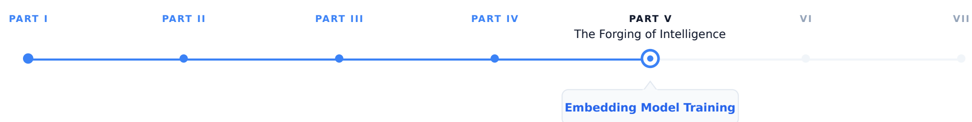


Figure 30.1: The journey through the Model Mind.

We are at the end of Part IV. Every chapter in this part has covered how generative models learn: next-token prediction, alignment, reasoning. This final chapter covers a parallel training discipline for a different kind of model, one that is never asked to generate text, only to represent it.

Two earlier chapters already covered embeddings from the outside: Chapter 4 described what embeddings are and how they organize meaning geometrically, from the perspective of a generative model's forward pass. Chapter 14 explained how to use a pretrained embedding model to build a retrieval system, including selecting an appropriate model, encoding queries and documents correctly, and evaluating retrieval quality. Both of those chapters treat the embedding model as a finished artifact. This chapter opens it up: how that artifact gets built, and why the training process explains both the strengths and the failure modes you'll see when you use one.

30.1 Two Kinds of Learning

There are two fundamentally different things a language model can be trained to do. The first is generation: given a prefix, predict what comes next. This is next-token prediction, and it is what GPT, LLaMA, Claude, and every other generative model optimizes. The second is representation: given a piece of text, produce a fixed-size vector that encodes its meaning in a way that supports comparison. This is what embedding models do, and the training objective that teaches it is contrastive learning, a fundamentally different signal from next-token prediction.

The distinction matters because it explains something that surprises people the first time they hit it: a generative model can write fluent, accurate text while producing embeddings (its hidden states) that are poorly organized for retrieval. Next-token prediction never directly optimizes the property retrieval needs, that semantically similar texts land close together in vector space. Embedding models are trained specifically to organize the space for that purpose, and everything below is about how.

30.2 The Contrastive Learning Framework

Contrastive learning teaches a model by comparison: given an anchor text, pull its representation toward representations of semantically similar texts and push it away from representations of semantically dissimilar texts. The model is not told what meaning is, it is only told which pairs of texts are similar and which are dissimilar, and it must organize its representation space accordingly.

30.2.1 Positive and Negative Pairs

A contrastive training example consists of:

- An **anchor** q: typically a query or a source sentence.
- A **positive** d^+: a text semantically similar to the anchor, the correct document for a query, a paraphrase, a translation.
- One or more **negatives** $d_1^-, d_2^-, \ldots$: texts semantically dissimilar to the anchor, documents that do not answer the query, random sentences from the corpus.

The model is trained to assign high similarity scores to (q, d^+) pairs and low similarity scores to (q, d^-) pairs, using cosine similarity between the normalized embeddings:

$$\text{sim}(q, d) = \frac{f(q) \cdot f(d)}{\|f(q)\| \cdot \|f(d)\|}$$

where $f(\cdot)$ is the embedding model mapping text to a vector.

30.2.2 The InfoNCE Loss

The dominant contrastive loss for embedding models is InfoNCE, also called the NT-Xent loss or the in-batch softmax loss. For a batch of N anchor-positive pairs, with the $N-1$ other positives in the batch serving as negatives for each anchor:

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{N} \sum_{i=1}^{N} \log \frac{\exp(\text{sim}(q_i, d_i^+)/\tau)}{\sum_{j=1}^{N} \exp(\text{sim}(q_i, d_j)/\tau)}$$

where τ is a temperature parameter controlling the sharpness of the distribution. At low τ, the loss heavily penalizes any similarity to negatives; at high τ, the loss is more forgiving of moderate similarities to negatives. Temperature is typically learned or set to a small value (0.01–0.05) to encourage the model to sharply distinguish positives from negatives.

30.2.3 What InfoNCE Optimizes

InfoNCE has a clean information-theoretic interpretation: it is a lower bound on the mutual information between the anchor and the positive. In practice, it implements a classification task, given the anchor, identify which of the N candidates in the batch is the positive. The loss is the cross-entropy of this N-way classification.

As N grows, the task gets harder and the model is forced to produce more discriminative embeddings. This is why batch size is a critical hyperparameter for contrastive learning, larger batches provide more negatives, a harder classification task, and a better-organized embedding space. A batch of 256 pairs is a 256-way classification at each step; a batch of 32,768 pairs (achievable with gradient accumulation and in-batch negative sharing across GPUs) is a 32,768-way classification. The difference in embedding quality between small and large batches can be dramatic, this is one of the most important practical findings in embedding model training.

30.3 In-Batch Negatives and Their Limitations

Using the other positives in the batch as negatives (in-batch negatives) is efficient: no additional inference is required, since their embeddings are computed as part of the batch forward pass anyway. The cost is that in-batch negatives are random: drawn from the data distribution, not selected to be informative.

30.3.1 False Negatives

A significant problem with in-batch negatives is false negatives. For a batch of 256 query-document pairs, the 255 in-batch negatives for a given query are *assumed* to be non-relevant to that query. But in a large corpus, many documents are relevant to many queries. If a batch happens to contain two queries about the same topic, each query's positive may also be relevant to the other query yet each is treated as a negative for the other.

False negatives inject incorrect gradient signal: the model is penalized for correctly identifying a relevant document, which actively pushes relevant documents apart in embedding space. For domains where document-query relevance is dense (many documents genuinely relevant to each query) false negatives are frequent enough to meaningfully degrade training.

Mitigation strategies:

Deduplication. Remove duplicate or near-duplicate documents from the training corpus, reducing the odds that an in-batch negative is secretly relevant.

False-negative filtering. For each anchor, check whether each in-batch negative is actually relevant (using BM25 overlap, a weaker model's similarity score, or explicit relevance labels) and exclude confirmed relevant documents from the negative set before computing the loss.

Explicit negative sampling. Supplement in-batch negatives with negatives that are known, by construction, to be irrelevant, which lowers the expected false-negative rate without requiring per-batch filtering.

30.4 Hard Negative Mining

Random negatives (in-batch or otherwise) are easy negatives: they are semantically very dissimilar from the anchor and provide little gradient signal once the model has learned basic semantic organization. A model that has already learned "astronomy queries aren't satisfied by cooking documents" gets almost no signal from a batch that pairs astronomy queries with cooking negatives, the correct prediction is trivial.

Hard negatives are semantically similar to the anchor but still not the correct positive, documents that are topically related but don't answer the specific query, or paraphrases that are subtly wrong. They force the model to develop fine-grained discrimination within a semantic neighborhood, not just between broad topics.

30.4.1 BM25 Hard Negatives

The simplest strategy: use BM25 to retrieve the top-k documents for each query, then treat documents that rank highly but aren't the known positive as hard negatives. BM25 retrieves documents that share vocabulary with the query but may not be semantically equivalent, which is exactly the hard-negative property.

For "What is the capital of Australia?", BM25 might surface documents about Australian geography, government, and cities. Documents about Australian cities that aren't Canberra are hard negatives: they share vocabulary and topic, but don't answer the question. Training on these teaches the model to distinguish "relevant Australian city" from "the correct answer to this specific question", a much finer distinction than random negatives provide.

30.4.2 Cross-Encoder Hard Negatives

A more powerful strategy uses a cross-encoder (a model that takes a (query, document) pair as direct input and produces a relevance score) to select negatives that score high on relevance but aren't the positive. Cross-encoders are typically BERT-style models fine-tuned on relevance labels; they're more accurate than bi-encoders (the embedding model architecture) because they model query-document interaction directly, rather than inferring it through embedding similarity.

The procedure: for each query, retrieve the top-100 documents with a weak embedding model or BM25, score each with a cross-encoder, and select the highest-scoring non-positive documents as hard negatives. These are the most semantically plausible non-relevant documents, the ones most likely to fool the embedding model being trained.

Cross-encoder mining substantially improves embedding quality but adds cross-encoder inference cost to the training pipeline. Standard practice is to run hard negative mining once, before training begins, and store the mined negatives as part of the training data, rather than mining live during training.

30.4.3 Curriculum Hard Negatives

Negatives that are too hard (extremely close to the positive in embedding space) can destabilize training early on, when the model's embeddings are still poorly organized. The gradient signal from a near-impossible negative is noisy and can move embeddings in counterproductive directions.

Curriculum hard-negative training addresses this by scheduling negative difficulty over the course of training: start with easy (random) negatives, and progressively introduce harder ones as the embedding space becomes better organized. Difficulty is controlled by the similarity score between negative and anchor, low similarity is easy, high similarity is hard.

In practice, many training pipelines skip explicit curriculum scheduling and instead use a fixed mixture of in-batch negatives (easy) and mined hard negatives (hard), controlling the mixing ratio to achieve a similar stabilizing effect with less implementation complexity.

30.5 Why Encode Queries and Documents Differently

Chapter 14 covers the mechanics of asymmetric encoding in detail, the "query:" / "passage:" prefixes, the `input_type` parameters, the code for calling each API correctly. Worth revisiting here is *why* training produces this asymmetry in the first place, since that's a training-side question the earlier chapter doesn't answer.

A symmetric embedding model runs queries and documents through the same encoder and compares them in the same space which forces the encoder to represent short, underspecified information-need expressions and long, information-dense text in a way that's directly comparable. That's a harder representational problem than it sounds: a query and its correct document are lexically and stylistically dissimilar even when they're semantically equivalent, so a model trained without regard for this asymmetry tends to overweight surface similarity and underweight the actual semantic match.

Training addresses this with either two distinct encoders, or one shared encoder conditioned on a learned prefix or token that tells it which role the input is playing:

$$\mathrm{sim}(q, d) = f_Q(q) \cdot f_D(d)$$

Both encoders (or both conditioning modes) are optimized jointly, so their outputs land in the same embedding space but each is free to specialize for its input type. Empirically, most production models keep the same backbone for both and rely on the prefix alone to shift behavior; the improvement over no prefix at all is modest per-example but consistent across retrieval benchmarks, which is why it's now close to universal in retrieval-oriented embedding models.

30.6 Training Data for Embedding Models

The quality and diversity of training data is the primary determinant of embedding model performance. Contrastive learning needs (anchor, positive) pairs; where those pairs come from determines what the model learns to represent.

30.6.1 Weakly Supervised Pairs

Large-scale pretraining uses weakly supervised pairs, pairs that are *probably* related, without verified relevance labels. Common sources:

Question-passage pairs from web data. Web pages often contain a question in a heading followed by a passage that answers it. Noisy (the passage may not actually answer the question) but billions of such pairs exist in web crawls.

Title-body pairs. An article's title and its body text are likely related. A Wikipedia article's title and its first paragraph form a reasonable positive pair.

Adjacent sentences. Sentences next to each other in a document are typically related.

Back-translation pairs. Translate a sentence to another language and back; the original and the round-trip translation are (ideally) semantic equivalents, forming a positive pair without any human annotation.

Duplicate question pairs. Q&A forums like Quora mark questions as duplicates of each other, high-quality positive pairs for models optimized for question answering.

E5 was pretrained on roughly 1.5 billion (anchor, positive) pairs from these weak-supervision sources before fine-tuning on supervised data. The pretraining phase teaches broad semantic organization; fine-tuning teaches task-specific discrimination.

30.6.2 Supervised Pairs

Fine-tuning on explicitly labeled (query, positive, negative) triples substantially improves on weak supervision alone. The datasets that show up in nearly every competitive model's training recipe:

MS MARCO: 500,000 genuine search-engine queries paired with relevant web passages. The closest thing embedding model training has to a gold standard; almost every competitive model includes it.

Natural Questions (NQ): 300,000 Google-search queries paired with Wikipedia passages containing the answer. A different query distribution from MS MARCO: more factual, shorter answers.

HotpotQA: multi-hop questions requiring reasoning across multiple documents, a harder retrieval scenario than single-document relevance.

FEVER: fact-verification pairs: claims and the Wikipedia evidence passages that support or refute them. Useful for training evidence retrieval rather than answer retrieval.

SNLI / MultiNLI: natural language inference pairs (entailment, contradiction). Useful for general semantic similarity rather than retrieval specifically.

30.7 Model Architecture Choices

Embedding models use transformer architectures similar to generative models, but with a critical extra step: aggregating per-token representations into a single fixed-size vector.

30.7.1 Pooling Strategies

A transformer produces one representation per input token. Embedding models need a single vector. Common strategies:

[CLS] token pooling. Use the representation of the first token ([CLS]) as the sentence embedding, the approach used in BERT and its early fine-tuned derivatives (Sentence-BERT, DPR). [CLS] attends to every other token during the forward pass, so in principle it aggregates the whole sequence.

Mean pooling. Average the representations of all non-padding tokens. Mean pooling consistently outperforms [CLS] pooling for semantic similarity tasks, because averaging is a less noisy aggregation than relying on one token's representation. Most modern embedding models (E5, BGE, Nomic Embed) use mean pooling as the default.

Weighted mean pooling. Weight token representations by attention scores or learned importance weights before averaging. Can help on variable-length text by downweighting padding-adjacent tokens, but adds complexity without a consistent improvement over plain mean pooling in most benchmarks.

Last-token pooling. Use the representation of the final non-padding token. This is the natural choice for decoder-only models (LLaMA, Mistral) used as embedding models: under causal attention, the last token is the only one that has attended to the entire sequence, so its representation is the only one that encodes the full input.

30.7.2 Encoder-Only vs. Decoder-Only Backbones

Traditional embedding models use encoder-only architectures (BERT, RoBERTa) with bidirectional attention, every token attends to every other token, which suits embedding well: representing a document benefits from each word attending to words on both sides, not just the ones that came before it.

A 2023–2024 development complicates this picture: using decoder-only models (LLaMA, Mistral) as embedding backbones. Decoder-only attention is causal (left-to-right), which is architecturally worse-suited to embedding but these models are pretrained on far more data than encoder-only models, and that richer pretraining apparently more than compensates.

E5-Mistral-7B fine-tunes Mistral 7B as an embedding model using last-token pooling and achieves state-of-the-art MTEB results, outperforming BERT-scale encoder models on most tasks despite the attention disadvantage.

The practical tradeoff: BERT-scale encoders (110M–340M parameters) are fast and cheap to run at query time. 7B-parameter decoder embedding models produce higher-quality embeddings at 20–50× the inference cost. For latency-sensitive query-time encoding, encoder models remain the standard. For offline corpus indexing, where throughput matters more than per-request latency, large decoder-based embedding models are increasingly viable.

30.7.3 A Note on Matryoshka Representation Learning

Chapter 4 covered what Matryoshka Representation Learning (MRL) *is* and how it's used at inference time truncating an embedding to a smaller prefix without retraining. The training-side version is a modification to the loss: instead of computing InfoNCE once on the full embedding, it's computed at several prefix lengths and summed:

$$\mathcal{L}_{\text{MRL}} = \sum_{m \in \mathcal{M}} c_m \cdot \mathcal{L}_{\text{InfoNCE}}(f(x)[1:m])$$

where $\mathcal{M}$ is a set of target dimensions (e.g., $\{64, 128, 256, 512, 1024, 1536\}$) and $f(x)[1:m]$ is the first m dimensions of the embedding. This is what forces the model to pack the most important semantic information into the earliest dimensions, it's penalized at training time for any prefix length that fails to stand on its own, not just for the full vector.

30.8 The Two-Stage Training Pipeline

Modern embedding models are trained in two stages that mirror the pretraining/fine-tuning split for generative models.

30.8.1 Stage 1: Contrastive Pretraining on Weak Supervision

Train on billions of weakly supervised (anchor, positive) pairs using in-batch negatives. The goal is broad semantic organization, the large-scale geometry of the space. The signal is noisy but abundant.

Typical hyperparameters: - **Batch size**: as large as possible, 16,384–65,536 pairs per step for large-scale pretraining. Bigger batches mean more in-batch negatives, a harder effective task, and a better-organized space. - **Temperature**: 0.02–0.07, learned or fixed; lower values produce sharper embeddings with clearer cluster boundaries. - **Learning rate**: 1×10 to 5×10 with warmup, higher than generative fine-tuning, since the model is being trained on the contrastive objective from a relatively unshaped starting point, not lightly adjusted.

30.8.2 Stage 2: Fine-Tuning on Supervised Data with Hard Negatives

Fine-tune on a curated mixture of supervised retrieval datasets (MS MARCO, NQ, FEVER, etc.) with mined hard negatives. The goal shifts to fine-grained discrimination within the neighborhoods Stage 1 already established.

Key differences from Stage 1: - **Smaller batch size**: 256–1,024 pairs, supervised data is scarce relative to weak supervision. - **Hard negatives**: 1–7 per positive, mined via BM25 or cross-encoder retrieval. - **Lower learning rate**: 1×10 to 1×10 , preserving Stage 1's large-scale geometry while refining local discrimination. - **Task-specific prefixes**: query/passage asymmetric prefixes are added at this stage if the model uses them.

The two stages together produce models that are both broadly semantic (Stage 1) and precisely discriminative (Stage 2) a combination that's difficult to get from supervised data alone, since supervised retrieval datasets are far too small on their own to establish broad semantic organization.

30.9 Why Embedding Models Fail

Two failure modes are worth calling out specifically here because they trace directly back to training-data choices rather than anything you can fix at retrieval time.

30.9.1 Out-of-Domain Queries

Embedding models trained primarily on web retrieval data (MS MARCO, for instance, is derived from Bing search logs) perform well on general web queries and poorly on specialized domain queries, because they've simply never seen clinical notes, legal contracts, or scientific abstracts during training. The model organizes these texts by whatever surface-level semantic content it can infer which may not match the relevance structure a domain expert would assign.

Domain-adaptive fine-tuning (continuing training on domain-specific (query, passage) pairs, even weakly supervised ones) is the direct fix, because it's a training intervention: it changes what the model has seen, not how retrieval is run at query time.

30.9.2 Lexical Mismatch on Rare Terms

Embedding models handle synonymy well (queries and documents using different words for the same concept still retrieve correctly) but can fail on exact technical terms that are rare in the training corpus: product codes, proprietary terminology, neologisms. If a term barely appears during training, its embedding never gets enough gradient signal to settle into a useful position, and retrieval on that term becomes unreliable.

This is the complementary failure to BM25's: BM25 fails on synonymy but succeeds on exact lexical match, embedding models invert that. It's a training-side fact worth knowing, even though the fix (hybrid retrieval) is a retrieval-time decision.

30.10 Key Takeaways

- Embedding models are trained with contrastive objectives — not next-token prediction — because organizing the embedding space by semantic similarity is a property next-token prediction doesn't directly optimize.
- InfoNCE loss implements a softmax over in-batch similarities, minimizing the negative log-probability of identifying the correct positive among all negatives; batch size is the single most impactful hyperparameter, since it sets the difficulty of that classification task.
- In-batch negatives are efficient but introduce false negatives whenever a batch contains multiple genuinely relevant documents for the same query; deduplication and false-negative filtering mitigate this.
- Hard negatives — semantically similar but non-relevant documents — are what teaches fine-grained discrimination; BM25 mining, cross-encoder mining, and curriculum scheduling are the three standard approaches, each trading off mining cost against negative quality.
- Query/document asymmetry exists in training because a single encoder forced to represent both short queries and long documents identically tends to overweight surface similarity over semantic match; separate encoders or conditioning prefixes, trained jointly into one shared space, fix this.
- Pooling strategy (mean vs. CLS vs. last-token) and backbone choice (encoder-only vs. decoder-only) are training-architecture decisions with real quality/cost tradeoffs — mean-pooled encoders are cheap and fast, large decoder backbones with last-token pooling currently top MTEB at much higher inference cost.
- The two-stage pipeline — contrastive pretraining on billions of weak pairs, then supervised fine-tuning with hard negatives — produces models that are both broadly semantic and precisely discriminative, a combination supervised data alone is too scarce to achieve.
- Embedding model failures on out-of-domain queries and rare exact-match terms both trace back to what the training data did and didn't cover — domain-adaptive fine-tuning and hybrid retrieval are the respective fixes, one training-side and one retrieval-side.

30.11 Further Reading

- van den Oord, A., Li, Y., & Vinyals, O. (2018). *Representation Learning with Contrastive Predictive Coding.* arXiv. — Introduces InfoNCE and its mutual information lower bound interpretation; the foundational contrastive learning paper, with the batch-size/task-difficulty connection implicit here and made explicit in later work.

- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). *Dense Passage Retrieval for Open-Domain Question Answering.* EMNLP. — Introduces DPR and the bi-encoder architecture; the hard-negative mining procedure and MS MARCO training setup define the standard embedding model training recipe.

- Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP. — Adapts BERT for sentence-level embeddings via siamese networks and mean pooling; establishes that pooled BERT embeddings substantially outperform [CLS] pooling for semantic similarity.

- Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., & Wei, F. (2022). *Text Embeddings by Weakly-Supervised Contrastive Pre-training.* arXiv. — The E5 paper; the two-stage weakly-supervised then supervised training pipeline and the query/passage asymmetric prefix approach are the key contributions.

- Kusupati, A., Bhatt, G., Rege, A., Wallingford, M., Sinha, A., Ramanujan, V., Howard-Snyder, W., Chen, K., Kakade, S., Jain, P., & Farhadi, A. (2022). *Matryoshka Representation Learning.* NeurIPS. — Introduces MRL and the nested-dimension training objective.

- Wang, L., et al. (2023). *Improving Text Embeddings with Large Language Models.* arXiv. — E5-Mistral; demonstrates that decoder-only LLM backbones fine-tuned with synthetic training data outperform encoder-only models on MTEB; the synthetic (query, passage) data generation procedure using GPT-4 is the key methodological contribution.

Part VI

The Trial of the Answer

Part VII

The Keepers of the Waking Engine

[1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {transformer-based} generative models," in *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, 2022, pp. 521–538.

[2] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *arXiv preprint arXiv:2203.02155*, 2022.

[3] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," in *International conference on machine learning*, PMLR, 2023, pp. 10835–10866.

[4] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," *Advances in neural information processing systems*, vol. 30, 2017.

[5] D. M. Ziegler *et al.*, "Fine-tuning language models from human preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[7] L. Engstrom *et al.*, "Implementation matters in deep policy gradients: A case study on ppo and trpo," *arXiv preprint arXiv:2005.12729*, 2020.

[8] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *arXiv preprint arXiv:2305.18290*, 2023.

[9] M. G. Azar *et al.*, "A general theoretical paradigm to understand learning from human preferences," in *International conference on artificial intelligence and statistics*, PMLR, 2024, pp. 4447–4455.

[10] J. Hong, N. Lee, and J. Thorne, "Orpo: Monolithic preference optimization without reference model," in *Proceedings of the 2024 conference on empirical methods in natural language processing*, 2024, pp. 11170–11189.

[11] Z. Shao *et al.*, "Deepseekmath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[12] Y. Bai *et al.*, "Constitutional ai: Harmlessness from ai feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[13] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24824–24837, 2022.

[14] H. Lightman *et al.*, "Let's verify step by step," in *International conference on learning representations*, 2024, pp. 39578–39601.